Foundation Models and Generative AI



Angshuman Paul

Assistant Professor

Department of Computer Science & Engineering

Syllabus

1. Introduction to Foundation Models

- Transformers & Attention: Introduction to Transformers, Self-Attention, Cross-Attention, Perceiver (Arbitrary I/O with Transformers), Self-Attention & Non-Parametric Transformers
- Diffusion: Sampling, Diffusion Models, Latent Diffusion Models (LDM)
- Applications: LLMs, LVMs, and Applications in Vision

2. Foundation Model Architectures

- Transformers in Language: GPT, BERT, RetNet, State Space Models
- Transformers in Vision: ViT, MLP-Mixer, Conformer, Vision-Language (VL) models, CLIP
- Architectures: Dual-Encoder, Fusion, Encoder-Decoder, Adapted LLM, Unified Architectures

3a. Multi-Modal & Generative Foundation Models

- Training & Inference: Training Objectives, Contrastive Learning, Efficient Inference Techniques, Pre-training, Fine-tuning & Parameter-Efficient Fine-tuning (LoRA, QLoRA), Flash Attention, Retrieval-Augmented Generation (RAG)

3b. Evaluation & Benchmarking

- Evaluation Protocols and Standard Benchmarks, Hallucinations, Bias & Fairness, Privacy, Memorization, Machine Unlearning, Prompt Engineering

Course Logistics

- Instructor: Angshuman Paul
- Contact: apaul@iitj.ac.in
- Class Timings: Monday, Wednesday, Friday: 8-8.50 AM
- Office Hours: Friday: 9.30 – 11 AM

Evaluation Scheme (Tentative)

- Mini Competition + Quiz +Assignments +Viva (40%)
- Minor (20%)
- Major (40%)
- Slides will be shared every Sunday

Mini Competition & Assignments

- Details will be announced next week

Books and Study Materials

- Book:
  - Introduction to Foundation Models: Pin-Yu Chen , Sijia Liu, 2025
  - The Little Book of Generative AI Foundations: An Intuitive Mathematical Primer: Tianhua Chen, 2026
  - Foundation Models for Natural Language Processing — Pre-trained Language Models Integrating Media, Gerhard Paaß & Sven Giesselbach, Springer, 2023.

- Other useful books:
  - Generative AI and Foundation Models: A Gentle Technical Introduction
  - Christopher M. Bishop, Hugh Bishop (2023), Deep Learning Foundations and Concepts, Springer
  - I. Goodfellow, Y. Bengio, A. Courville (2016), Deep Learning, The MIT Press, 1st Edition.
  - A. Zhang, Z. Lipton, M. Li, A. Smola (2020) Dive into Deep Learning (Release 0.7.1), https://d2l.ai/d2l-en.pdf

- Other books & Online materials:
  - https://www.youtube.com/watch?v=y1fGlAECNFM&list=PLXV9Vh2jYcjbnv67sXNDJiO8MWLA3ZJKR
  - https://www.youtube.com/watch?v=Ub3GoFaUcds&list=PLoROMvodv4rOCXd21gf0CF4xr35yINeOy&index=1
  - https://www.youtube.com/watch?v=skWhn8W9P_Y&list=PLZ2ps__7DhBa5xCmncgH7kPqLqMBq7xlu
  - https://www.youtube.com/watch?v=XZ0PMRWXBEU&list=PLoROMvodv4rPOWA-omMM6STXaWW4FvJT8

Prerequisites

- DL: Theory and Coding

- Basics of
  - Linear Algebra
  - Calculus
  - Probability

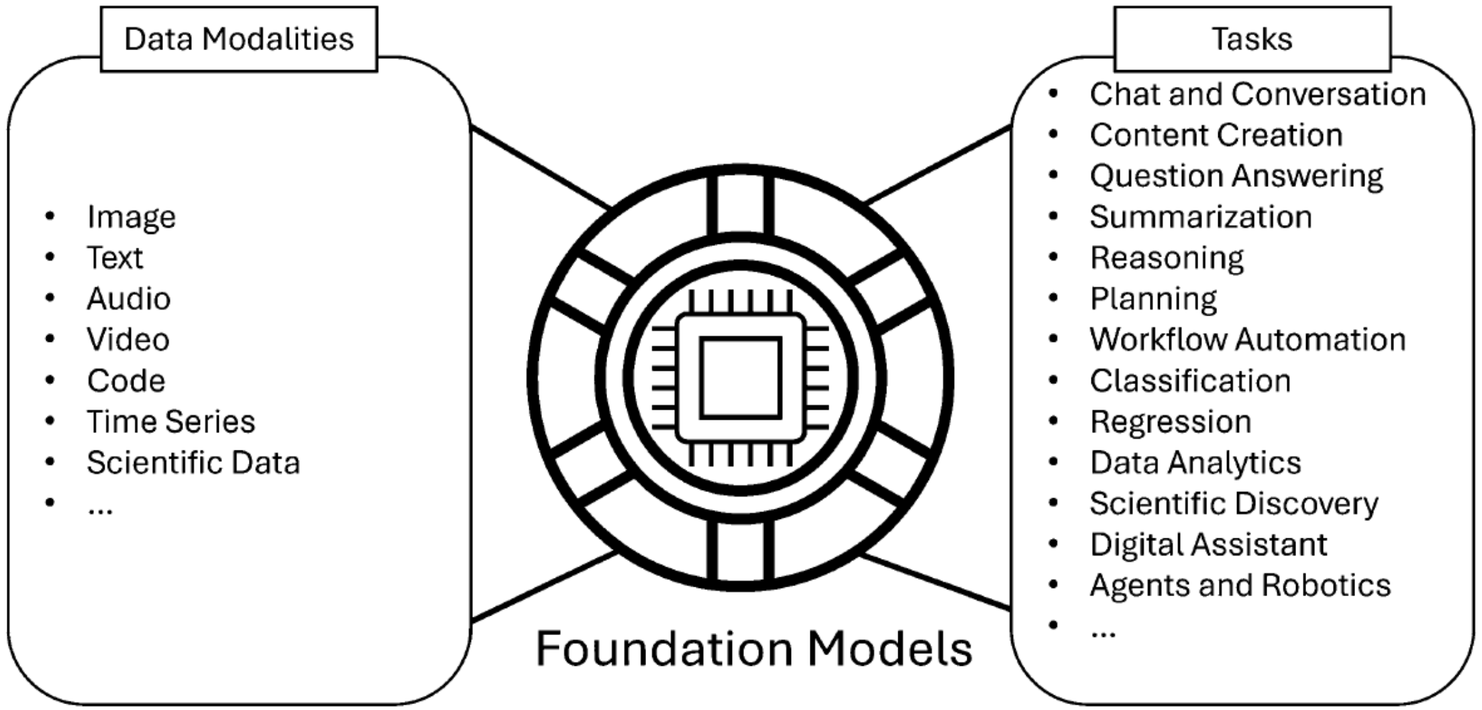
Why Should We Study This Course?

What is a Foundation Model?

What is a Foundation Model?

- Foundation models are high-capacity neural networks (e.g., neural networks with billions of trainable parameters)
- Trained with large-scale data (e.g., the entire text data scraped from the Internet)
- Once a foundation model is trained, it can be used to solve various downstream machine learning tasks.

What is a Foundation Model?



What is Generative AI?

- *Generative AI* (GenAI) refers to machine learning technology and applications featuring generative capabilities, such as large language models (LLMs) that can understand human users' instructions (also known as the "context") and generate the desired content and actions.

What is generative AI?

Generative AI refers to models that learn the underlying structure of data well enough to produce new, original examples — text, images, audio, or code — rather than just labeling or classifying what already exists.

Discriminative models

Learn a boundary between categories.

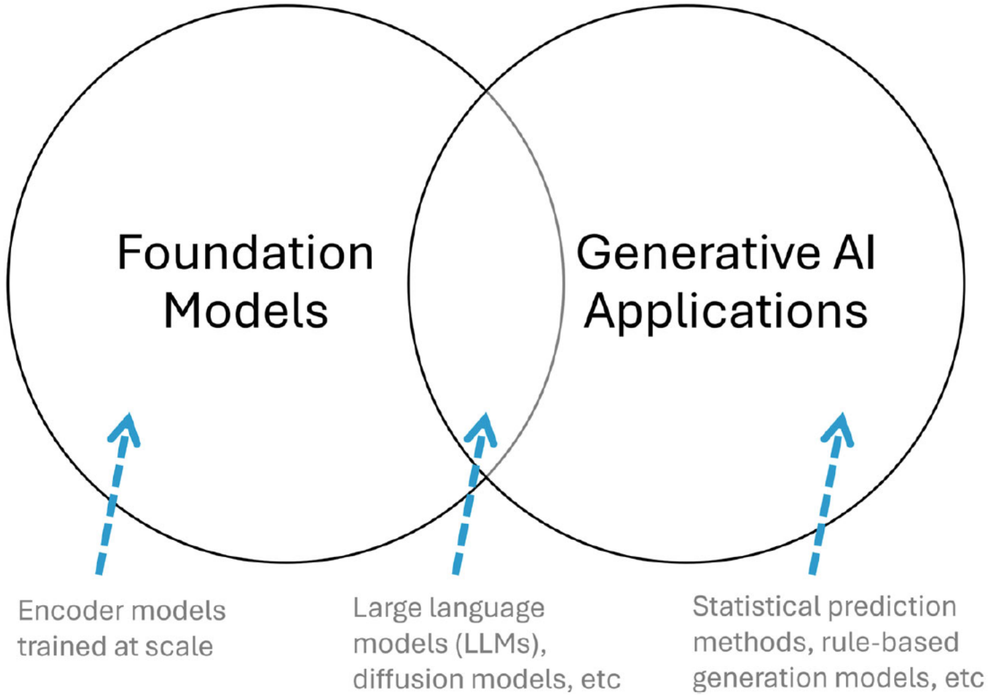
"Is this a cat or a dog?"

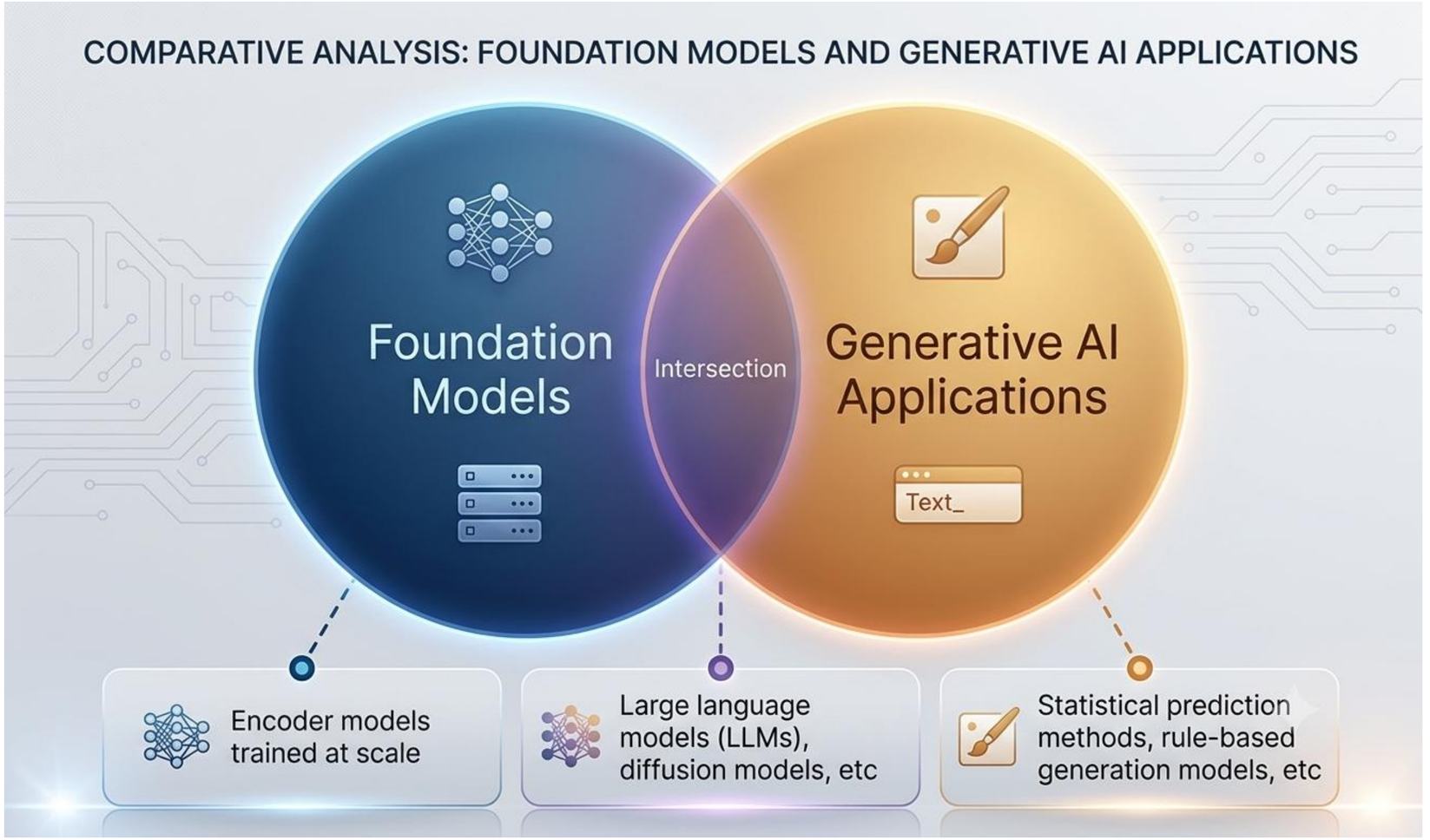
Generative models

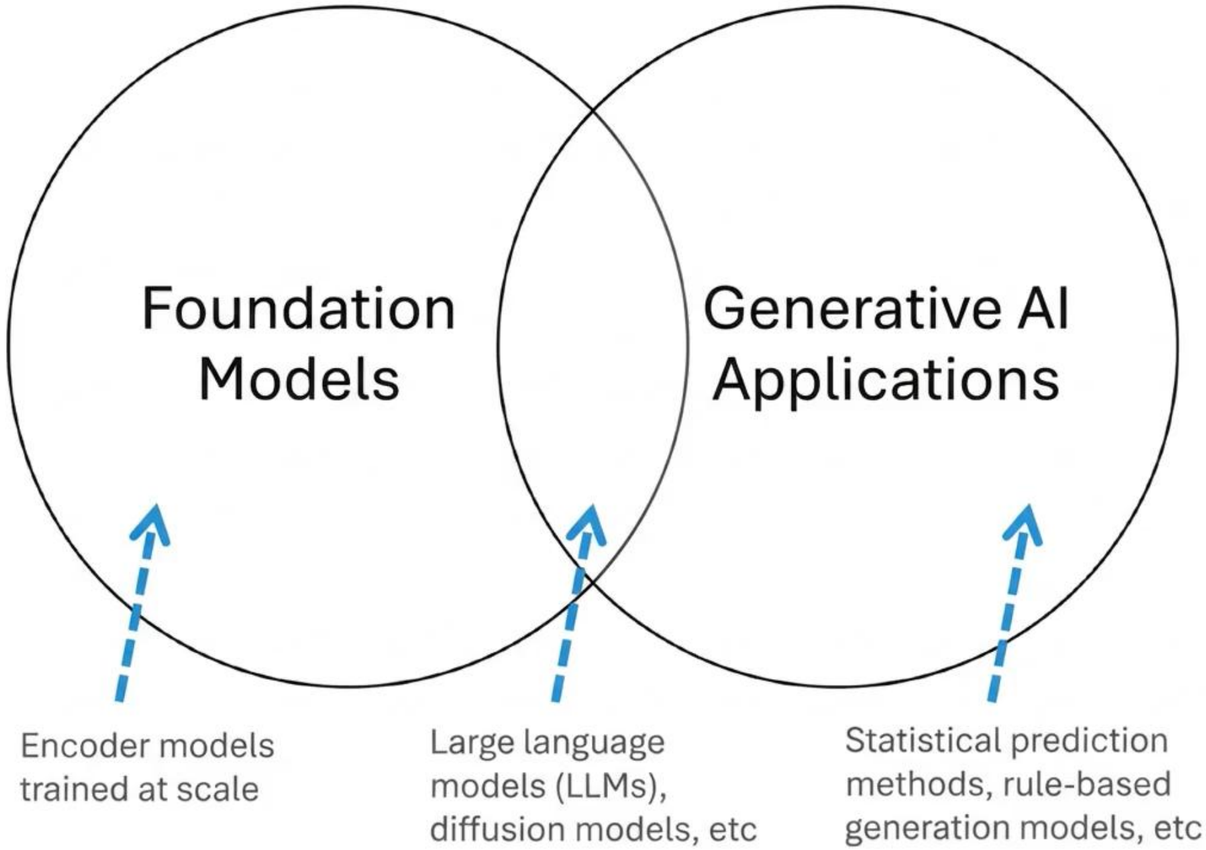
Learn the data distribution itself.

"Draw me a cat."

What is Generative AI?




COMPARATIVE ANALYSIS: FOUNDATION MODELS AND GENERATIVE AI APPLICATIONS
Foundation Models
Intersection
Generative AI Applications
Text_
Encoder models trained at scale
Large language models (LLMs), diffusion models, etc
Statistical prediction methods, rule-based generation models, etc


Foundation Models
Generative AI Applications
Encoder models trained at scale
Large language models (LLMs), diffusion models, etc
Statistical prediction methods, rule-based generation models, etc

Why now? Landmark breakthroughs

A handful of systems, released within a few years of each other, moved generative AI from a research topic to a everyday tool.

Text

ChatGPT

A conversational large language model that showed foundation models could follow instructions and reason across nearly any topic.

Image

Stable Diffusion

An open, efficient diffusion model that turns text prompts into high-quality images, run entirely from noise.

Image

DALL·E

One of the first models to reliably turn natural-language descriptions into coherent, creative images.

Deep Generative Model Families

Adversarial

GANs

Two networks compete: a generator that creates fakes, and a discriminator that tries to catch them. Competition drives both to improve.

Reconstructive

Variational Autoencoders

An encoder compresses data into a compact code; a decoder reconstructs it. Variational autoencoders add structure for generating new samples.

Denoising

Diffusion models

Data is gradually noised into randomness, and the model learns to reverse the process — denoising step by step to generate new samples.

Invertible

Flow-based models

A sequence of invertible transformations maps data to a simple base distribution and back. Each step is invertible, so the model can compute the exact likelihood of the data.

Sequential

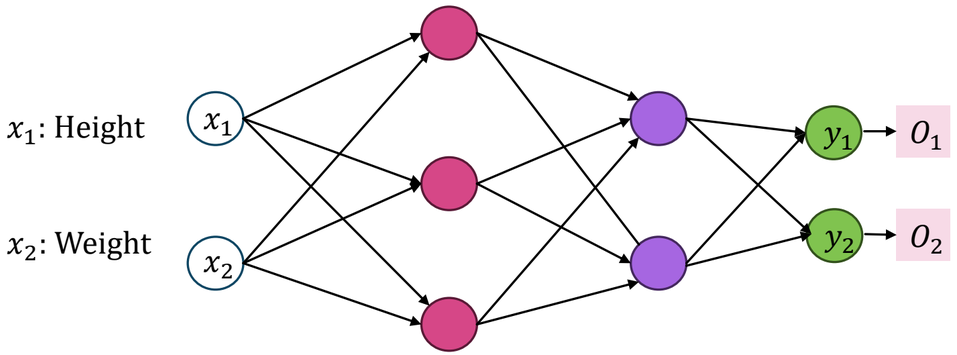
Autoregressive models

Data is generated one element at a time, each conditioned on everything generated so far. Exact likelihoods are tractable, but generation is inherently sequential.

Generative Model Families

Can you name a generative model that is not designed using neural networks?

Neural Networks: A Quick Recap



Predicted output

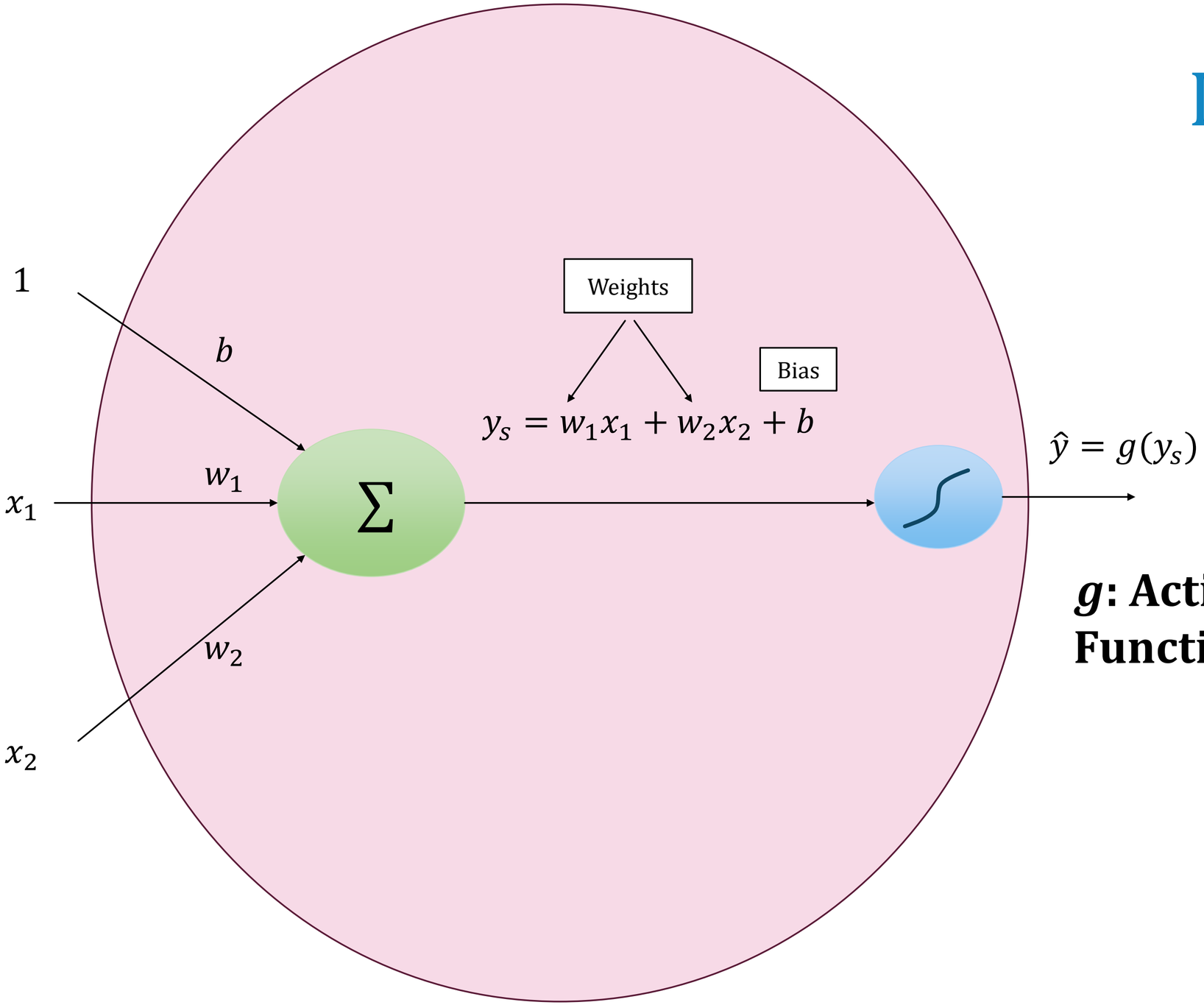
$$\boldsymbol{y_1 = \alpha_1 x_1 + \alpha_2 x_2 + \alpha_3}$$
$$\boldsymbol{y_2 = \beta_1 x_1 + \beta_2 x_2 + \beta_3}$$

- There is no guarantee that y_i will be 0 or 1
- So, we do softmax

$$o_i = \frac{\exp(y_i)}{\sum_{\forall j} \exp(y_j)}$$

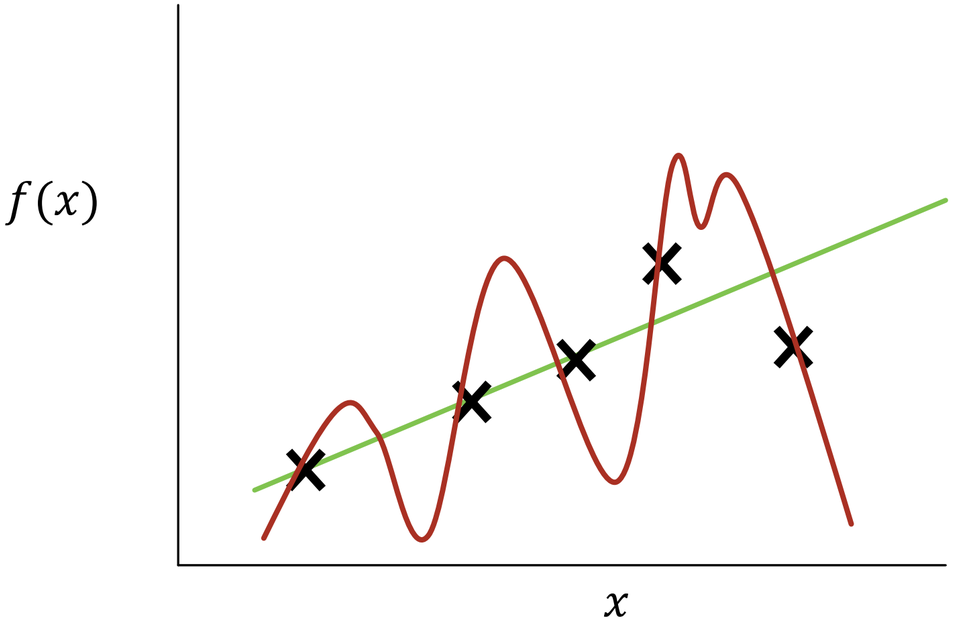
- $o_1 = \frac{\exp(y_1)}{\exp(y_1)+\exp(y_2)}$
- $o_2 = \frac{\exp(y_2)}{\exp(y_1)+\exp(y_2)}$

Inside a Node



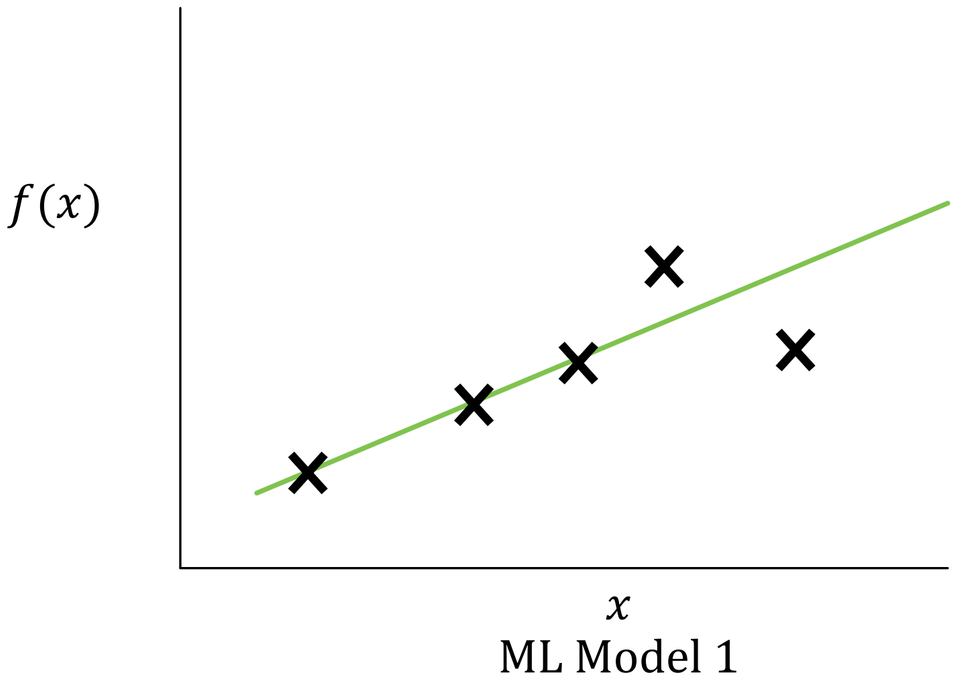
g: Activation Function

Bias and Variance

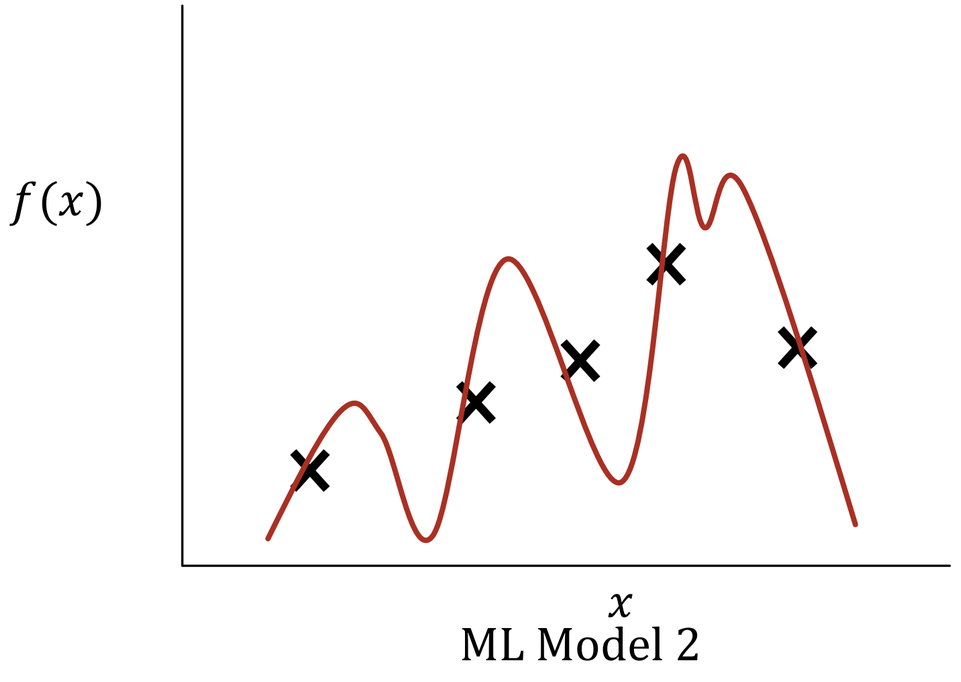


- Bias is the difference between the average prediction of our model and the correct value which we are trying to predict. Model with high bias pays very little attention to the training data and oversimplifies the model. It leads to high error on training and test data.
- Variance is the variability of model's prediction across different sets of data

Fitting Possible Curves on the Training Data



ML Model 1

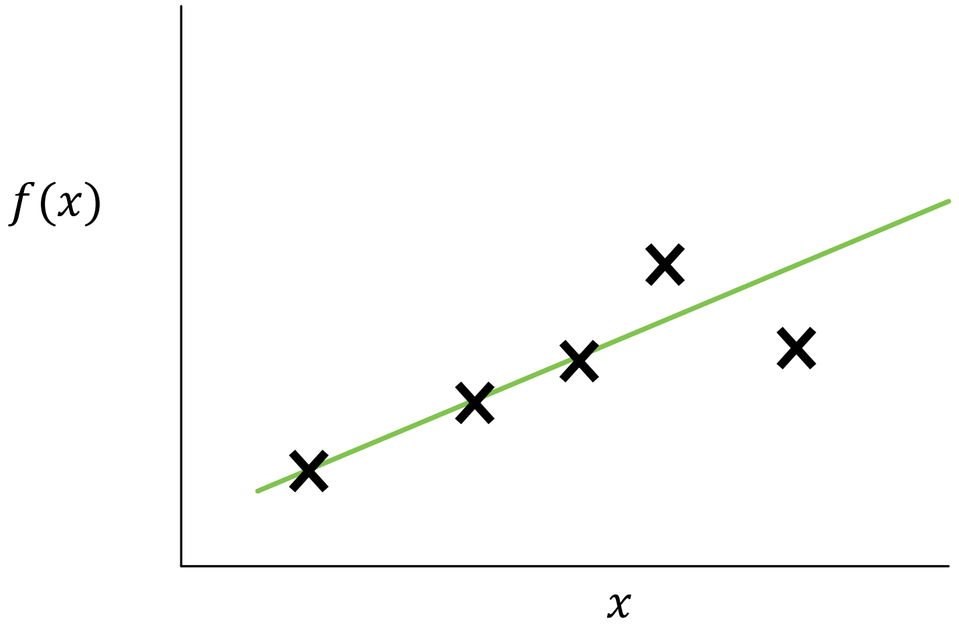


ML Model 2

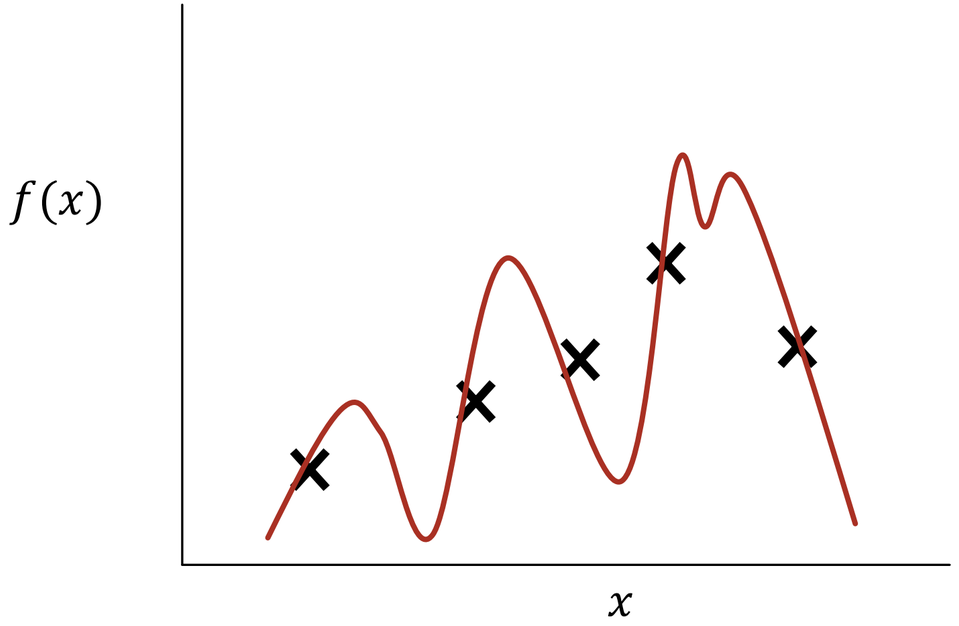
- More assumption: simple model/ simple curve
  - More bias

- Ranking of bias (high to low)
  - Green curve (model 1)
  - Red curve (model 2)

Fitting Possible Curves on the Training Data

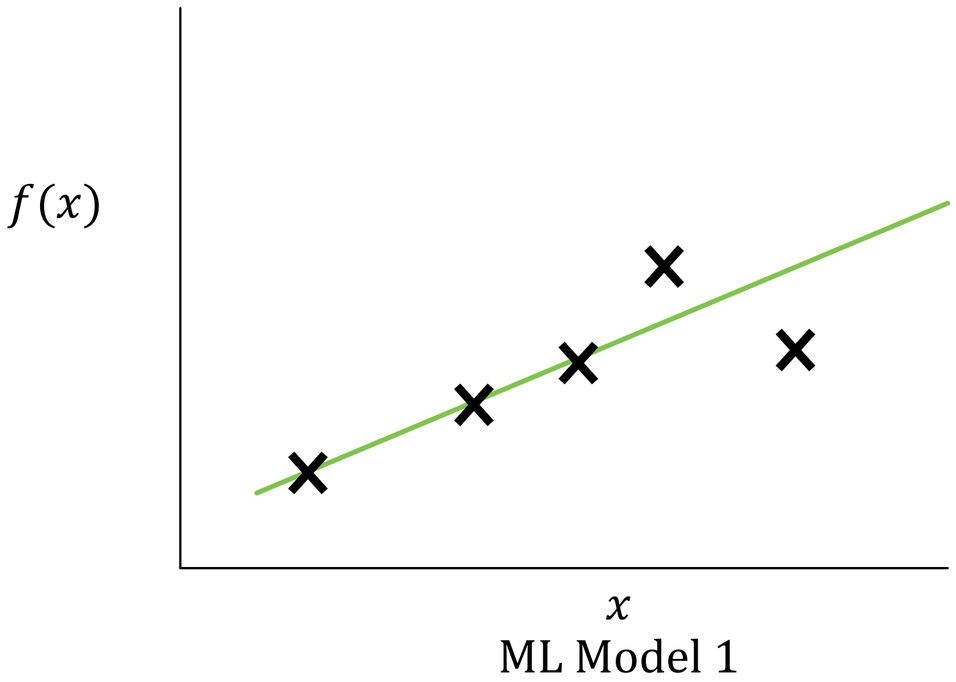


ML Model 1

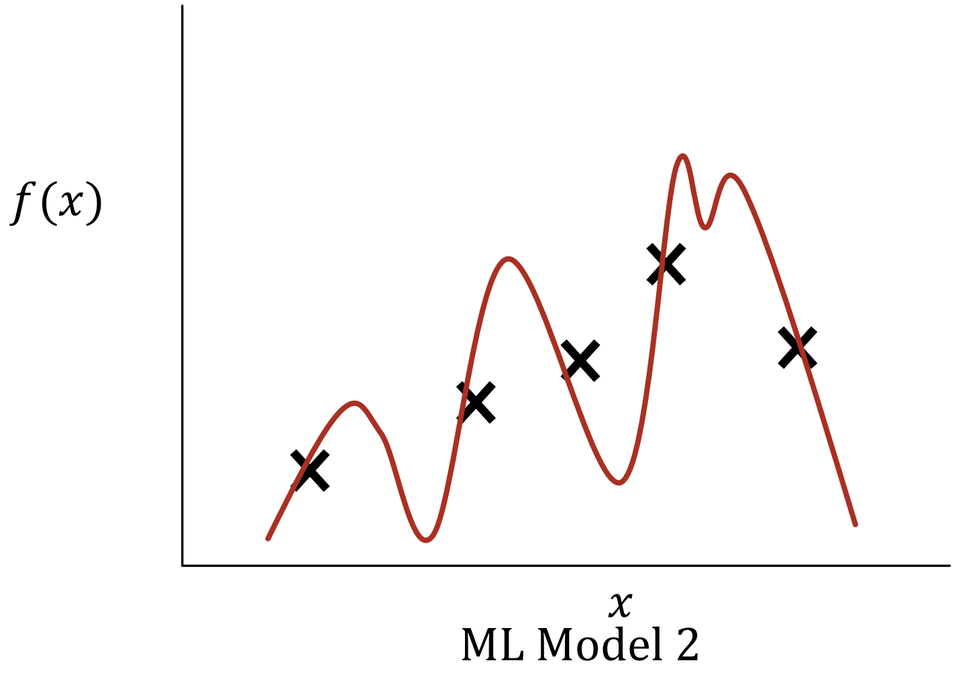


ML Model 2

Fitting Possible Curves on the Training Data



ML Model 1

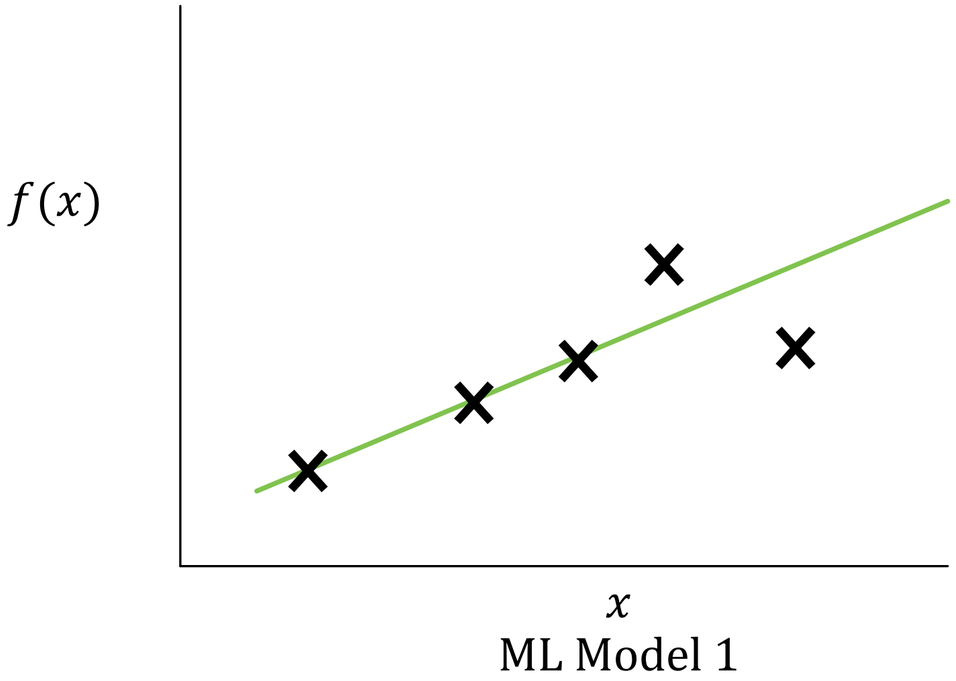


ML Model 2

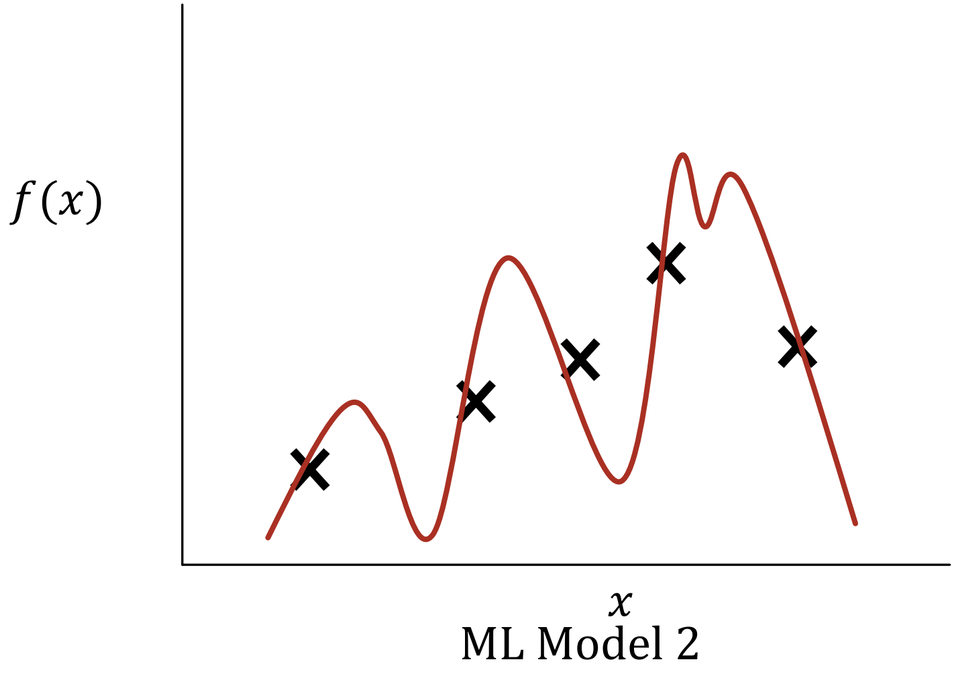
- More assumption: simple model/ simple curve
  - More bias

- Ranking of bias (high to low)
  - Green curve (model 1)
  - Red curve (model 2)

Fitting Possible Curves on the Training Data



ML Model 1

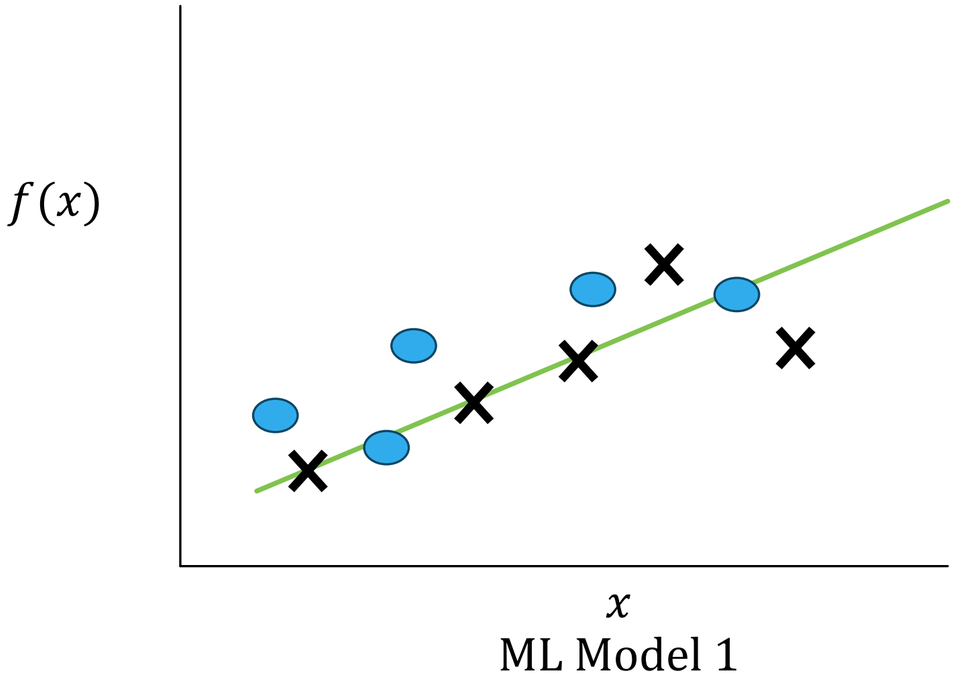


ML Model 2

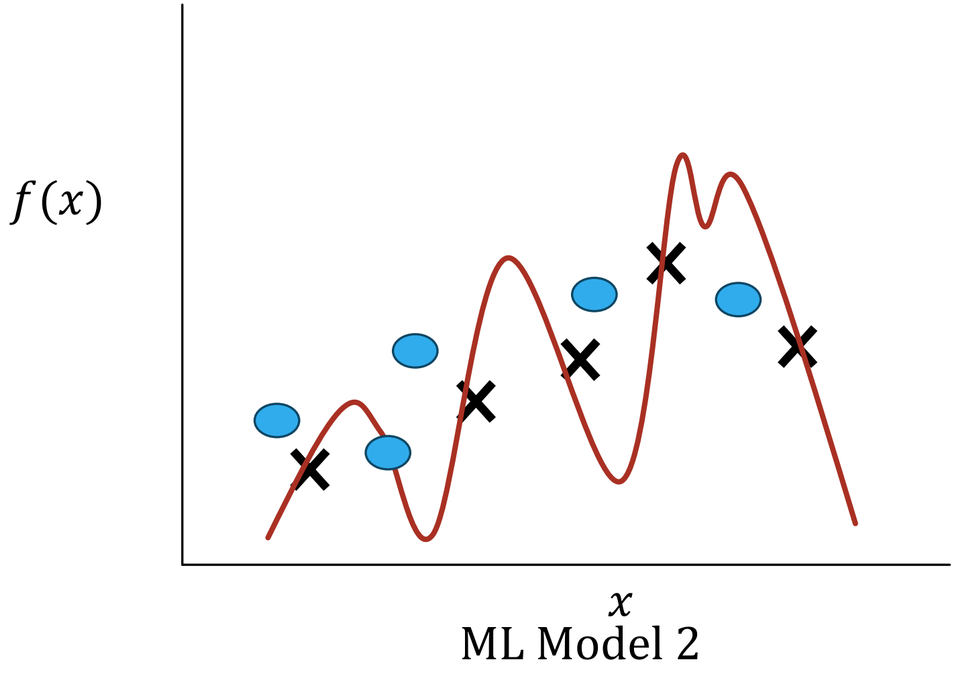
- Ranking of bias (high to low)
  - Green curve (model 1)
  - Red curve (model 2)

- Which curve has more error when compared to training examples?
- Ranking of training error (high to low)
  - Green curve
  - Red curve

Now, Let's See How They Perform on Test Data 1



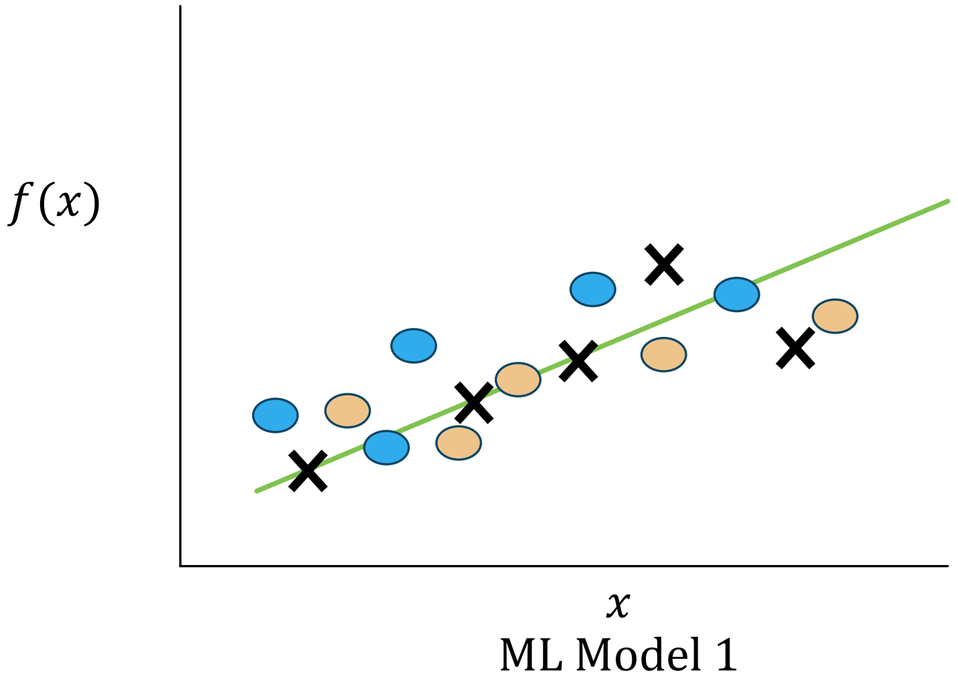
ML Model 1



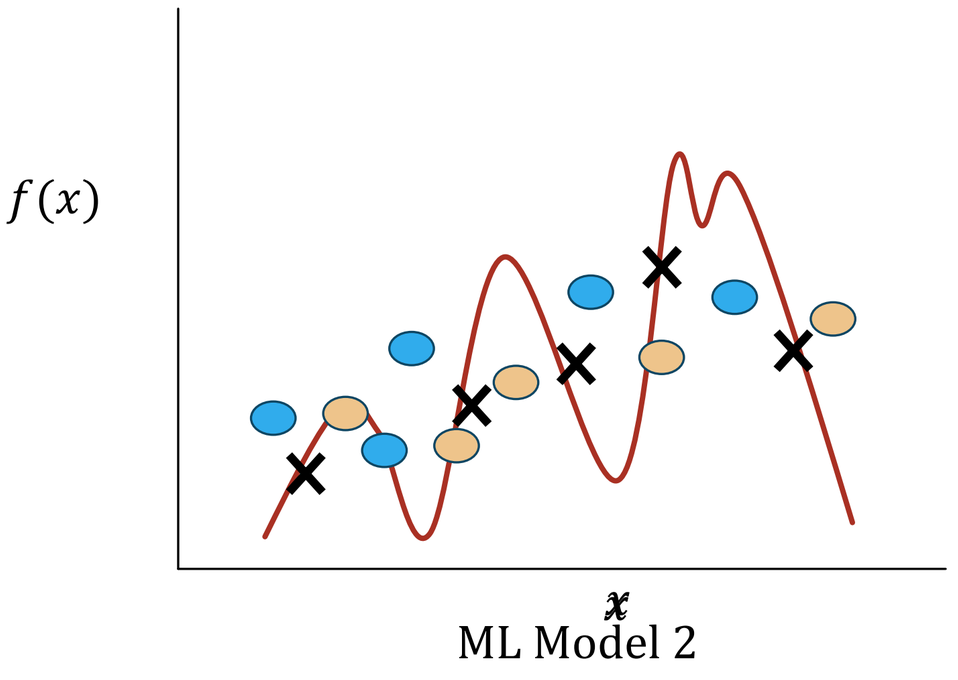
ML Model 2

- Model 1
  - Similar to its performance on training data
  - Test error $e_g(1)$
- Model 2
  - Very different from its performance on training data
  - Test error $e_r(1)$

Now, Let's See How They Perform on Test Data 2



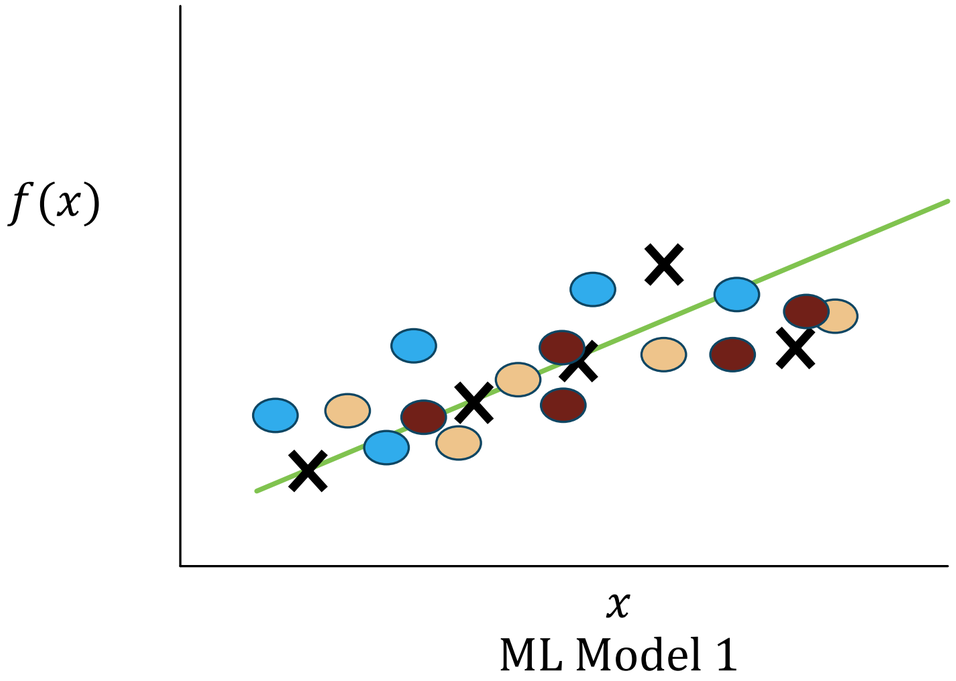
ML Model 1



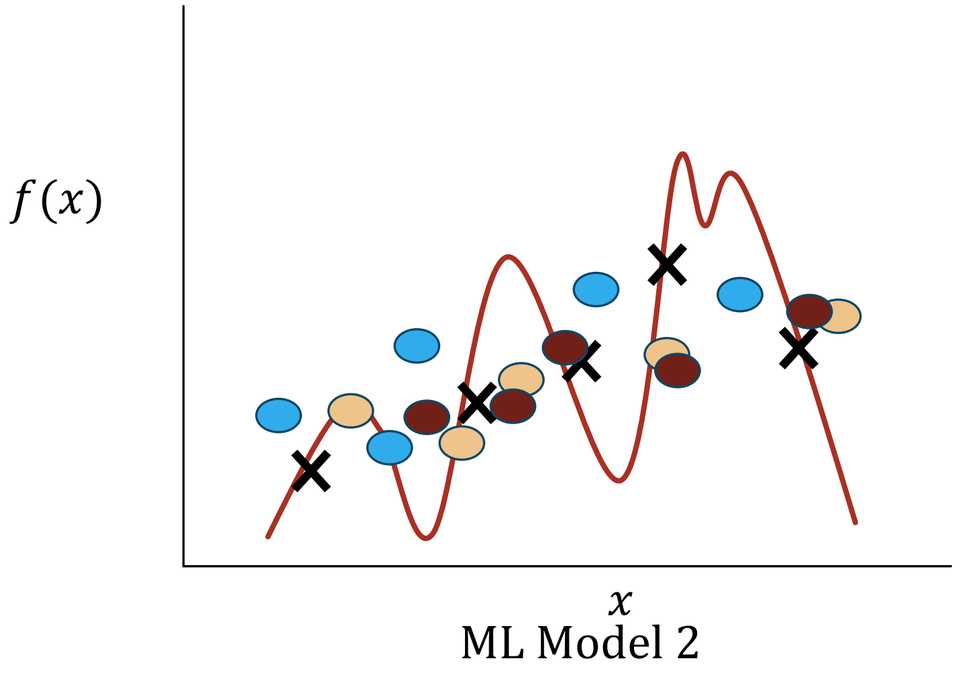
ML Model 2

- Model 1
  - Similar to its performance on training data, and test data 1
  - Test error $e_g(2)$
- Model 2
  - Very different from its performance on training data, and test data 1
  - Test error $e_r(2)$

Now, Let's See How They Perform on Test Data 3

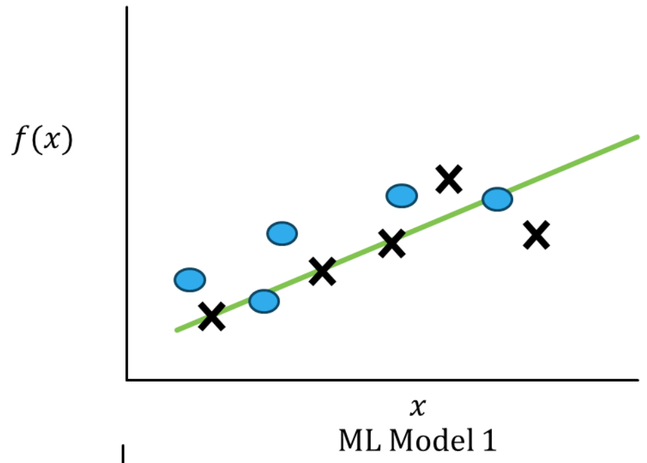


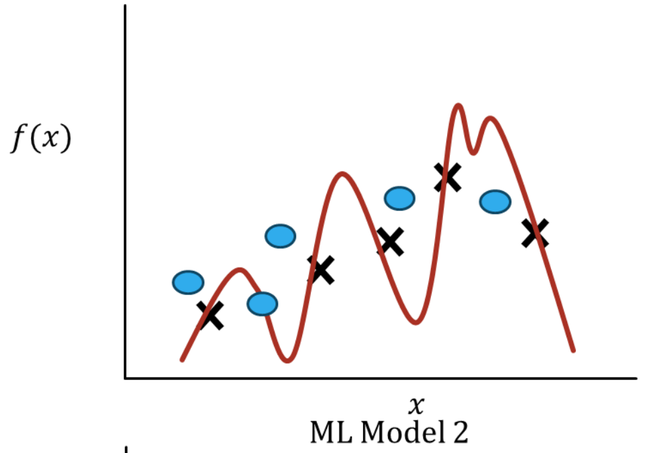
ML Model 1

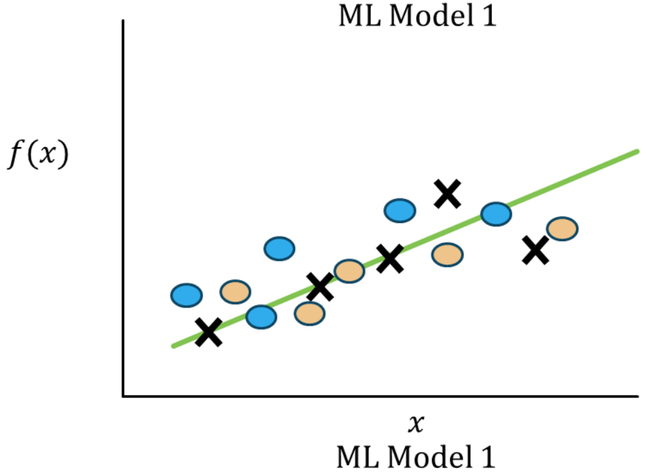


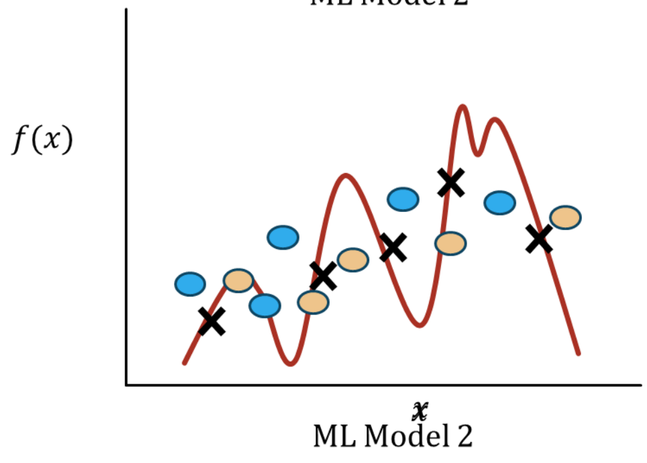
ML Model 2

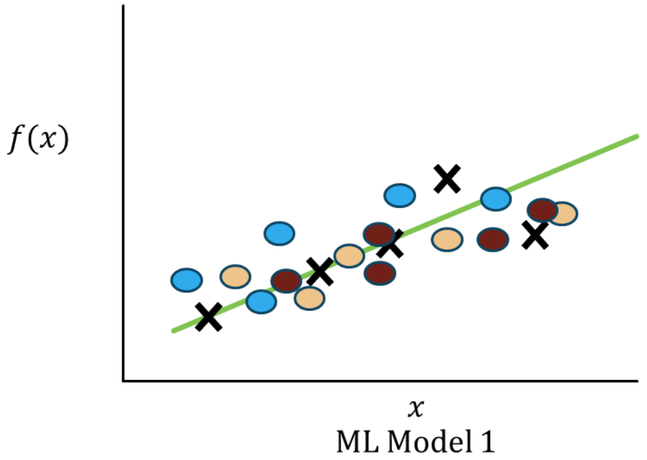
- Model 1
  - Similar to its performance on training data, test data 1, and test data 2
  - Test error $e_g(3)$
- Model 2
  - Very different from its performance on training data, test data 1, and test data 2
  - Test error $e_r(3)$

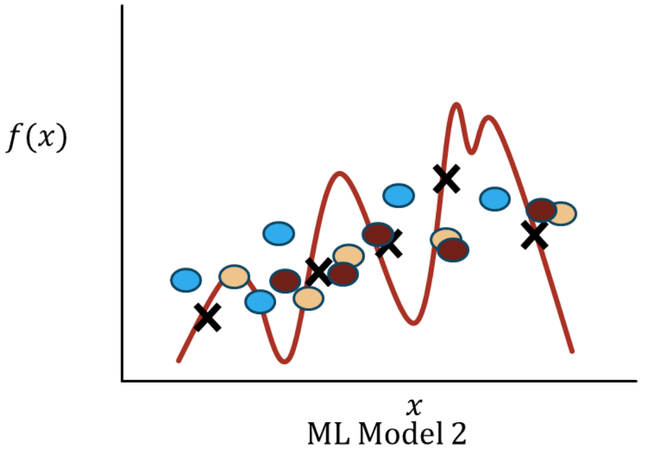




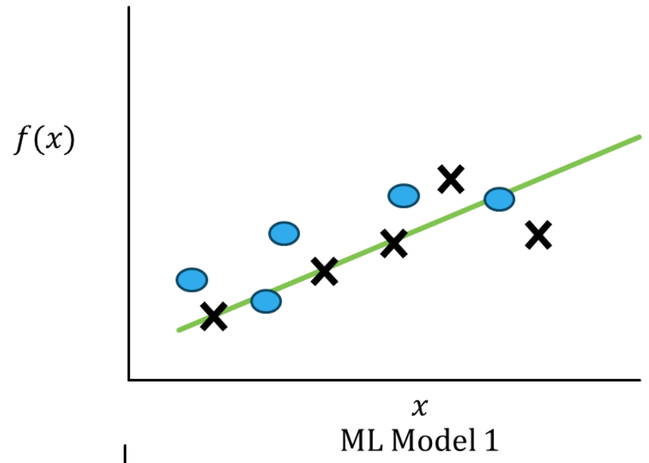


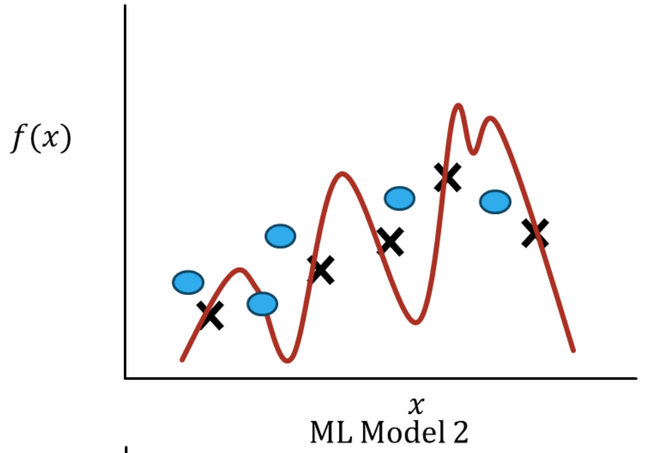


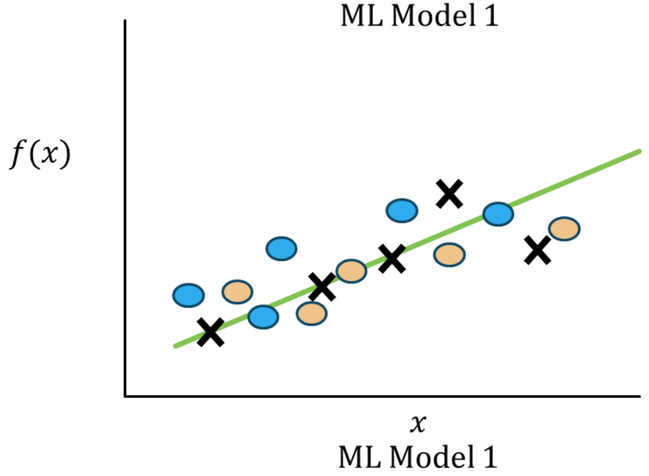


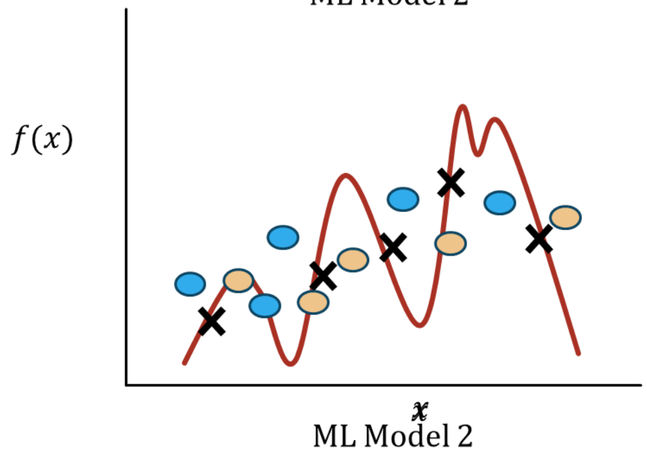


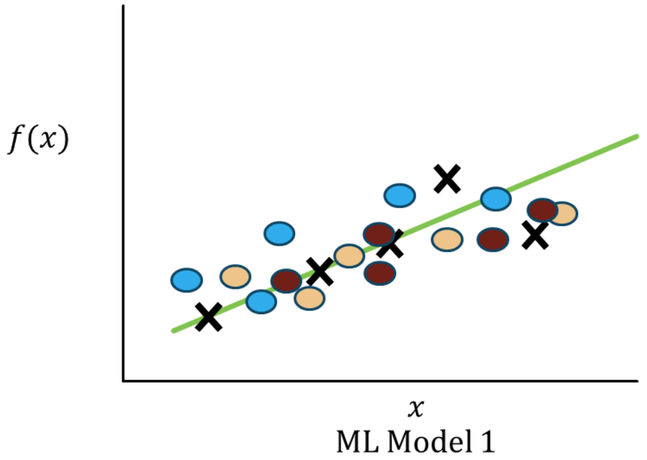
So, What do We Observe?

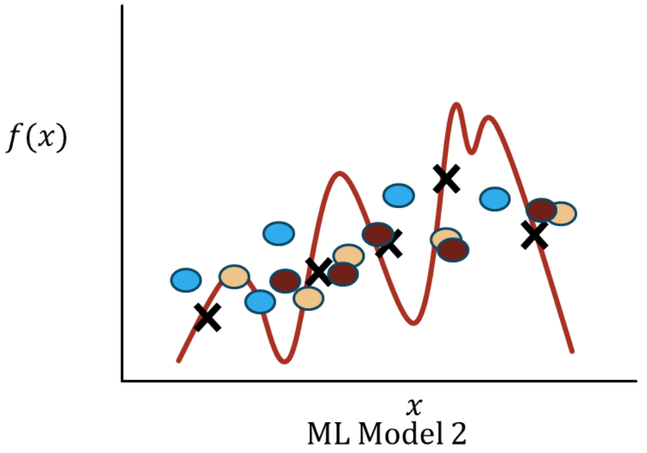






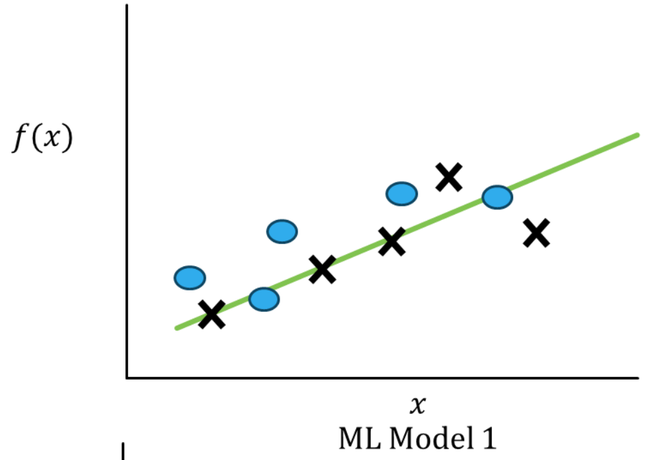


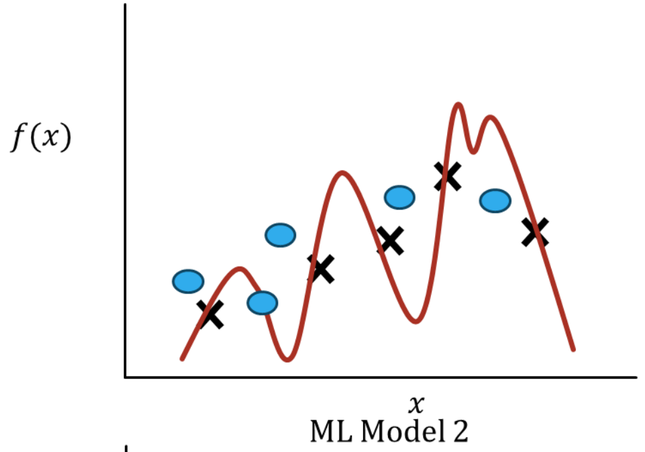


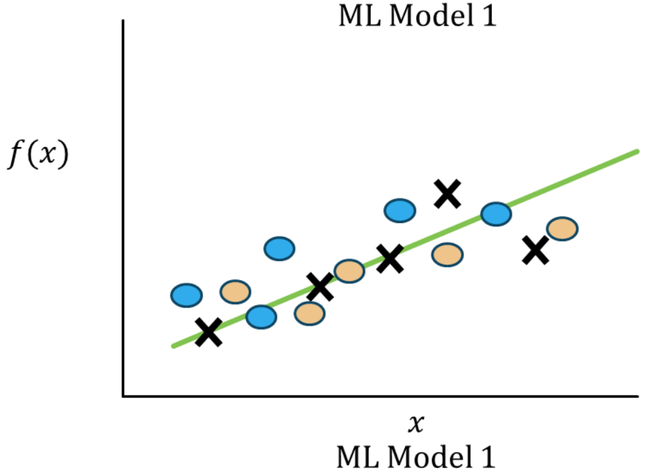


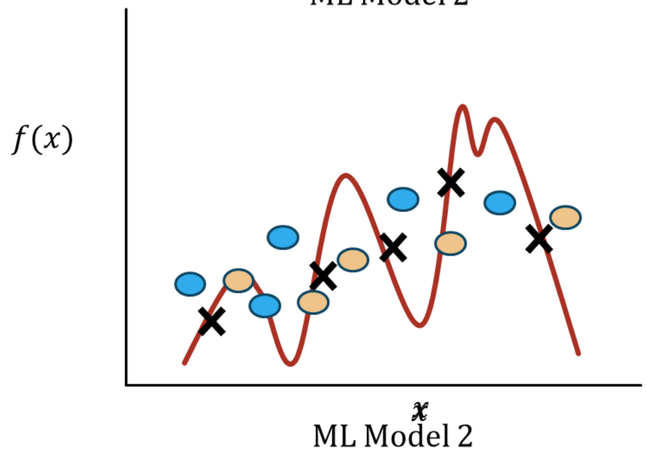
So, What do We Observe?

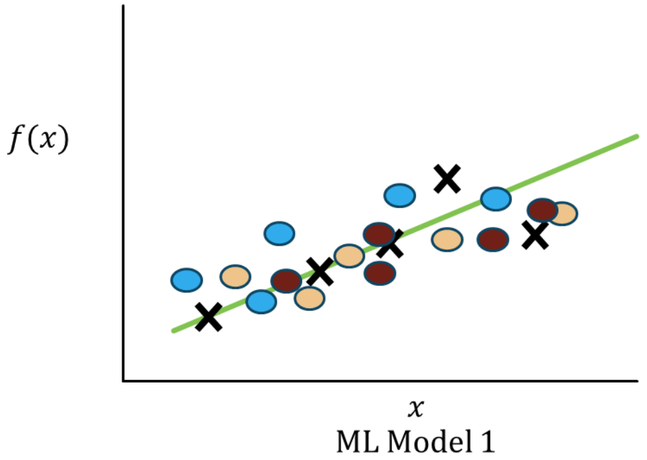
- The three sets of test data are similar to the training data with slight differences
- For the green curve (model 1), the errors in the test data $e_g(1), e_g(2), e_g(3)$ are similar to each other and similar to training error
  - Variance of the error is low
  - Low variance
  - High bias

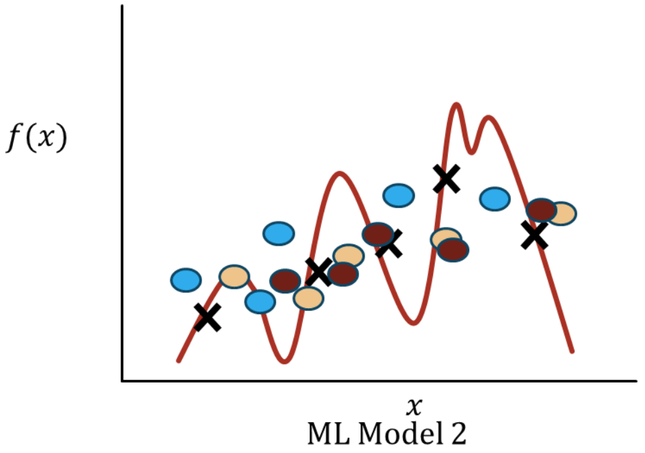








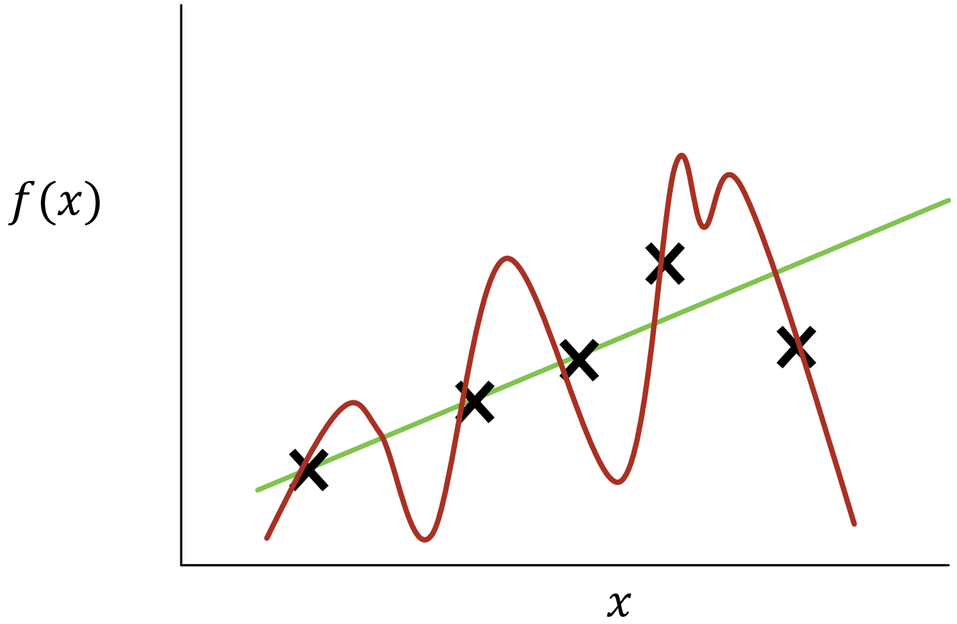




So, What do We Observe?

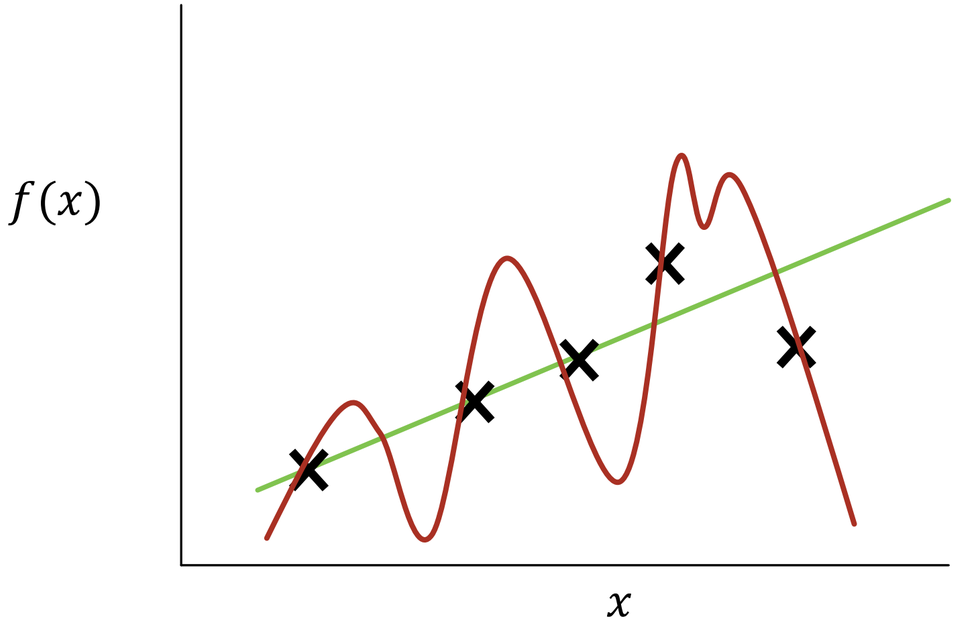
- The three sets of test data are similar to the training data with slight differences
- For the red curve (model 2), the errors in the test data $e_r(1), e_r(2), e_r(3)$ are not similar to each other and not similar to training error
  - Variance of the error is high
  - High variance
  - Low bias

Bias and Variance



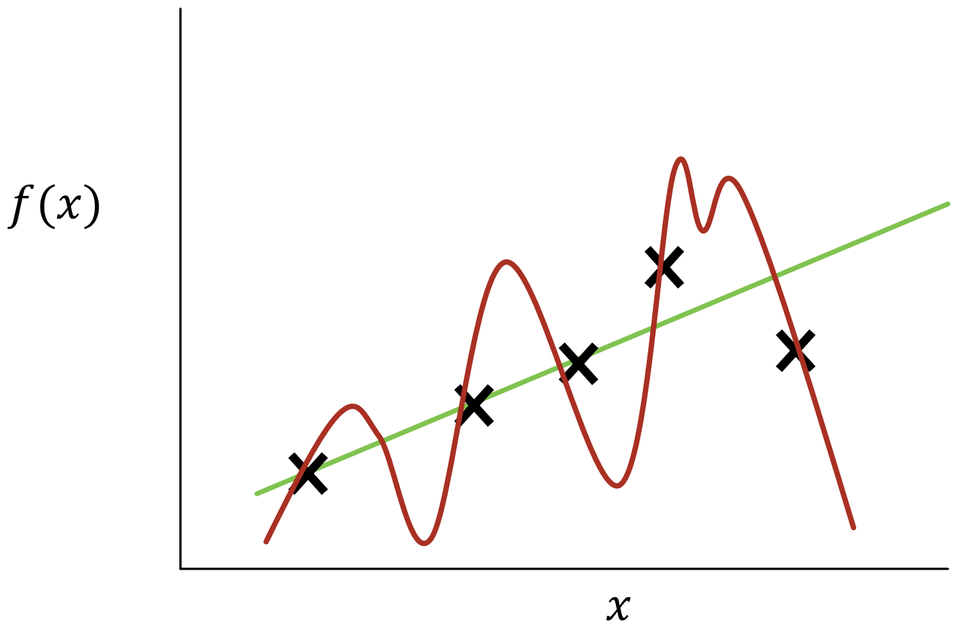
- More assumption: simple model/ simple curve
  - More bias, less variance
- Less assumption: complex model/ complex curve
  - Less bias, more variance

Impact of Bias and Variance



- More assumption: simple model/ simple curve
  - Less parameters
  - More bias, less variance
  - Underfitting
  - Consistent but relative poorer performance across datasets
- Less assumption: complex model/ complex curve
  - More parameters
  - Less bias, more variance
  - Overfitting
  - Inconsistent performance across datasets

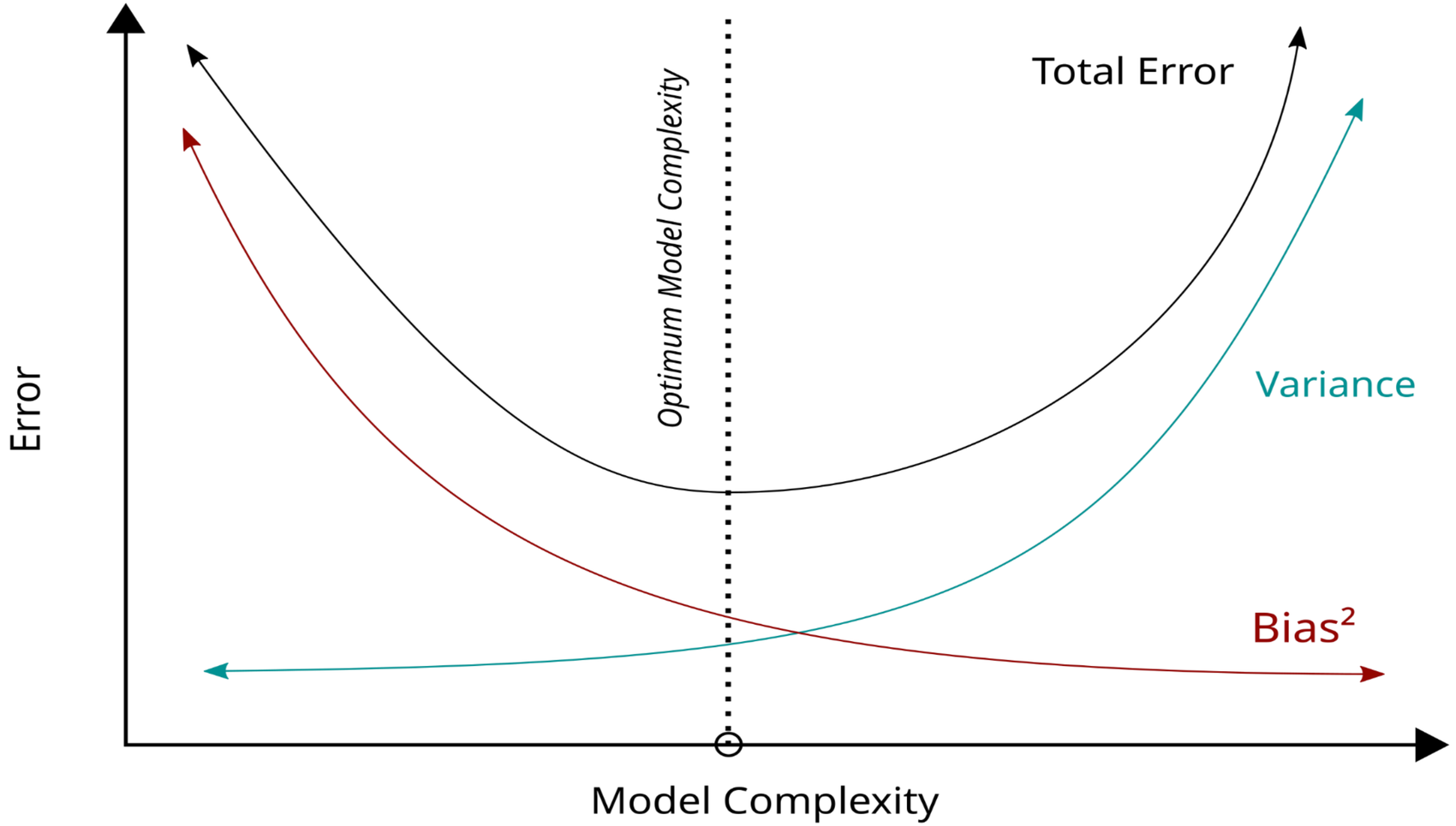
Impact of Bias and Variance



So, we want low bias and low variance in any model. But we can't achieve these at the same time

- More assumption: simple model/ simple curve
  - Less parameters
  - More bias, less variance
  - Underfitting
  - Consistent but relative poorer performance across datasets
- Less assumption: complex model/ complex curve
  - More parameters
  - Less bias, more variance
  - Overfitting
  - Inconsistent performance across datasets

Errors



https://en.wikipedia.org/wiki/Bias%E2%80%93variance_tradeoff#/media/File:Bias_and_variance_contributing_to_total_error.svg

Convolutional Neural Networks

Type: conv - Stride: 1 Padding: 0



Input Output

https://towardsdatascience.com/what-is-transposed-convolutional-layer-40e5e6e31c11

How Convolution Works

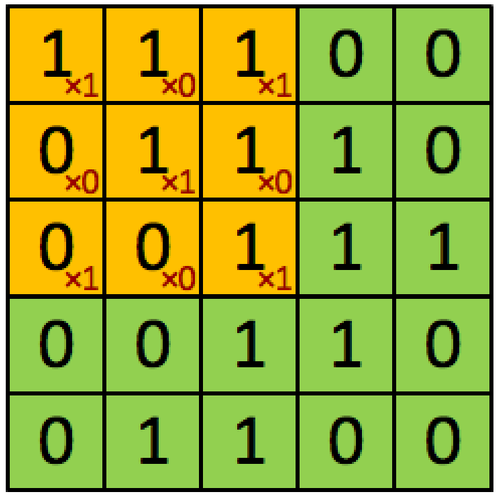
1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

Image (I)

1	0	1
0	1	0
1	0	1

Kernel (K)

How Convolution Works



Image

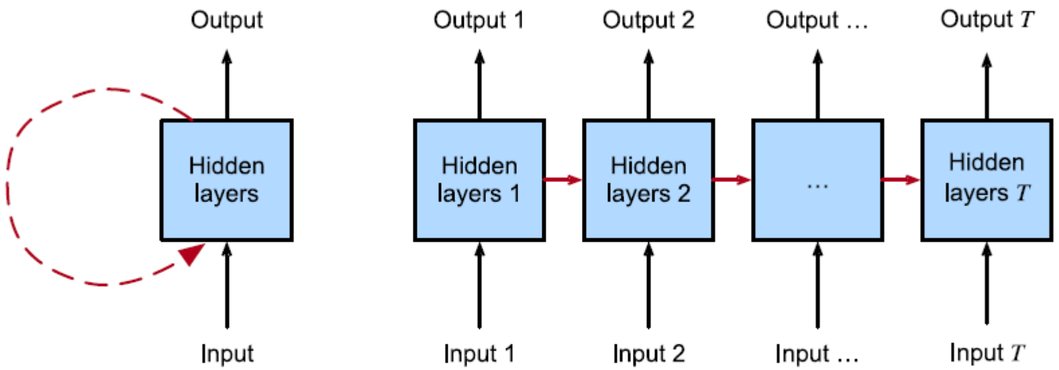
Convolved Feature

s

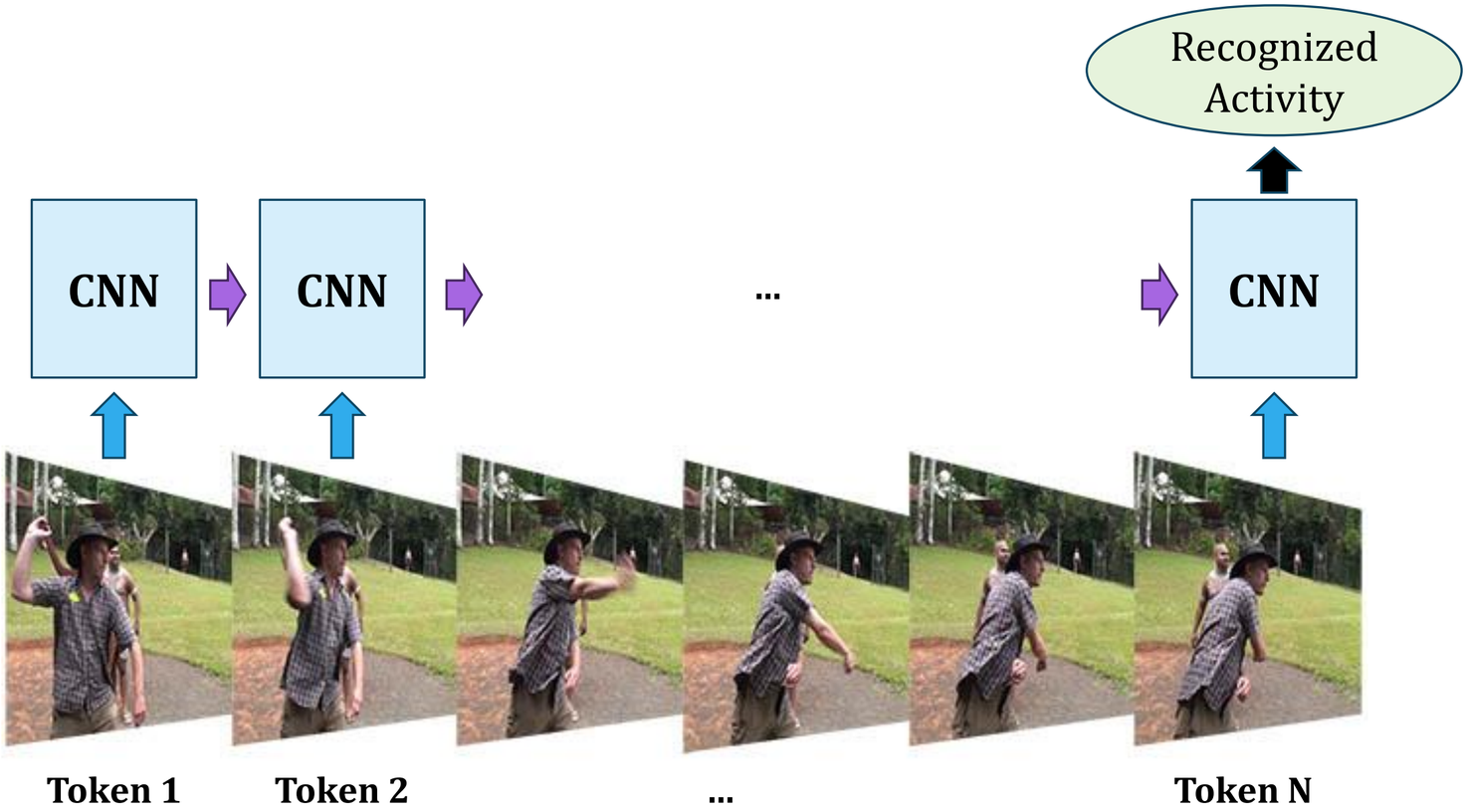
Recurrent Neural Networks

- Recurrent neural networks (RNNs) are deep learning models that capture the dynamics of sequences via recurrent connections

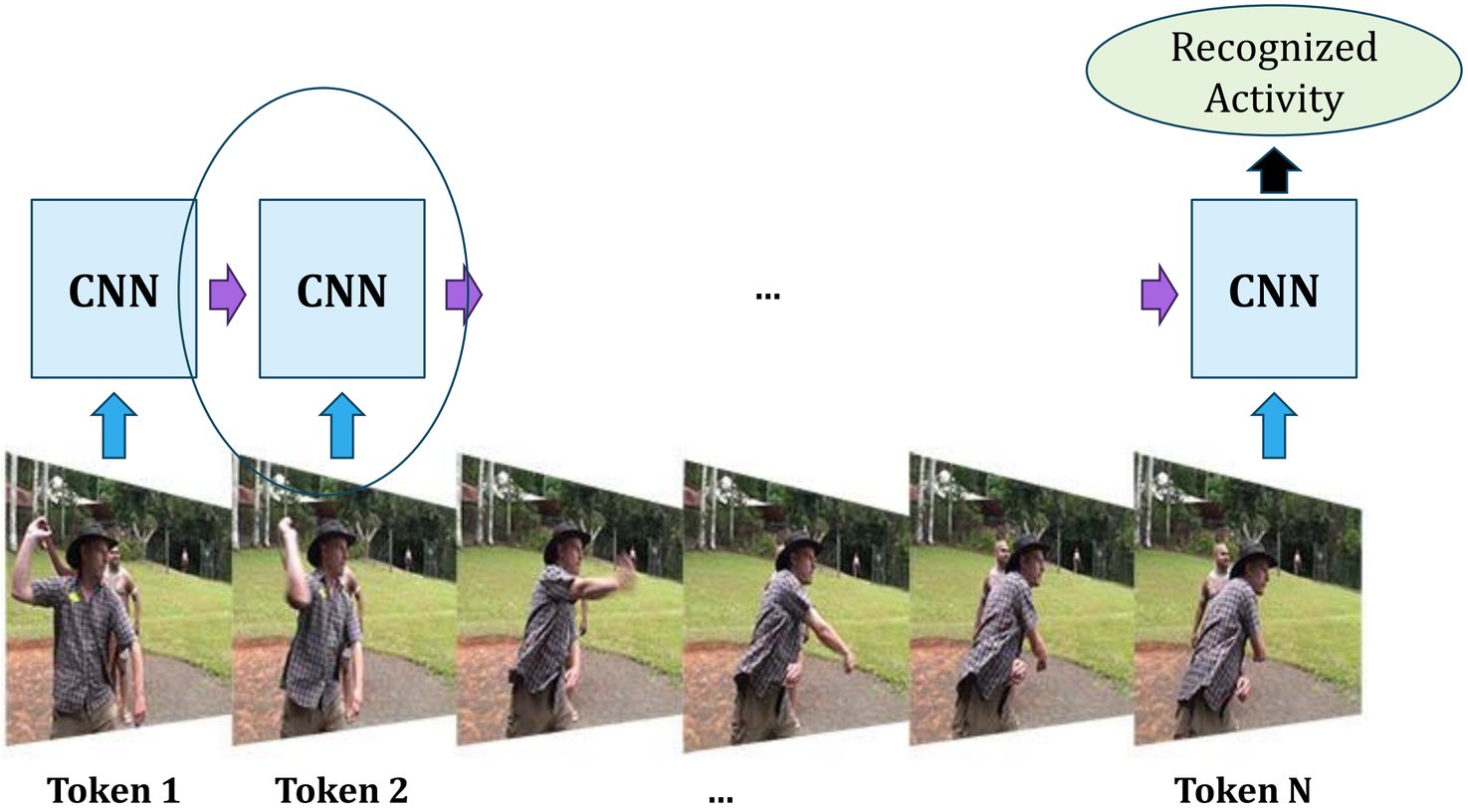
- Can be thought of as cycles in the network of nodes



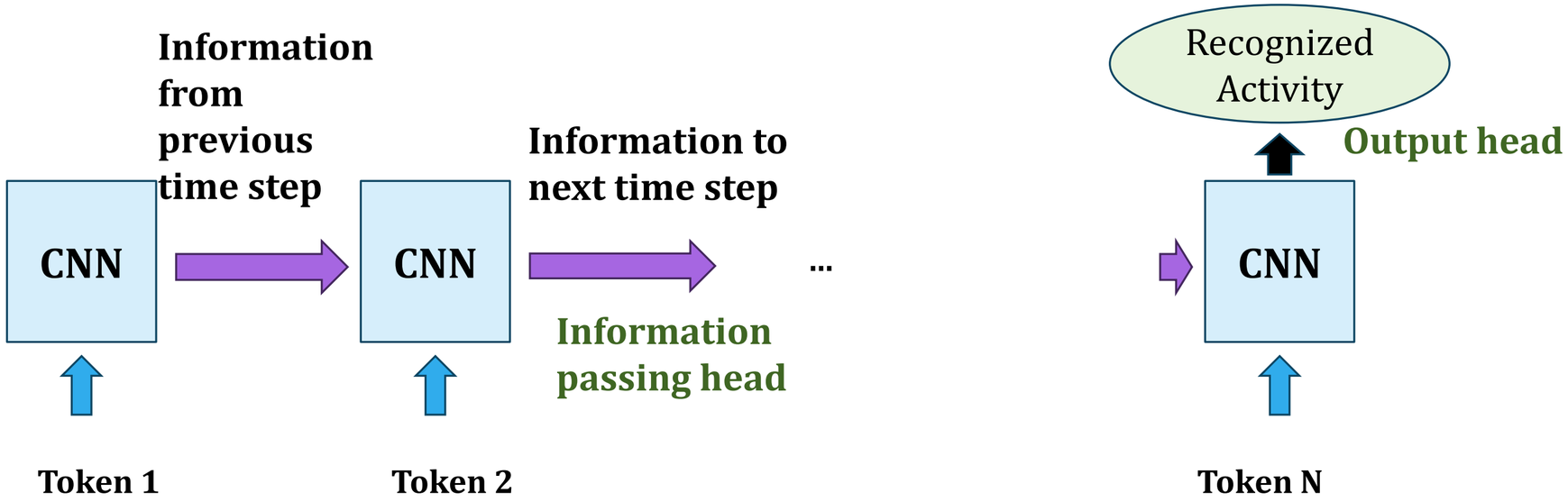
On the left recurrent connections are depicted via cyclic edges. On the right, we unfold the RNN over time steps. Here, recurrent edges span adjacent time steps, while conventional connections are computed synchronously.



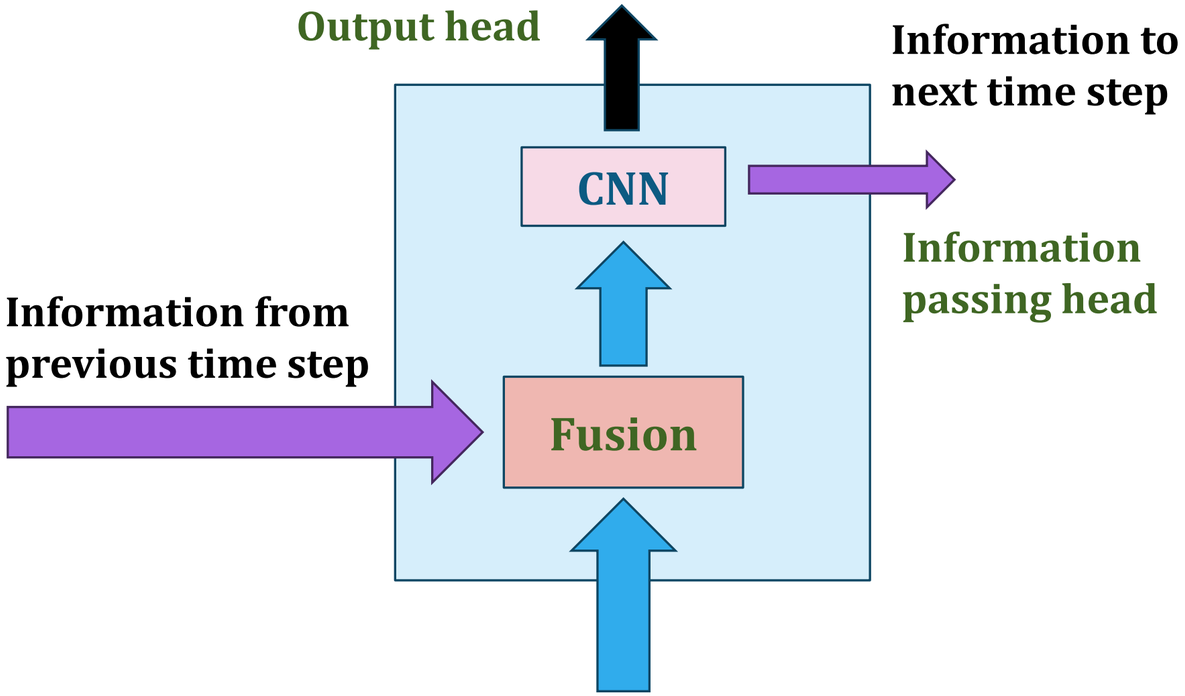
Parameter sharing across frames (time steps)



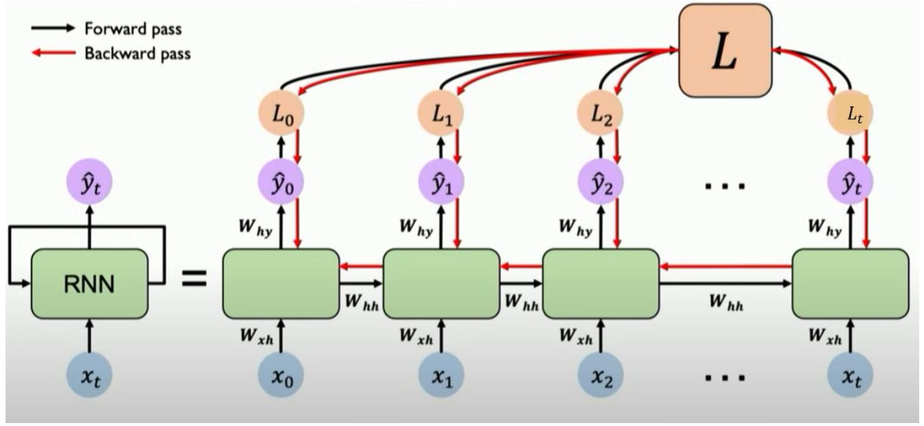
The CNN model needs to have two heads: one for information passing, one for output

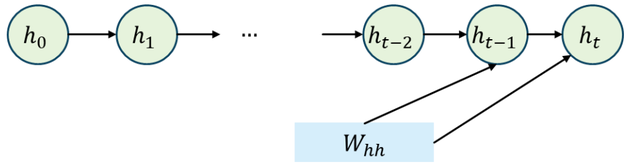

Information from previous time step
Information to next time step
CNN
CNN
...
Information passing head
Recognized Activity
Output head
CNN
Token 1
Token 2
Token N

...


Output head
Information to
next time step
CNN
Information
passing head
Information from
previous time step
Fusion
Token 2

Back Propagation Through Time





$$\boldsymbol{h_t = \phi(x_t W_{xh} + h_{t-1} W_{hh} + b_h)}$$

$$\boldsymbol{\hat{y}_t = h_t W_{hy} + b_o}$$

Total Loss

$$L = \frac{1}{t}\sum_{i=1}^{t} L_i$$

$$\frac{\partial L}{\partial W_{hh}} = \frac{1}{t}\sum_{i=1}^{t} \frac{\partial L_i}{\partial W_{hh}} = \frac{1}{t}\sum_{i=1}^{t} \frac{\partial L_i}{\partial H_i}\frac{\partial H_i}{\partial W_{hh}}$$

For $i = t$, let us try to calculate $\frac{\partial L_t}{\partial W_{hh}}$

$$\frac{\partial L_t}{\partial W_{hh}} = \frac{\partial L_t}{\partial h_t}\frac{\boldsymbol{\partial h_t}}{\boldsymbol{\partial W_{hh}}}$$

h_t depends on W_{hh}

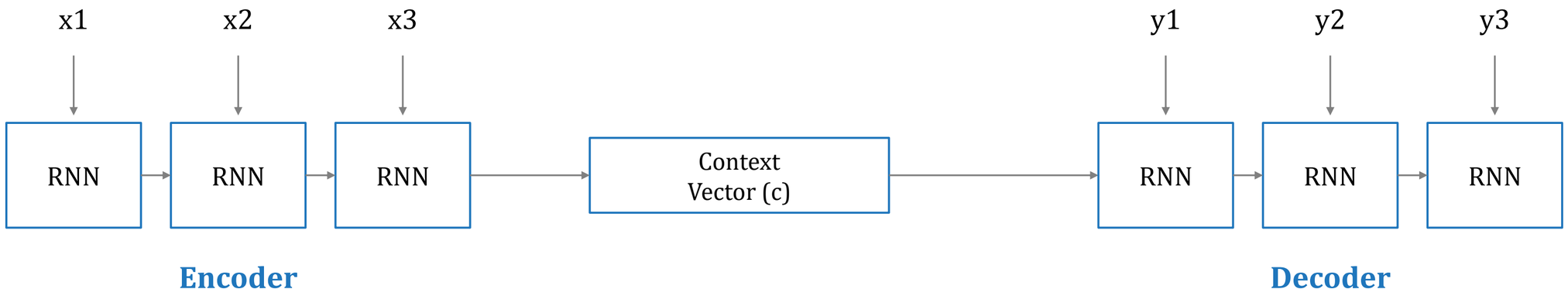
$$h_t = \phi(x_t W_{xh} + h_{t-1} W_{hh} + b_h)$$

h_{t-1} also depends on W_{hh}. So,

$$\boldsymbol{\frac{\partial h_t}{\partial W_{hh}} = \frac{\partial \phi(x_t W_{xh} + h_{t-1} W_{hh} + b_h)}{\partial W_{hh}} + \sum_{i=\tau}^{t-1}\left(\prod_{j=i+1}^{t} \frac{\partial \phi(x_t W_{xh} + h_{t-1} W_{hh} + b_h)}{\partial h_{j-1}}\right)\frac{\partial \phi(x_i W_{xh} + h_{i-1} W_{hh} + b_h)}{\partial W_{hh}}}$$

Vanishing/ exploding gradient problems

Encoder–Decoder RNN Architecture

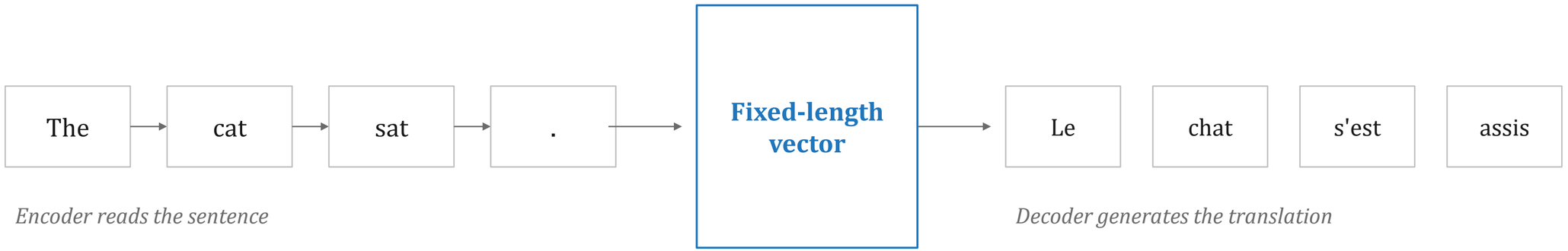


The encoder reads the input sequence step by step and compresses it into a fixed-length context vector, which the decoder then uses to generate the output sequence one token at a time.

The Fixed-Length Bottleneck

Standard encoder-decoder RNNs compress an entire source sentence into a single fixed-length vector

The decoder must then reconstruct the full translation from that one vector — nothing else is available to it



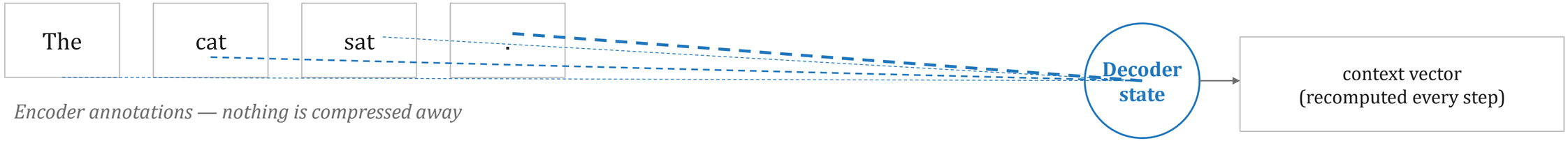
The problem

A single vector has fixed capacity. It cannot hold arbitrarily much information without losing detail. As sentences get longer, translation quality drops sharply.

Let the Decoder Look Back at the Source (Bahdanau et al.)

Instead of forcing everything through one vector, keep an annotation for every source word

The decoder learns, at each step, which source words matter right now



Encoder annotations — nothing is compressed away

Alignment scores → softmax → attention weights

The mechanism, in three steps

1. Score

A small feedforward network scores how relevant each source word is to the decoder's current state.

2. Weight

Scores are passed through softmax, turning them into attention weights that sum to 1.

3. Blend

A weighted sum of the source annotations becomes the context vector the decoder actually uses.

Bahdanau et al. — *Neural Machine Translation by Jointly Learning to Align and Translate*. ICLR 2015. arXiv:1409.0473

From Alignment to Self-Attention

Bahdanau's decoder-to-encoder attention is the direct ancestor of the self-attention used in the Transformer

Concept	Bahdanau Attention (2014)	Self-Attention (Transformer)
What plays the query role	The decoder's current hidden state	Every token's query vector
What plays the key/value role	Encoder annotations (RNN hidden states)	Every token's key and value vectors
Scoring function	A small feedforward network ("additive attention")	Scaled dot product
Still needs an RNN?	Yes — attention sits on top of encoder/decoder RNNs	No — attention replaces recurrence entirely

Introduction to Transformers

Why Sequences are Hard for Neural Networks

Any sequence model must answer: **which other tokens matter for understanding this one?**

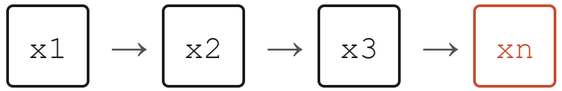
The dependency can be *local* or *arbitrarily far apart* — a pronoun referring back forty words earlier.

```
The trophy didn't fit in the suitcase
because it was too big.
```

Resolving it requires connecting back to trophy — 8 tokens away.

Recurrence and Convolution: Challenges

Recurrent networks



Information must pass through every intermediate step — sequential, hard to parallelize, and long-range signal decays.

Convolutional networks



A single filter only sees a local window — reaching a distant token needs many stacked layers, growing depth and path length.

Attention connects any two positions in a single step — in parallel.

A BRIEF HISTORY

From Alignment Scores to "Attention Is All You Need"

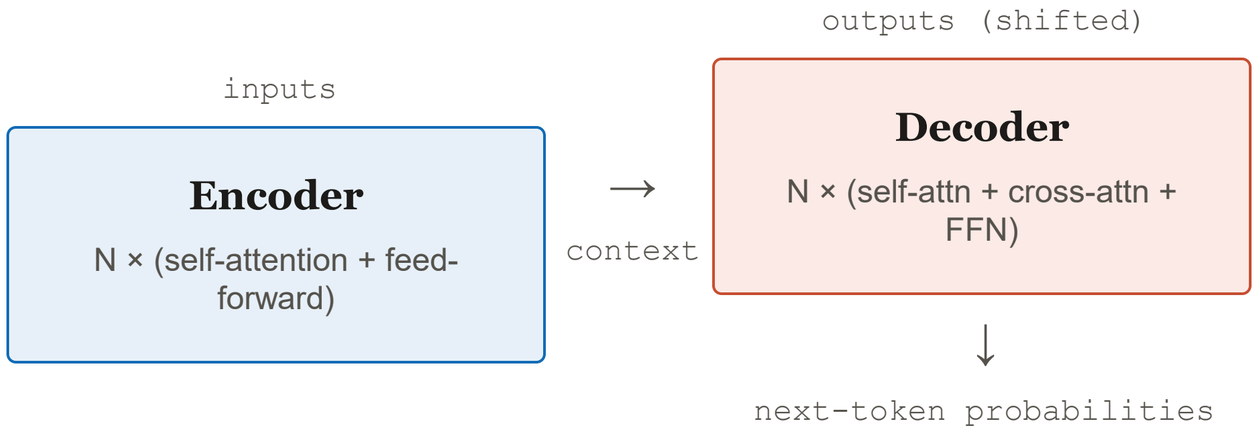
2015

Bahdanau et al. add a learned *alignment* mechanism to RNN translation — the seed of attention.

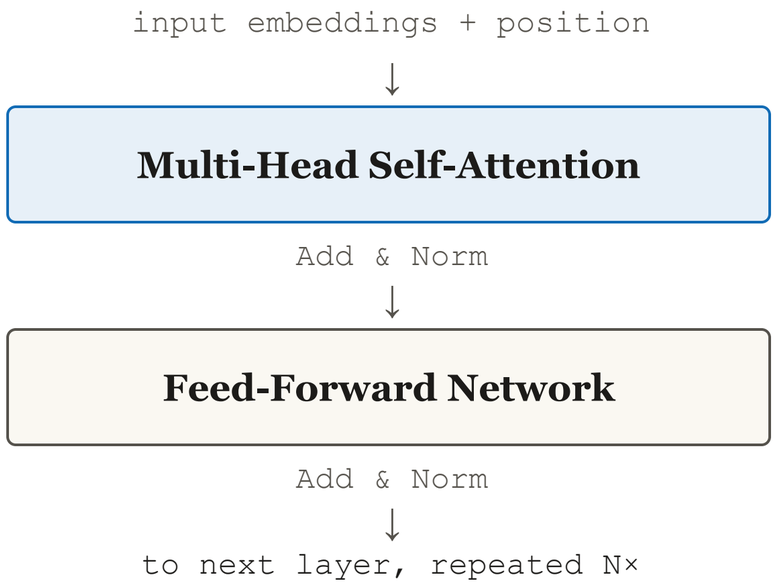
2017

Vaswani et al. remove recurrence entirely — a model built purely from attention and feed-forward layers.

The Transformer: Encoder–decoder Overview



Inside an Encoder Block

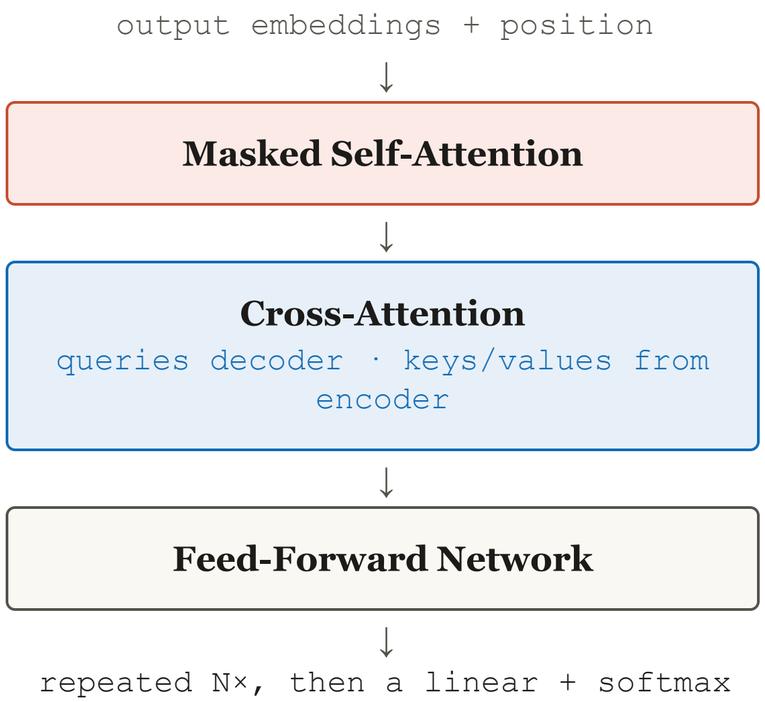


Every token attends to **every other token in the same sequence** — this is self-attention.

Residual connections + layer norm keep gradients stable through many stacked layers.

The feed-forward network applies the same two-layer MLP independently to each position.

Inside a Decoder Block



Masked self-attention: token i can only attend to positions $\leq i$, preserving autoregressive generation.

Cross-attention lets the decoder query the encoder's representations — this is how the output stays grounded in the input.

Question

If an RNN needs n steps to connect the first and last token, a CNN (1D) roughly needs _____ stacked layers

Question

If an RNN needs n steps to connect the first and last token, a CNN (1D) roughly needs log(n) stacked layers

Question

If an RNN needs n steps to connect the first and last token, and a CNN needs log(n) stacked layers — how many steps does self-attention need, and why?

Self-attention: the Core Idea

Every token asks a question — *"what am I looking for?"* — and every token (including itself) offers an answer.

The token gathers a weighted mix of everyone's answers, weighted by how relevant each answer is to its question.

Path length: O(1). Fully parallel across positions.

`"trophy"`	asks	*what is the subject here?*
`"it"`	asks	*what do I refer to?*
`"big"`	asks	*what am I describing?*

Each question is matched against every token's "offer" — the highest-matching tokens dominate the mix.

Attention Mechanism

- Mimics the process of retrieval of a value v_i based on key k_i from a database for a query q

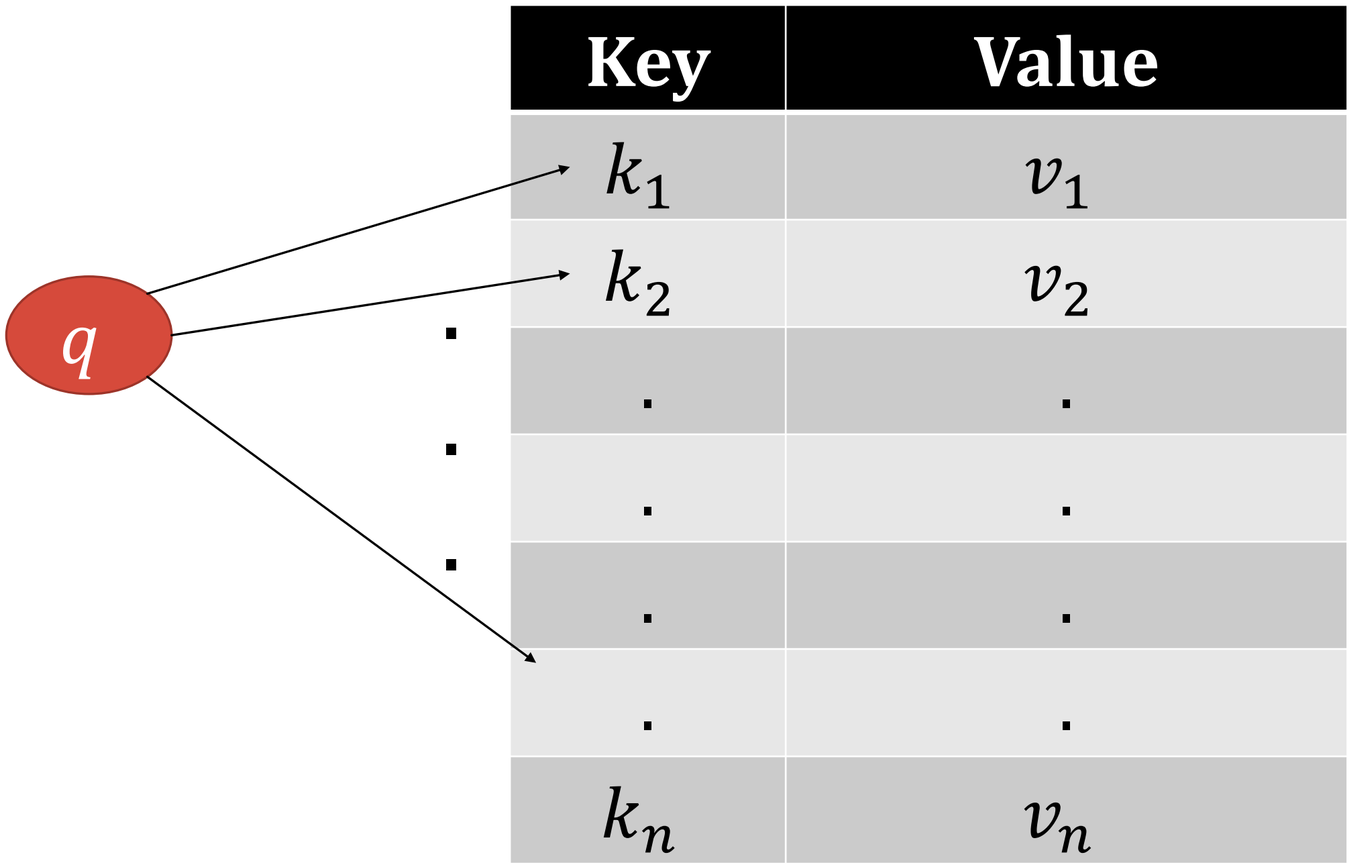
Attention Mechanism

- Mimics the process of retrieval of a value v_i based on key k_i from a database for a query q

Key	Value
k_1	v_1
k_2	v_2
.	.
.	.
.	.
.	.
k_n	v_n

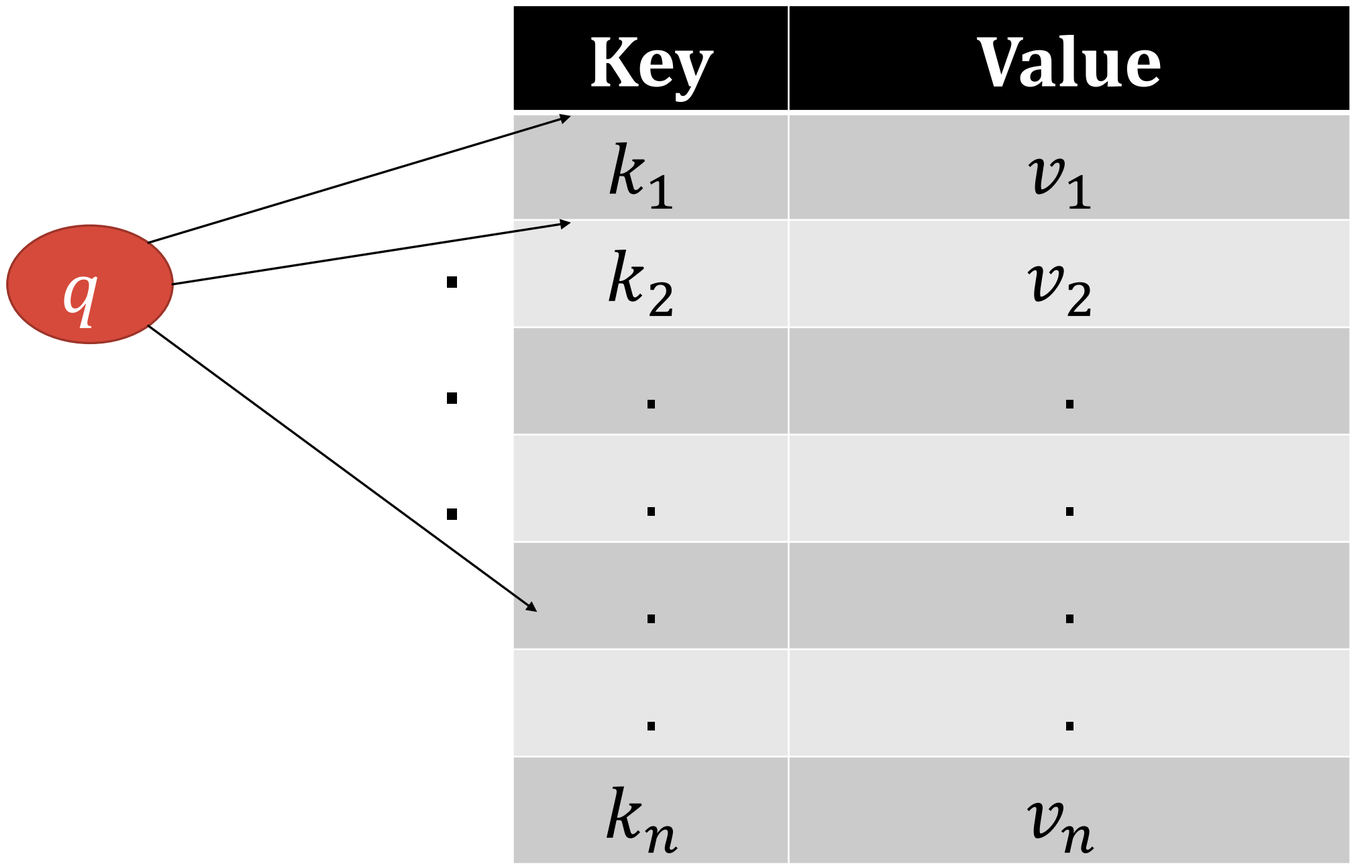
Attention Mechanism

- Mimics the process of retrieval of a value v_i based on key k_i from a database for a query q



Attention Mechanism

- Mimics the process of retrieval of a value v_i based on key k_i from a database for a query q



Compute similarity between query and keys

Attention Mechanism

- Mimics the process of retrieval of a value v_i based on key k_i from a database for a query q



Similarity	Key	Value
0.01	k_1	v_1
0.03	k_2	v_2
.	.	.
.	.	.
0.87	$\boldsymbol{k_m}$	$\boldsymbol{v_m}$
.	.	.
k_n	k_n	v_n

Compute similarity between query and keys

Attention Mechanism

- Mimics the process of retrieval of a value v_i based on key k_i from a database for a query q

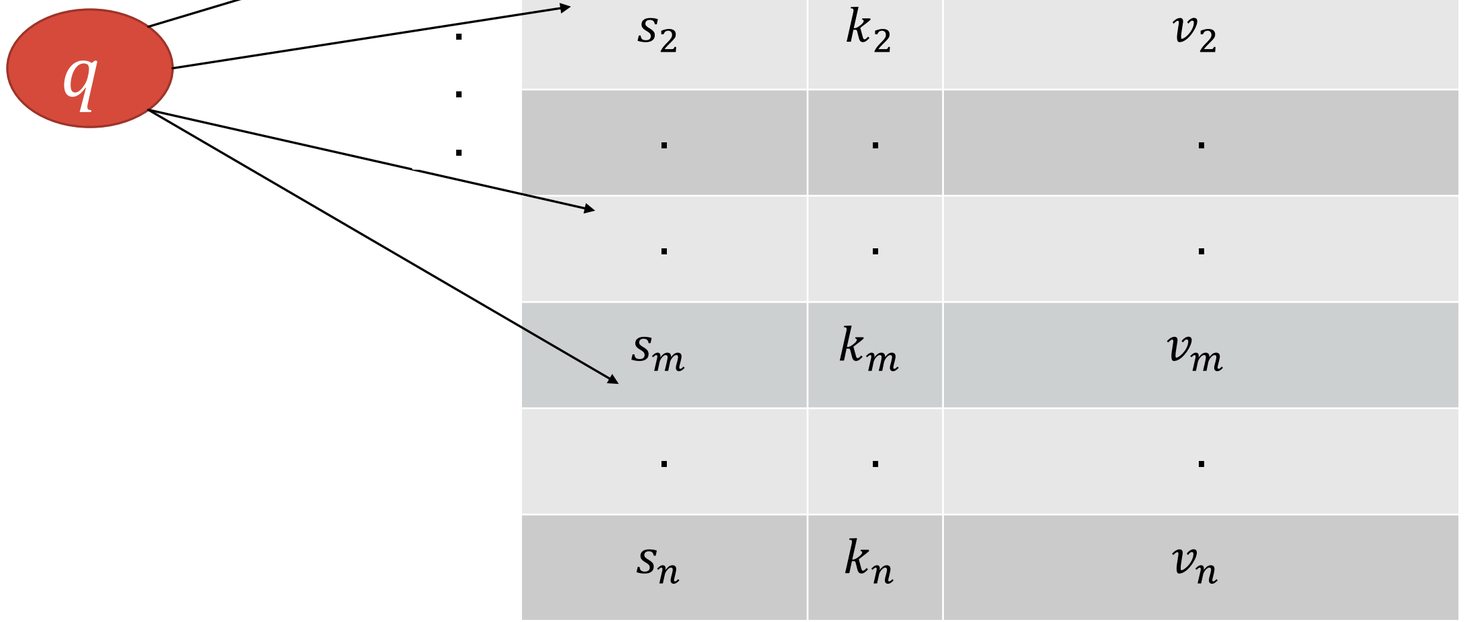


Similarity	Key	Value
0.01	k_1	v_1
0.03	k_2	v_2
.	.	.
.	.	.
0.87	$\boldsymbol{k_m}$	$\boldsymbol{v_m}$
.	.	.
k_n	k_n	v_n

v_m is the output of the query

Attention Mechanism

- What else can we do to get the output?



Similarity	Key	Value
s_1	k_1	v_1
s_2	k_2	v_2
.	.	.
.	.	.
s_m	k_m	v_m
.	.	.
s_n	k_n	v_n

Attention Mechanism

- Instead of a single output, we may take a weighted combination of the outputs



Similarity	Key	Value
s_1	k_1	v_1
s_2	k_2	v_2
.	.	.
.	.	.
s_m	k_m	v_m
.	.	.
s_n	k_n	v_n

$$Output = \sum_i softmax(s_i)v_i$$

Attention Mechanism

- Instead of a single output, we may take a weighted combination of the outputs



Similarity	Key	Value
s_1	k_1	v_1
s_2	k_2	v_2
.	.	.
.	.	.
s_m	k_m	v_m
.	.	.
s_n	k_n	v_n

$$Output = \sum_i softmax(s_i)v_i$$

$$= \sum_i softmax(similarity(q, k_i)) \times v_i$$

How to Calculate Similarity?

- Instead of a single output, we may take a weighted combination of the outputs

$similarity(q, k_i) = q^T k_i$ **Dot product**

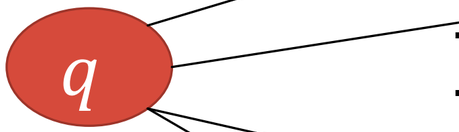
$similarity(q, k_i) = \frac{q^T k_i}{\sqrt{d}}$ **Scaled Dot product**

d is the dimensionality

$similarity(q, k_i) = q^T W k_i$ **General Dot product**

Attention Mechanism

- Attention value can be computed using similarity scores and values



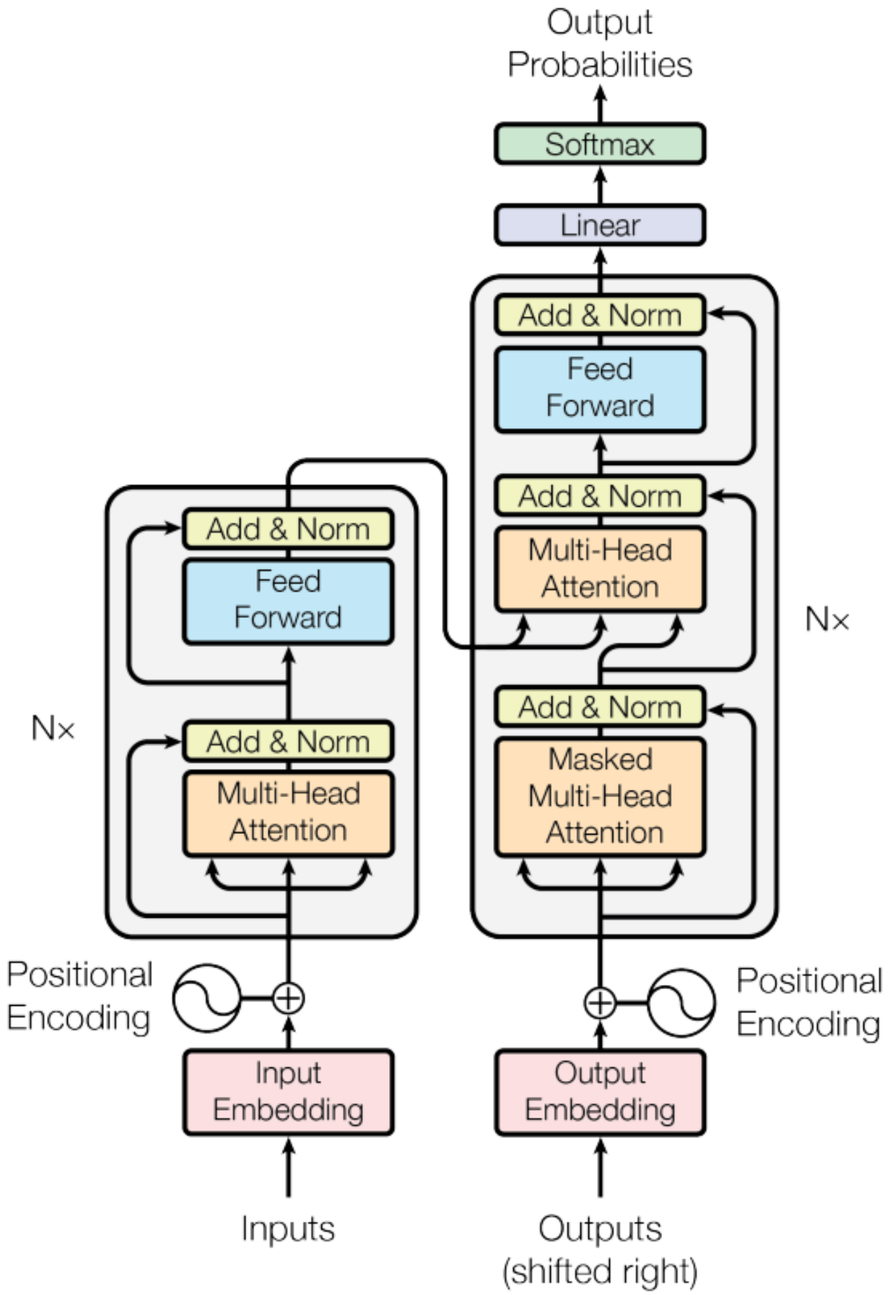
Similarity	Key	Value
s_1	k_1	v_1
s_2	k_2	v_2
.	.	.
.	.	.
s_m	k_m	v_m
.	.	.
s_n	k_n	v_n

$$Attention\ value = \sum_i softmax(s_i)v_i$$

Transformer for NLP

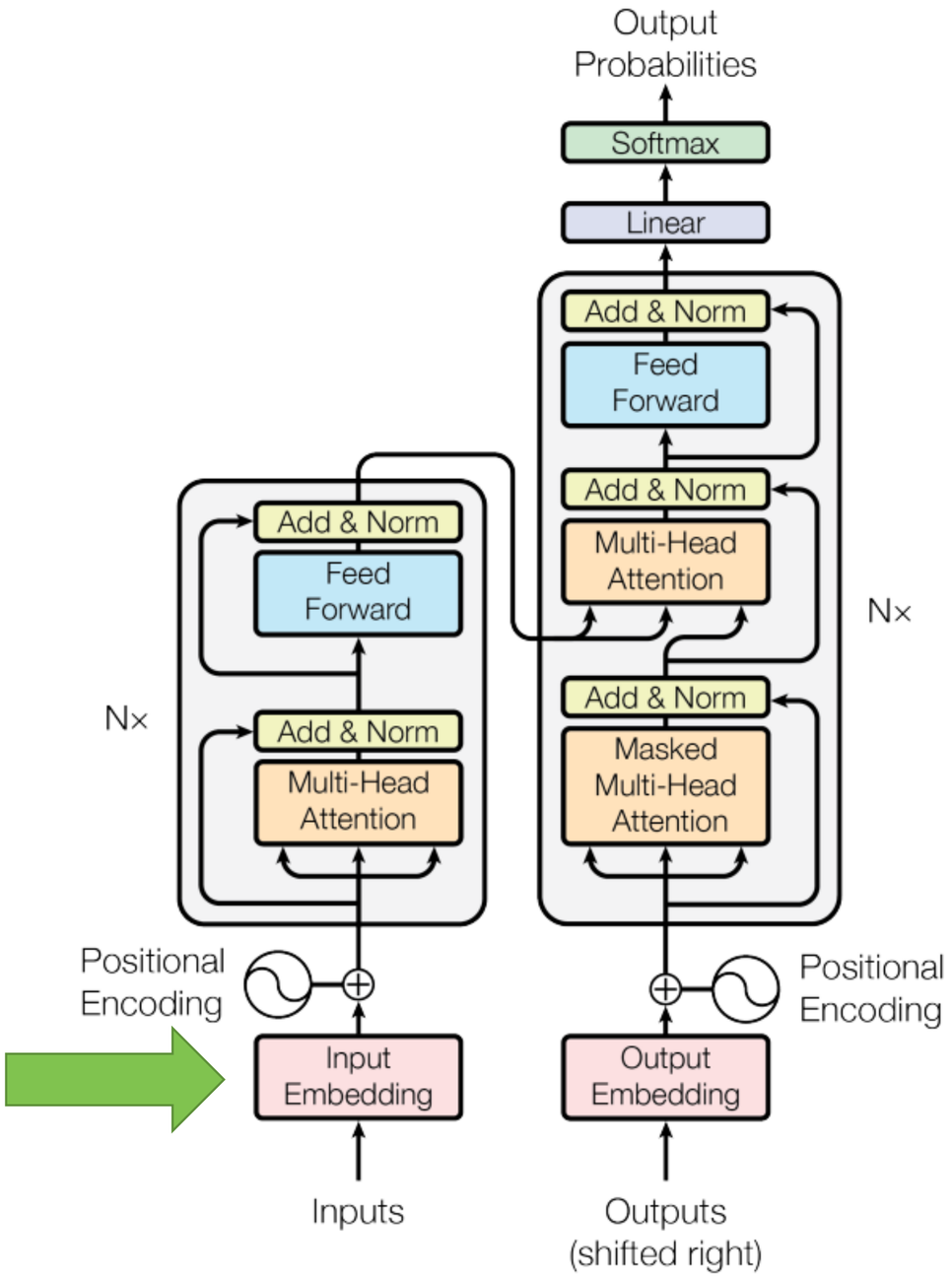
Where there is a will, there …….

Can an ML model complete the sentence?



https://arxiv.org/pdf/1706.03762.pdf

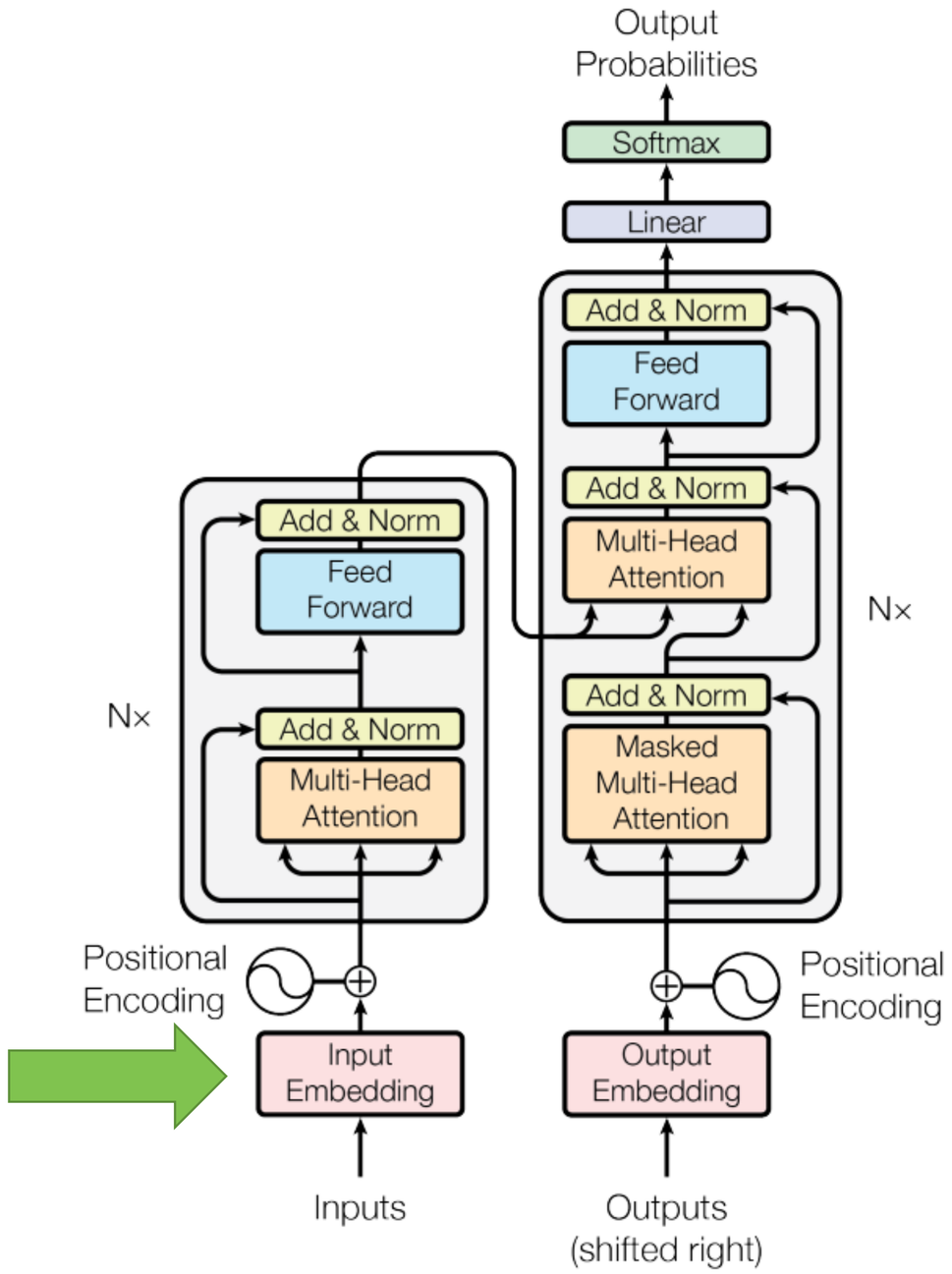
Transformer for NLP



Word embedding: Vector representation of words

https://medium.com/@hari4om/word-embedding-d816f643140

Transformer for NLP

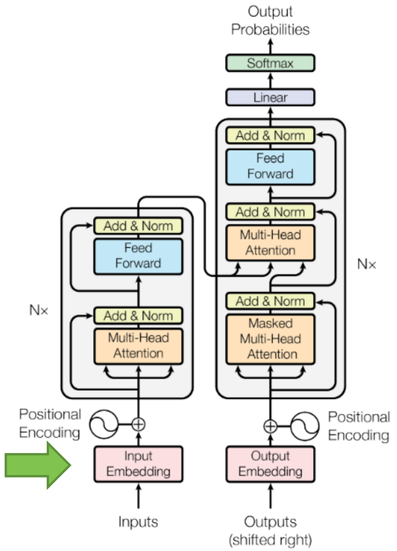


Where there is a will, there …….

Each word is assigned an index based on the dictionary of words

Word	Index
a	0
..	..
is	882
..	..
..	..
there	3582
..	..
where	11442
..	..

https://medium.com/@hari4om/word-embedding-d816f643140



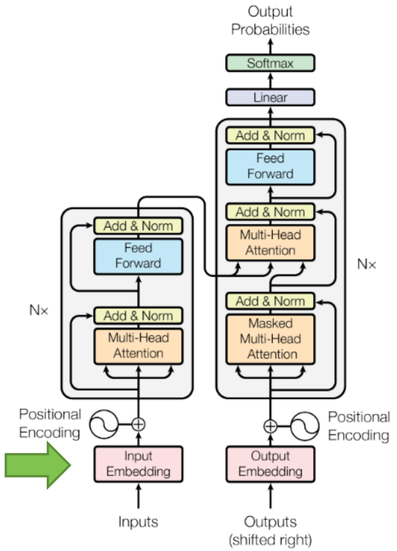
Transformer for NLP

Word	Index
a	0
..	..
is	882
..	..
..	..
there	3582
..	..
where	11442
..	..

Where there is a will, there …….

(11442 3582 882 … 3582)

https://medium.com/@hari4om/word-embedding-d816f643140



Word	Index
a	0
..	..
is	882
..	..
..	..
there	3582
..	..
where	11442
..	..

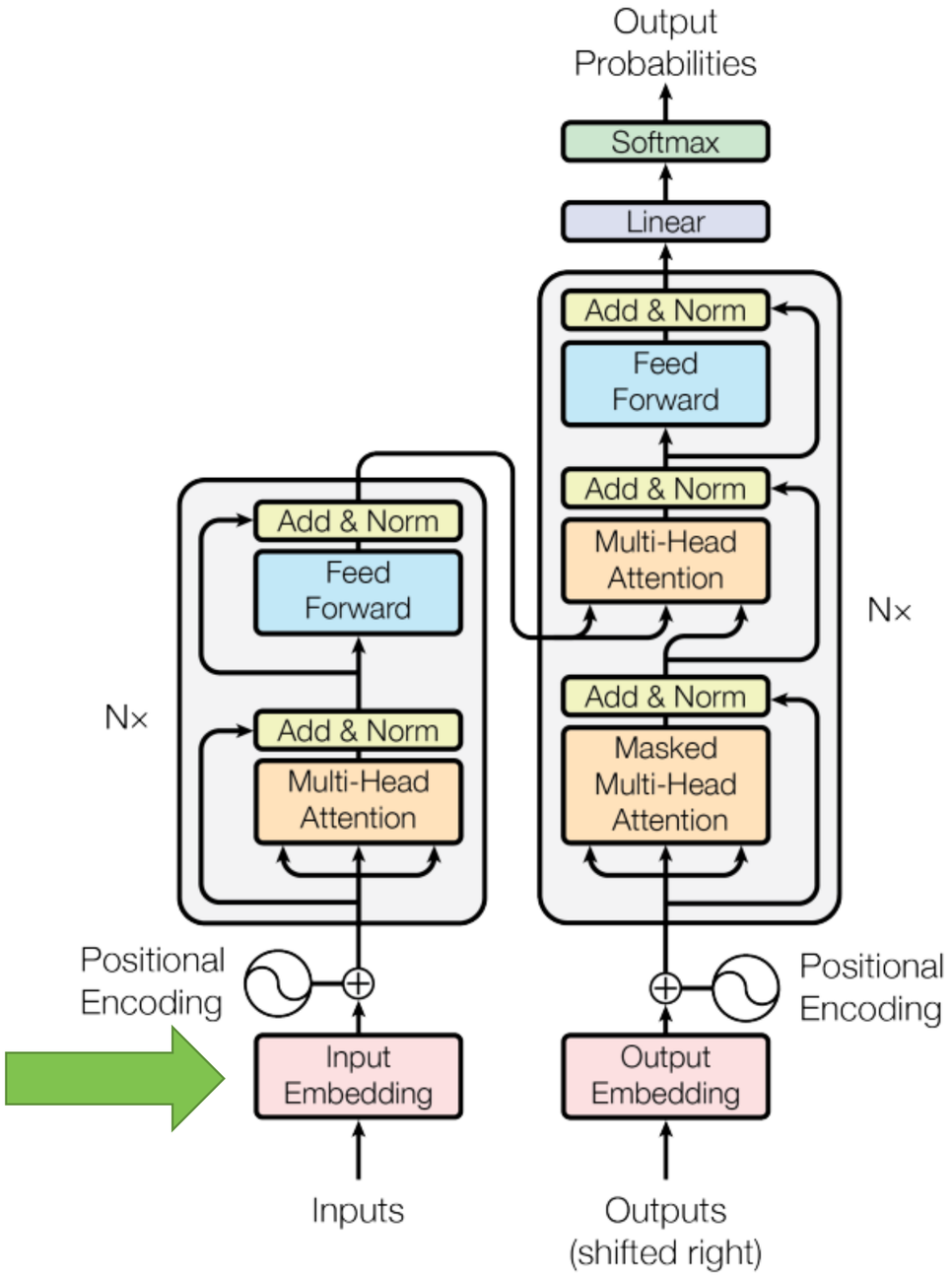
Transformer for NLP

Where there is a will, there …….

(11442 3582 882 … 3582)

Input

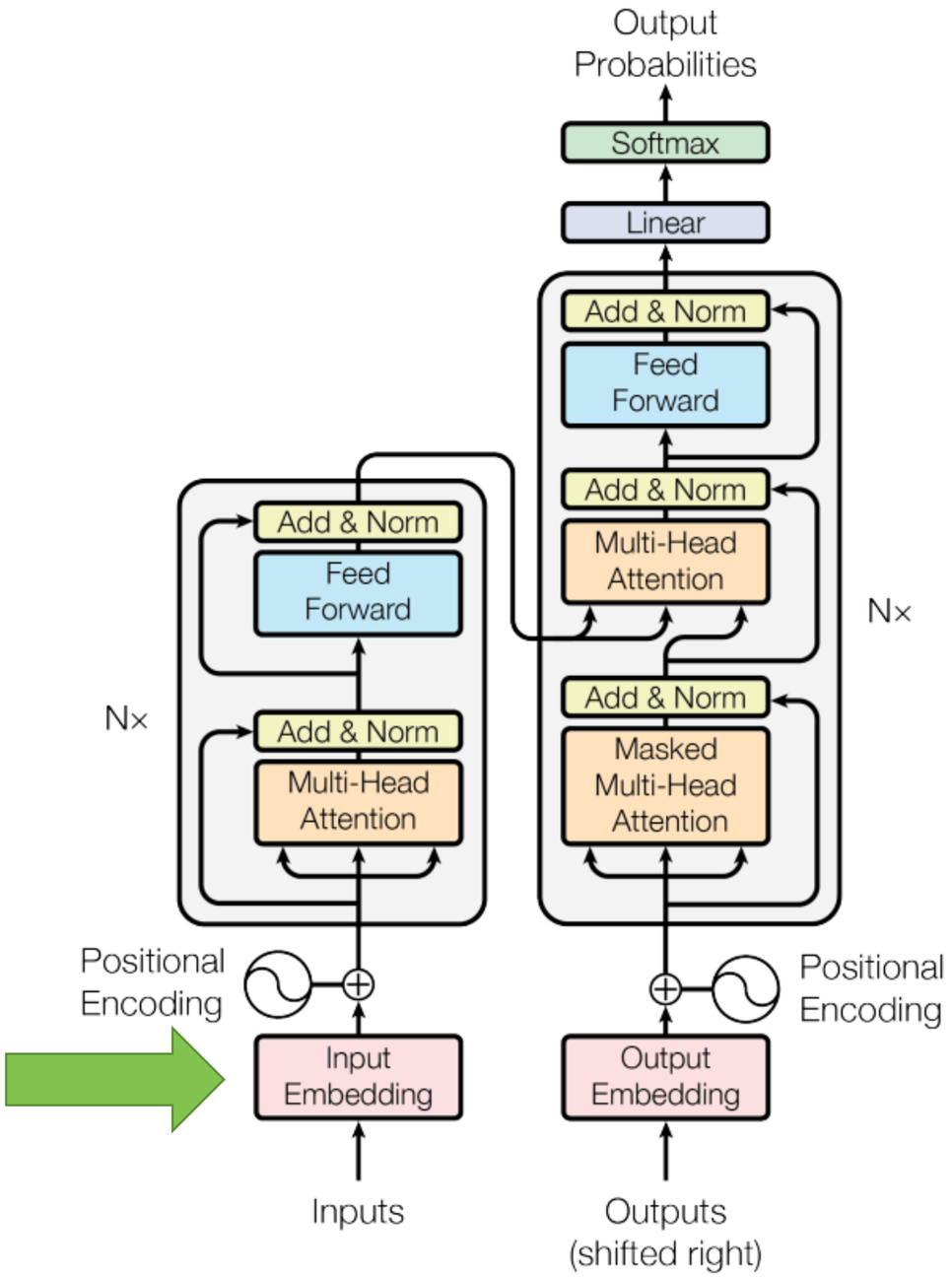
https://medium.com/@hari4om/word-embedding-d816f643140



Word Embedding

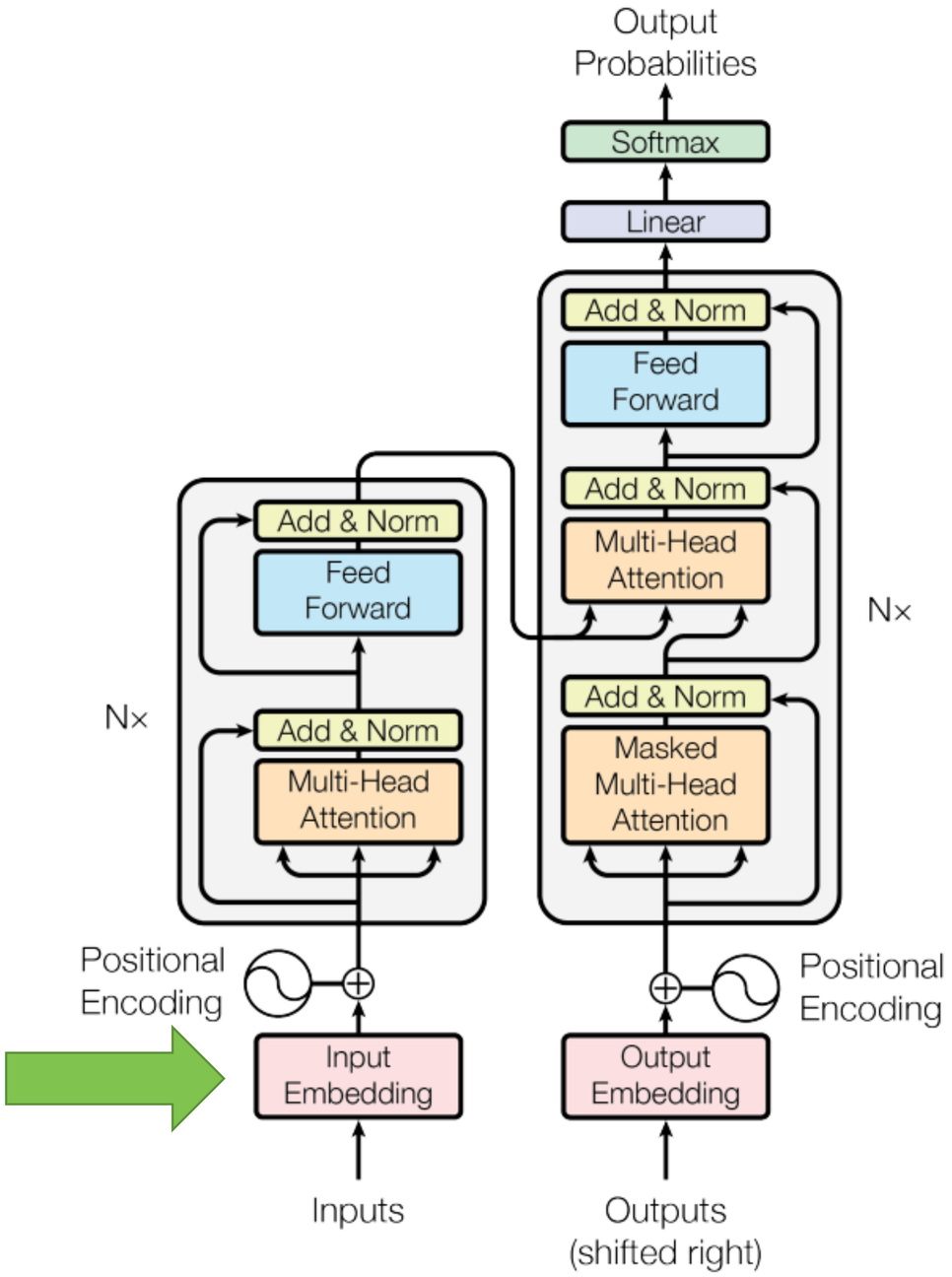
	living being	feline	human	gender	royalty	verb	plural
cat →	0.6	0.9	0.1	0.4	−0.7	−0.3	−0.2
kitten →	0.5	0.8	−0.1	0.2	−0.6	−0.5	−0.1
dog →	0.7	−0.1	0.4	0.3	−0.4	−0.1	−0.3
houses →	−0.8	−0.4	−0.5	0.1	−0.9	0.3	0.8

man →	0.6	−0.2	0.8	0.9	−0.1	−0.9	−0.7
woman →	0.7	0.3	0.9	−0.7	0.1	−0.5	−0.4
king →	0.5	−0.4	0.7	0.8	0.9	−0.7	−0.6
queen →	0.8	−0.1	0.8	−0.9	0.8	−0.5	−0.9



Word Embedding

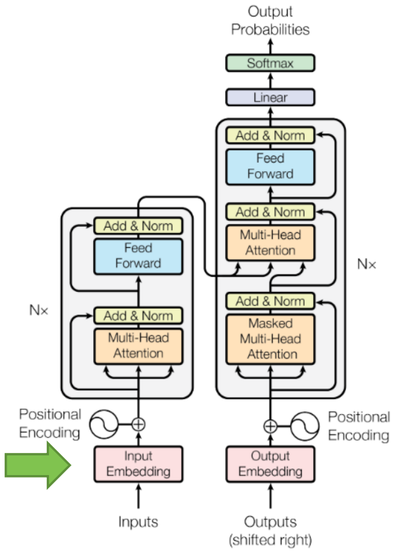
Word embedding: Learnt during training for transformer



Word Embedding

	Feature 1	Feature 2	Feature 3	Feature 4	Feature 5	Feature 6	Feature 7
cat →	0.6	0.9	0.1	0.4	−0.7	−0.3	−0.2
kitten →	0.5	0.8	−0.1	0.2	−0.6	−0.5	−0.1
dog →	0.7	−0.1	0.4	0.3	−0.4	−0.1	−0.3
houses →	−0.8	−0.4	−0.5	0.1	−0.9	0.3	0.8
man →	0.6	−0.2	0.8	0.9	−0.1	−0.9	−0.7
woman →	0.7	0.3	0.9	−0.7	0.1	−0.5	−0.4
king →	0.5	−0.4	0.7	0.8	0.9	−0.7	−0.6
queen →	0.8	−0.1	0.8	−0.9	0.8	−0.5	−0.9

After Word Embedding



Word	Index
a	0
..	..
is	882
..	..
..	..
there	3582
..	..
where	11442
..	..

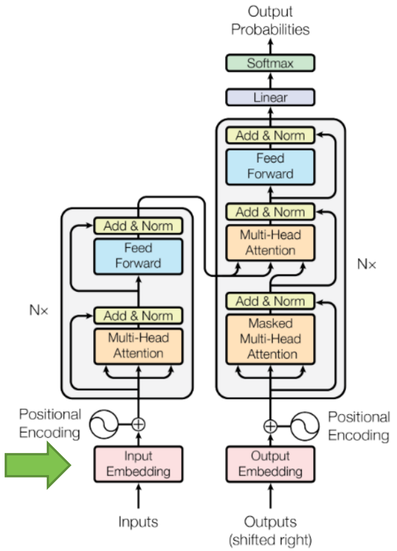


Where there is a will, there …….

(11442 3582 882 … 3582)

0.30	0.17		0.20
0.22	0.29		0.52
0.17	0.10	…	0.18
0.65	0.32		0.69
0.12	0.11		0.72

https://medium.com/@hari4om/word-embedding-d816f643140



After Word Embedding

Word	Index
a	0
..	..
is	882
..	..
..	..
there	3582
..	..
where	11442
..	..

	Feature 1	Feature 2	Feature 3	Feature 4	Feature 5	Feature 6	Feature 7
cat →	0.6	0.9	0.1	0.4	−0.7	−0.3	−0.2
kitten →	0.5	0.8	−0.1	0.2	−0.6	−0.5	−0.1
dog →	0.7	−0.1	0.4	0.3	−0.4	−0.1	−0.3
houses →	−0.8	−0.4	−0.5	0.1	−0.9	0.3	0.8
man →	0.6	−0.2	0.8	0.9	−0.1	−0.9	−0.7
woman →	0.7	0.3	0.9	−0.7	0.1	−0.5	−0.4
king →	0.5	−0.4	0.7	0.8	0.9	−0.7	−0.6
queen →	0.8	−0.1	0.8	−0.9	0.8	−0.5	−0.9

Where there is a will, there …….

(11442 3582 882 … 3582)

0.30	0.17		0.20
0.22	0.29		0.52
0.17	0.10	…	0.18
0.65	0.32		0.69
0.12	0.11		0.72
e_0	e_1		e_n

https://medium.com/@hari4om/word-embedding-d816f643140

Word (Token) Embedding in Transformer

A token embedding is a lookup: every vocabulary entry has its own row in a learned matrix, and looking up a token means reading off that row.

A note on tokens

In practice, BPE (byte-pair encoding) builds a vocabulary of subword pieces. Common words often stay whole, but rarer words get split into smaller chunks. For this example, we simplify: each token is a whole word (or punctuation mark), so the mechanics of embedding lookup are easier to follow.

The sentence we'll trace through this deck:

"Pizza is my favourite food."

The Vocabulary

15 word-level tokens, listed alphabetically (here, I have taken words first, punctuation grouped at the end). The order of tokens in the vocabulary will actually depend on BPE. Row position carries no meaning; only the learned values do.

Token	Index
burger	0
cricket	1
favourite	**2**
food	**3**
football	4

Token	Index
is	**5**
my	**6**
pasta	7
pizza	**8**
sport	9

Token	Index
the	10
,	11
.	**12**
!	13
?	14

vocab_size = 15 — highlighted rows are the tokens used in our sentence

The Full Embedding Matrix

d_model = 4 → every row is a 4-dimensional learnable vector. 15 rows × 4 dimensions = 60 learnable parameters.

burger	[0.29, 0.24, 0.35, -0.12]	**is**	[0.01, 0.04, -0.02, 0.02]
cricket	[-0.22, 0.41, 0.09, 0.33]	**my**	[0.05, -0.02, 0.10, 0.03]
favourite	[0.40, 0.31, -0.15, 0.22]	**pasta**	[0.31, 0.26, 0.38, -0.15]
food	[0.18, 0.27, 0.09, -0.05]	**pizza**	[0.33, 0.29, 0.41, -0.18]
football	[-0.19, 0.46, 0.15, 0.28]	**sport**	[-0.20, 0.44, 0.12, 0.30]

the	[0.02, 0.03, -0.01, 0.05]
,	[0.00, 0.01, -0.01, 0.00]
.	[0.00, 0.00, 0.00, 0.00]
!	[0.01, -0.01, 0.01, 0.01]
?	[-0.01, 0.01, 0.00, -0.01]

green = food *purple = sport* *gray = punctuation* *highlighted = used in our sentence*

At the beginning of the training, these entries start with small random numbers, like any other weight matrix in neural networks.

WORKED EXAMPLE

From Sentence to Embedding Matrix

"Pizza is my favourite food."

1. **Tokenize (word-level)**

```
["Pizza", "is", "my", "favourite", "food", "."]
```

2. **Convert to indices**

```
[8, 5, 6, 2, 3, 12]
```

3. **Look up each row**

```
embedding_matrix[8], embedding_matrix[5], ...
```

4. **Stack into a matrix**

```
shape (6 tokens × 4 dimensions)
```

5. **Scale by √d_model**

```
multiply every value by √4 = 2
```

WORKED EXAMPLE

The Looked-Up Vectors

Each token's row, pulled straight from the embedding matrix — no computation, just a lookup.

pizza (8)	`[ 0.33,  0.29,  0.41, -0.18]`
is (5)	`[ 0.01,  0.04, -0.02,  0.02]`
my (6)	`[ 0.05, -0.02,  0.10,  0.03]`
favourite (2)	`[ 0.40,  0.31, -0.15,  0.22]`
food (3)	`[ 0.18,  0.27,  0.09, -0.05]`
. (12)	`[ 0.00,  0.00,  0.00,  0.00]`

Stacked together, these 6 rows form a (6 × 4) matrix — the direct input to positional encoding, and from there, the first attention layer.

THE KEY IDEA

Every Row Is a Learnable Vector

The embedding matrix is not computed for transformer. Every single number in it is an independent, trainable parameter, just like a weight anywhere else in the network.

```
embedding_matrix[8]  =  [ 0.33,  0.29,  0.41, -0.18]
```

4 independent learnable numbers — this row happens to represent "pizza" only because that's the index our vocabulary assigned to it.

Starts as noise

At initialization, every row is small random values — no meaning yet.

Gets a gradient when used

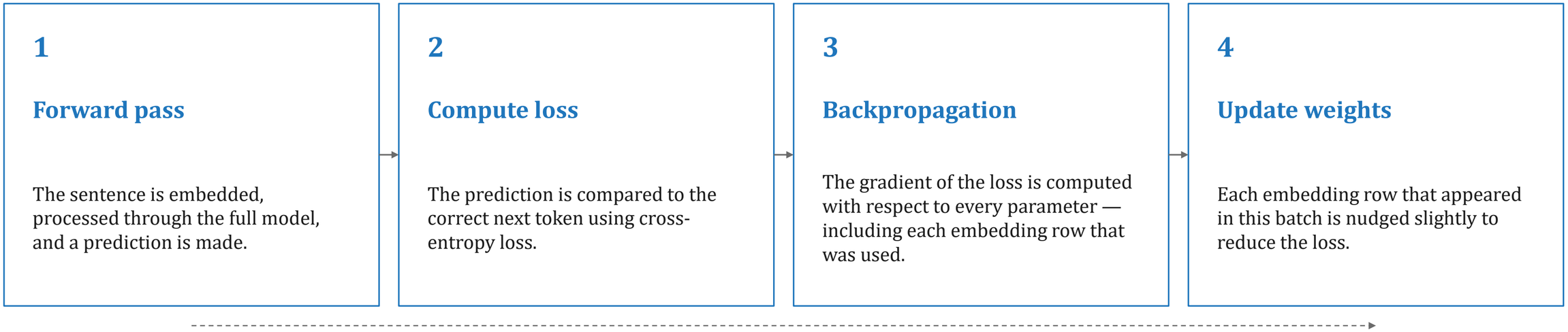
A row only receives an update on steps where its token actually appears in the batch.

Total learnable size

vocab_size × d_model — e.g. 15 × 4 = 60 parameters here; ~15M in a real model.

Training the Embedding Matrix

The embedding matrix starts as random numbers. It only becomes meaningful through the same training loop used for every other parameter in the model.



Repeat this loop over the entire training corpus, many times — only the rows for tokens actually seen in a batch get updated on that step.

Why Similar Words End Up with Similar Vectors

Nobody tells the model "pizza" and "pasta" are related. It's a byproduct of how they're used.

"My favourite food is ___ " • *"I ordered a ___ for dinner"* • *"We had ___ last night"*

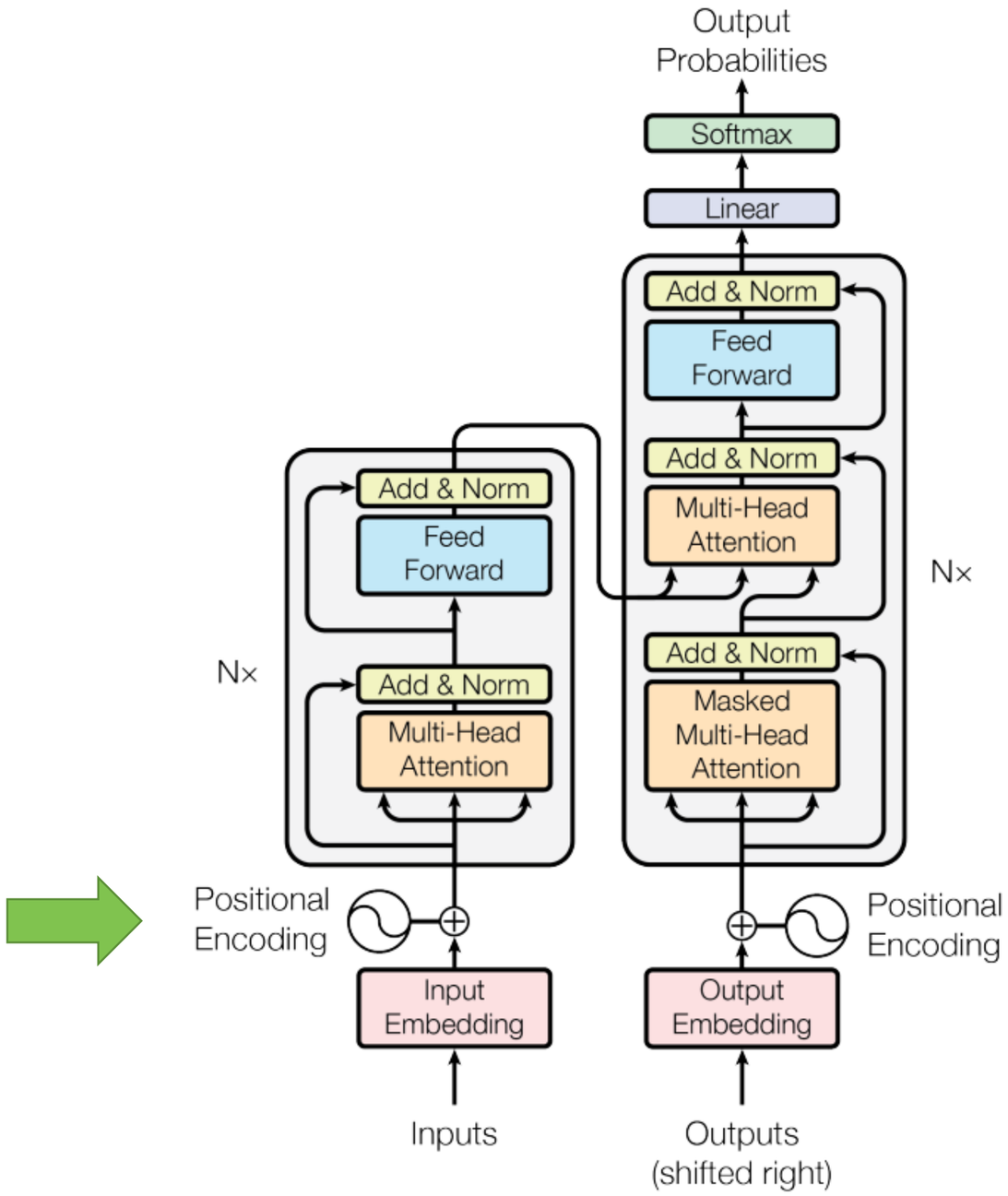
pizza, pasta, and burger all appear in similar surrounding contexts across the training data

↓

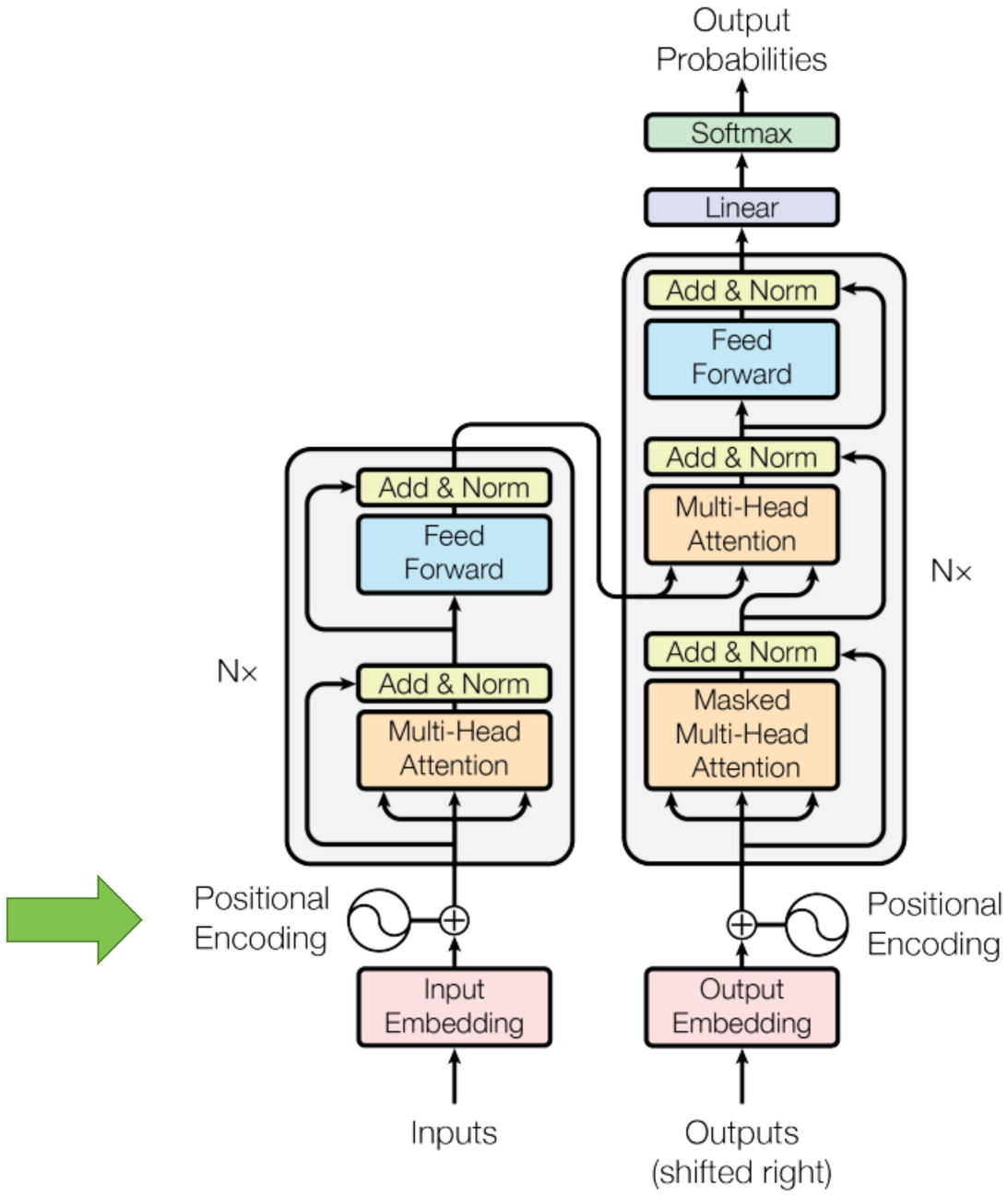
Gradient descent finds it useful to give them similar embeddings — whatever adjustment helps the model handle "pizza" in a context also tends to help with "pasta" in a similar context.

Result: after enough training, semantically related words cluster together in the embedding space — purely because that arrangement makes next-token prediction easier.

Positional Embedding



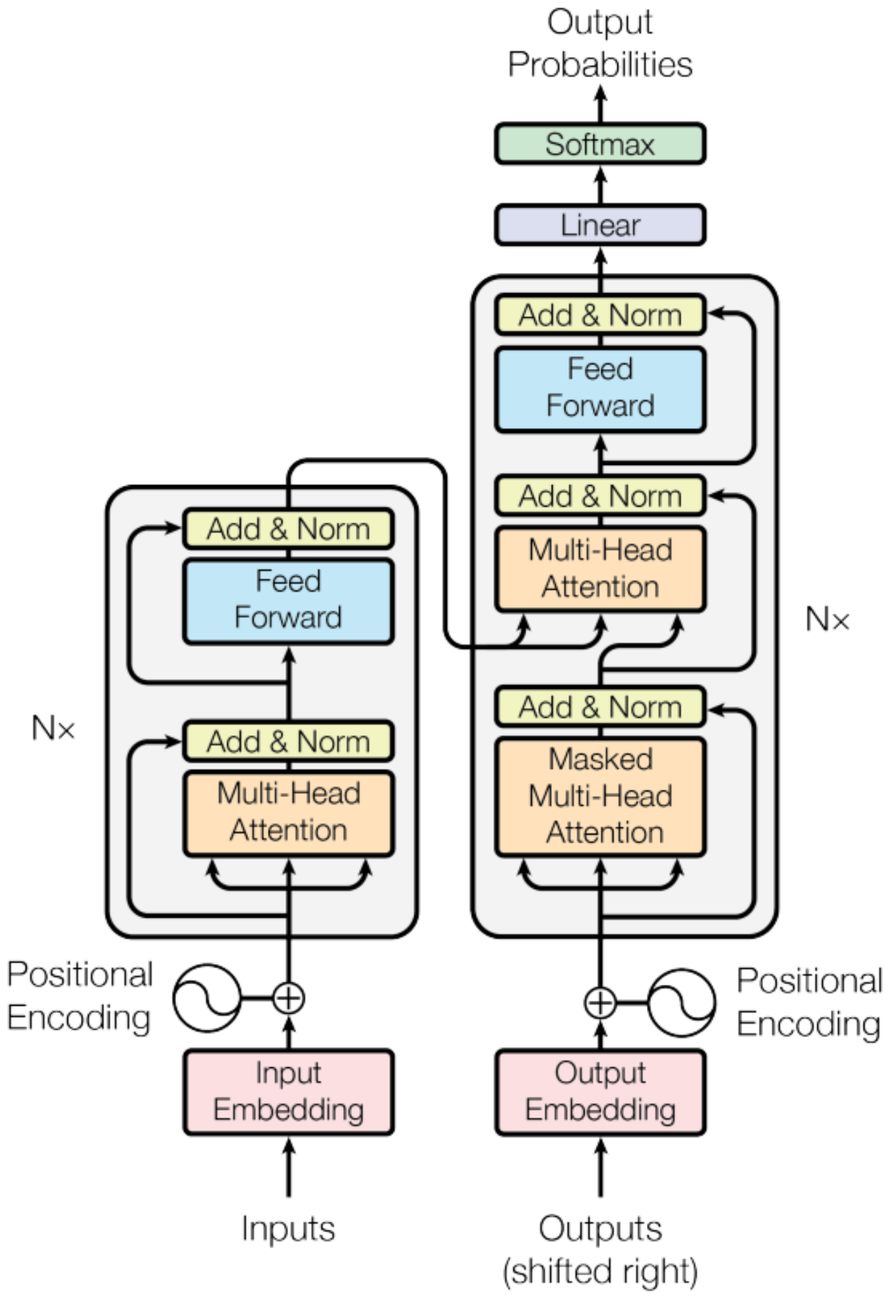
Although the sky is cloudy, it may not rain



Positional Embedding

Although the sky is cloudy, it may not rain

Although the sky is not cloudy, it may rain

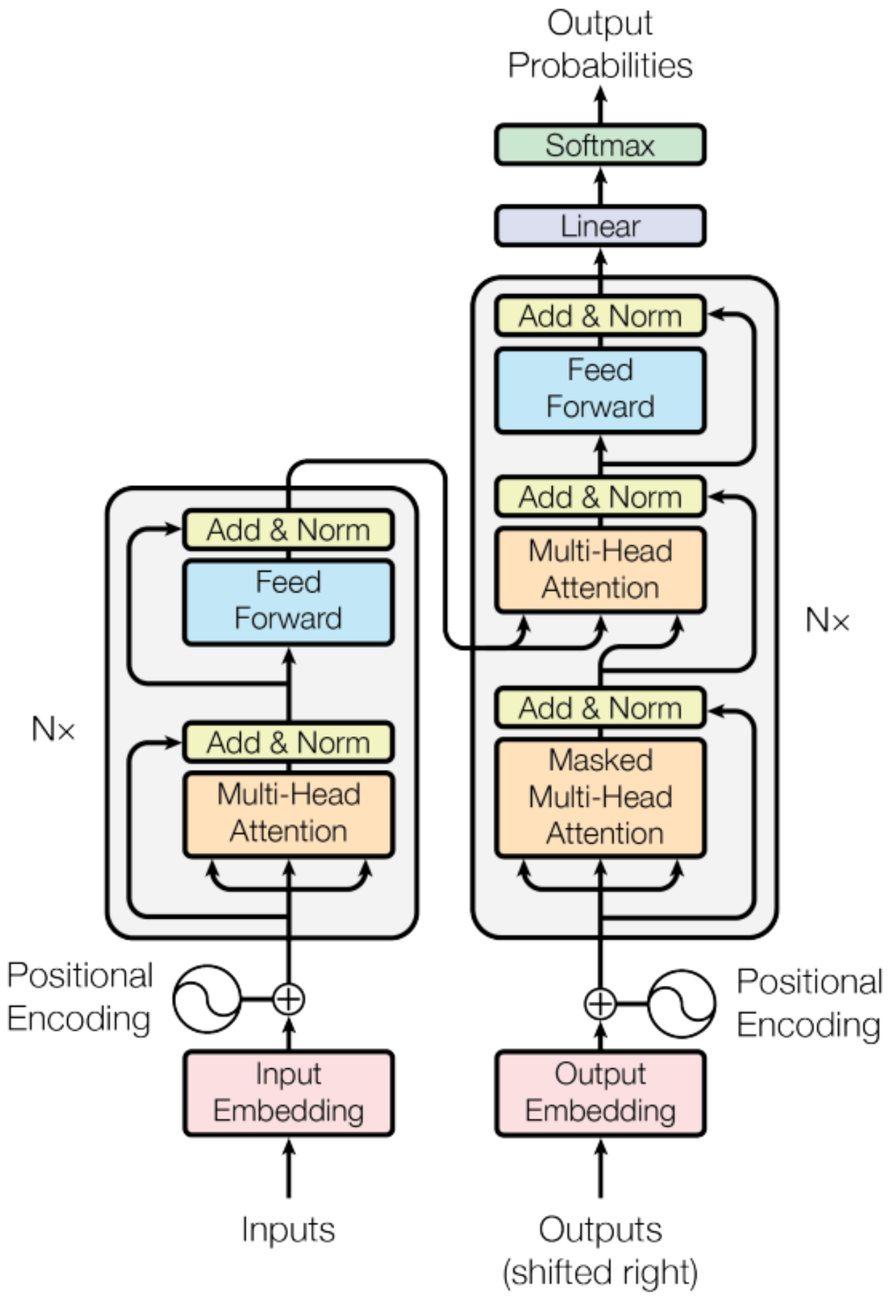


Positional Embedding

Although the sky is cloudy, it may not rain

Although the sky is not cloudy, it may rain

The position of the word ‘not’ changes the meaning of the sentence



Positional Embedding

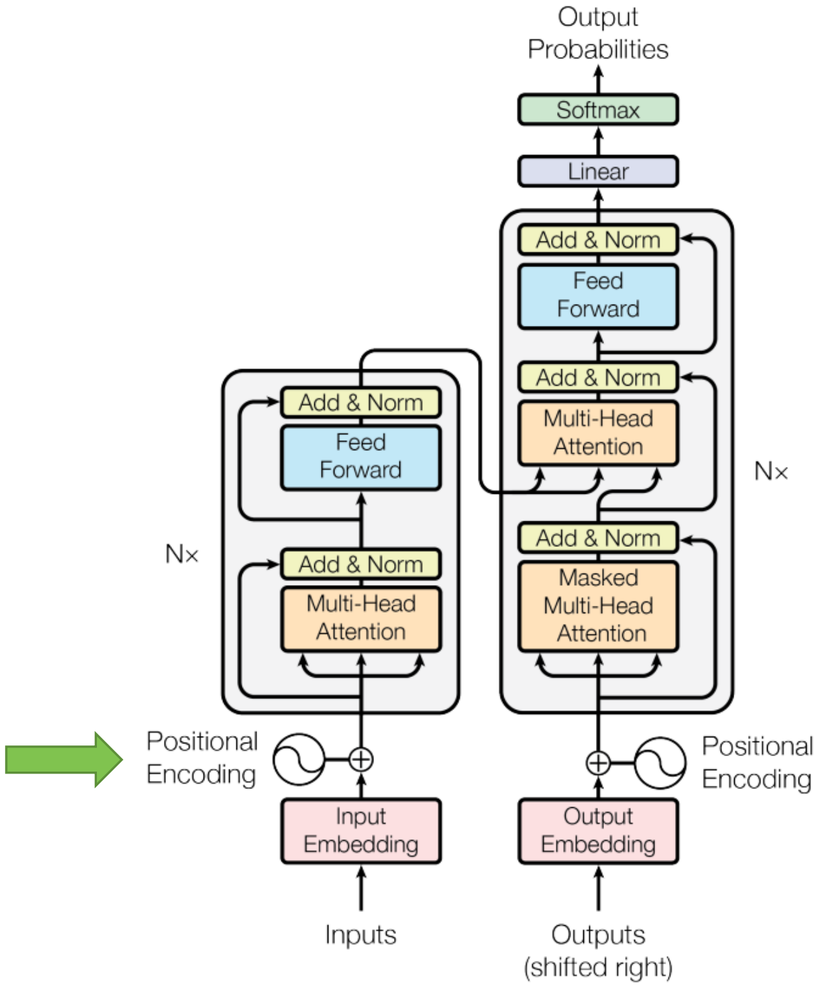
Although the sky is cloudy, it may not rain

Although the sky is not cloudy, it may rain

The position of the word ‘not’ changes the meaning of the sentence

So, position information should be incorporated with word embedding

Positional Embedding



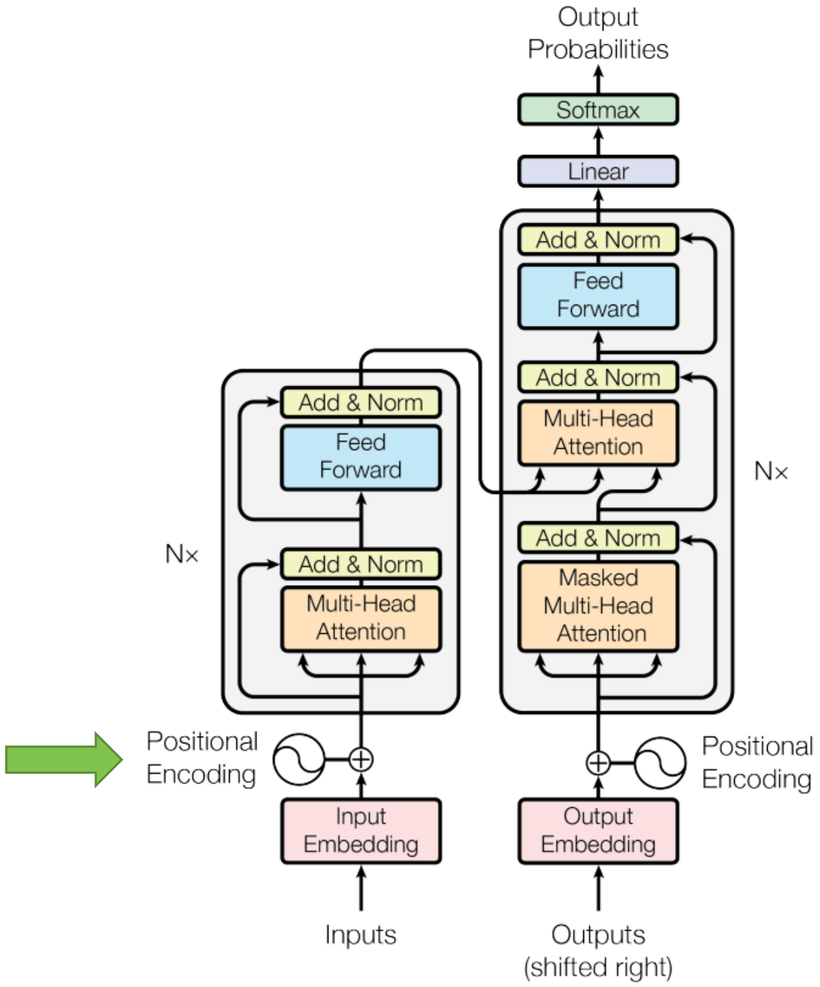
Where there is a will, there …….

(11442 3582 882 … 3582)

e_0	e_1		e_n
0.30	0.17		0.20
0.22	0.29		0.52
0.17	0.10	. . .	0.18
0.65	0.32		0.69
0.12	0.11		0.72

https://medium.com/@hari4om/word-embedding-d816f643140

Positional Embedding

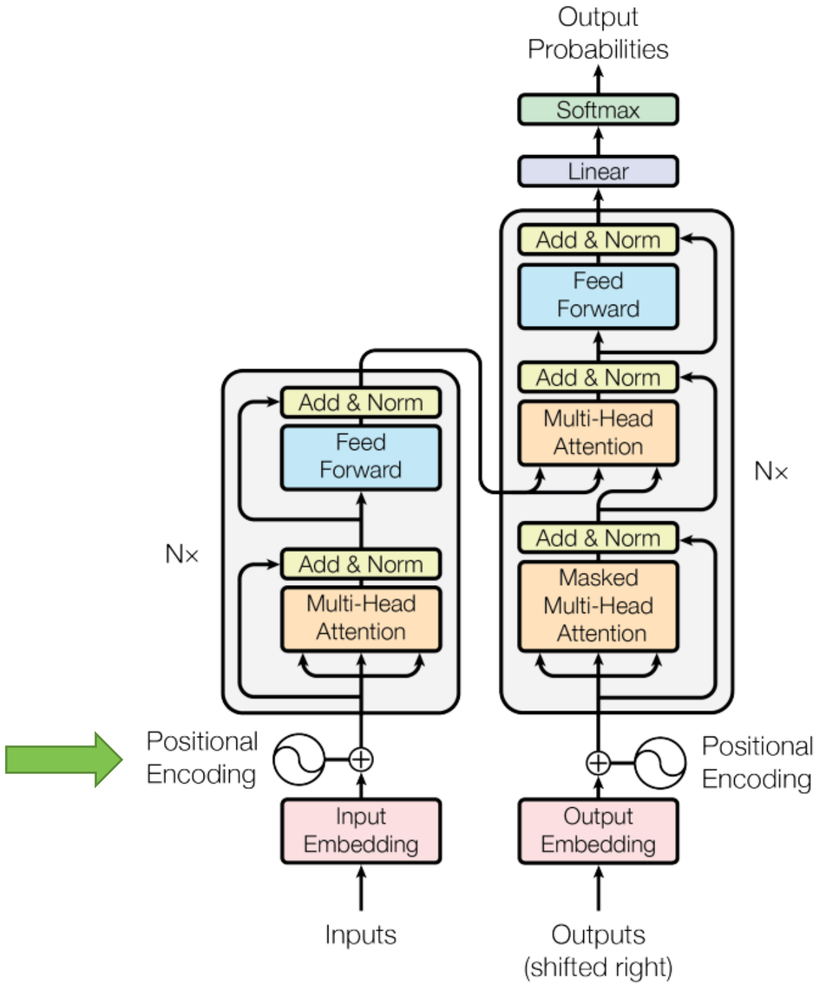


Where there is a will, there …….

(11442 3582 882 … 3582)

0.30		0.17				0.20	
0.22		0.29				0.52	
0.17		0.10		. . .		0.18	
0.65		0.32				0.69	
0.12		0.11				0.72	
e_0	p_0	e_1	p_1			e_n	p_n

https://medium.com/@hari4om/word-embedding-d816f643140



Positional Embedding

Where there is a will, there …….

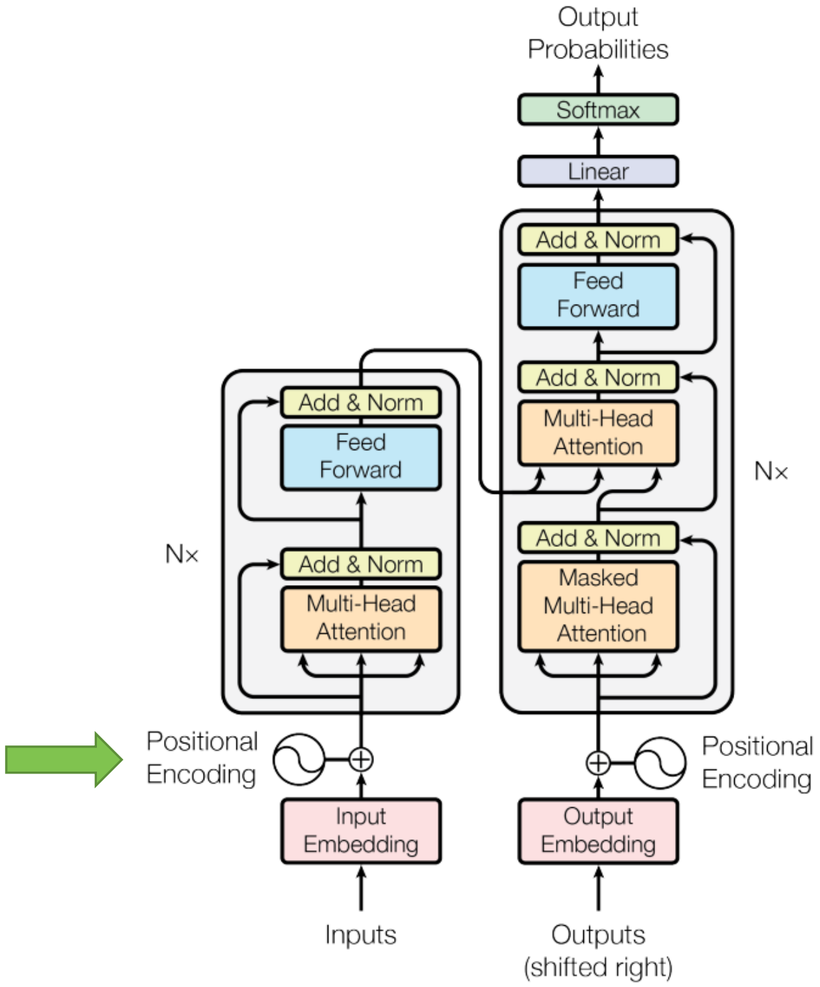
(11442 3582 882 … 3582)

0.30		0.17				0.20	
0.22		0.29				0.52	
0.17		0.10		. . .		0.18	
0.65		0.32				0.69	
0.12		0.11				0.72	
e_0	p_0	e_1	p_1			e_n	p_n

How to get p_i?

https://medium.com/@hari4om/word-embedding-d816f643140

Positional Embedding



Where there is a will, there …….

(11442 3582 882 … 3582)

0.30		0.17				0.20	
0.22		0.29				0.52	
0.17		0.10		. . .		0.18	
0.65		0.32				0.69	
0.12		0.11				0.72	
e_0	p_0	e_1	p_1			e_n	p_n

How to get p_i? Vaswani *et al.*

$$PE(pos, 2i) = \sin\left(\frac{pos}{1000^{\frac{2i}{d}}}\right) \text{ Dimesnions at even positions}$$

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{1000^{\frac{2i}{d}}}\right) \text{ Dimensions at odd positions}$$

https://medium.com/@hari4om/word-embedding-d816f643140

Positional Embedding

$$PE(pos, 2i) = \sin\left(\frac{pos}{1000^{\frac{2i}{d}}}\right)$$ Dimesnions at even positions

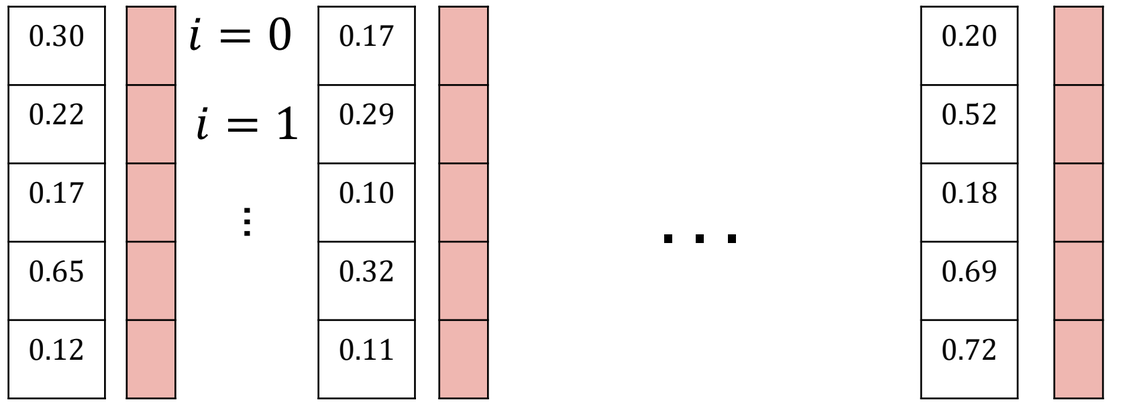
$$PE(pos, 2i+1) = \cos\left(\frac{pos}{1000^{\frac{2i}{d}}}\right)$$ Dimensions at odd positions

pos: position of the token in the sequence
i: Dimension index

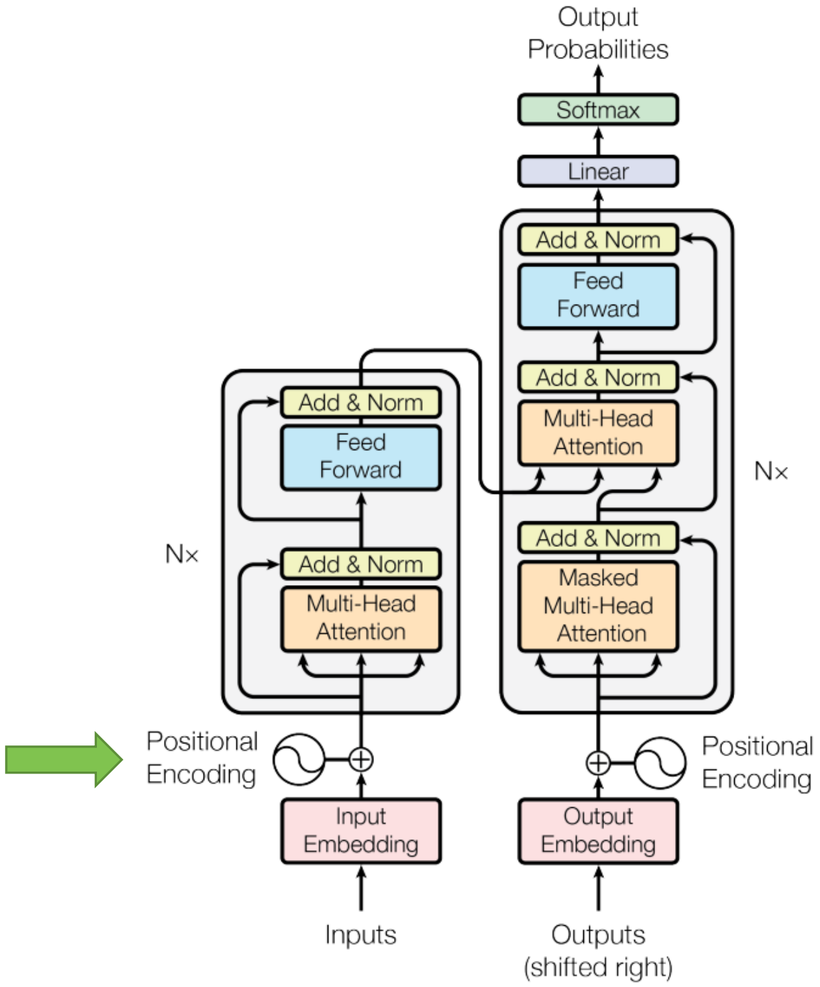
https://medium.com/@hari4om/word-embedding-d816f643140

Positional Embedding

$pos = 0$ $pos = 1$ ...



https://medium.com/@hari4om/word-embedding-d816f643140



Positional Embedding

Where there is a will, there …….

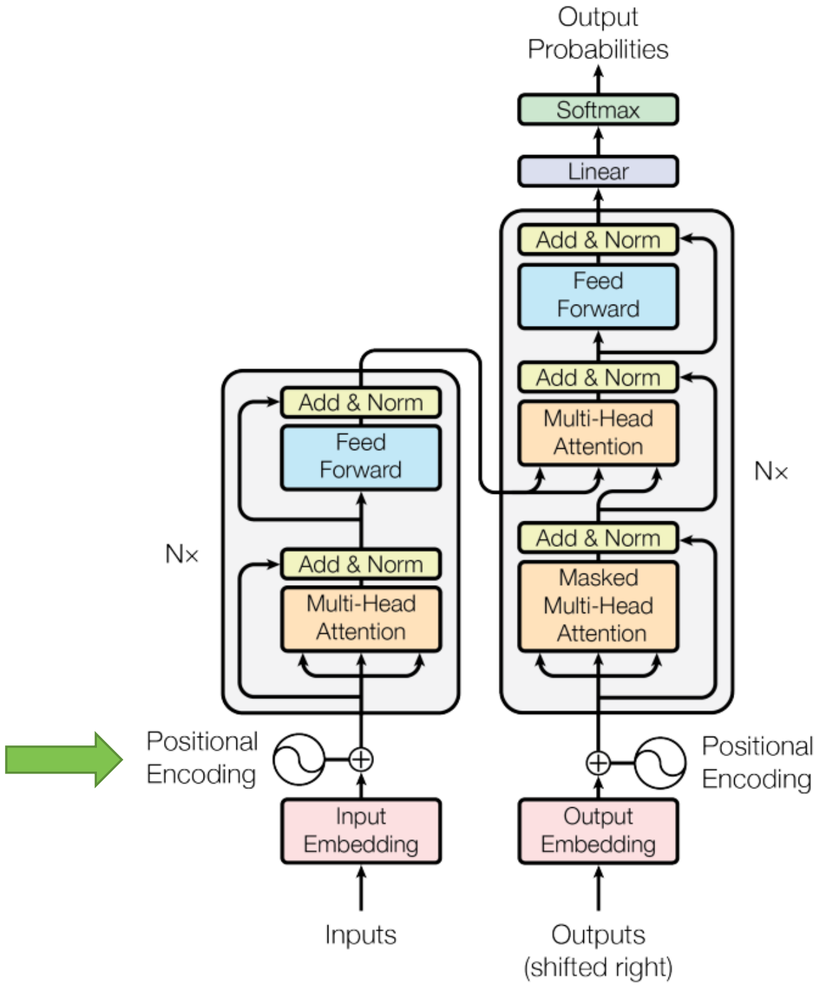
(11442 3582 882 … 3582)

0.30		0.17				0.20	
0.22		0.29				0.52	
0.17		0.10		. . .		0.18	
0.65		0.32				0.69	
0.12		0.11				0.72	
e_0	p_0	e_1	p_1			e_n	p_n

How to combine e_i and p_i?

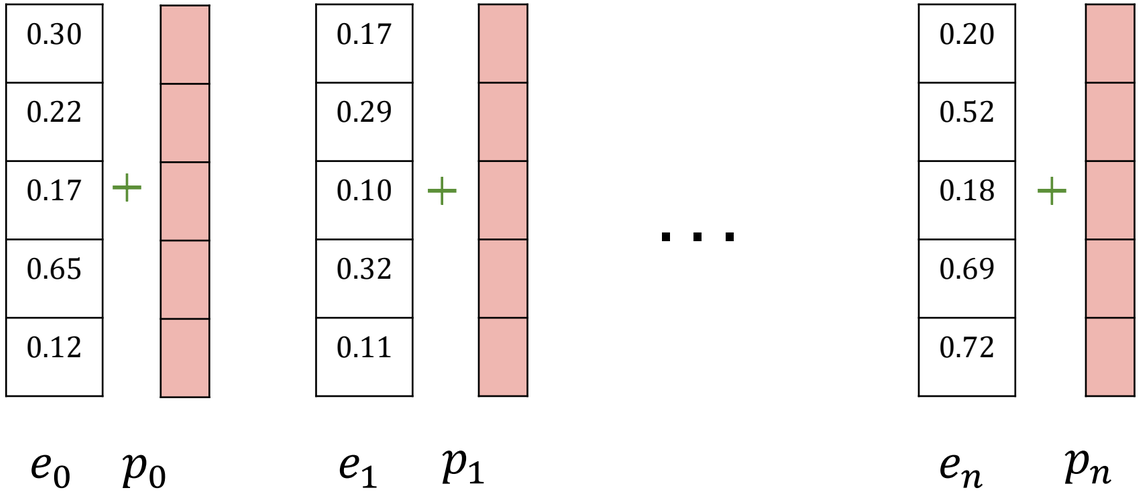
https://medium.com/@hari4om/word-embedding-d816f643140

Positional Embedding

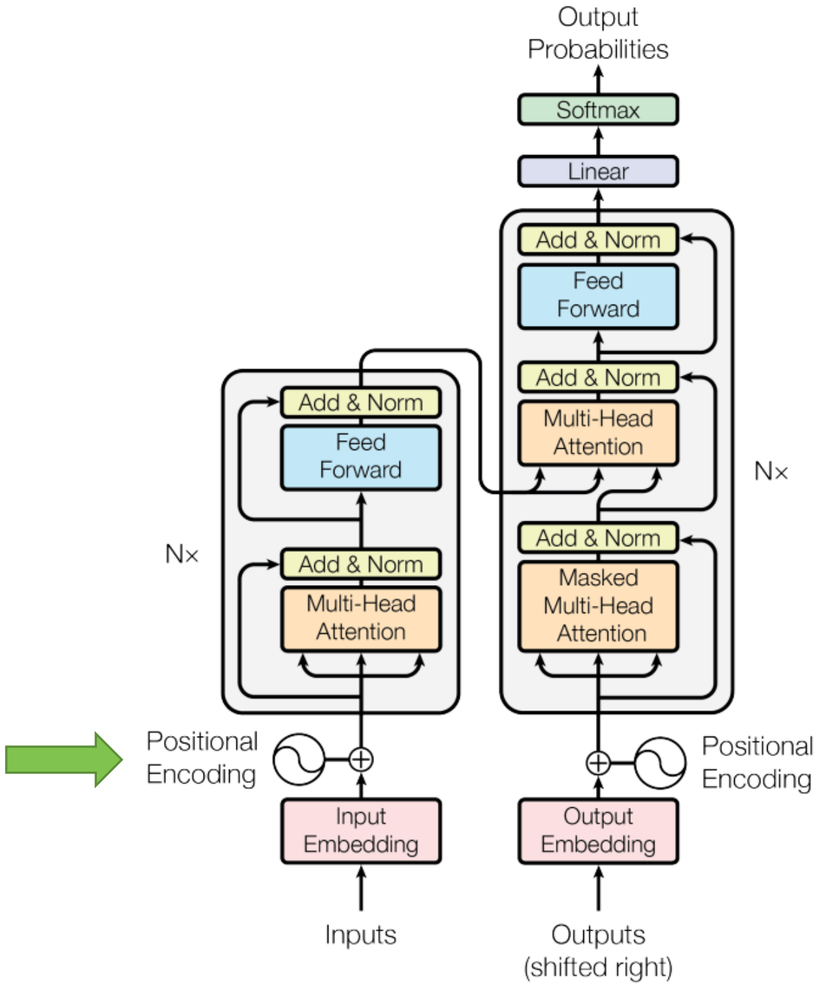


Where there is a will, there …….

(11442 3582 882 … 3582)



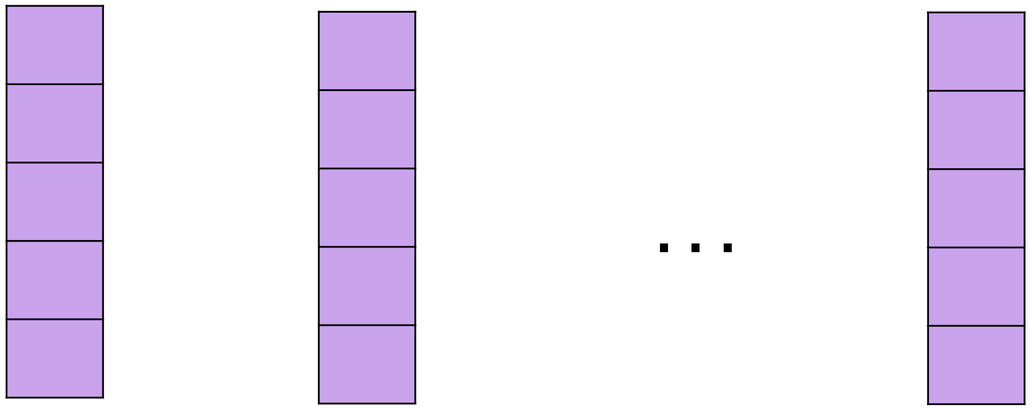
https://medium.com/@hari4om/word-embedding-d816f643140



Positional Embedding

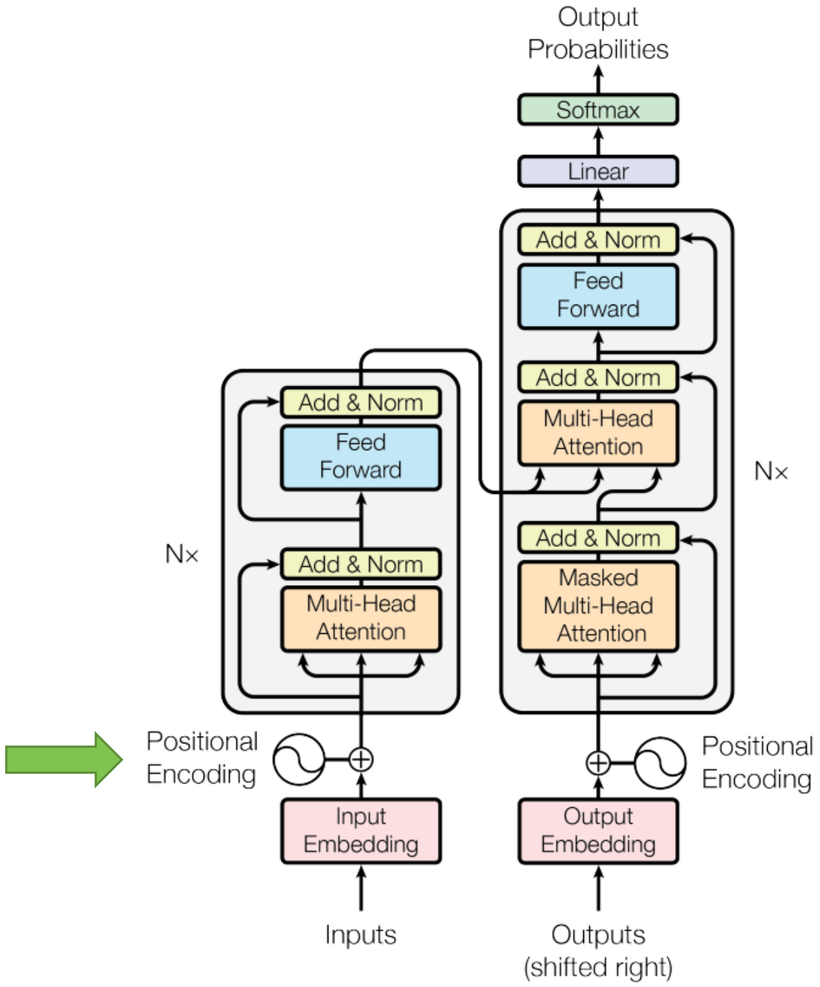
Where there is a will, there …….

(11442 3582 882 … 3582)



ep_0 ep_1 ep_n

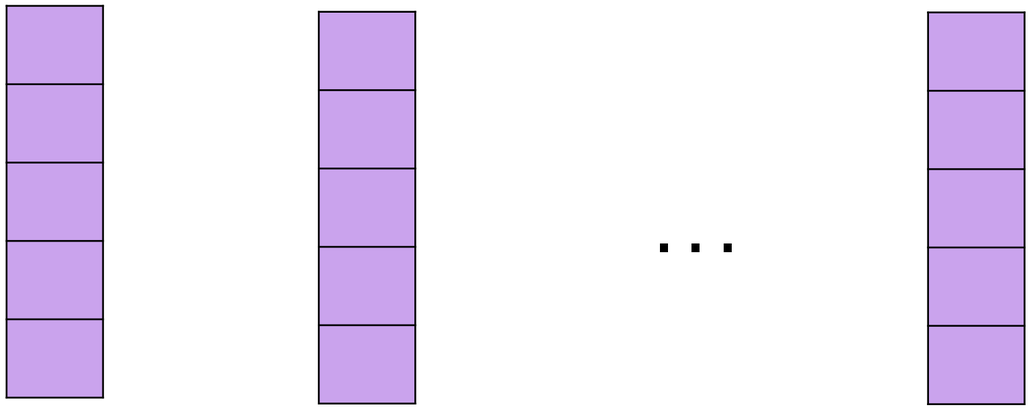
https://medium.com/@hari4om/word-embedding-d816f643140



Positional Embedding

Where there is a will, there …….

(11442 3582 882 … 3582)



ep_0 ep_1 ep_n

https://medium.com/@hari4om/word-embedding-d816f643140

Visualizing Positional Encodings



x → position · y → embedding dimension

Low dimensions oscillate quickly across positions; high dimensions oscillate slowly.

The combination gives every position a unique, smoothly-varying fingerprint the model can learn to use.

Learned vs. Fixed Position Embeddings

Sinusoidal (fixed)

No parameters. Extrapolates reasonably to sequence lengths longer than seen in training.

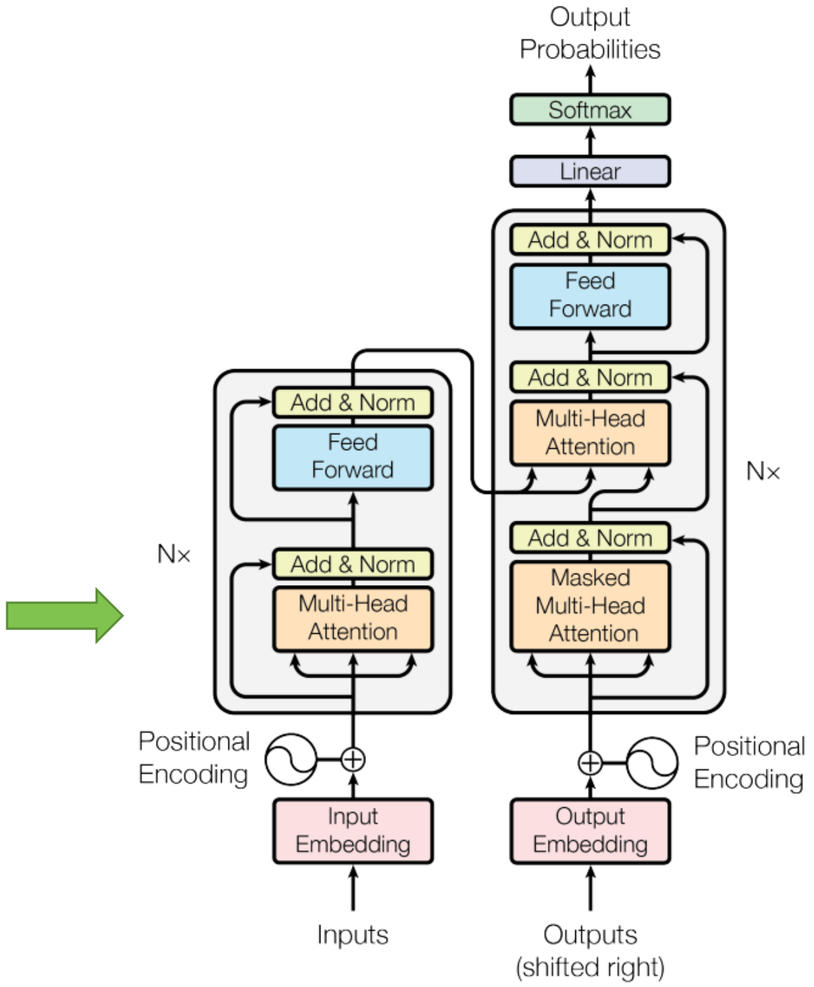
Learned absolute

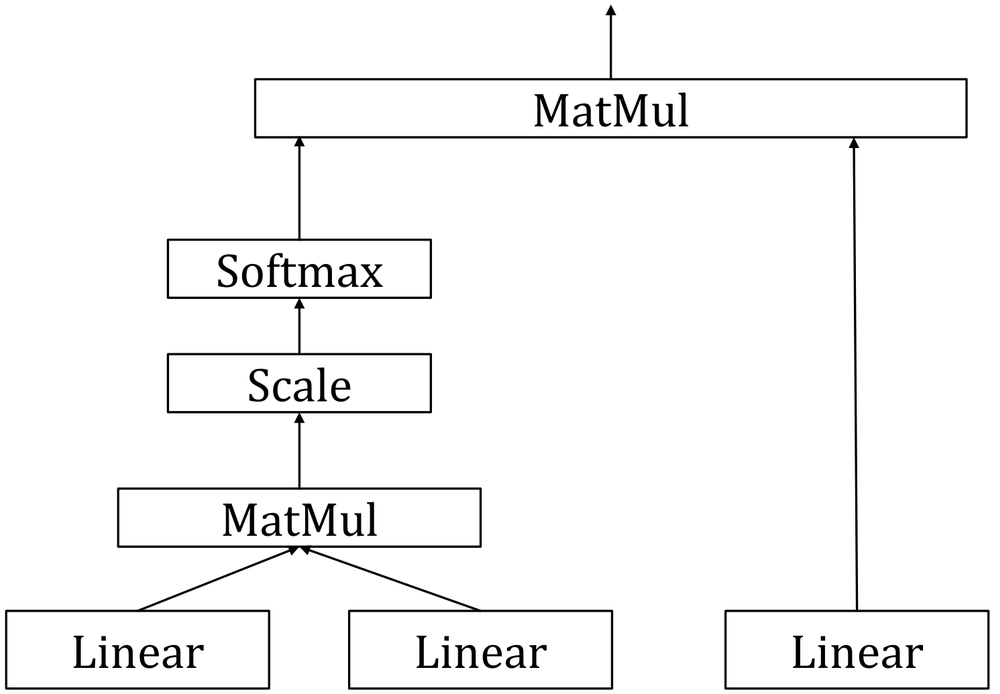
A trainable embedding per position (BERT, GPT-2). Flexible, but capped at the max training length.

Relative (RoPE, ALiBi)

Encode relative offset directly into attention scores — the dominant choice in modern LLMs.

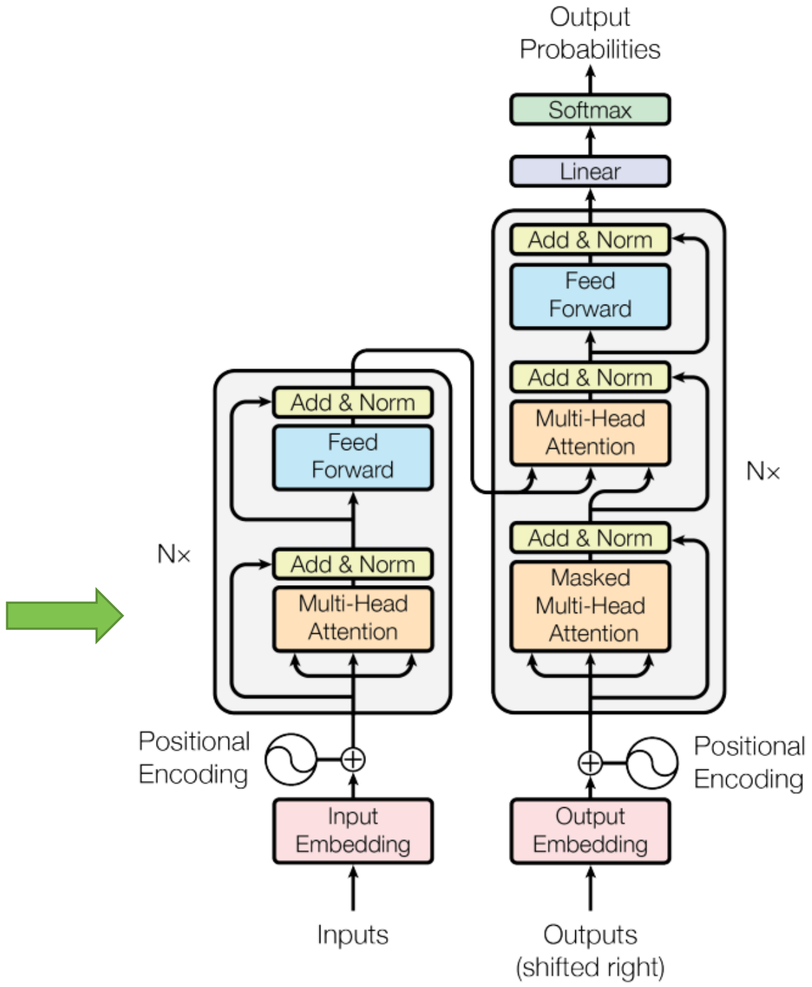
Multi-head Attention

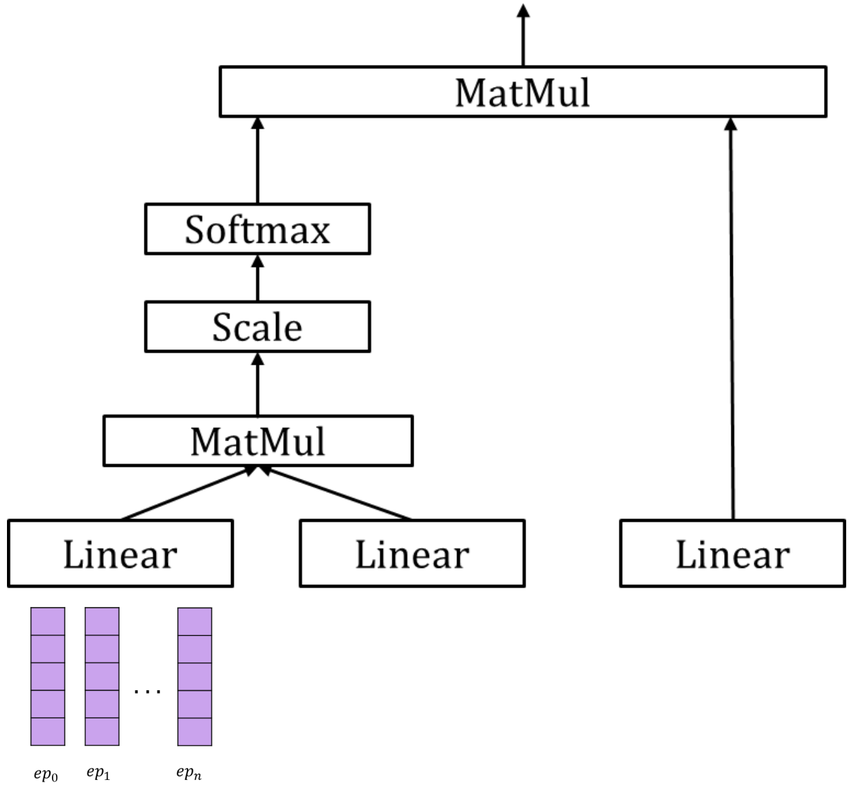




https://medium.com/@hari4om/word-embedding-d816f643140

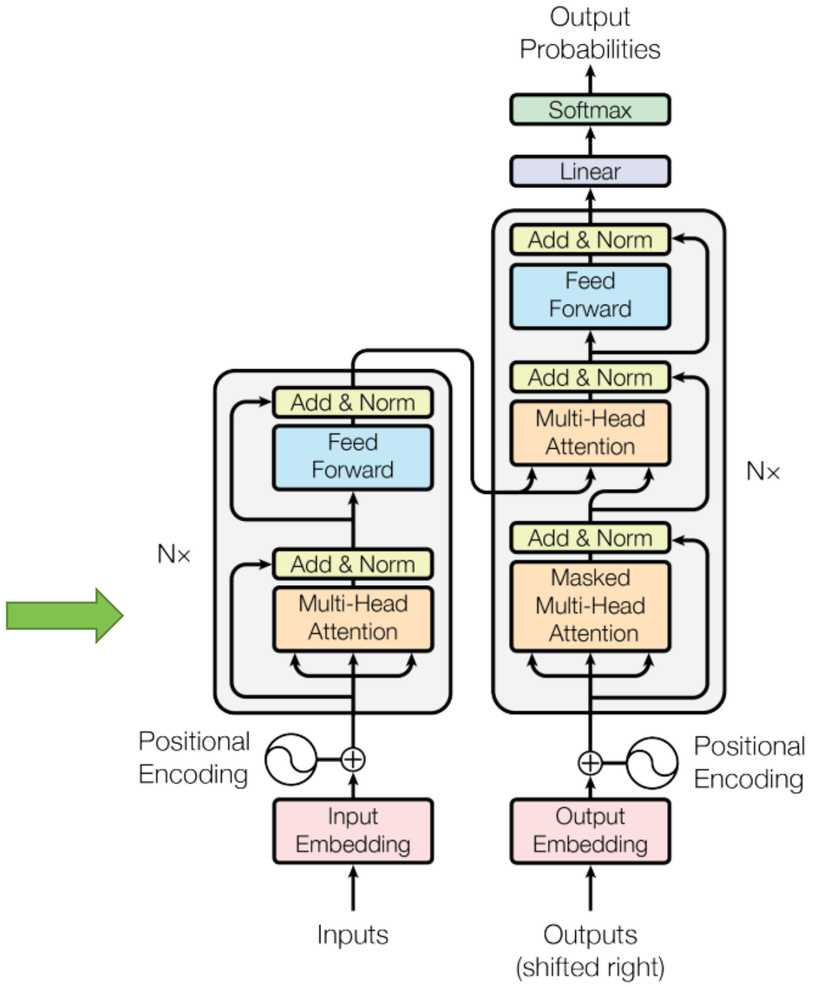
Multi-head Attention

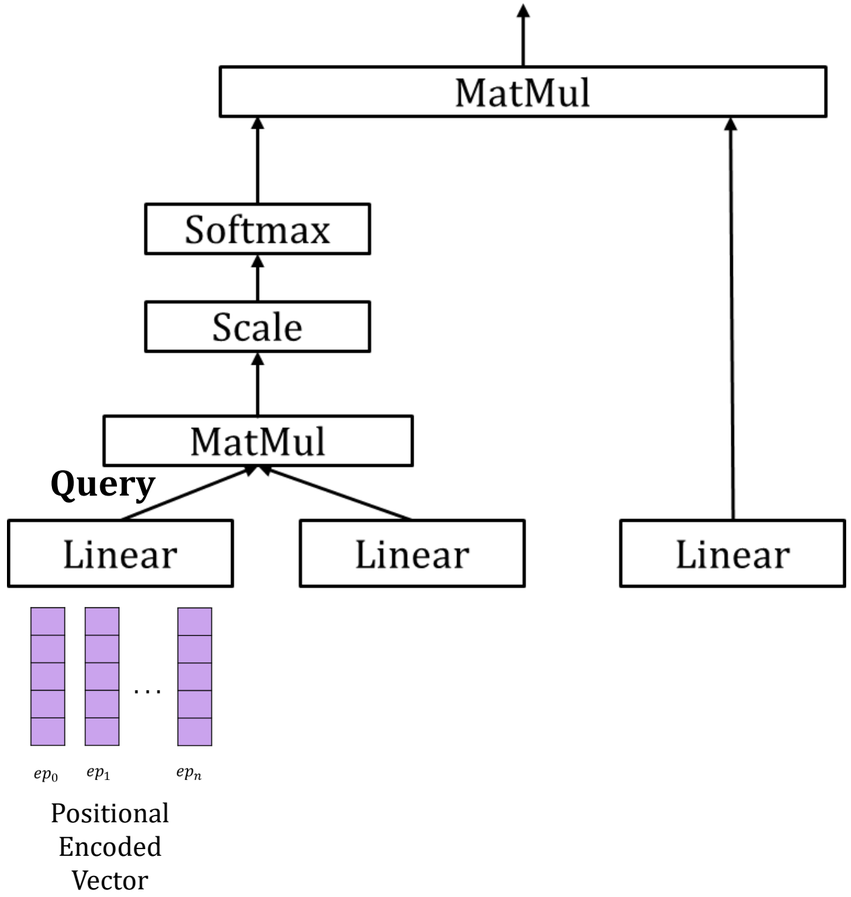




https://medium.com/@hari4om/word-embedding-d816f643140

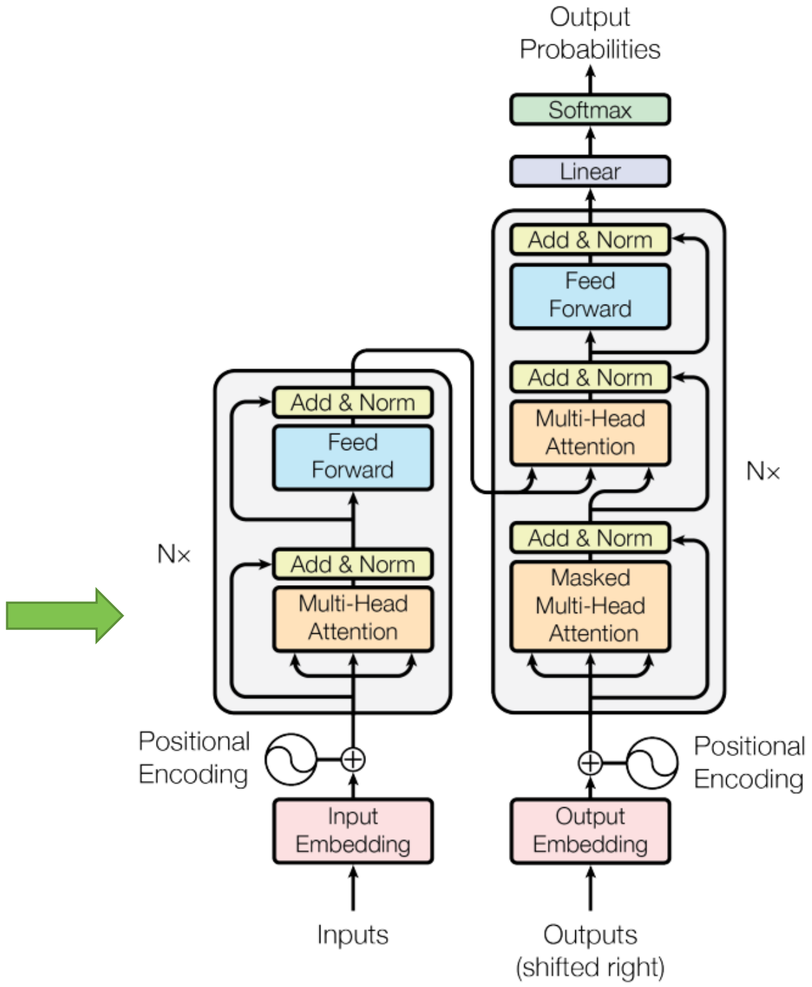
Multi-head Attention

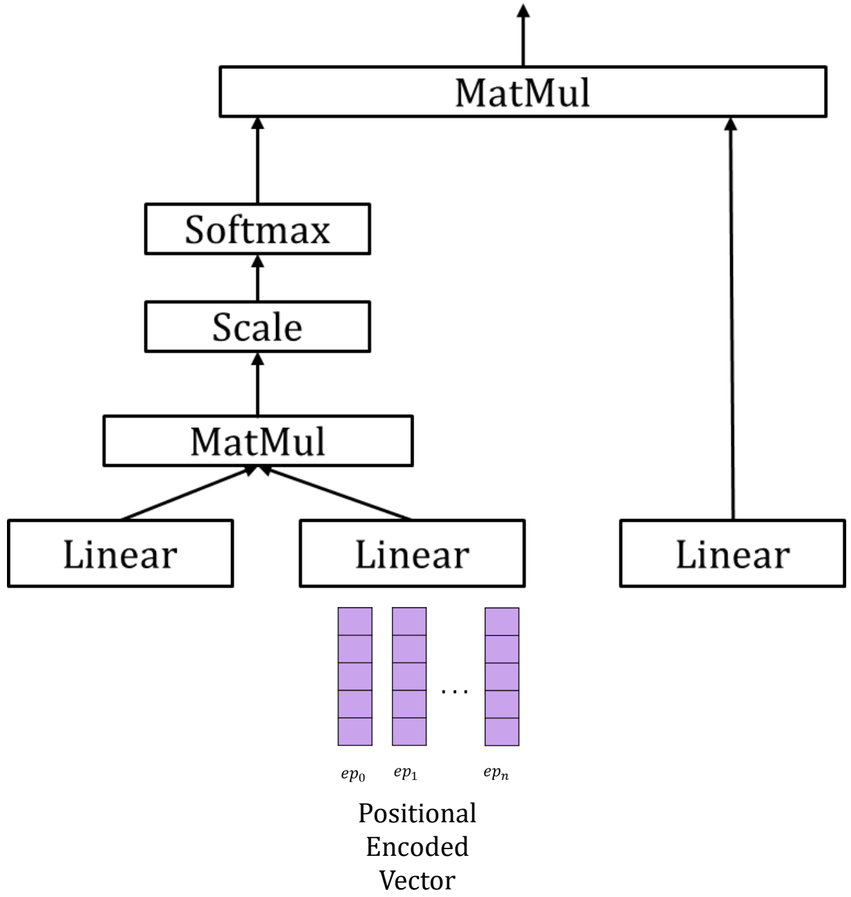




https://medium.com/@hari4om/word-embedding-d816f643140

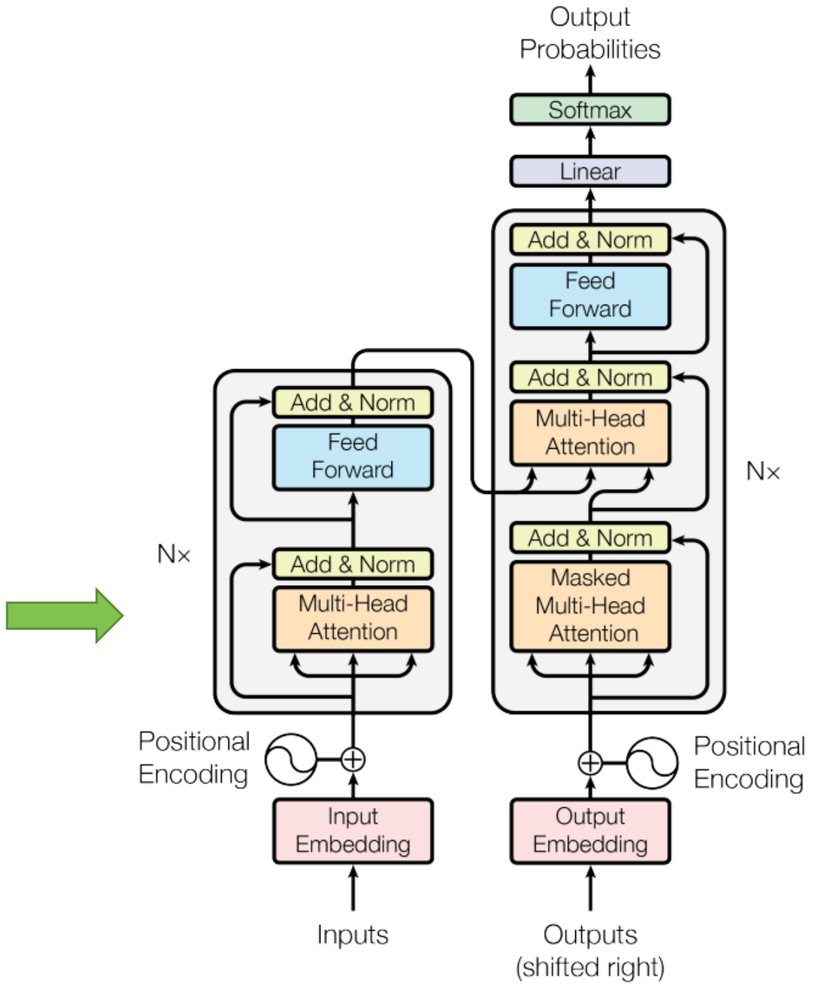
Multi-head Attention

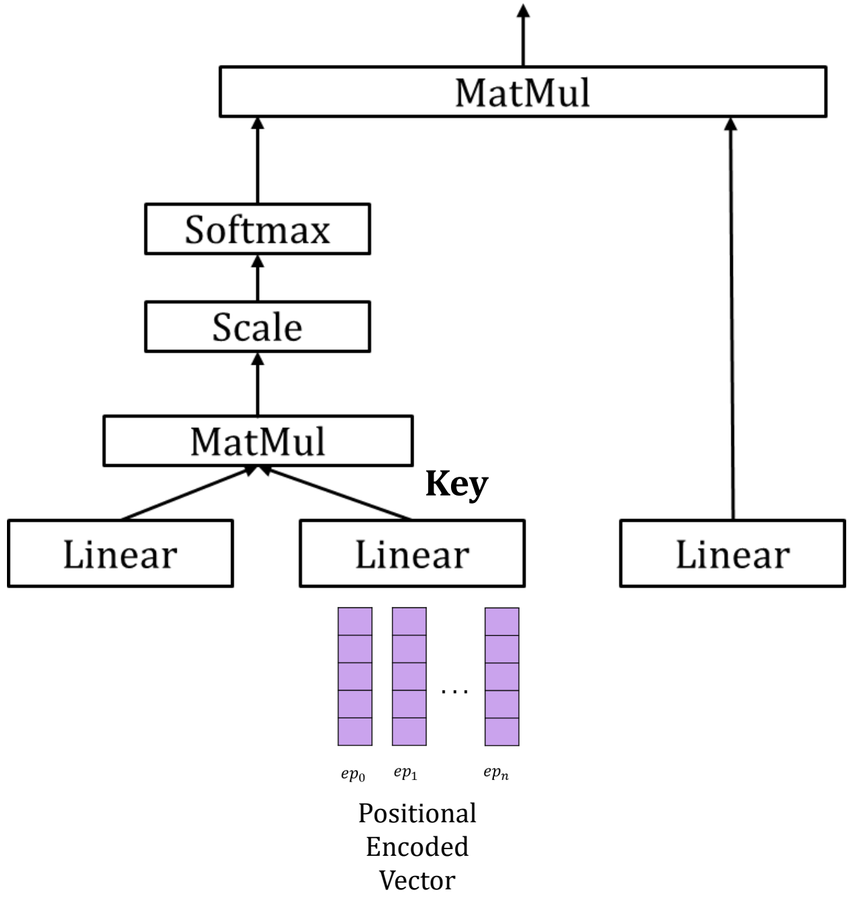




https://medium.com/@hari4om/word-embedding-d816f643140

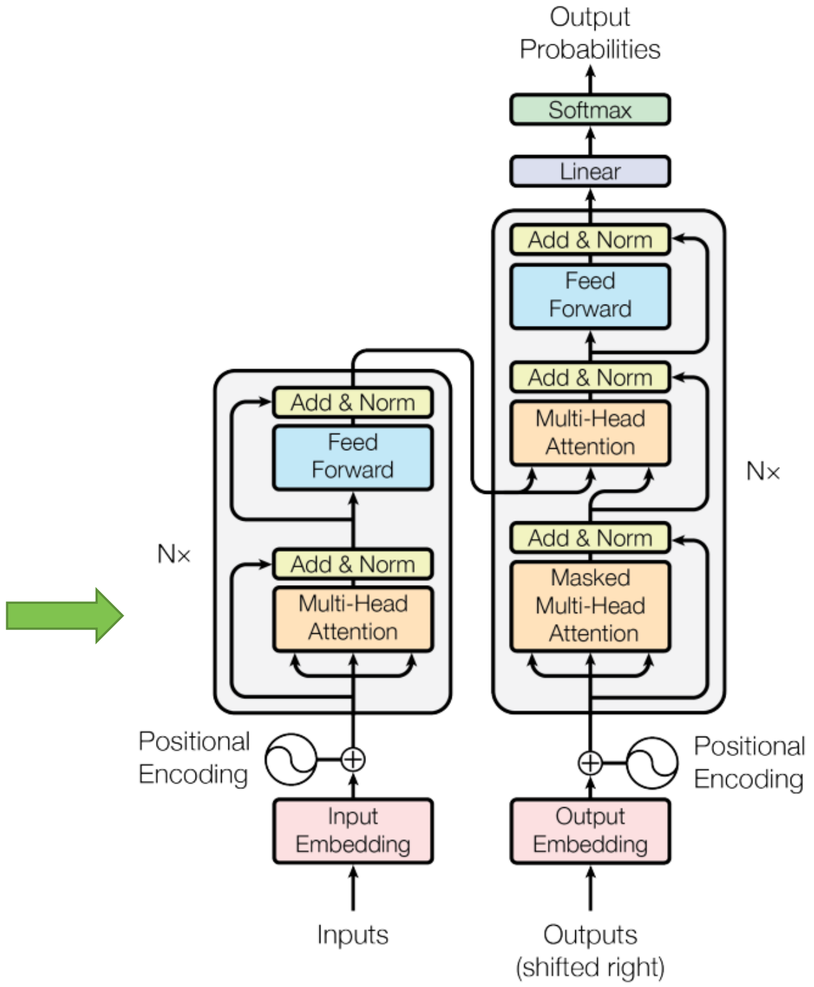
Multi-head Attention

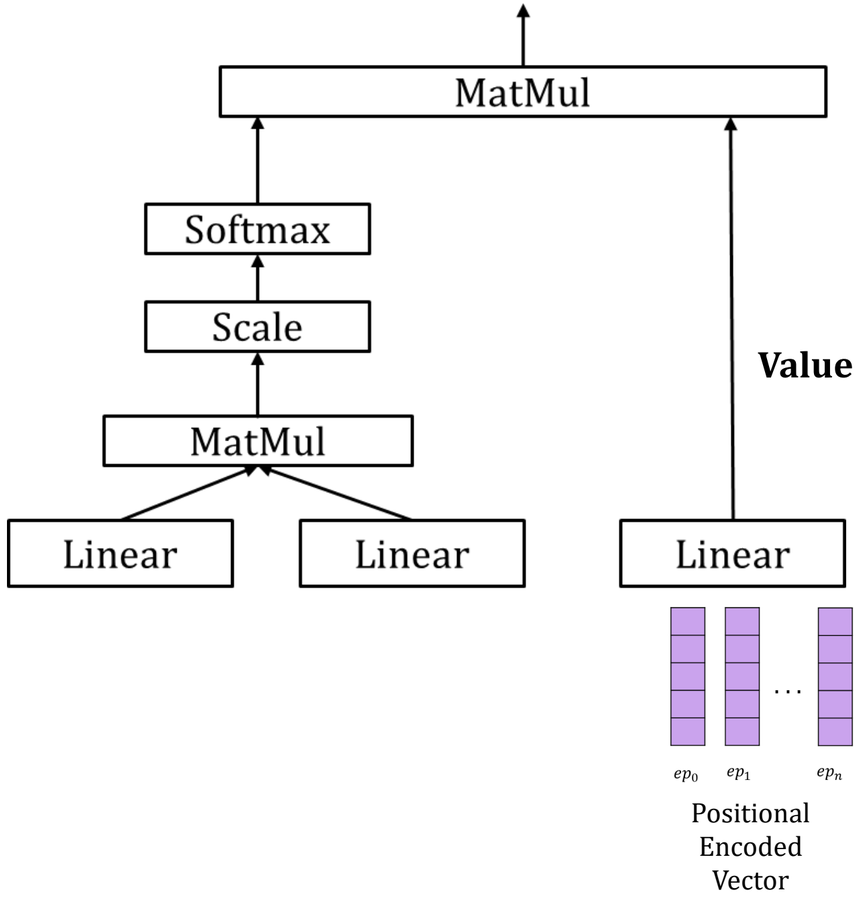




https://medium.com/@hari4om/word-embedding-d816f643140

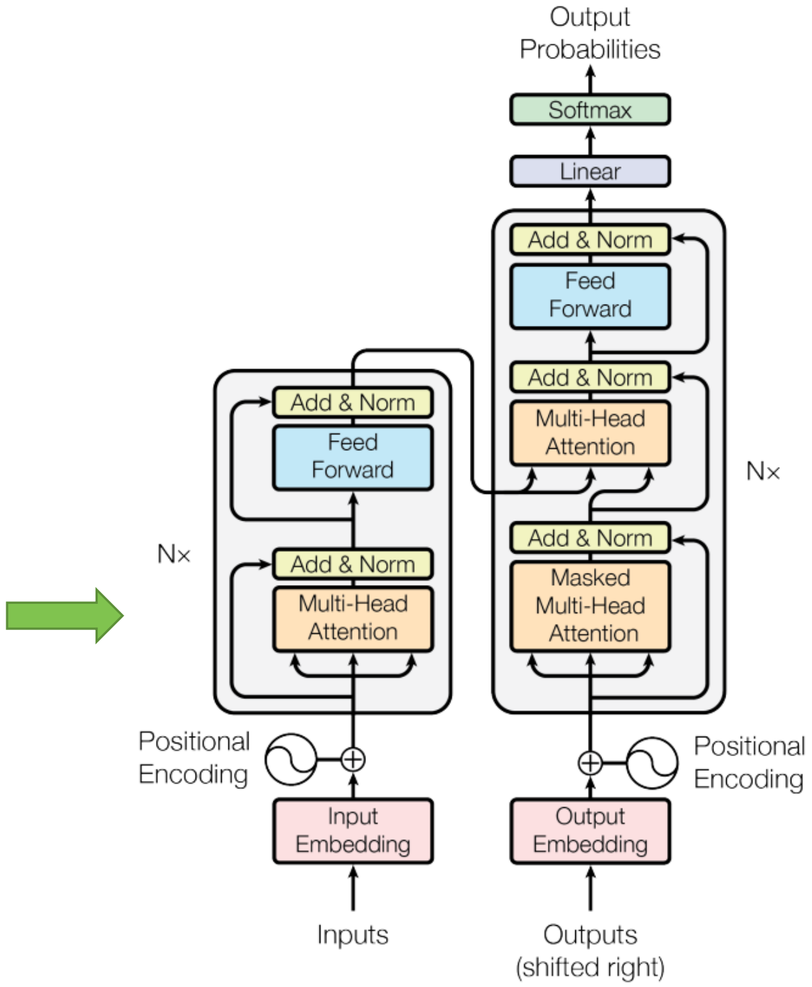
Multi-head Attention

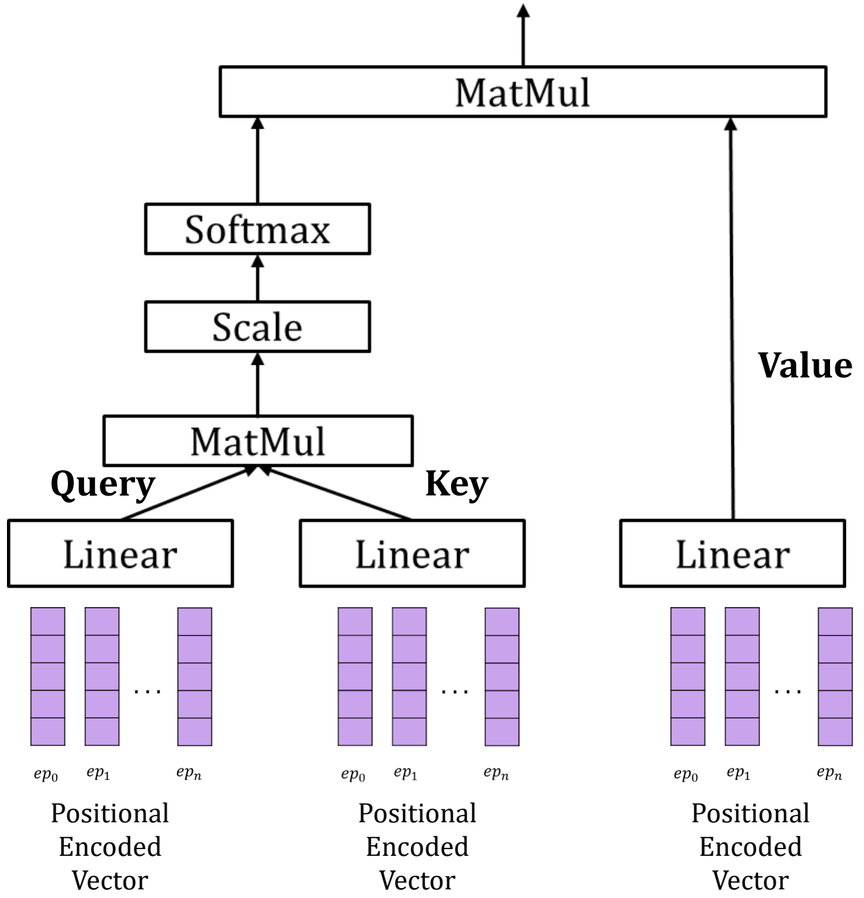




https://medium.com/@hari4om/word-embedding-d816f643140

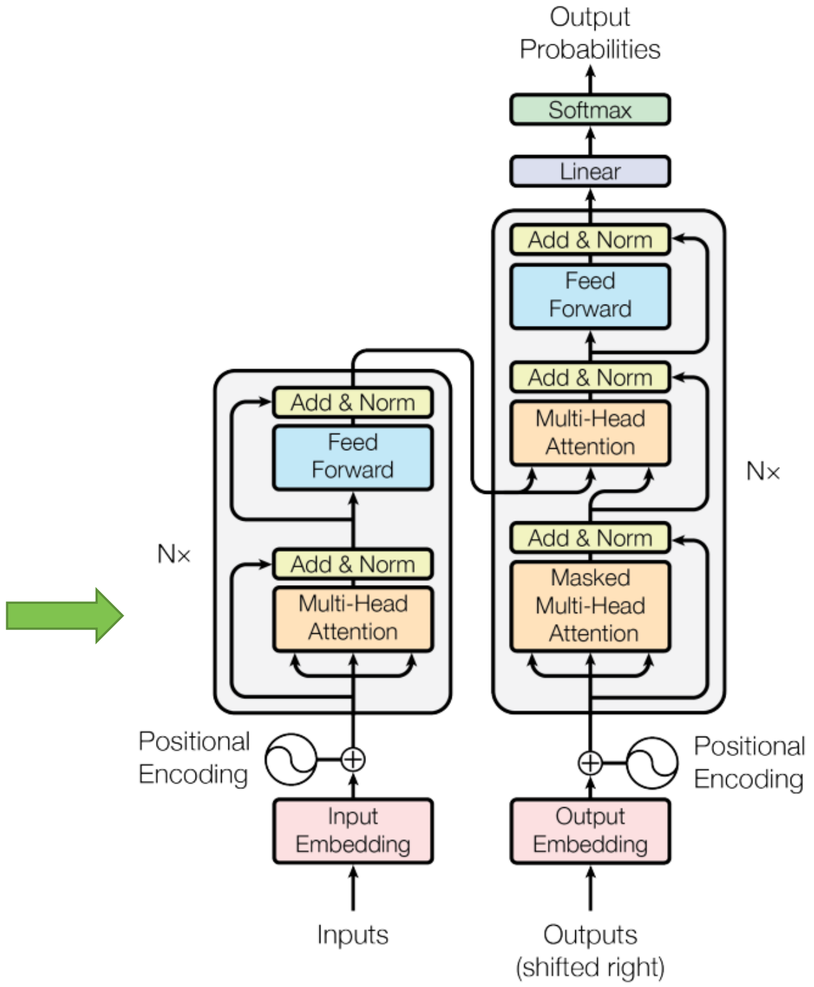
Multi-head Attention



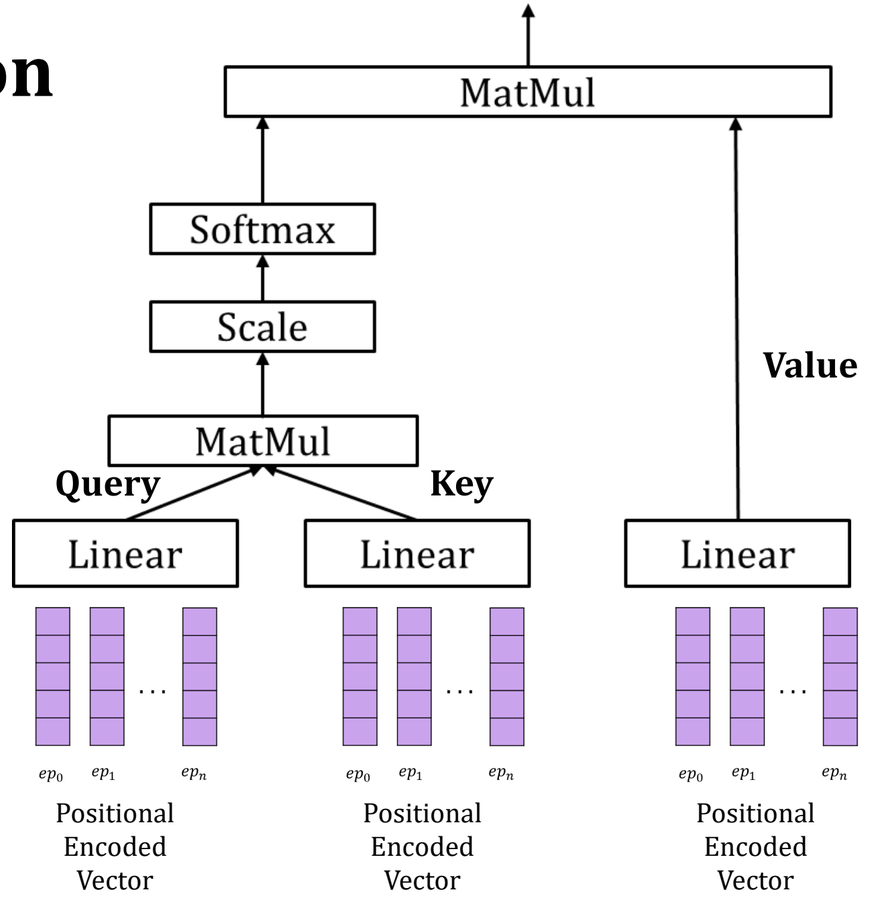


https://medium.com/@hari4om/word-embedding-d816f643140

Multi-head Attention

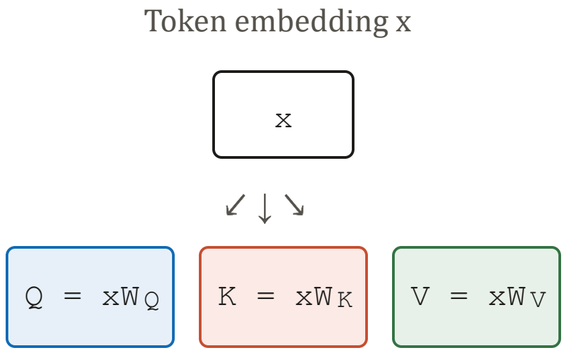


Self-Attention



https://medium.com/@hari4om/word-embedding-d816f643140

Query, Key, Value: Three Views of Every Token



Query — what this token is looking for.

Key — what this token has to offer, for matching.

Value — the actual content passed along once matched.

Scaled Dot-product Attention

$$\text{Attention}(Q,K,V) = \text{softmax}\left(QK^T / \sqrt{d_k} \right) V$$

QK^T

similarity of every query to every key

$/ \sqrt{d_k}$

rescales so gradients stay stable

softmax

turns scores into weights that sum to 1

$\cdot V$

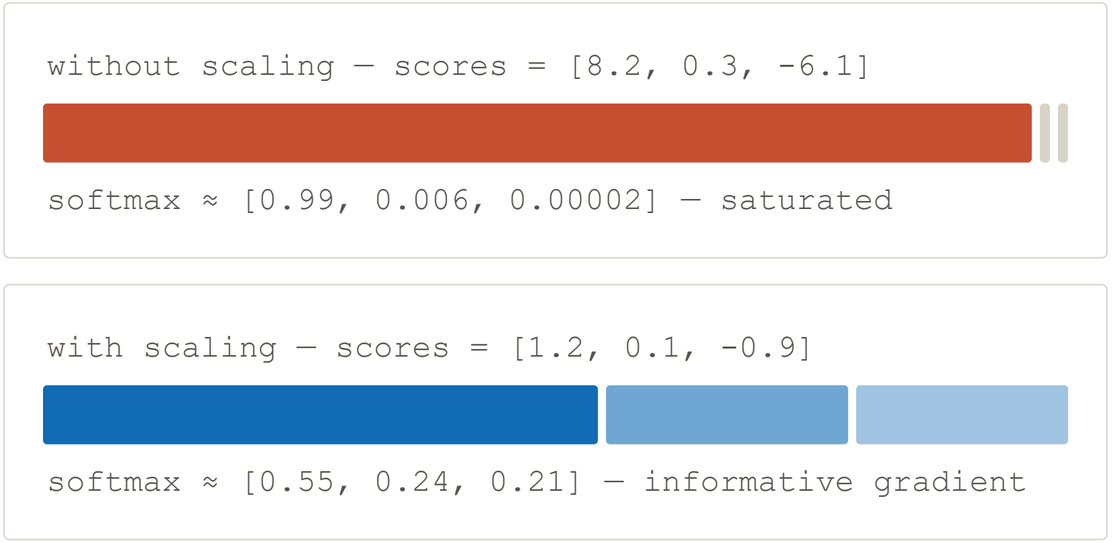
weighted sum of value vectors

Why Divide by $\sqrt{d_k}$?

If q, k have independent, unit-variance entries, each dot product $q \cdot k$ has variance proportional to d_k.

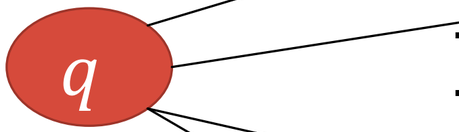
Large-magnitude scores push softmax toward one-hot outputs; gradients through softmax vanish.

Dividing by $\sqrt{d_k}$ keeps the variance ≈ 1, irrespective of the dimension.



Attention Mechanism

- Attention value can be computed using similarity scores and values



Similarity	Key	Value
s_1	k_1	v_1
s_2	k_2	v_2
.	.	.
.	.	.
s_m	k_m	v_m
.	.	.
s_n	k_n	v_n

$$Attention\ value = \sum_i softmax(s_i)v_i$$

Worked Example

Two tokens, $d_k = 2$ · $q = [1, 0]$ · $k_1 = [1, 0]$, $k_2 = [0, 1]$ · $v_1 = [1, 2]$, $v_2 = [3, 0]$

Worked Example

Two tokens, $d_k = 2$ · $q = [1, 0]$ · $k_1 = [1, 0]$, $k_2 = [0, 1]$ · $v_1 = [1, 2]$, $v_2 = [3, 0]$

$$\text{scores} = q \cdot k_1 = 1,\ q \cdot k_2 = 0$$

$$\text{scaled} = [1, 0] / \sqrt{2} = [0.71, 0]$$

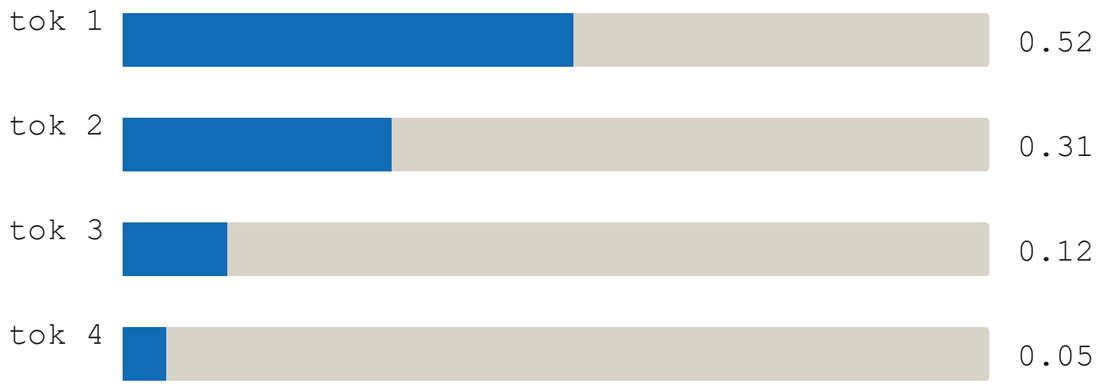
$$\text{softmax}([0.71, 0]) \approx [0.67, 0.33]$$

$$\text{output} = 0.67 \cdot [1,2] + 0.33 \cdot [3,0] \approx [1.66, 1.33]$$

Softmax Turns Scores into a Distribution

$$\text{softmax}(z)_i = e^{z_i} / \Sigma_j e^{z_j}$$

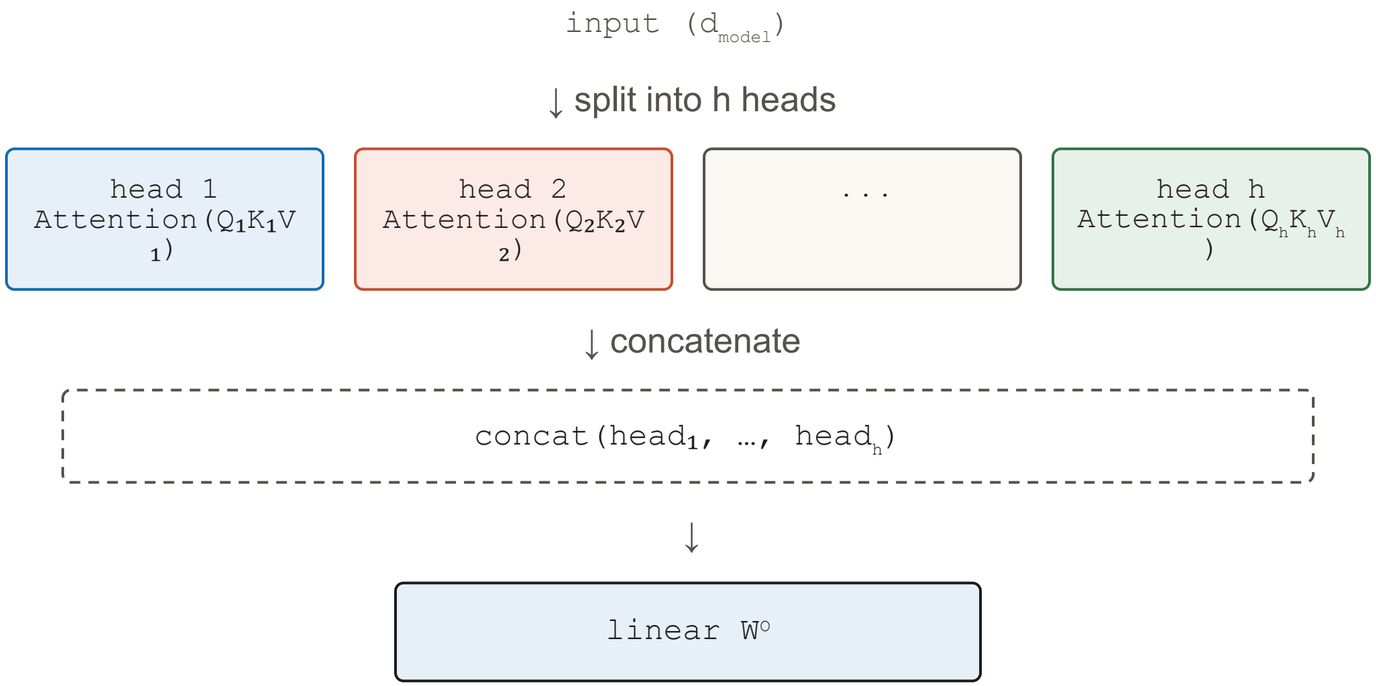
Non-negative, sums to exactly 1, and preserves the ranking of the input scores — a valid probability distribution over positions to attend to.

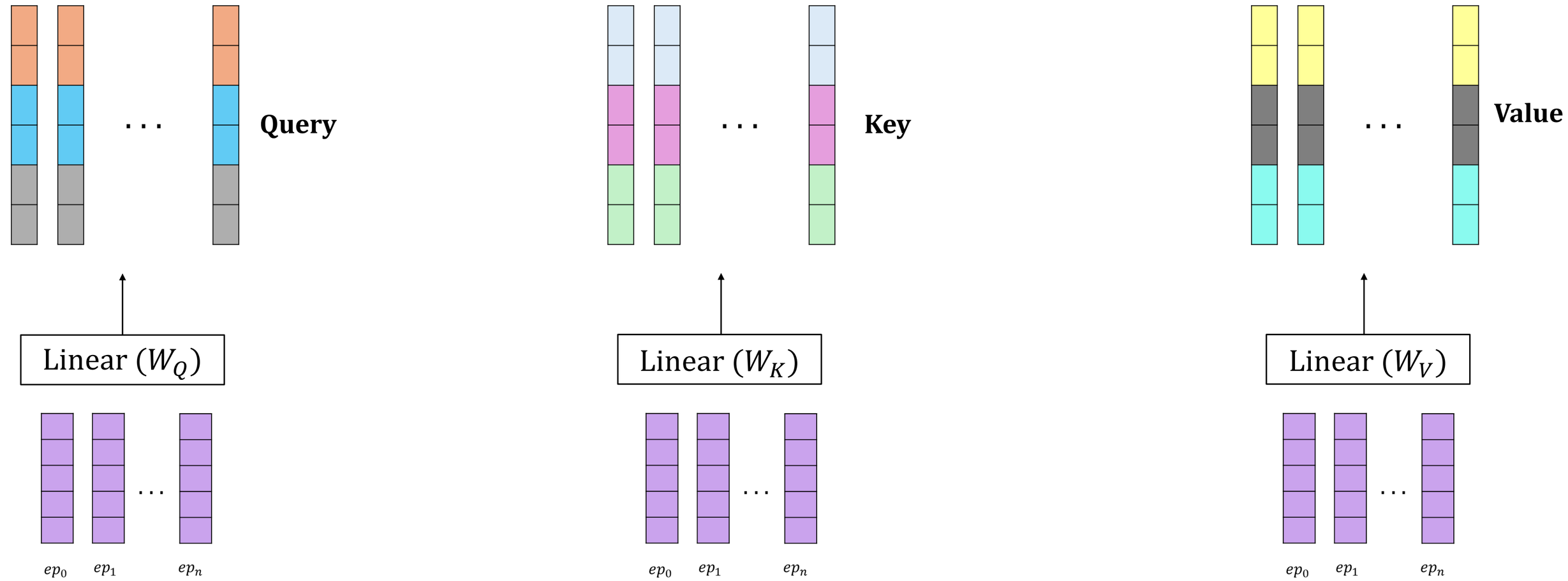


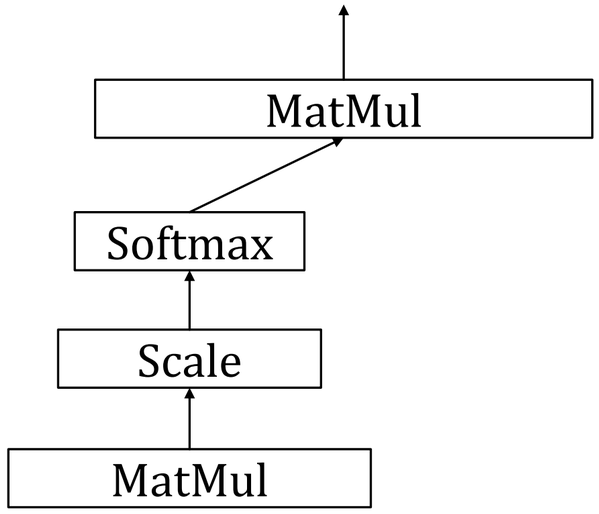
Question

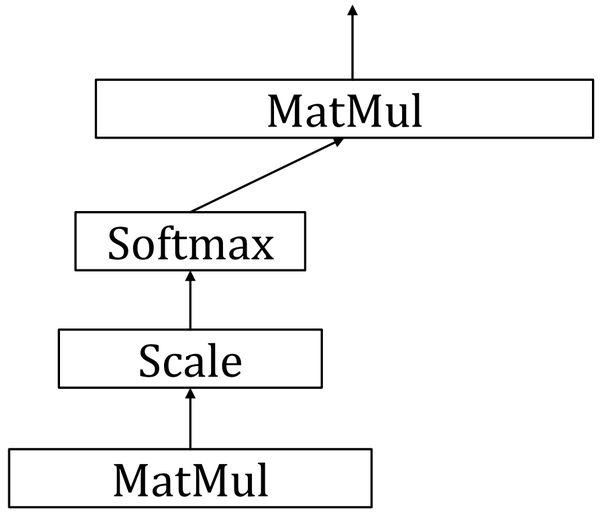
Self-attention computes a weighted sum over ALL positions with no notion of order. If you randomly shuffled the input tokens, what would happen to the output?

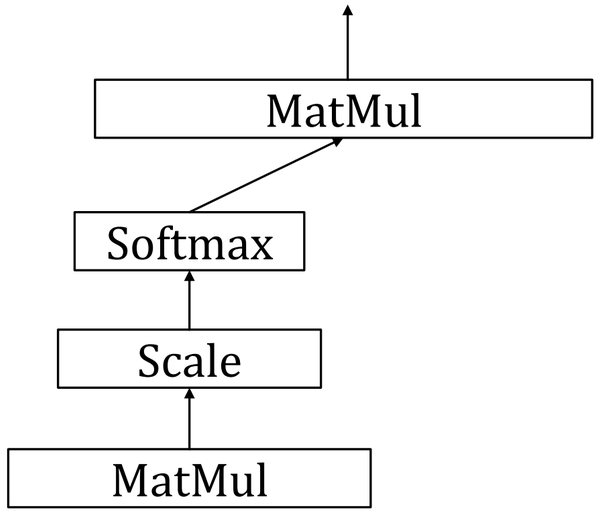
Multi-head Attention Architecture

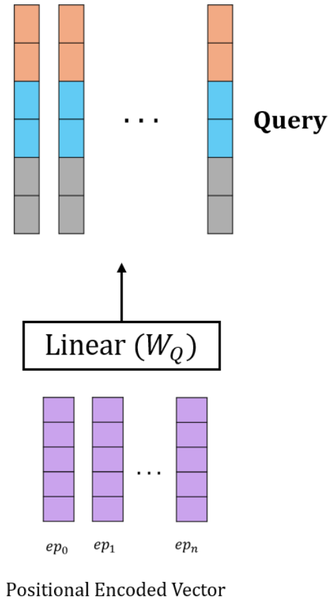


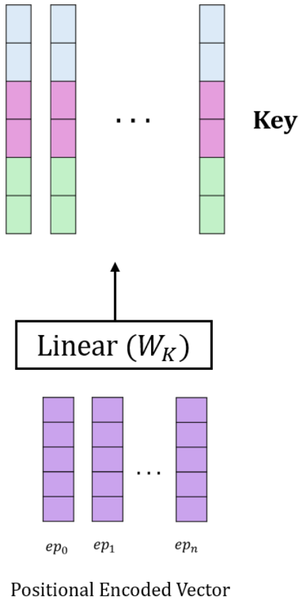

Query
Key
Value
Linear (W_Q)
Linear (W_K)
Linear (W_V)
ep_0 ep_1 ep_n
Positional Encoded Vector
Positional Encoded Vector
Positional Encoded Vector

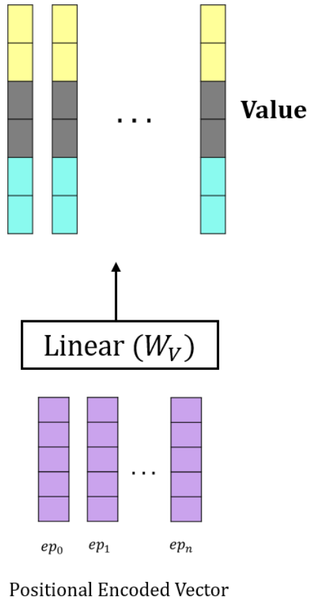

MatMul
Softmax
Scale
MatMul

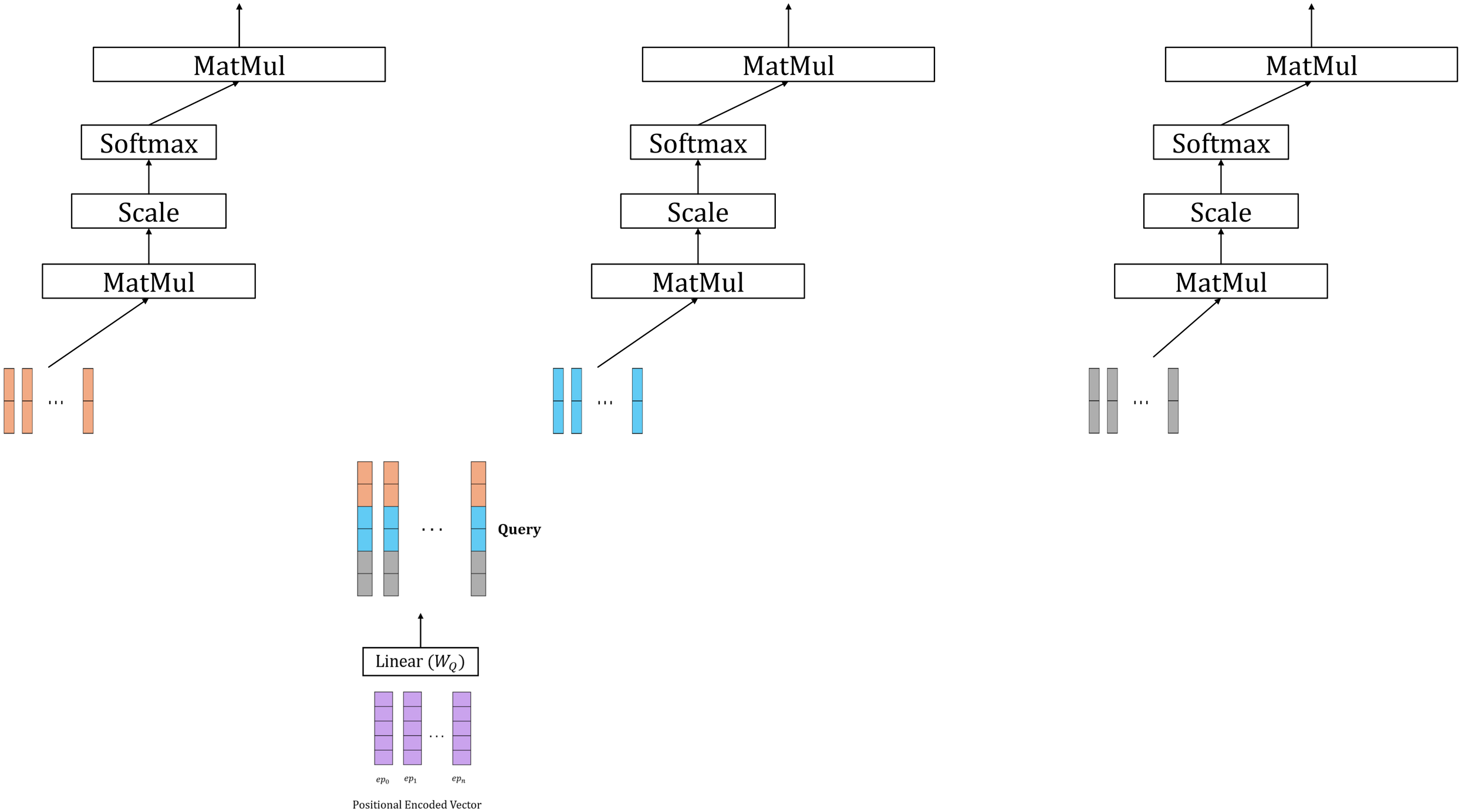

MatMul
Softmax
Scale
MatMul

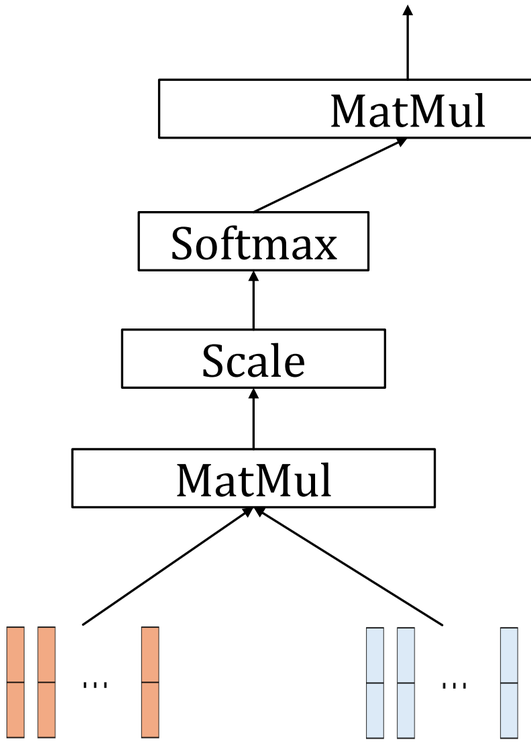

MatMul
Softmax
Scale
MatMul

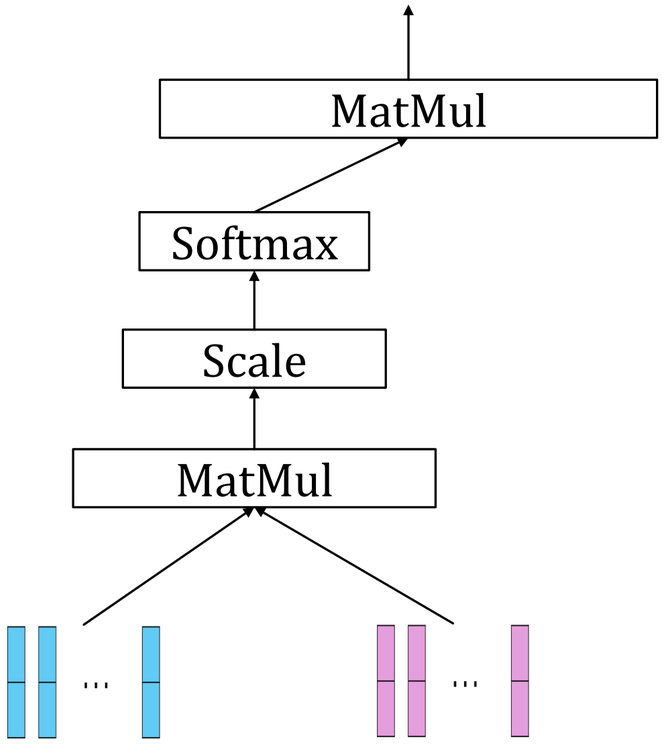

Query
Linear (W_Q)
ep_0 ep_1 ep_n
Positional Encoded Vector

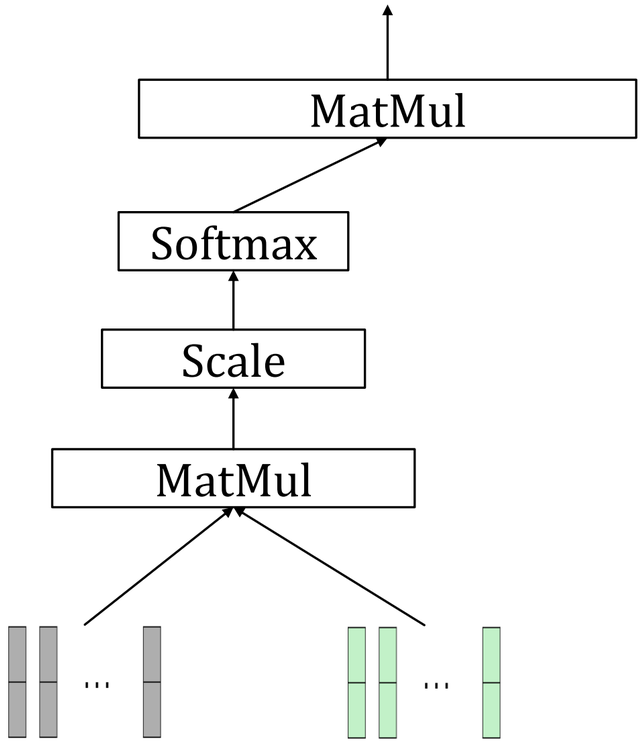

Key
Linear (W_K)
ep_0 ep_1 ep_n
Positional Encoded Vector

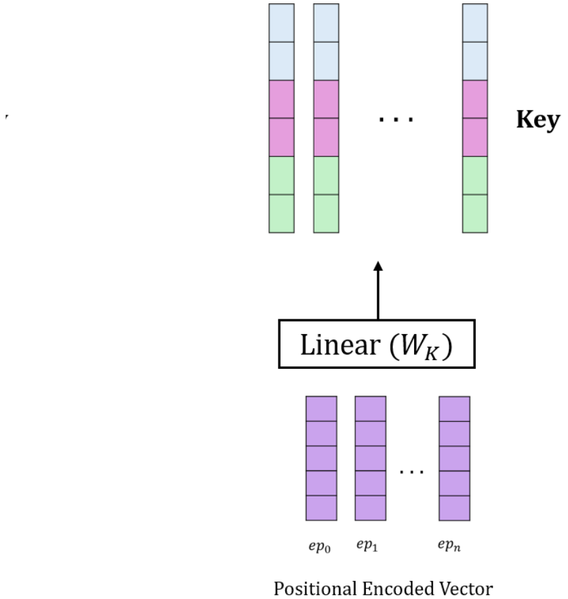

Value
Linear (W_V)
ep_0 ep_1 ep_n
Positional Encoded Vector

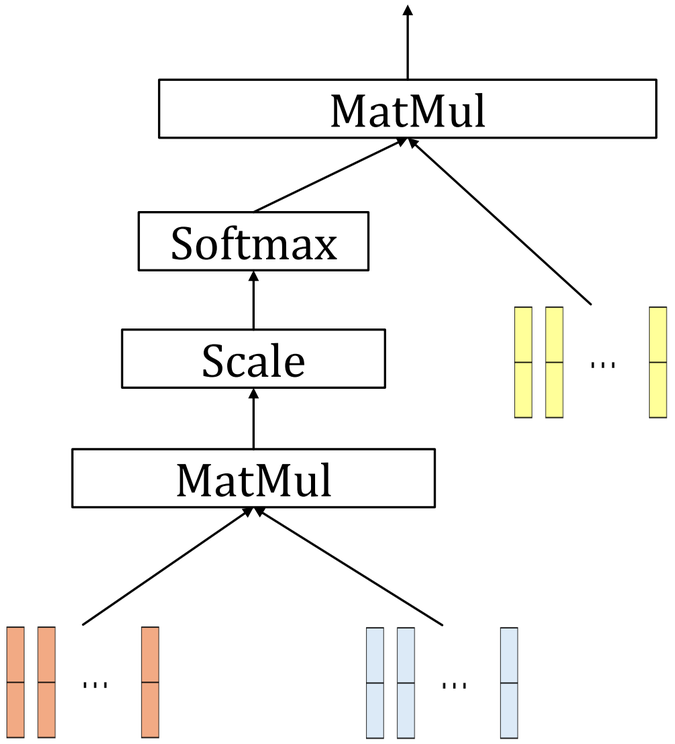

MatMul
Softmax
Scale
MatMul
MatMul
Softmax
Scale
MatMul
MatMul
Softmax
Scale
MatMul
Query
Linear (W_Q)
ep_0
ep_1
ep_n
Positional Encoded Vector

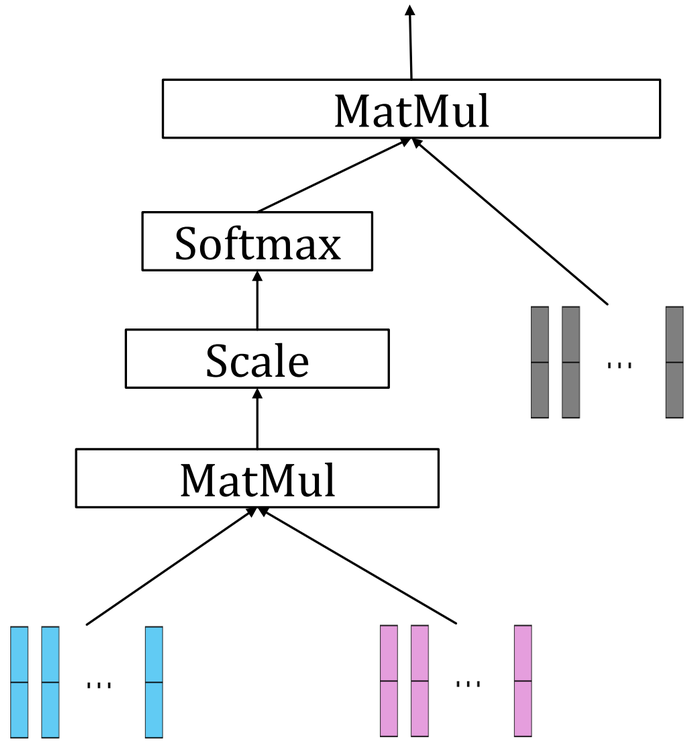

MatMul
Softmax
Scale
MatMul

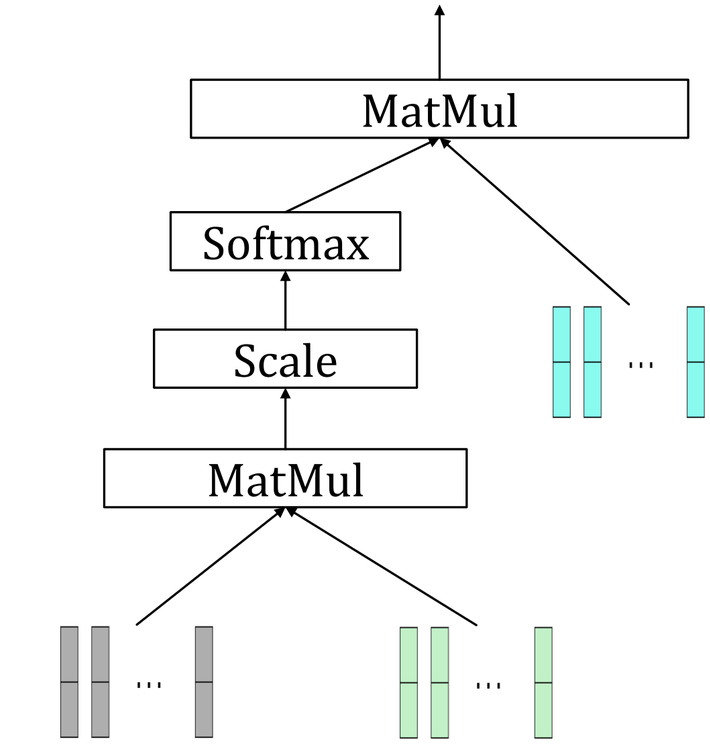

MatMul
Softmax
Scale
MatMul

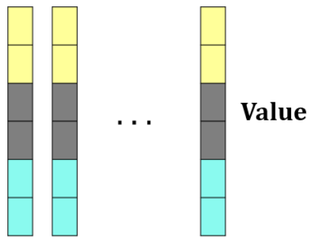

MatMul
Softmax
Scale
MatMul

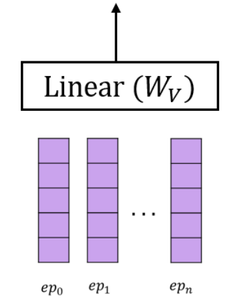

Key
Linear (W_K)
ep_0
ep_1
ep_n
Positional Encoded Vector



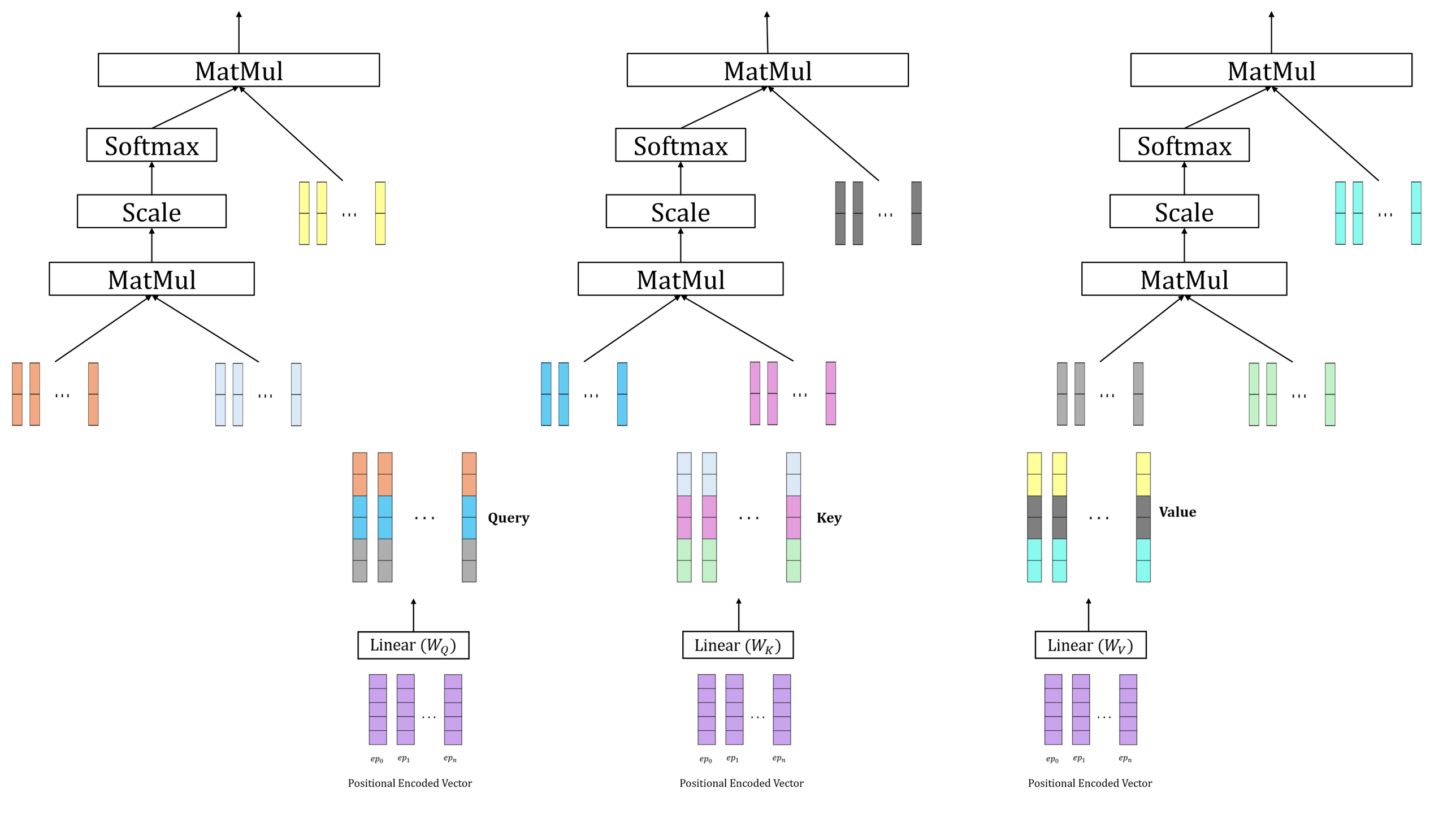


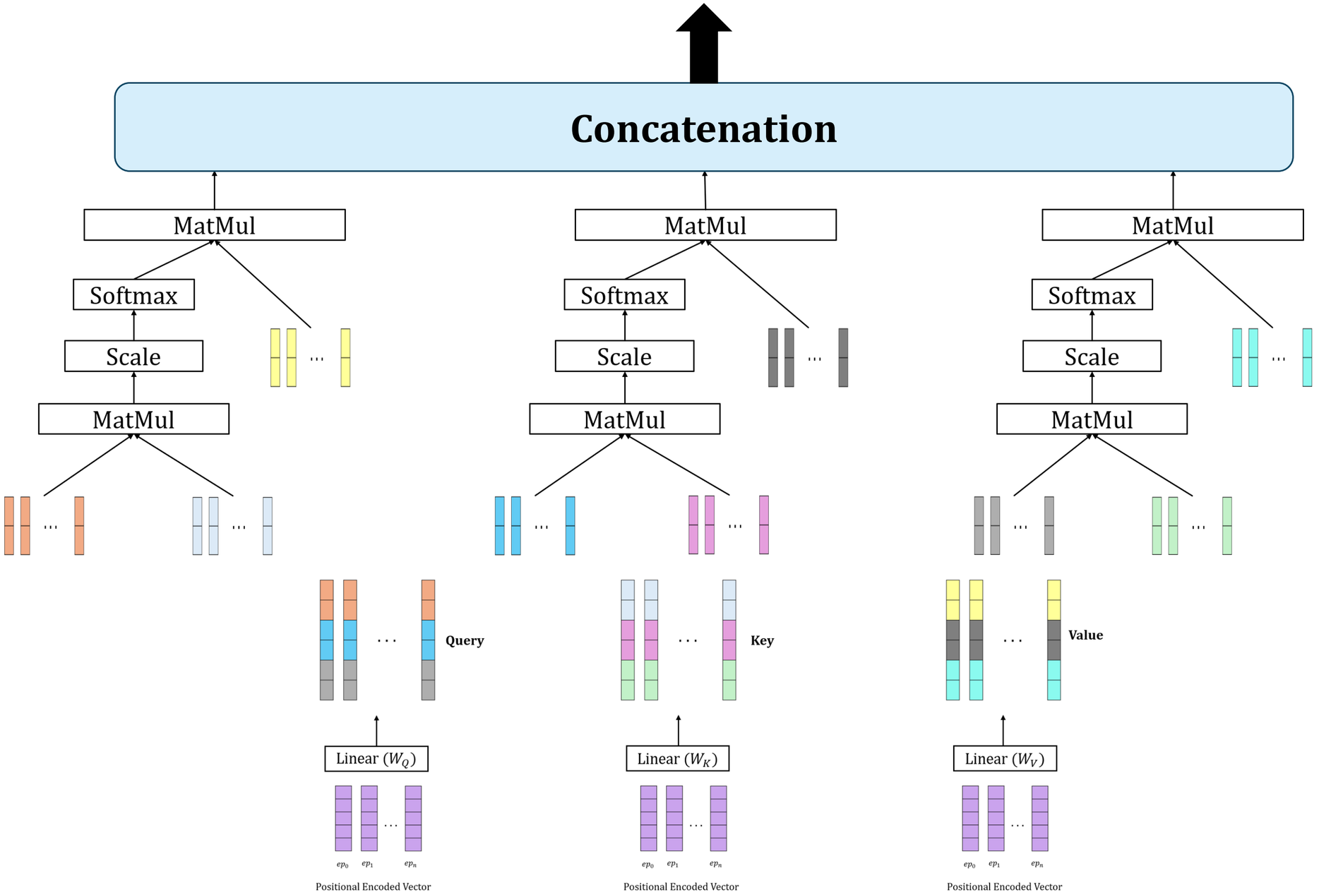






Positional Encoded Vector


MatMul
Softmax
Scale
MatMul
MatMul
Softmax
Scale
MatMul
MatMul
Softmax
Scale
MatMul
Query
Key
Value
Linear (W_Q)
Linear (W_K)
Linear (W_V)
ep_0 ep_1 ep_n
ep_0 ep_1 ep_n
ep_0 ep_1 ep_n
Positional Encoded Vector
Positional Encoded Vector
Positional Encoded Vector


Concatenation
MatMul
Softmax
Scale
MatMul
MatMul
Softmax
Scale
MatMul
MatMul
Softmax
Scale
MatMul
Query
Key
Value
Linear (W_Q)
Linear (W_K)
Linear (W_V)
ep_0 ep_1 ep_n
Positional Encoded Vector
Positional Encoded Vector
Positional Encoded Vector

Multi-head Attention: Why One View isn’t Enough

A single attention head produces one weighted average per token — one notion of "relevance."

But language has many simultaneous relationships: syntax, coreference, adjacency, semantic similarity.

Run several attention operations in parallel, each in its own subspace.

Possibilities

- Head 1 — tracks adjacent-word patterns
- Head 2 — tracks coreference (pronouns → nouns)
- Head 3 — tracks subject–verb agreement
- Head h — whatever the data rewards

Multi-head Attention

$$\text{head}_i = \text{Attention}(QW_i^Q,\ KW_i^K,\ VW_i^V)$$

$$\text{MultiHead}(Q,K,V) = \text{concat}(\text{head}_1, \ldots, \text{head}_h)\ W^O$$

With $d_{model} = 512$ and $h = 8$ heads, each head works in $d_k = d_v = 64$ dimensions.

Total compute is comparable to single-head attention at full dimensionality — the split is nearly free.

What Different Heads Learn

Positional heads

Attend mostly to nearby tokens — a learned, soft version of a convolution window.

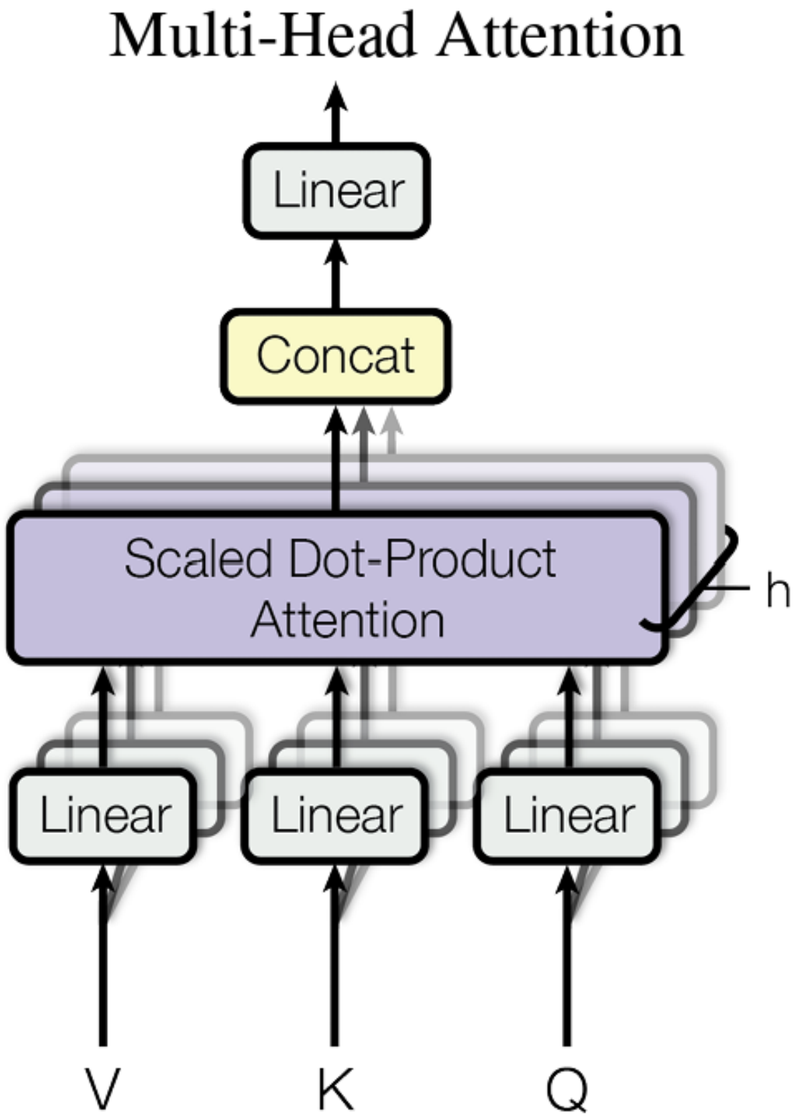
Syntactic heads

Track dependency-like relations — verbs attending to their subjects and objects.

Rare-token heads

Specialize in infrequent or highly informative tokens across the sequence.

No head is told what to specialize in — this structure emerges purely from training.



Multi-head Attention

NUMERICAL

Compute Attention

Given 3 tokens with $d_k = 2$:

$q_2 = [0, 1]$ $k_1 = [1, 1]$, $k_2 = [0, 1]$, $k_3 = [1, 0]$

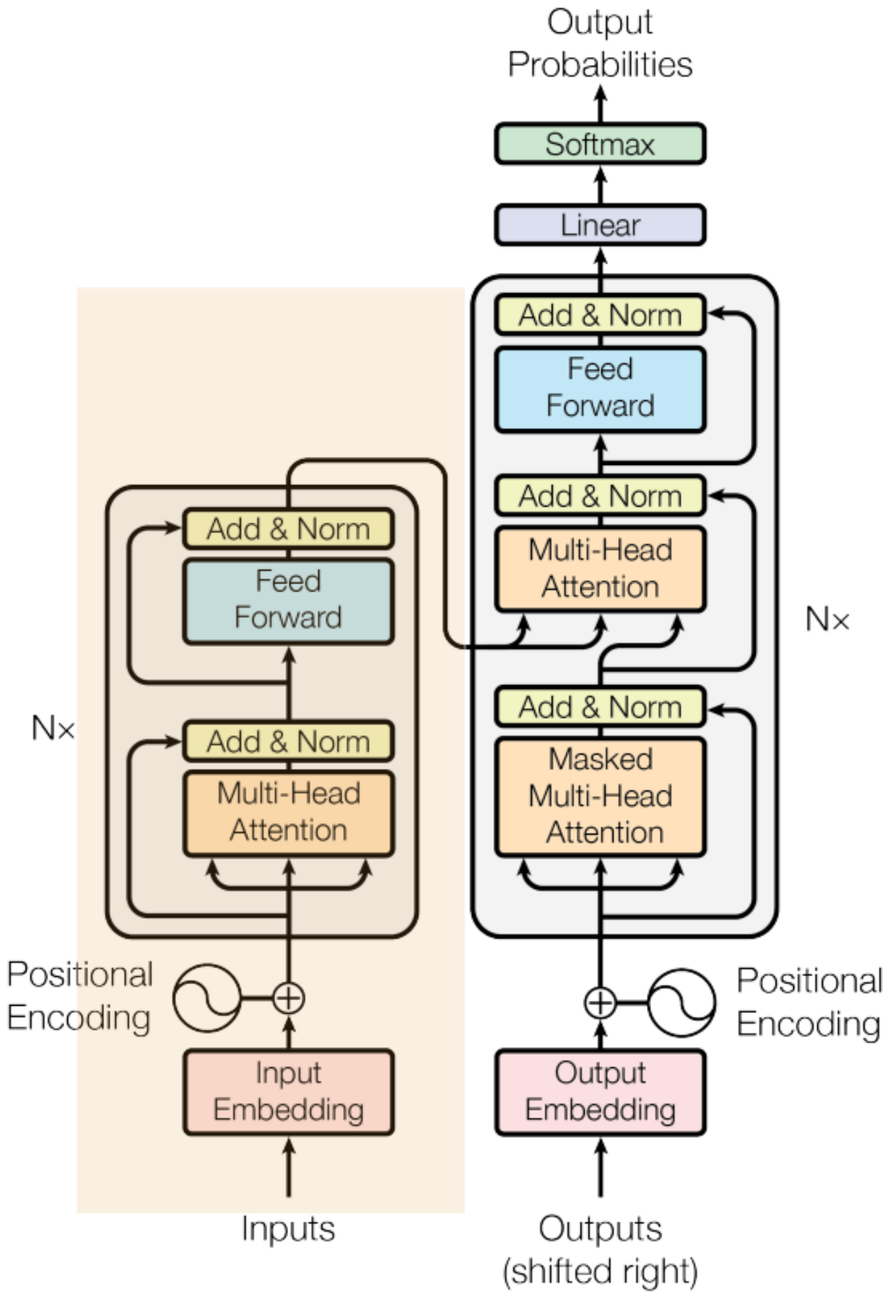
$v_1 = [1, 0]$, $v_2 = [0, 2]$, $v_3 = [2, 2]$

Compute the output vector for the query, using scaled dot-product attention.

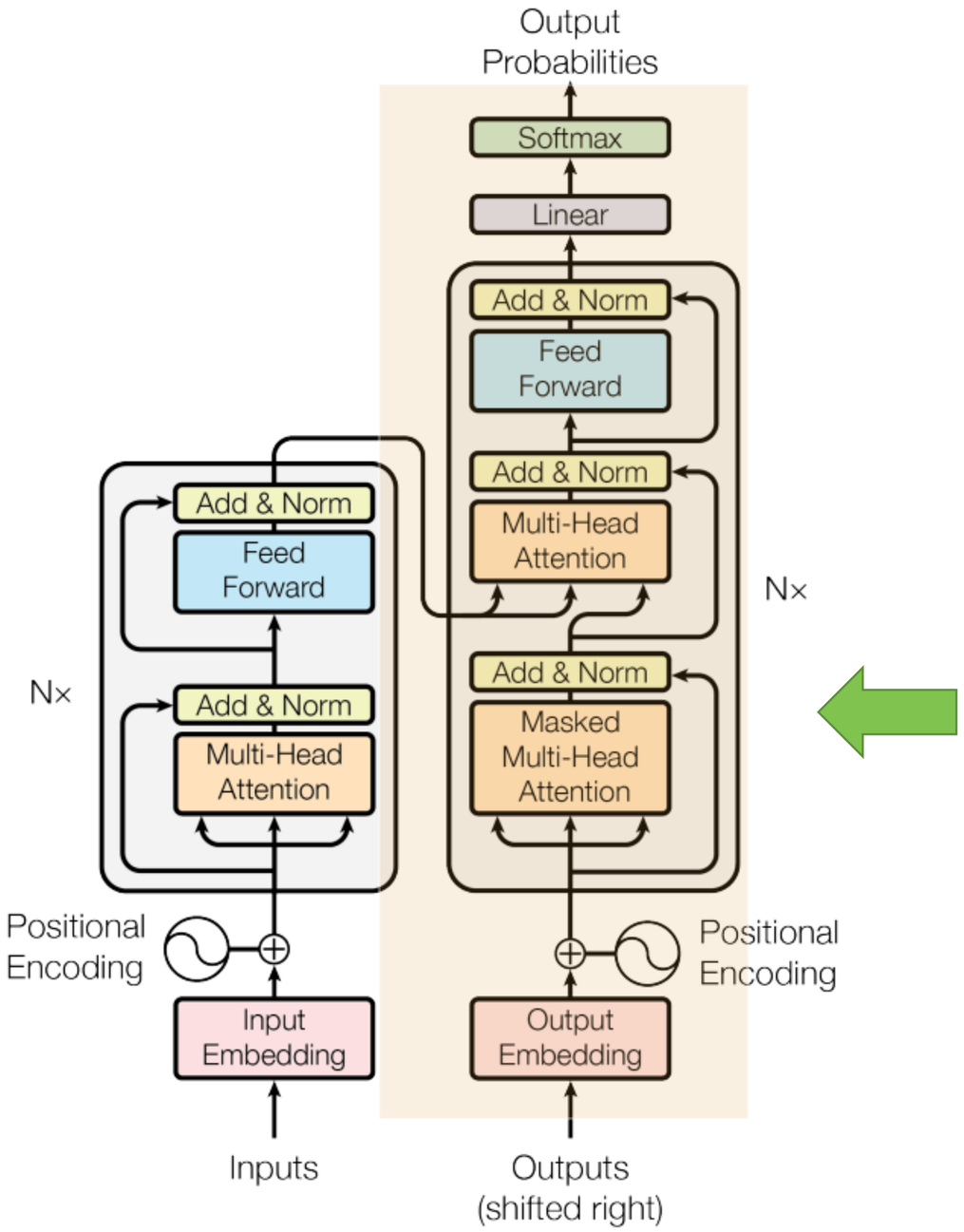
Solution

scores = [1, 1, 0] scaled = [0.71, 0.71, 0] softmax ≈ [0.40, 0.40, 0.20] output = 0.40[1,0]+0.40[0,2]+0.20[2,2] ≈ [0.80, 1.20]

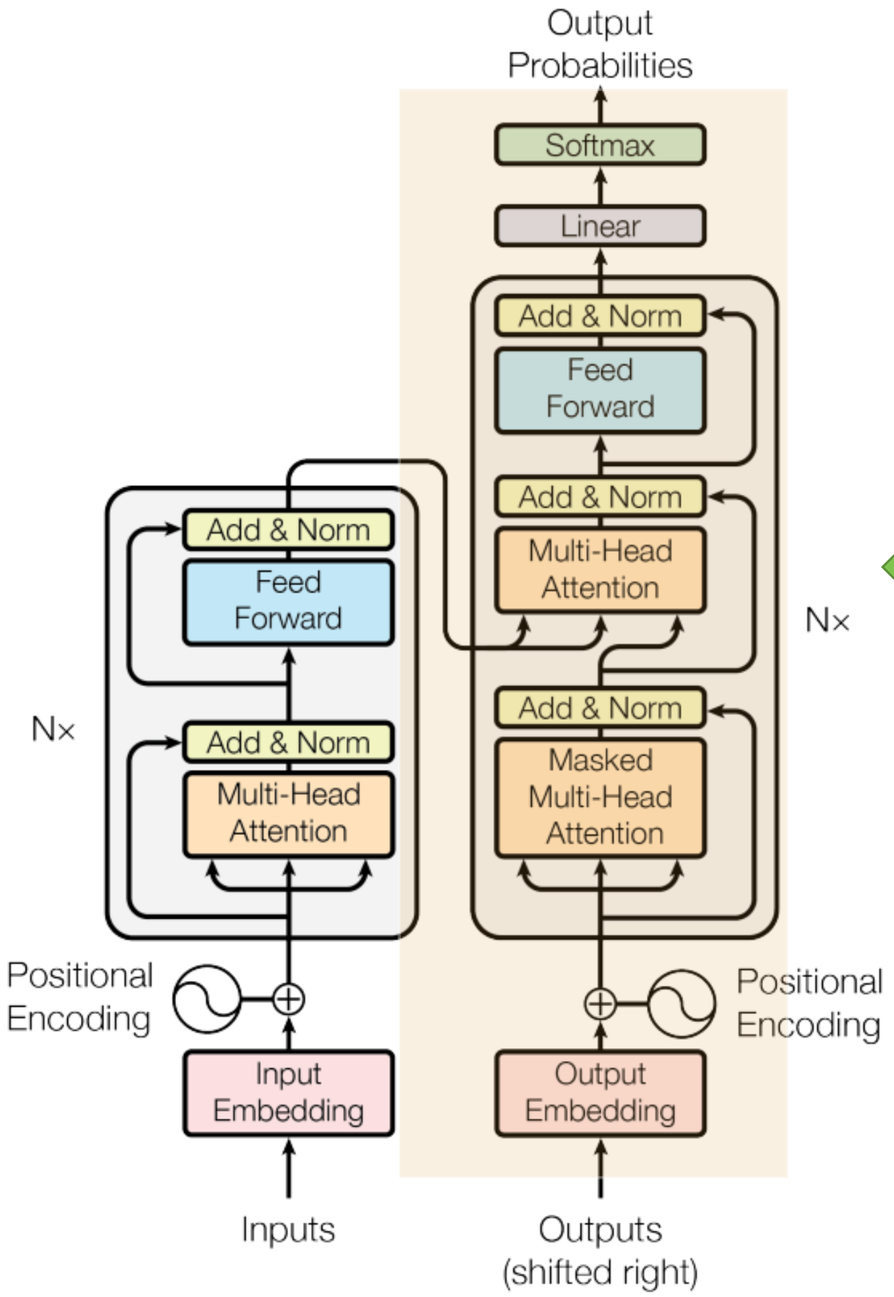
Encoder


Output Probabilities
Softmax
Linear
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Add & Norm
Masked Multi-Head Attention
Nx
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Nx
Positional Encoding
Positional Encoding
Input Embedding
Output Embedding
Inputs
Outputs (shifted right)

Decoder


Output Probabilities
Softmax
Linear
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Add & Norm
Masked Multi-Head Attention
Nx
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Nx
Positional Encoding
Positional Encoding
Input Embedding
Output Embedding
Inputs
Outputs (shifted right)

Decoder



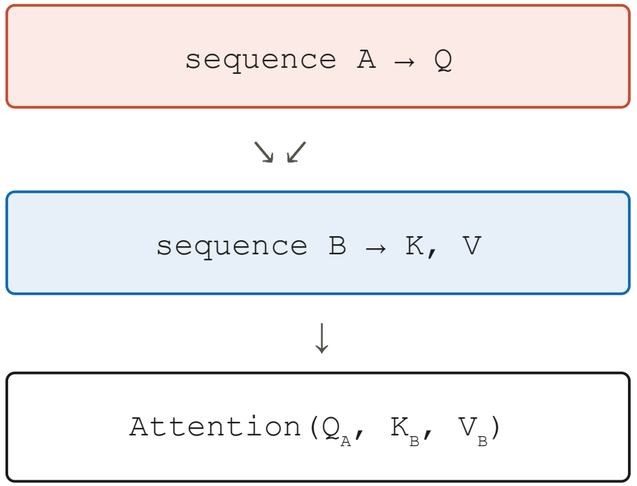
Cross-attention

Cross-attention: Letting One Sequence Query Another

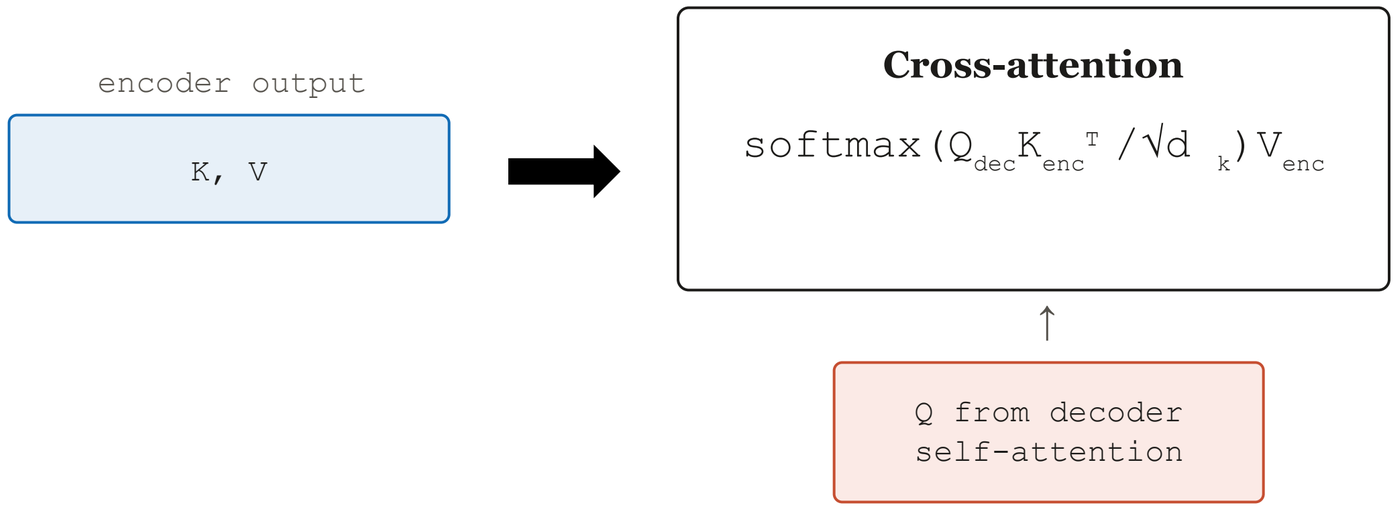
Self-attention: queries, keys, and values all come from the *same* sequence.

Cross-attention: queries come from one sequence, keys and values from another.

This is exactly what the decoder does to read the encoder's output — and the general mechanism behind any "look up information elsewhere" step.



Cross-attention Architecture



Self-attention vs. Cross-attention

	Self-attention	**Cross-attention**
Query source	same sequence	decoder sequence
Key/Value source	same sequence	a different sequence (encoder)
Sequence lengths	$n = n$	n_Q can $\neq n_{KV}$
Typical use	building context within a sequence	grounding generation in another input

Cross-attention: The equations

$$Q = X_{dec} W^{Q} \quad K = X_{enc} W^{K} \quad V = X_{enc} W^{V}$$

$$\mathrm{CrossAttn}(Q,K,V) = \mathrm{softmax}\left(QK^{T} / \sqrt{d_k} \right) V$$

If X_{dec} has n_Q tokens and X_{enc} has n_{KV} tokens, the attention weight matrix has shape $(n_Q \times n_{KV})$ — rectangular, not square.

Where Cross-attention Shows Up

Machine translation

Target-language tokens query source-language encoder states.

Image captioning

Caption tokens query image patch embeddings.

Speech recognition

Text tokens query audio frame representations.

Diffusion models

Image features query a text-prompt encoding (e.g. Stable Diffusion's U-Net).

NUMERICAL

Cross-attention Shapes

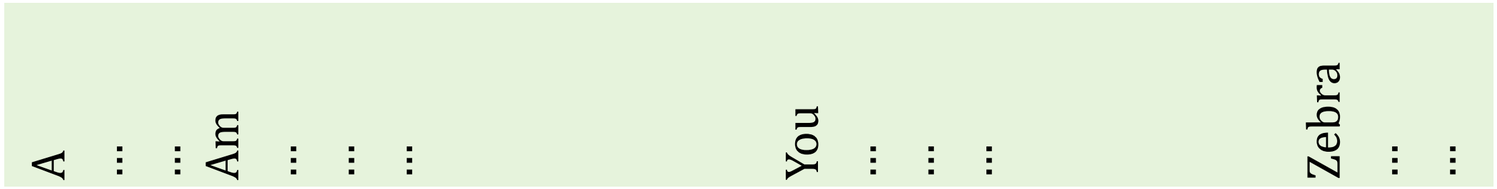
An encoder processes a 20-token source sentence. The decoder is generating token 7 of the target sentence. $d_{model} = 512$, $h = 8$ heads.

What is the shape of the cross-attention weight matrix for one head, at this decoding step, (a) if computed one query at a time, and (b) if computed for the whole 7-token decoder prefix at once?

Solution

$d_k = 512/8 = 64$ (a) one query: (1 × 20) (b) full prefix: (7 × 20) Values used: V has shape (20 × 64) output shape: (7 × 64) per head

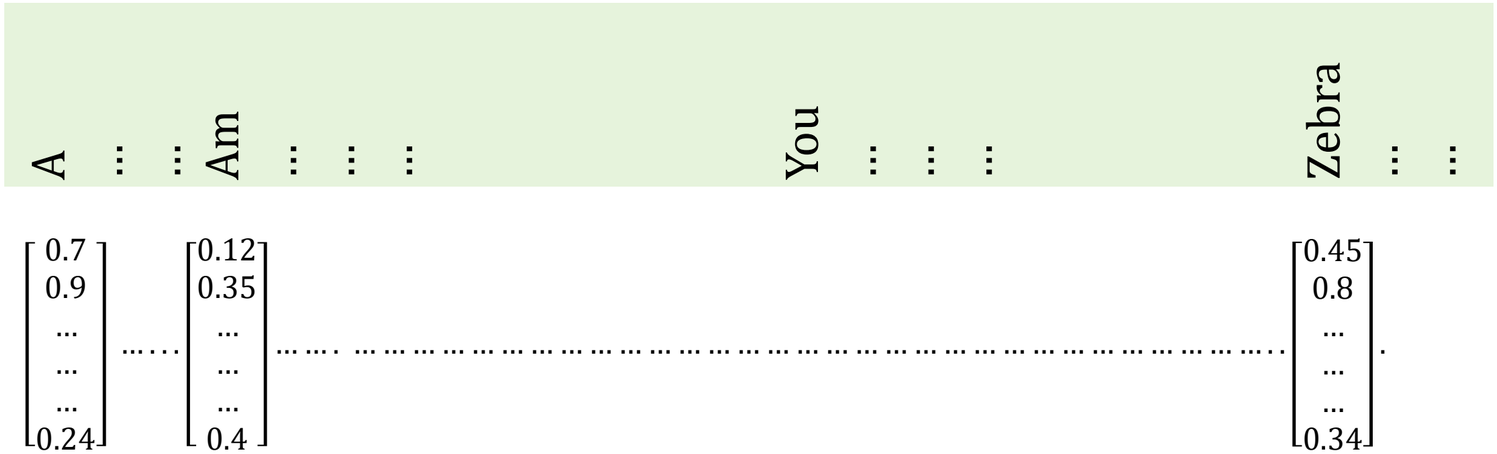
Transformer: Step by Step



The English vocabulary

(Let's say $\sim 50K$ words)

Transformer: Step by Step

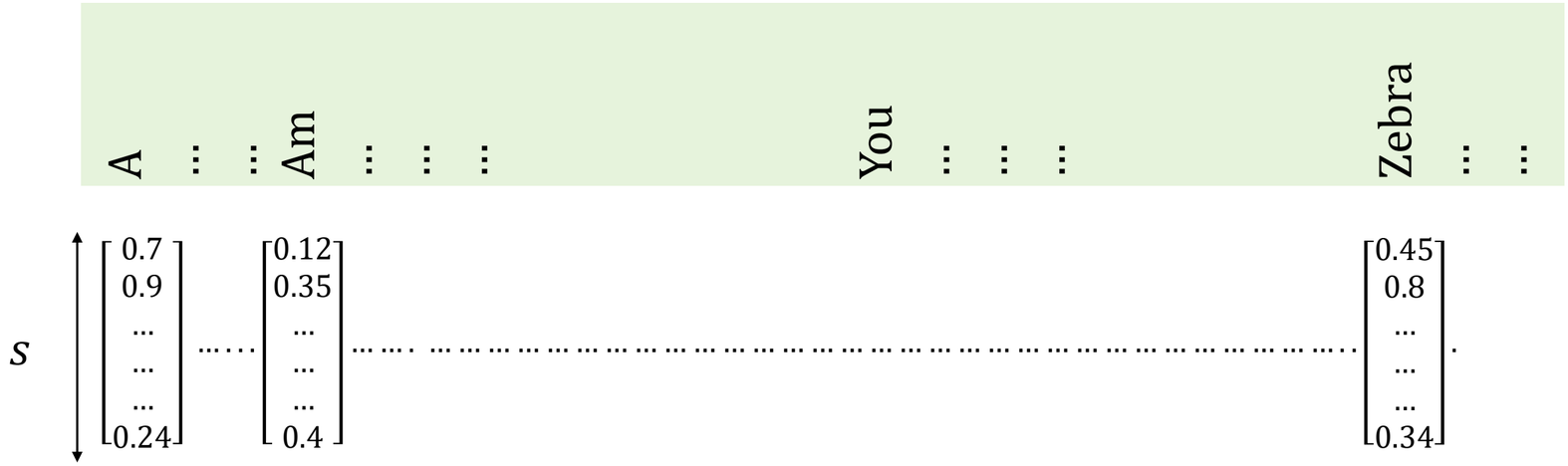


The English vocabulary (Let's say $\sim 50K$ words)

Each word converted to vector (Word embedding)

Learnt from data

Transformer: Step by Step



s: Embedding dimension

In GPT 3, the vocabulary has 50257 tokens

Embedding dimension: 12288

The English vocabulary (Let's say $\sim 50K$ words)

Each word converted to vector (Word embedding)

Learnt from data

Transformer: Step by Step

Consider the sentence translation problem. We want to translate the following sentence to Hindi

Where there is a will, there is a way

Transformer: Step by Step

Consider the sentence translation problem. We want to translate the following sentence to Hindi

Where there is a will, there is a way

Assume that the Transformer can take at most 20 token-long sentences as input.

Transformer: Step by Step

Where there is a will, there is a way

Assume that the Transformer can take at most 20 token-long sentences as input.

We first divide our input sentence into tokens and pad suitable number of additional tokens so that the total number of tokens become 20

Transformer: Step by Step

Where there is a will, there is a way

Assume that the Transformer can take at most 20 token-long sentences as input.

We first divide our input sentence into tokens and pad suitable number of additional tokens so that the total number of tokens become 20

Where there is a will , there is a way

Transformer: Step by Step

Where there is a will, there is a way

Assume that the Transformer can take at most 20 token-long sentences as input.

We first divide our input sentence into tokens and pad suitable number of additional tokens so that the total number of tokens become 20

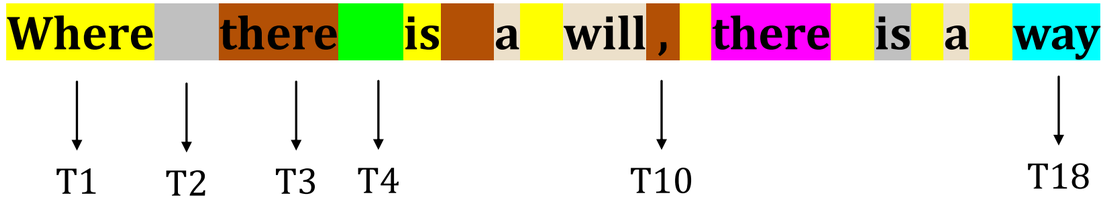
Where there is a will , there is a way

Transformer: Step by Step

Where there is a will, there is a way

Assume that the Transformer can take at most 20 token-long sentences as input.

We first divide our input sentence into tokens and pad suitable number of additional tokens so that the total number of tokens become 20

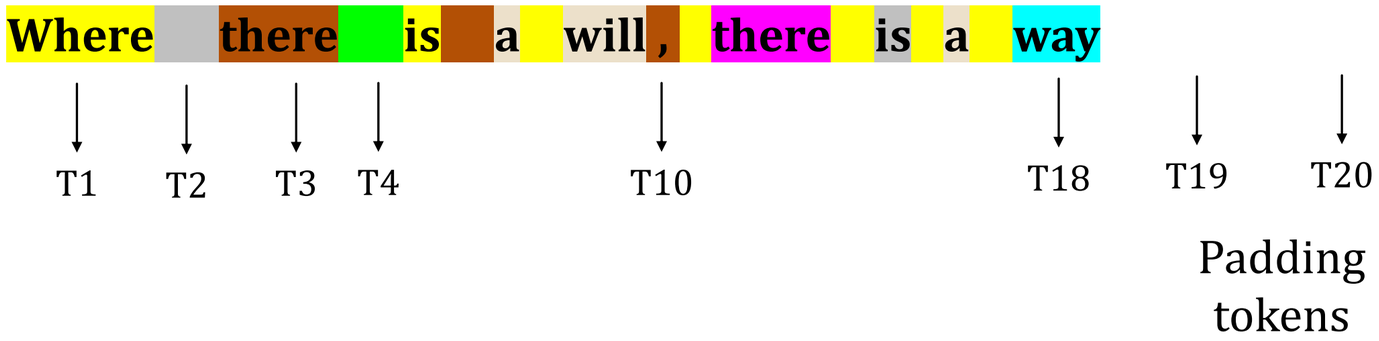


Transformer: Step by Step

Where there is a will, there is a way

Assume that the Transformer can take at most 20 token-long sentences as input.

We first divide our input sentence into tokens and pad suitable number of additional tokens so that the total number of tokens become 20



Transformer: Step by Step

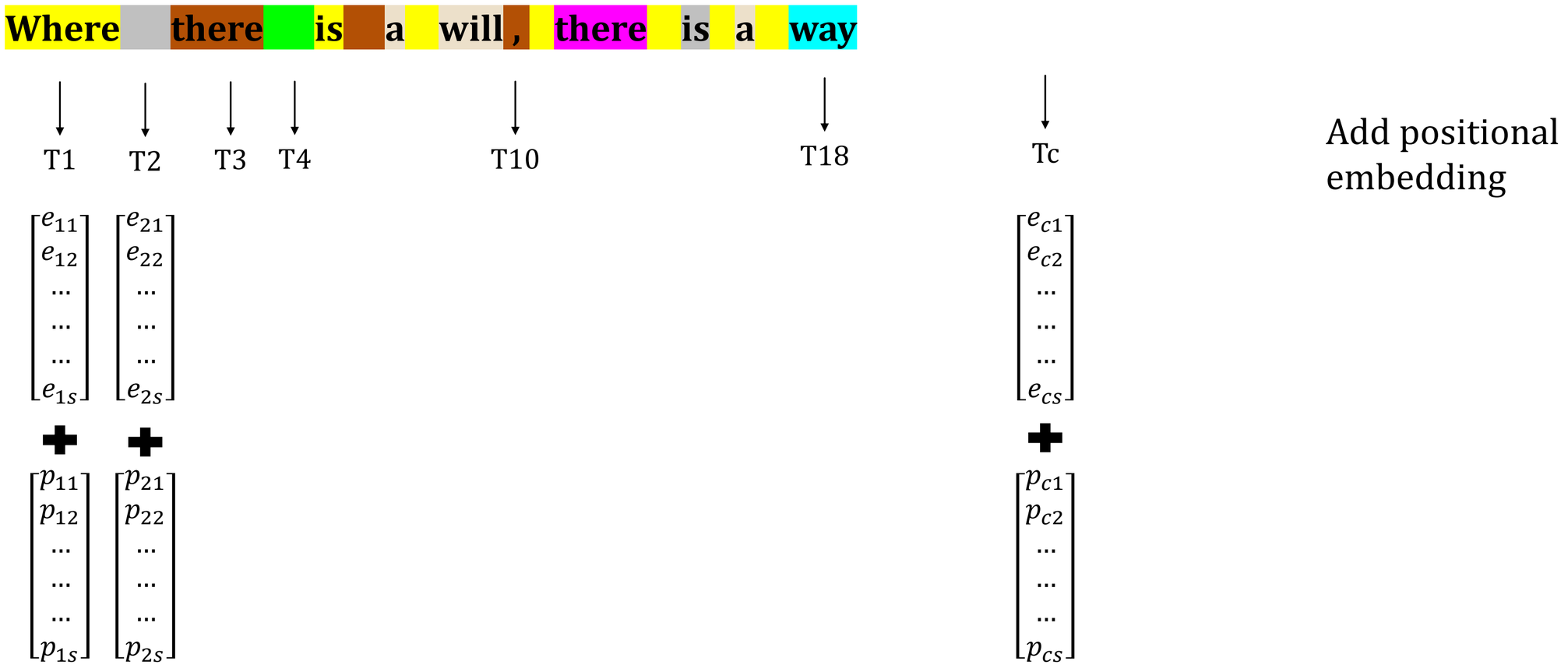


T1 T2 T3 T4 T10 T18 Tc

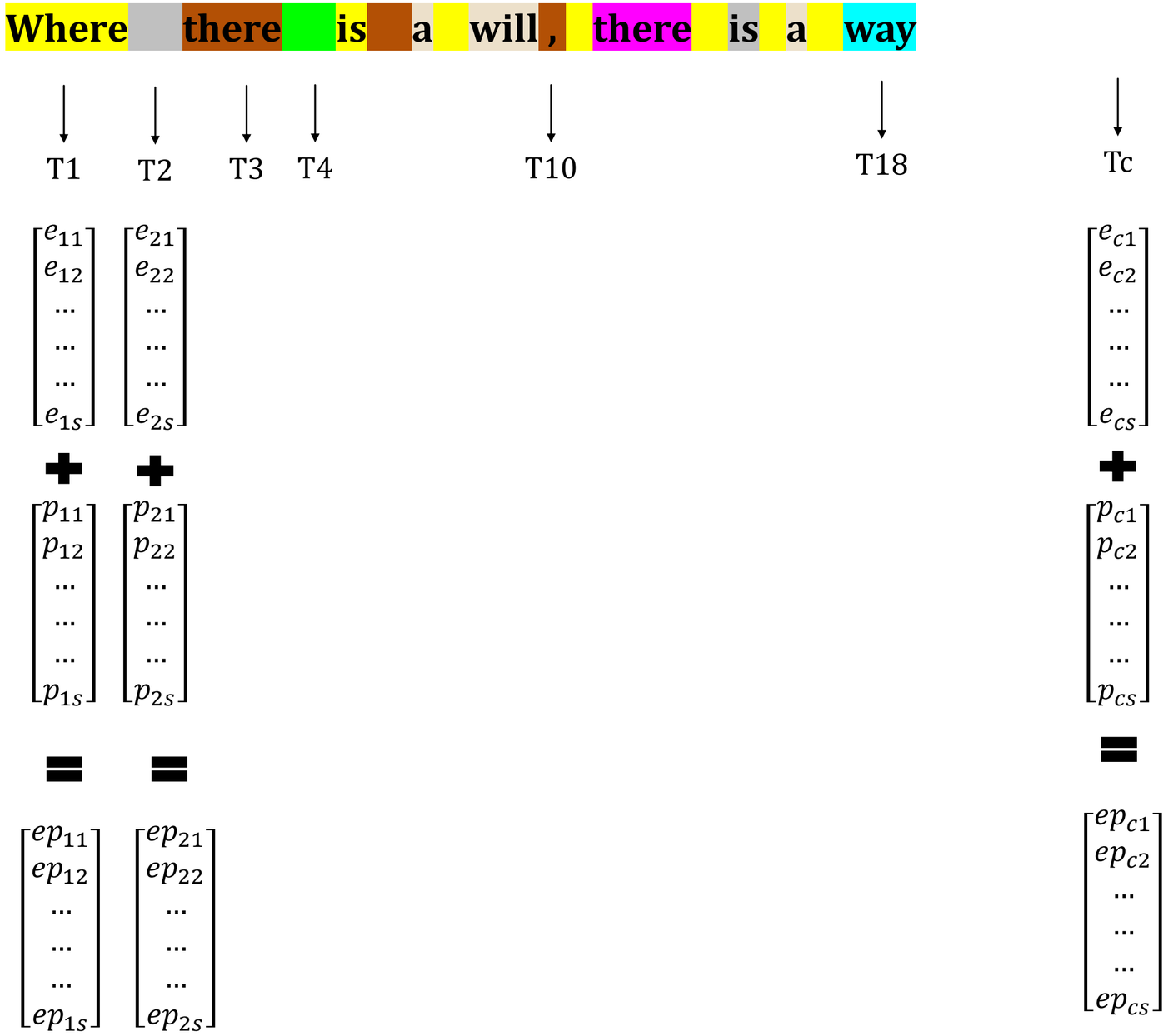
$$\begin{bmatrix} e_{11} \\ e_{12} \\ \dots \\ \dots \\ \dots \\ e_{1s} \end{bmatrix} \quad \begin{bmatrix} e_{21} \\ e_{22} \\ \dots \\ \dots \\ \dots \\ e_{2s} \end{bmatrix} \quad \begin{bmatrix} e_{c1} \\ e_{c2} \\ \dots \\ \dots \\ \dots \\ e_{cs} \end{bmatrix}$$

c is the max number of tokens

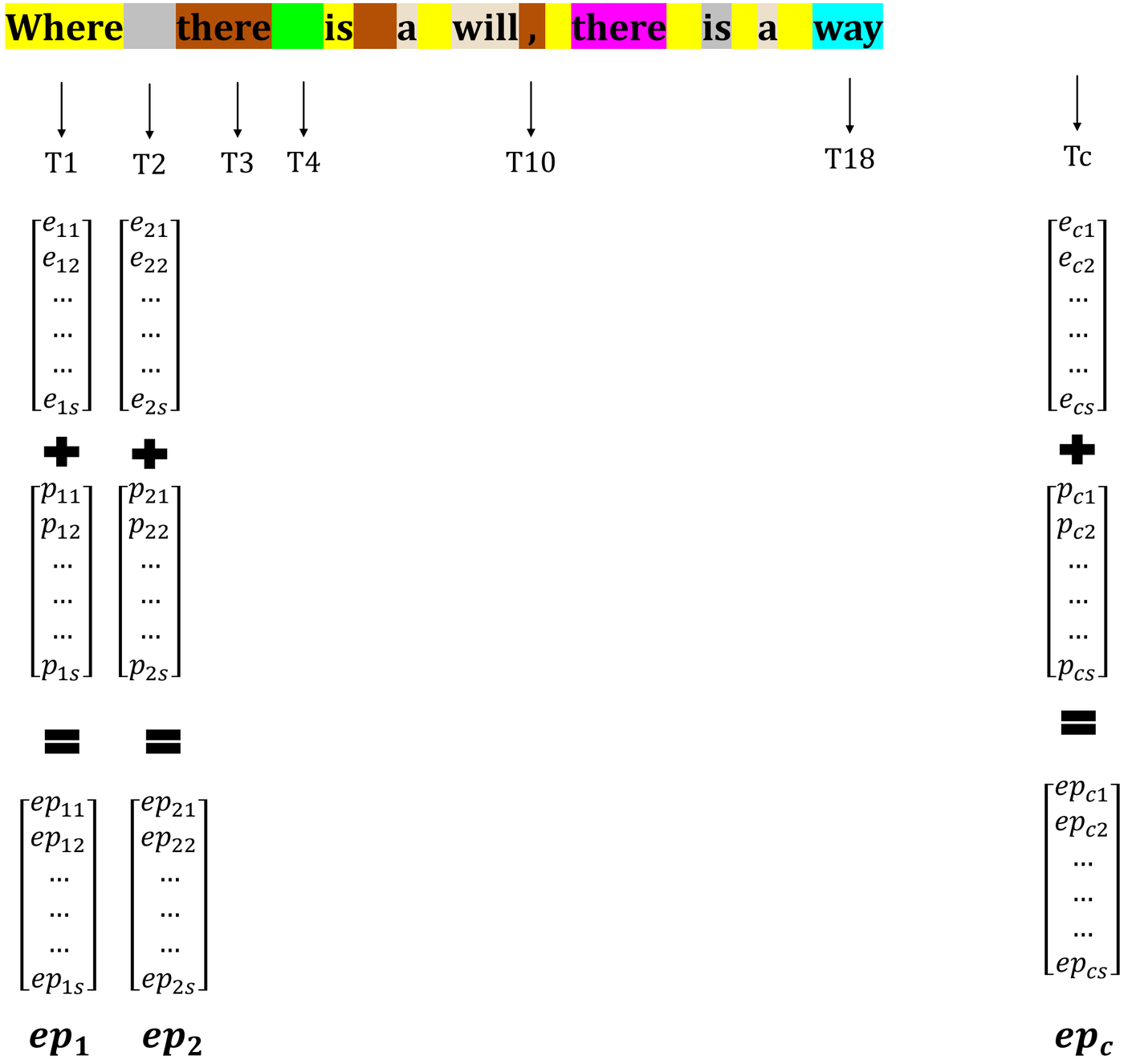
Transformer: Step by Step



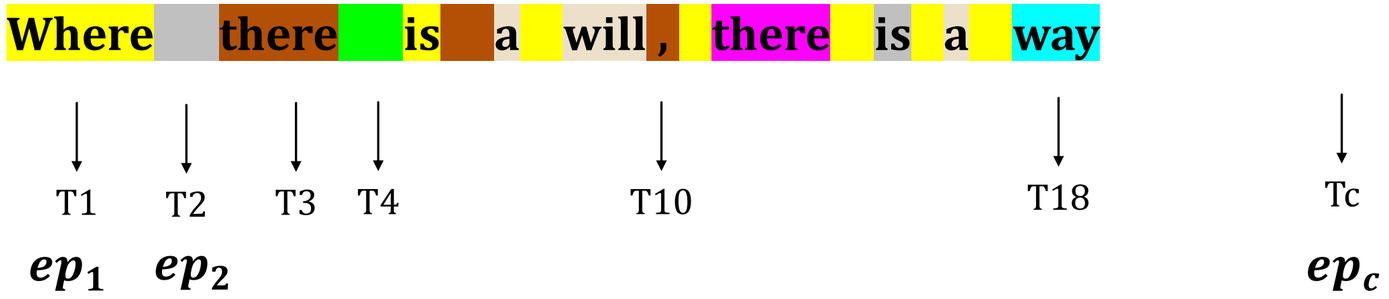
Add positional embedding



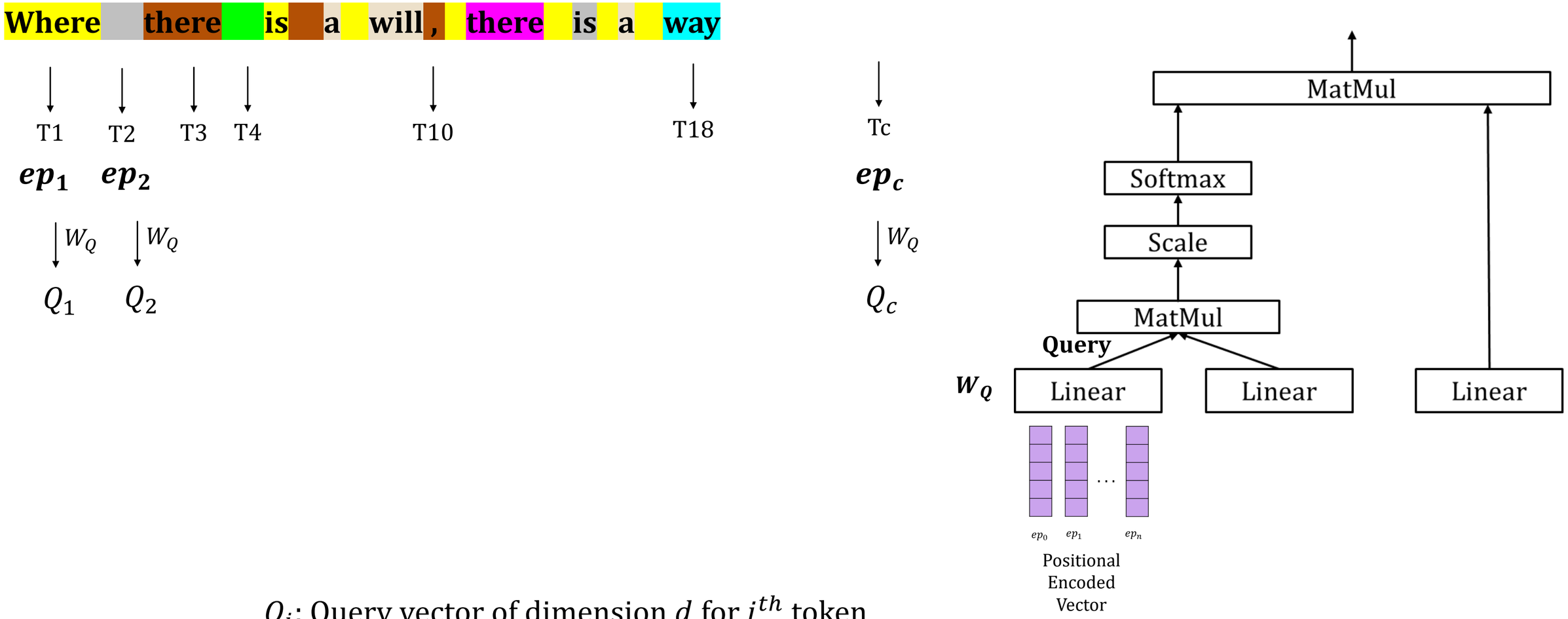
Transformer: Step by Step



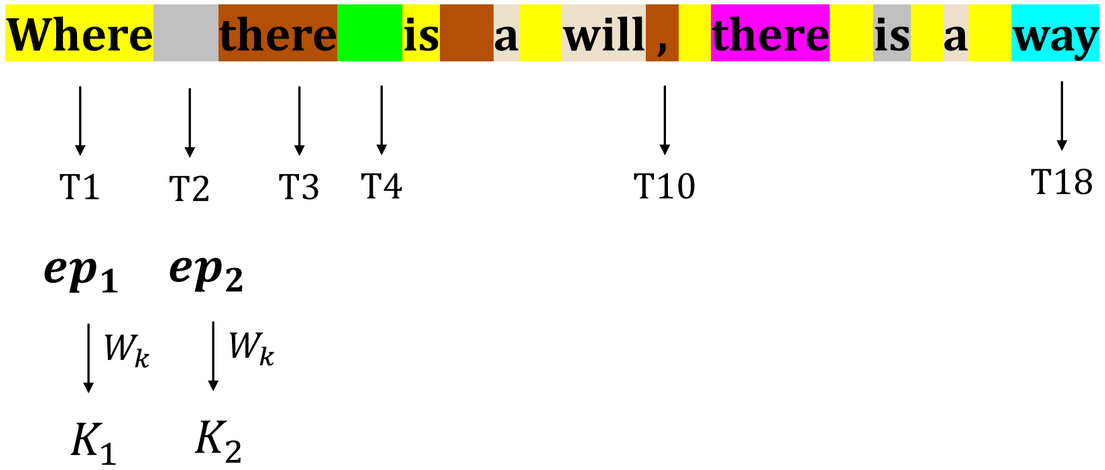
Transformer: Step by Step

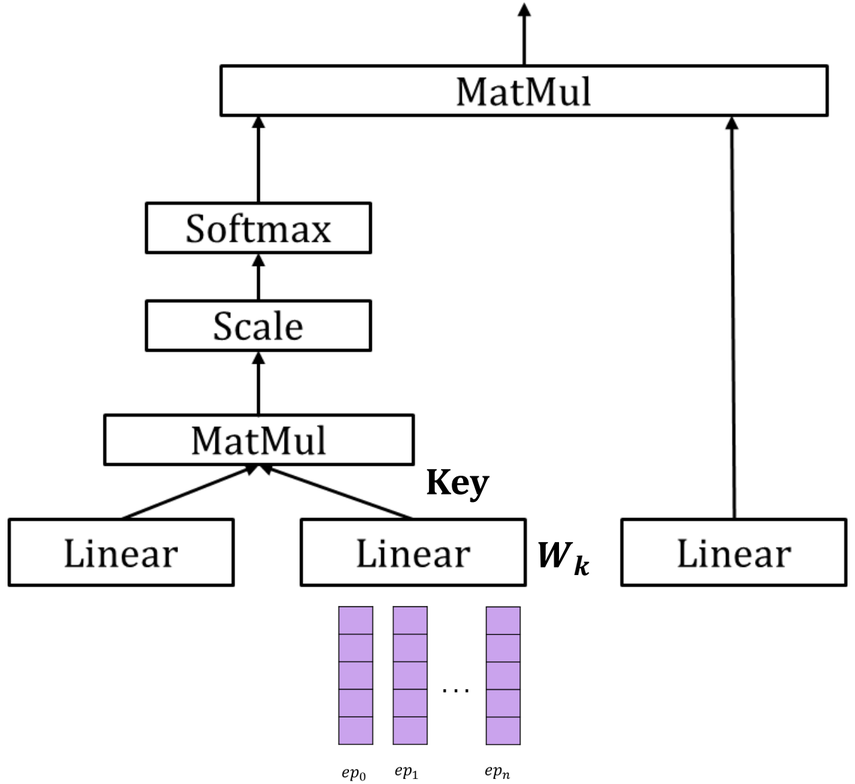


Transformer: Step by Step

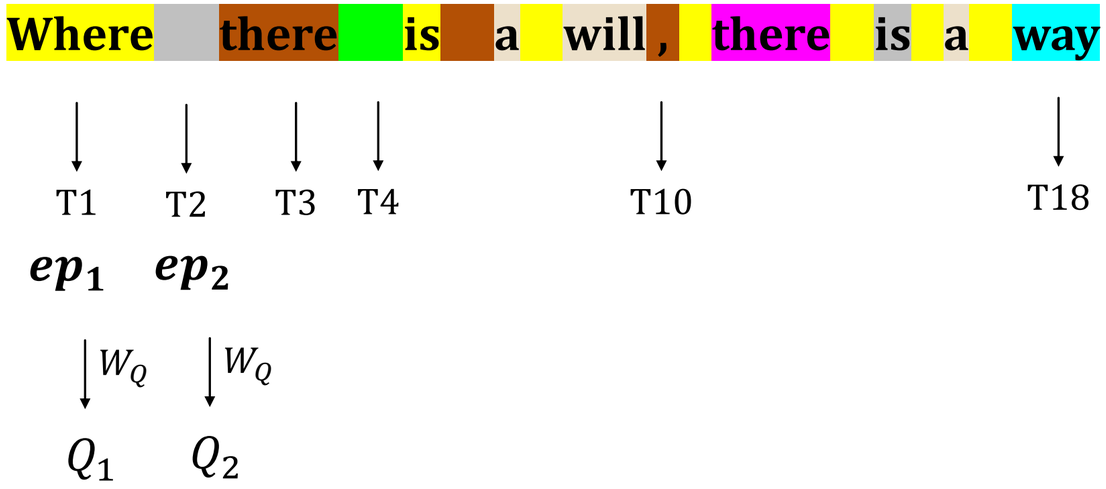


Q_i: Query vector of dimension d for i^{th} token

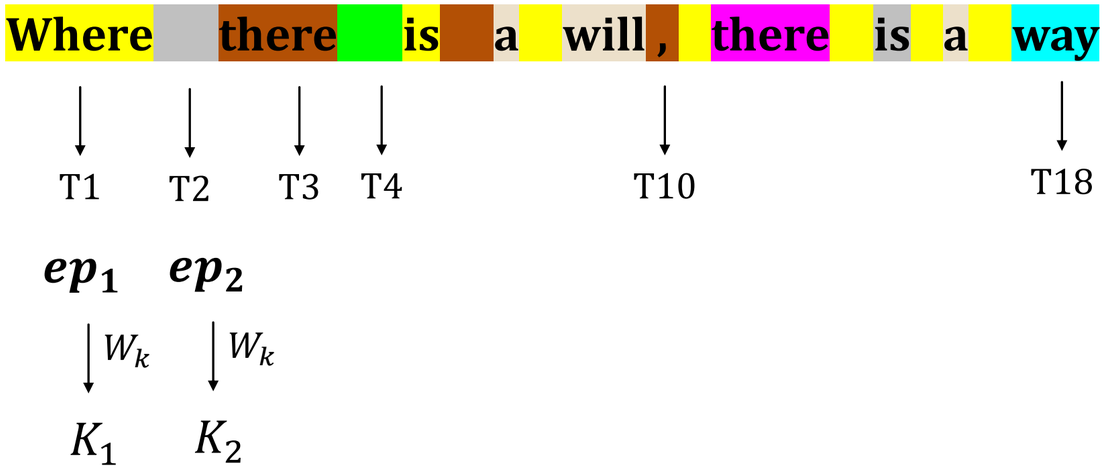




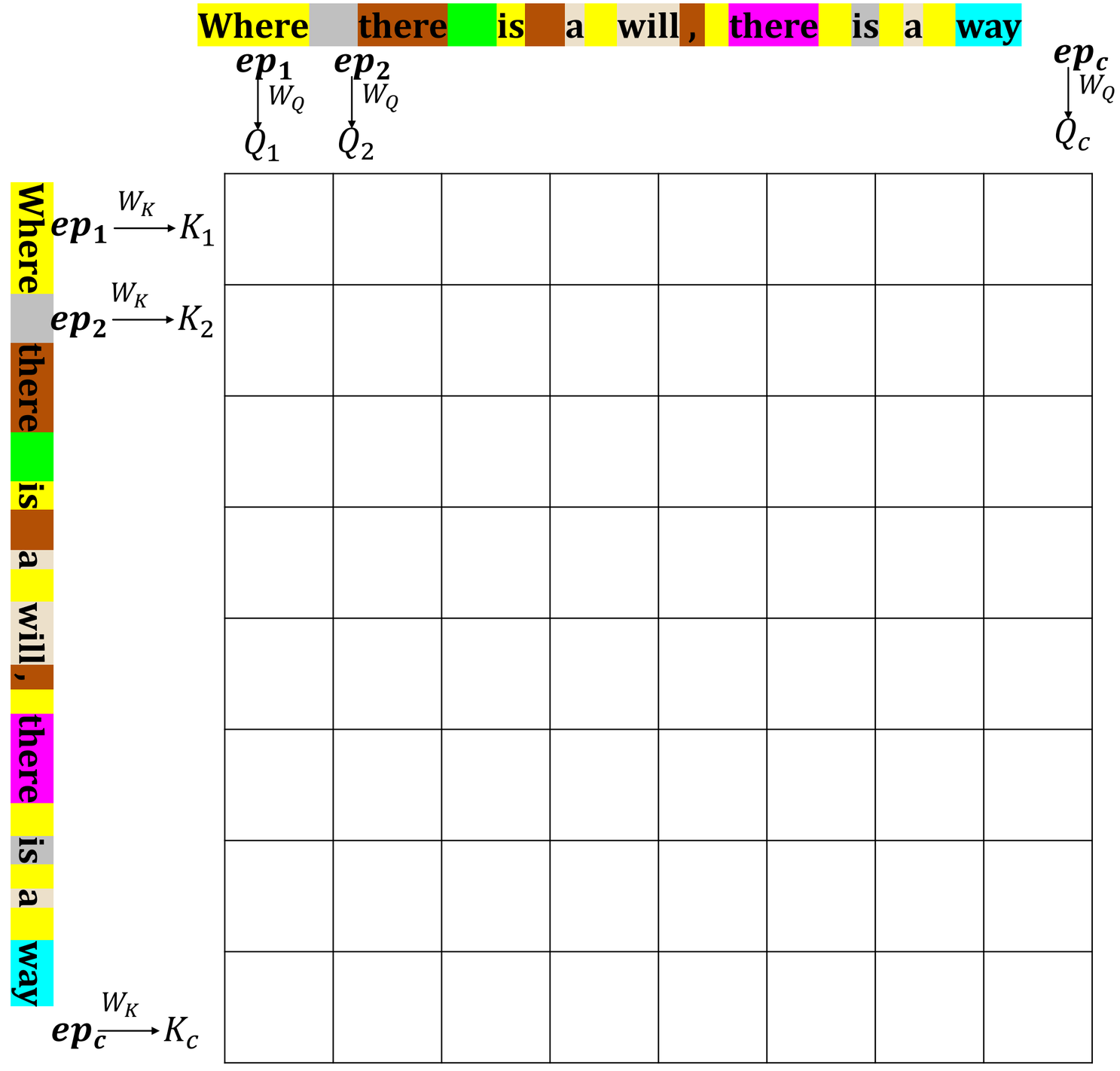
K_i: Key vector of dimension d for i^{th} token

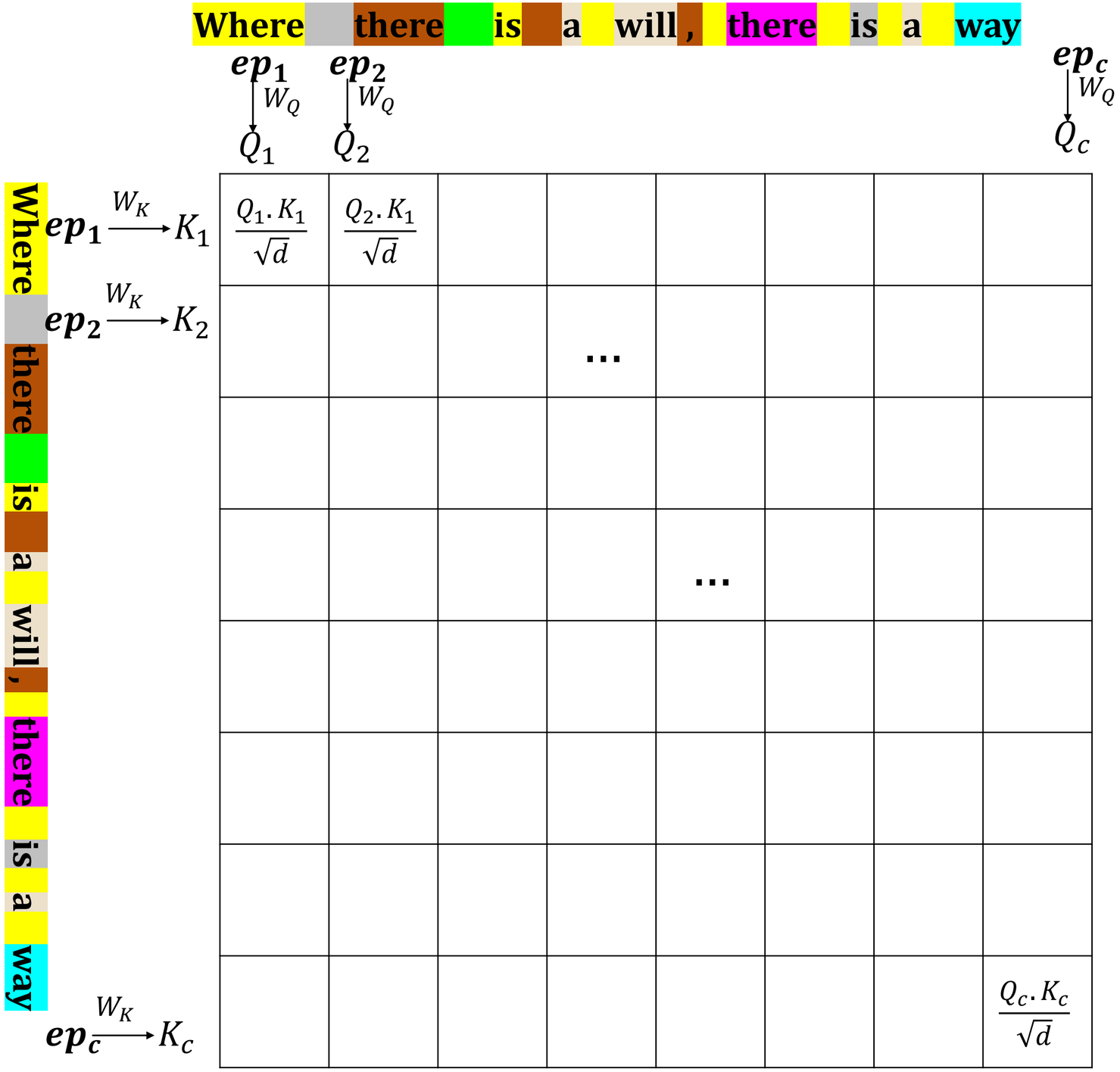


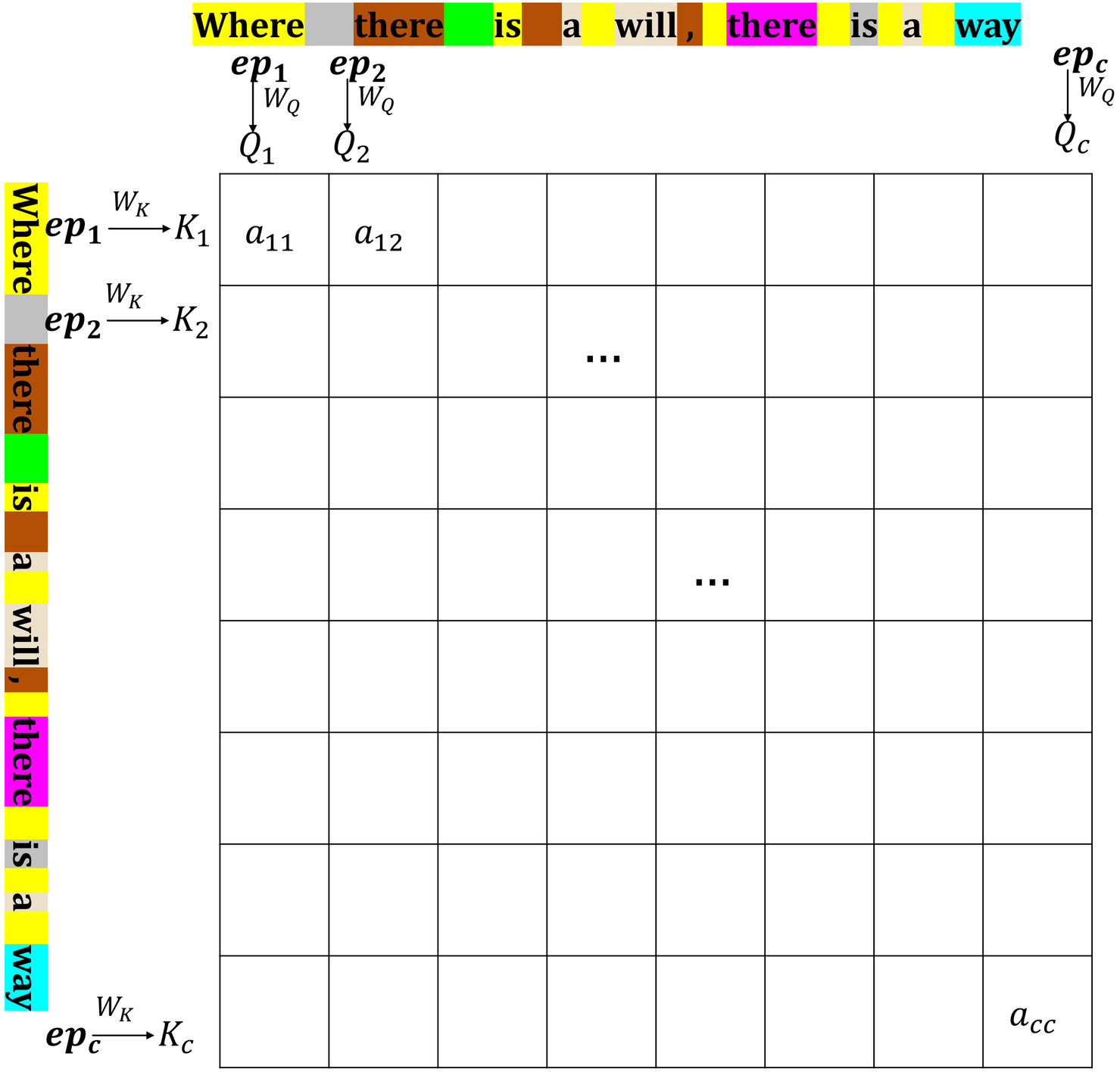
Q_i: Query vector of dimension d for i^{th} token



K_i: Key vector of dimension d for i^{th} token

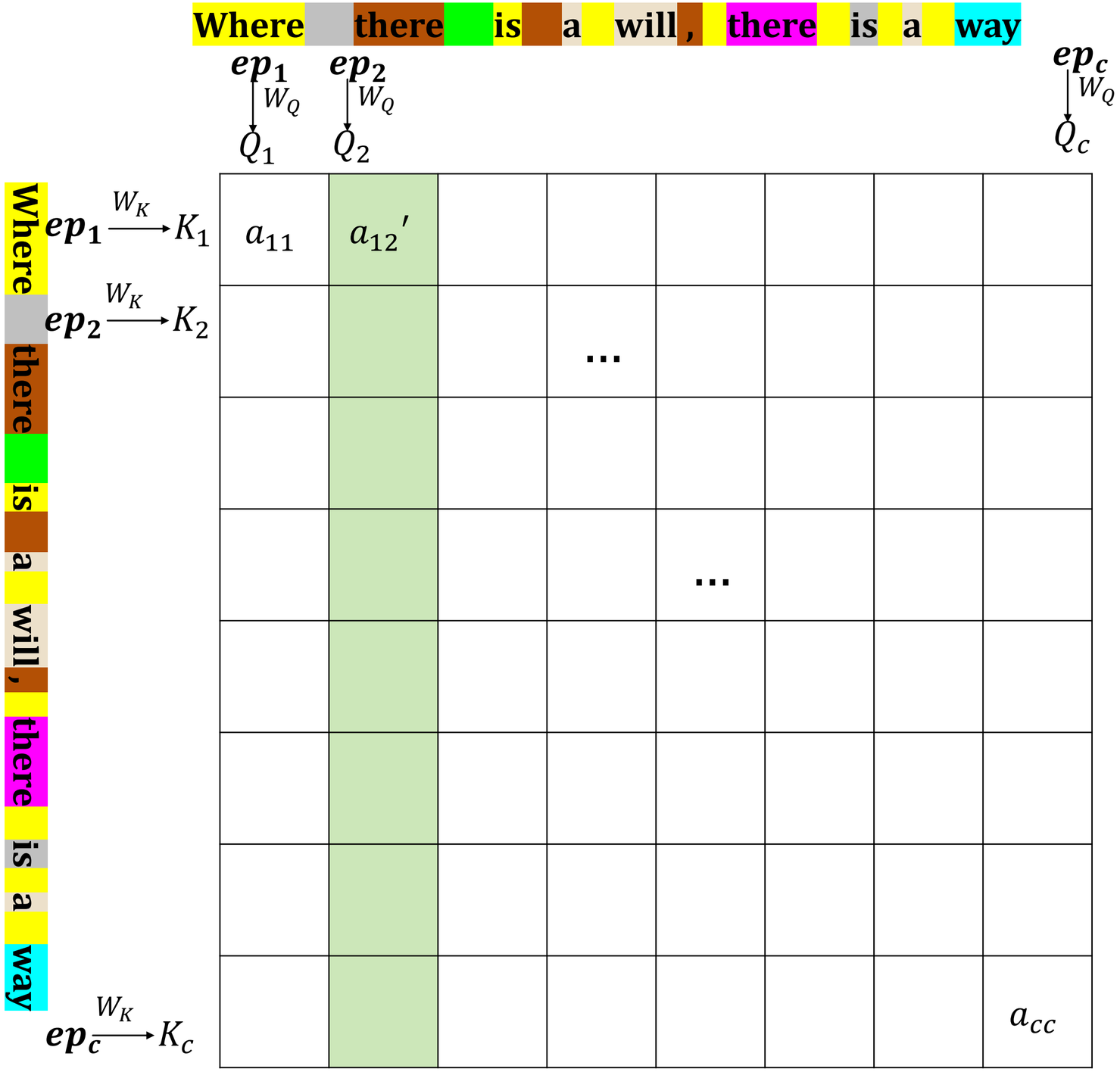

Where there is a will, there is a way
ep_1
ep_2
ep_c
W_Q
Q_1
Q_2
Q_c
W_K
K_1
K_2
K_c


Where there is a will, there is a way
ep_1 W_Q Q_1
ep_2 W_Q Q_2
ep_c W_Q Q_c
Where ep_1 W_K K_1
there ep_2 W_K K_2
is
a
will,
there
is
a
way
ep_c W_K K_c
$\frac{Q_1 . K_1}{\sqrt{d}}$
$\frac{Q_2 . K_1}{\sqrt{d}}$
...
...
$\frac{Q_c . K_c}{\sqrt{d}}$



Attention matrix

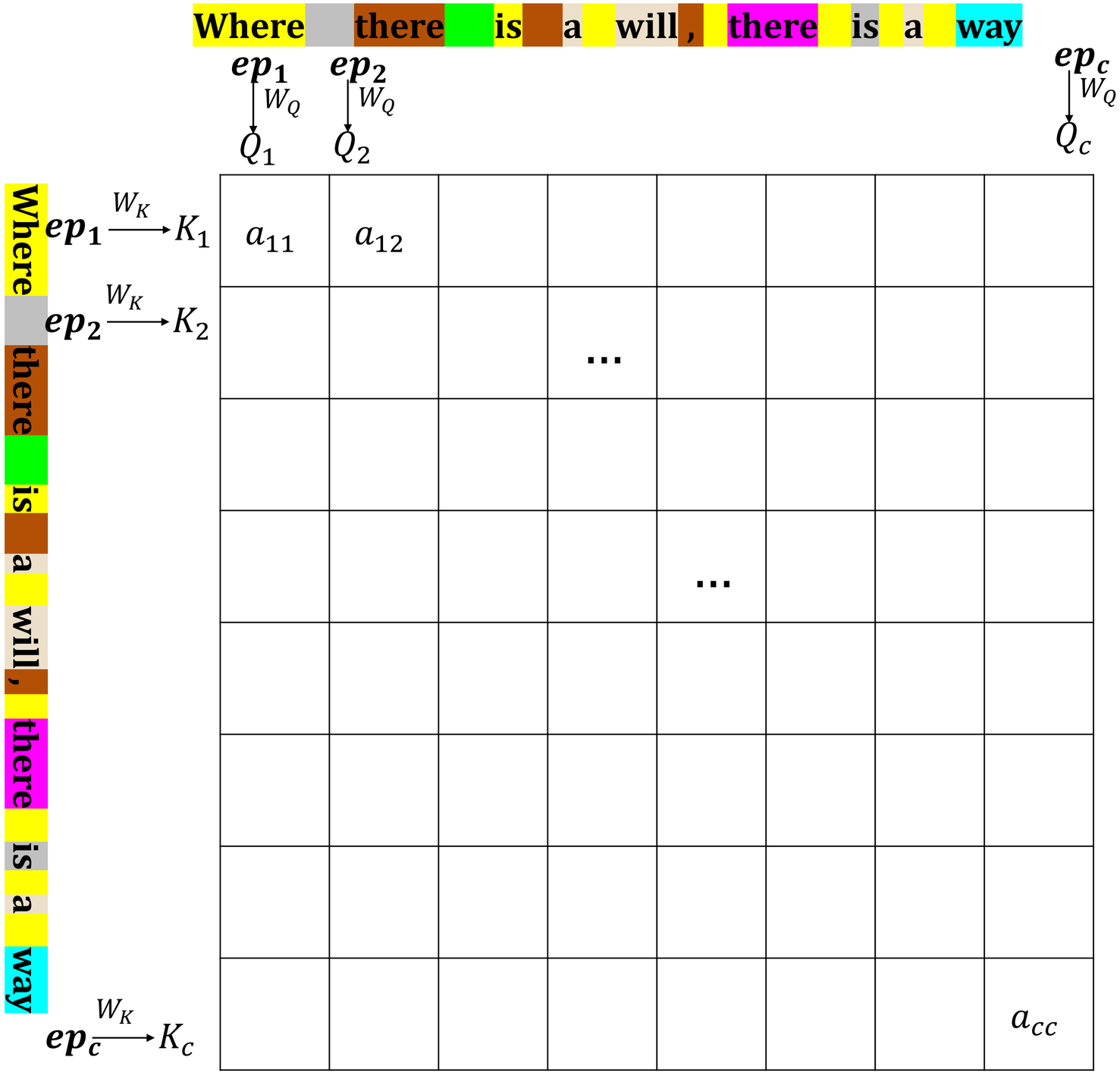
Entry (i, j) indicate some sort of correlation between the token of the ith row and the token of the jth row



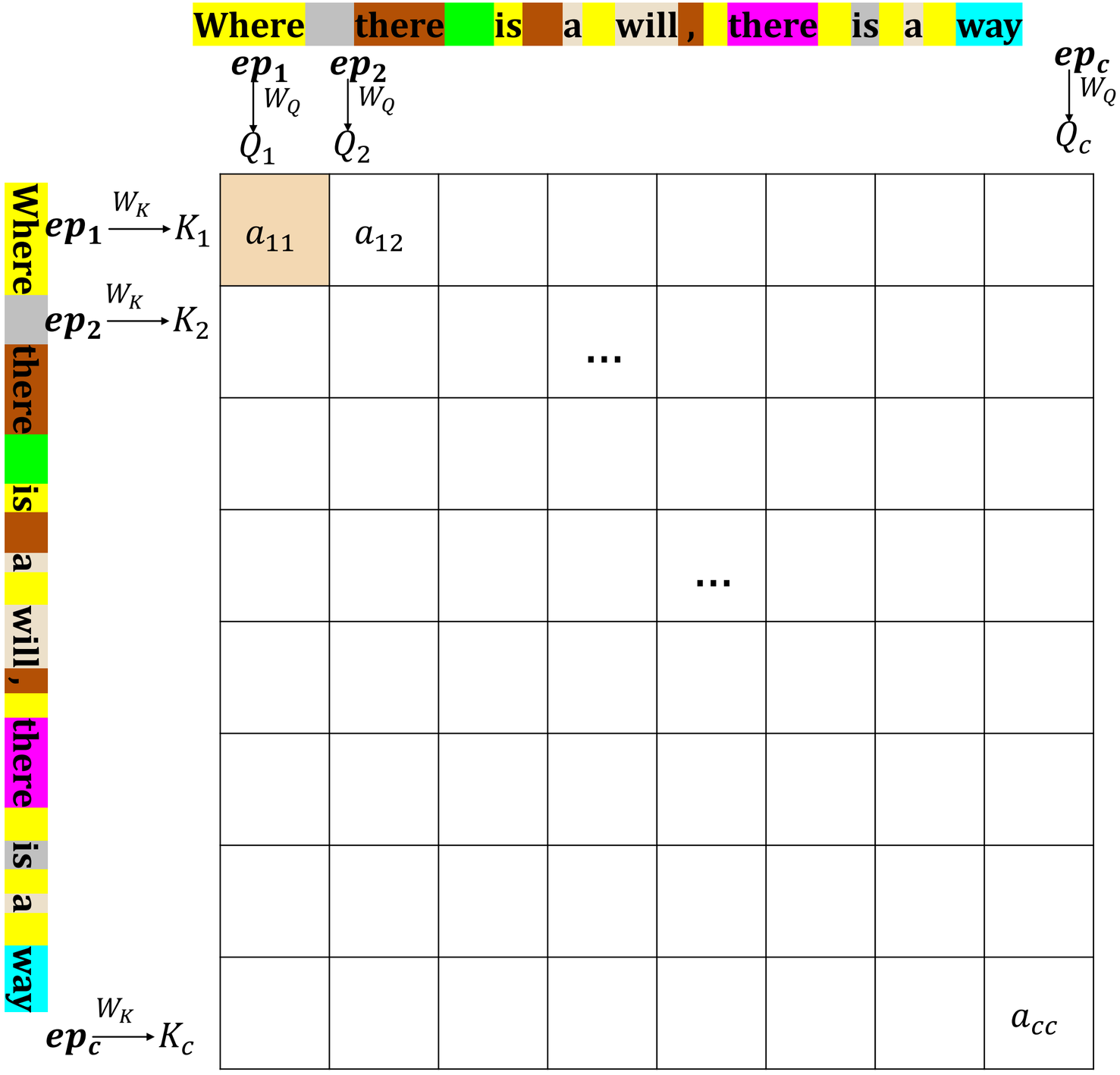
Attention matrix

For each query column, we apply softmax. As a result, each column kind of indicates the probabilities of the corresponding query to match with different keys

a'_{12}: Value of the entry after softmax

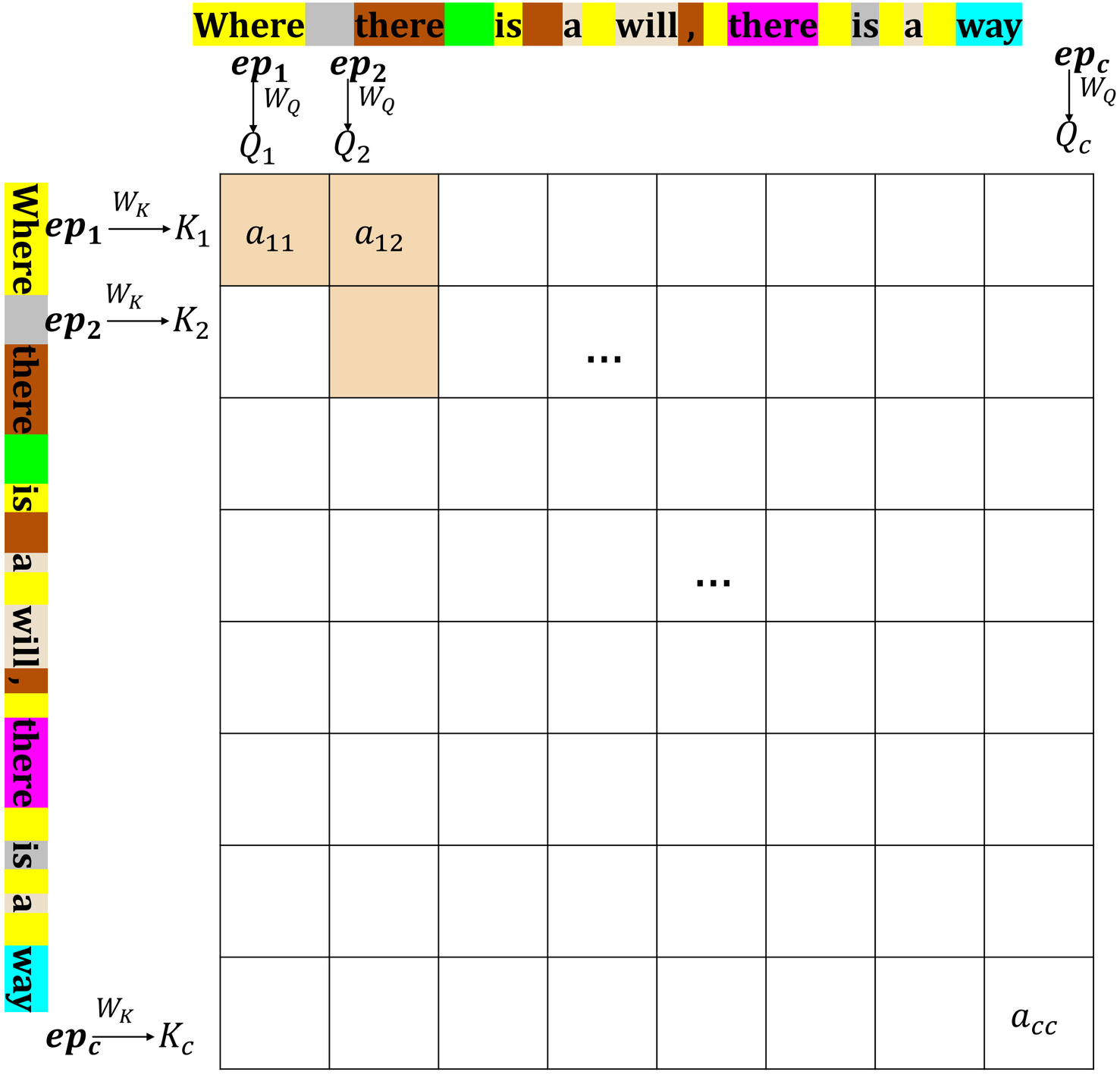


We may want that a query (a token) is influenced by only past keys (past tokens)



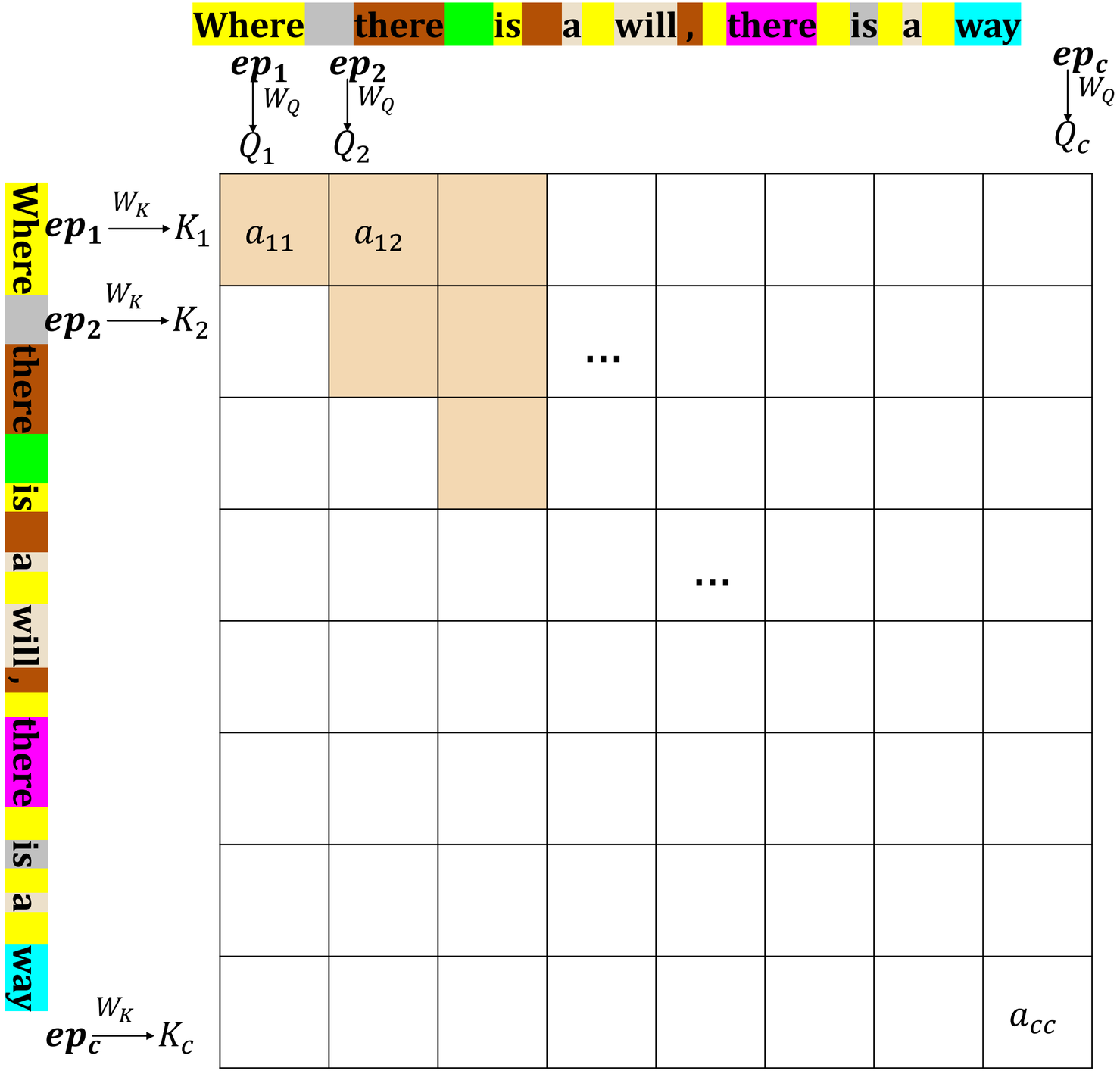
We may want that a query (a token) is influenced by only past keys (past tokens)

Q_1 is influenced by only K_1



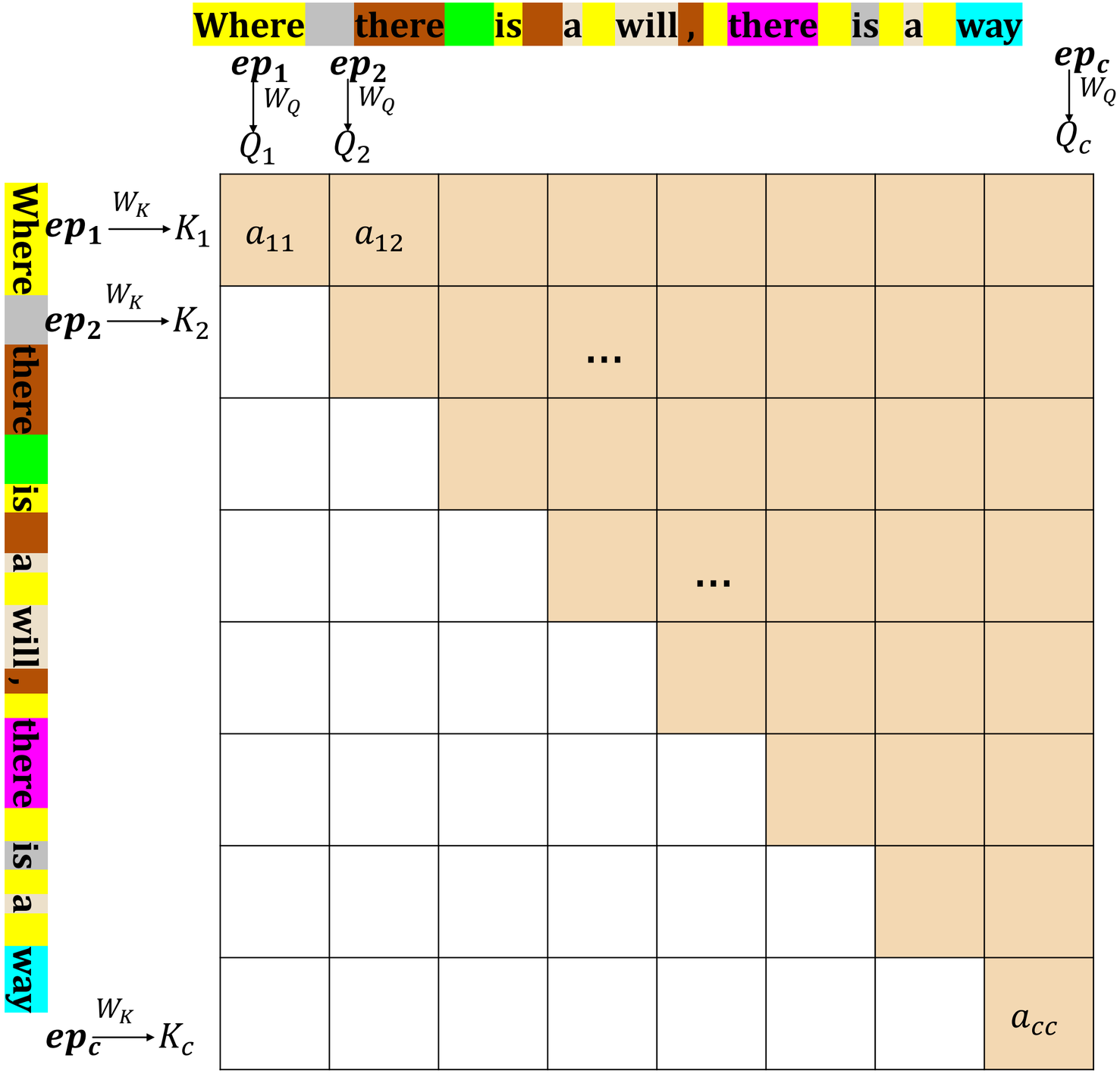
We may want that a query (a token) is influenced by only past keys (past tokens)

Q_1 is influenced by only K_1
Q_2 is influenced by only K_1, K_2



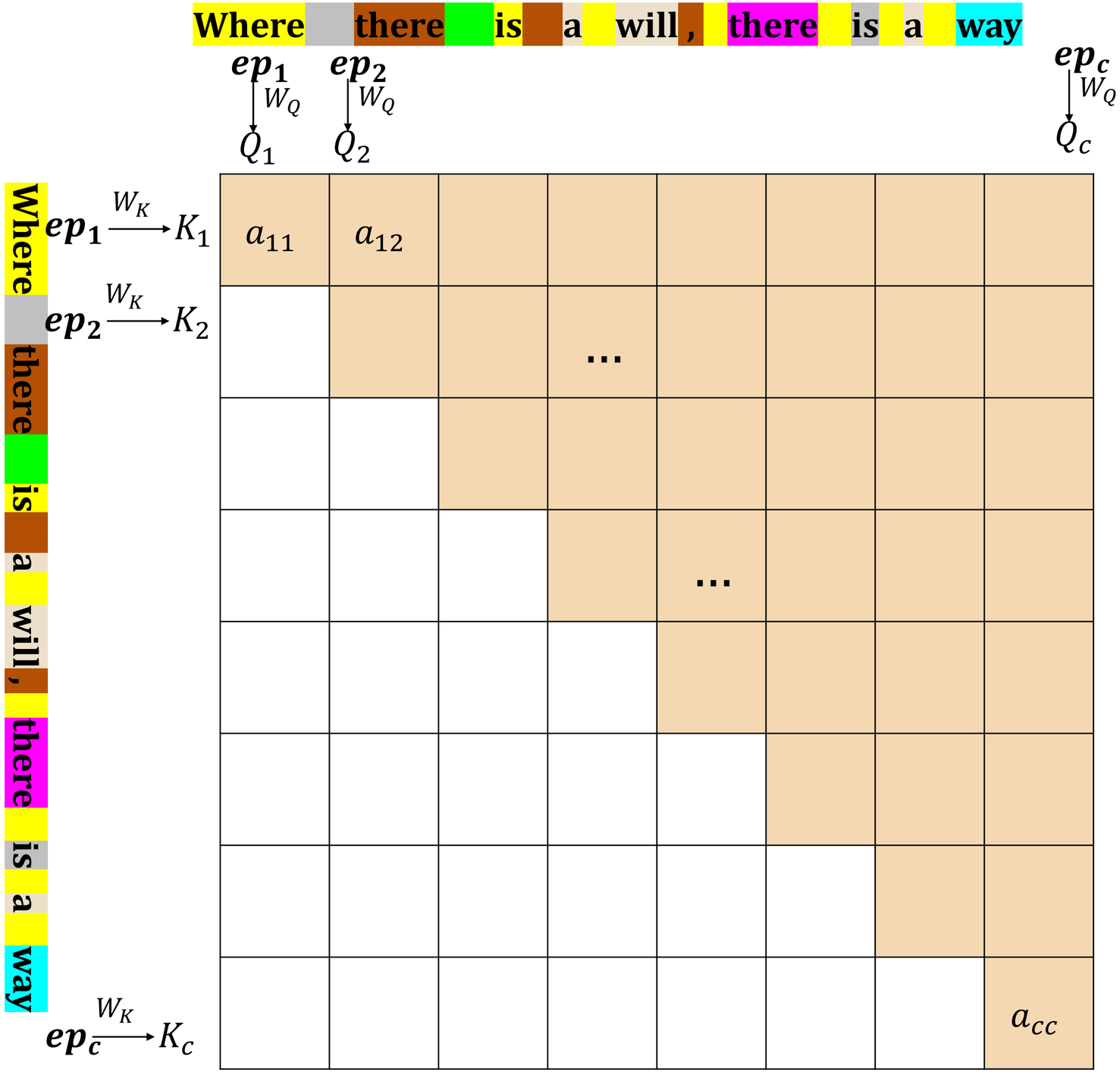
We may want that a query (a token) is influenced by only past keys (past tokens)

Q_1 is influenced by only K_1
Q_2 is influenced by only K_1, K_2
Q_3 is influenced by only K_1, K_2, K_3, and so on



We may want that a query (a token) is influenced by only past keys (past tokens)

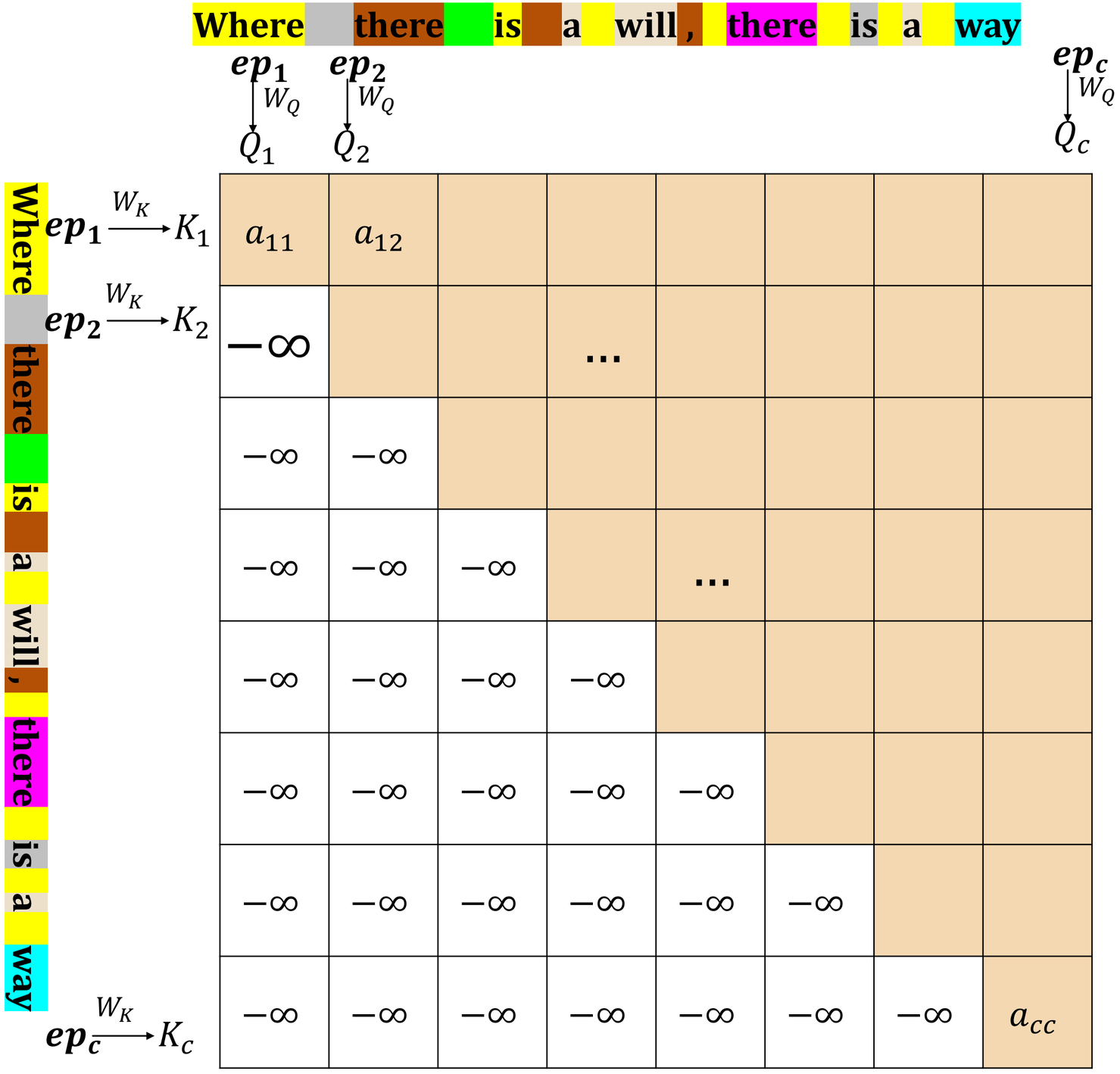
Q_1 is influenced by only K_1
Q_2 is influenced by only K_1, K_2
Q_3 is influenced by only K_1, K_2, K_3, and so on



We may want that a query (a token) is influenced by only past keys (past tokens)

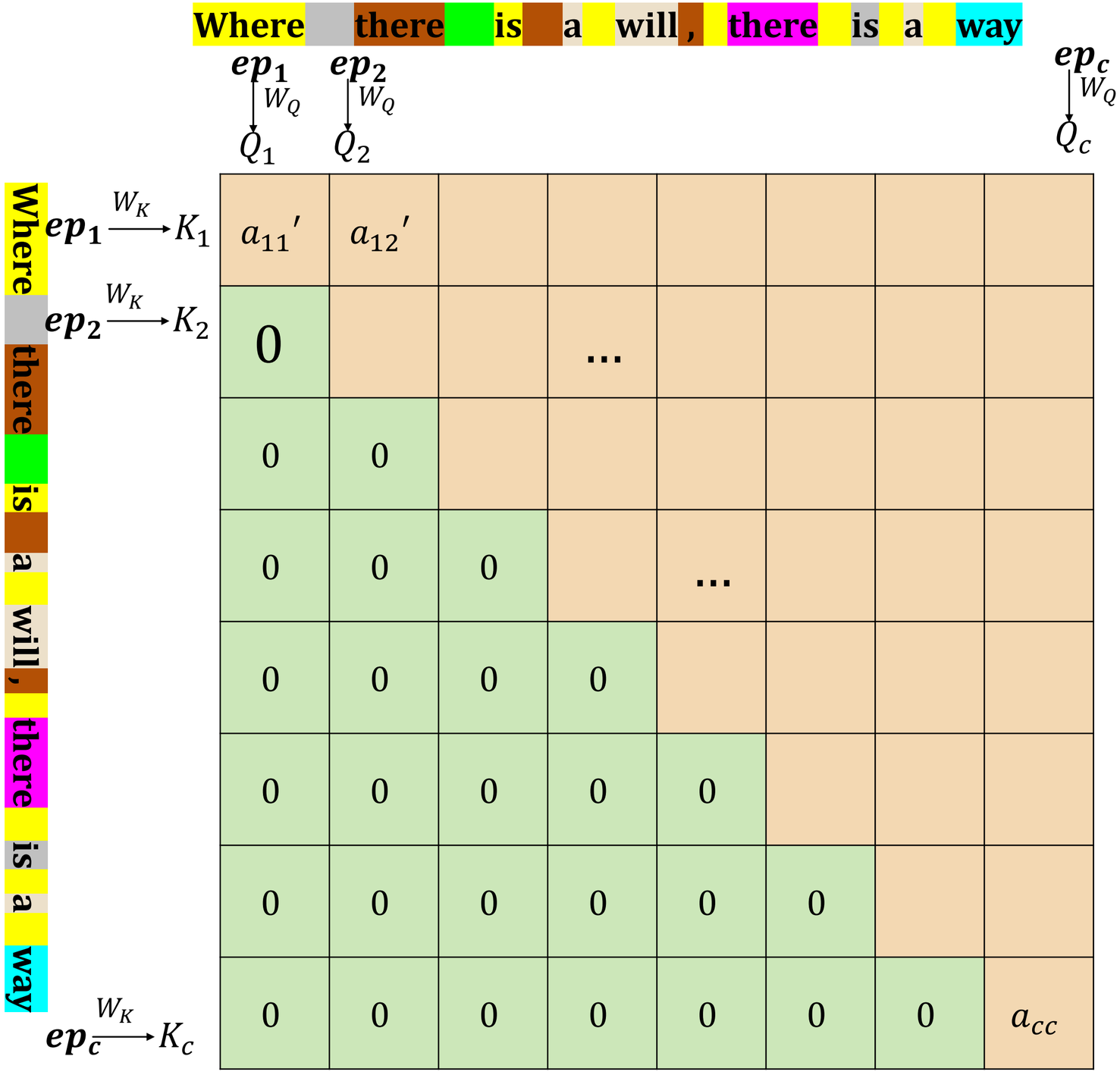
Q_1 is influenced by only K_1
Q_2 is influenced by only K_1, K_2
Q_3 is influenced by only K_1, K_2, K_3, and so on

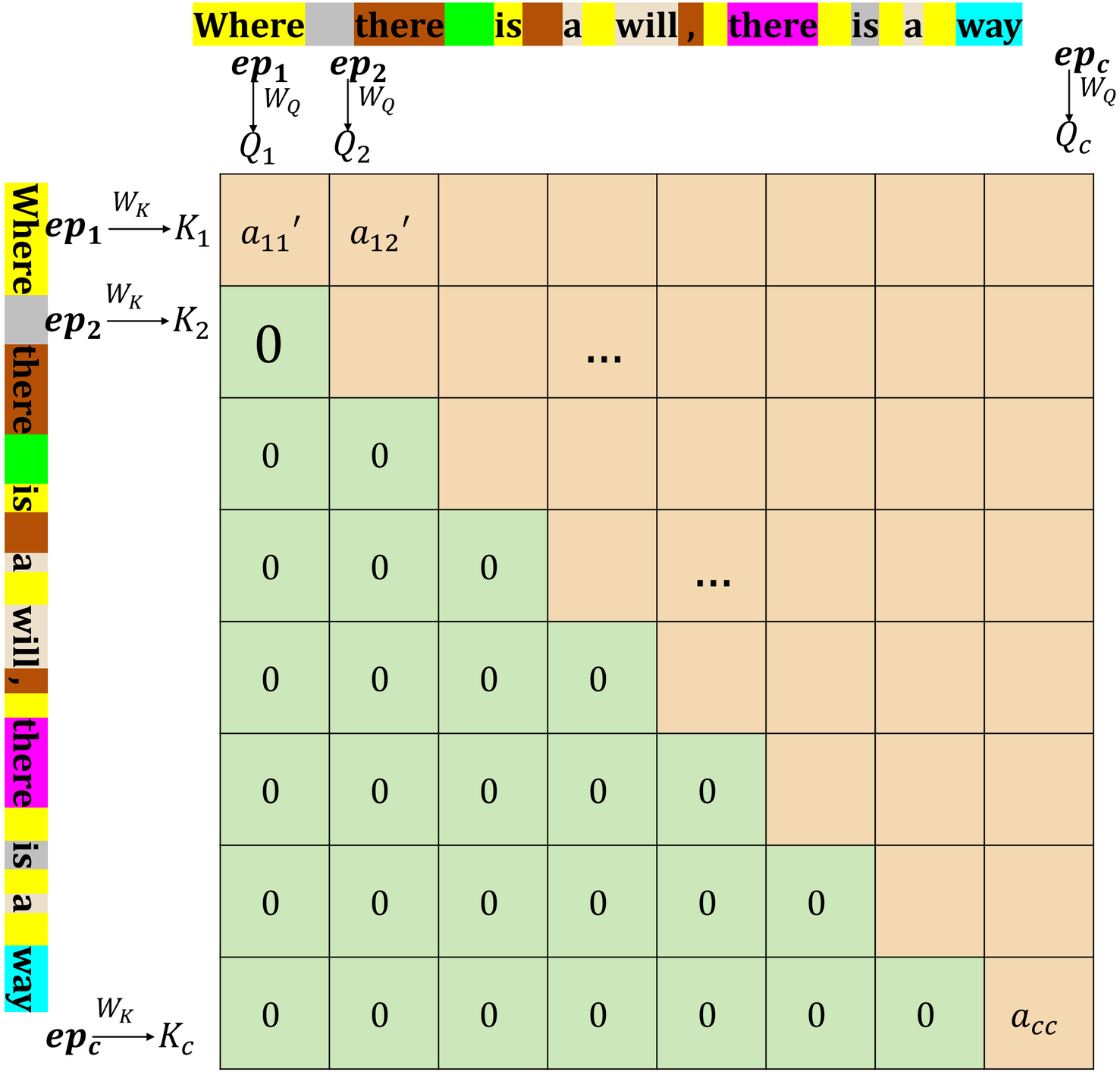
This is equivalent to masking the white region



This is equivalent to masking the white region

For that, we make the entries at the white boxes $-\infty$


Where there is a will, there is a way
ep_1 W_Q Q_1
ep_2 W_Q Q_2
ep_c W_Q Q_c
ep_1 W_K K_1
ep_2 W_K K_2
ep_c W_K K_c
a_{11}' a_{12}'
0 ...
0 0
0 0 0 ... 0 0 0

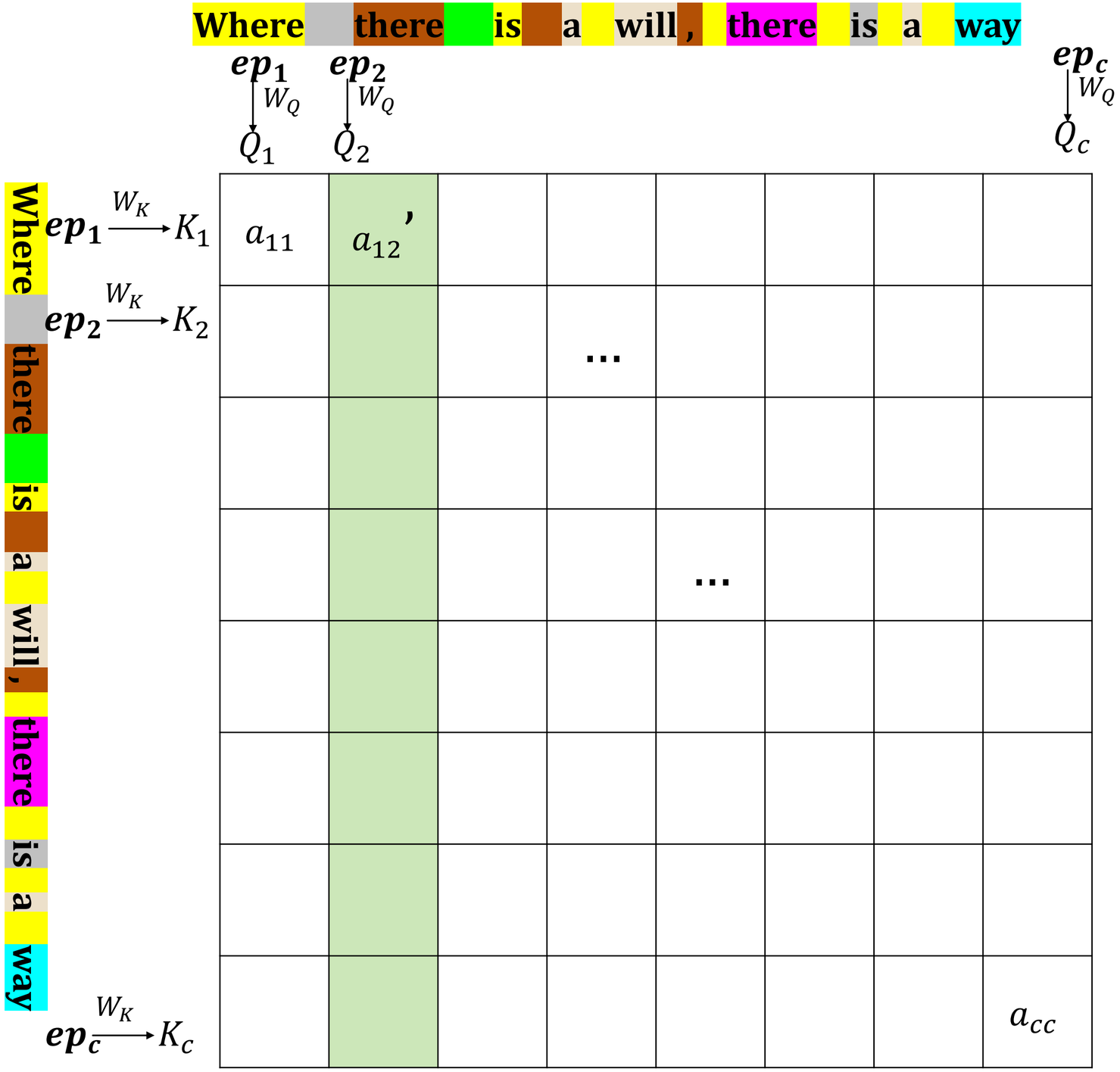


This is equivalent to masking the white region

For that, we make the entries at the white boxes $-\infty$

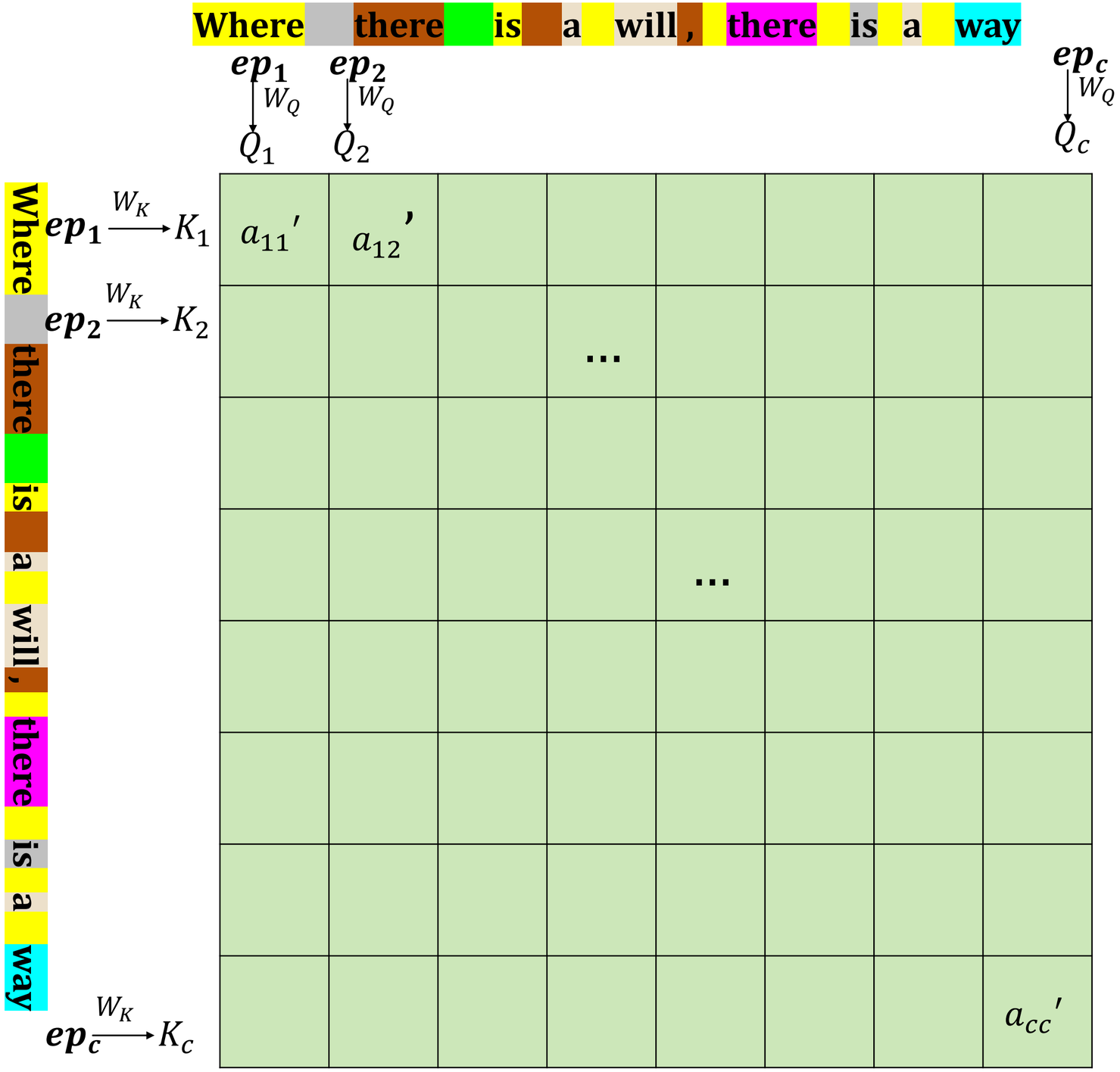
After column-wise softmax, these entries become 0

This is called
masked attention



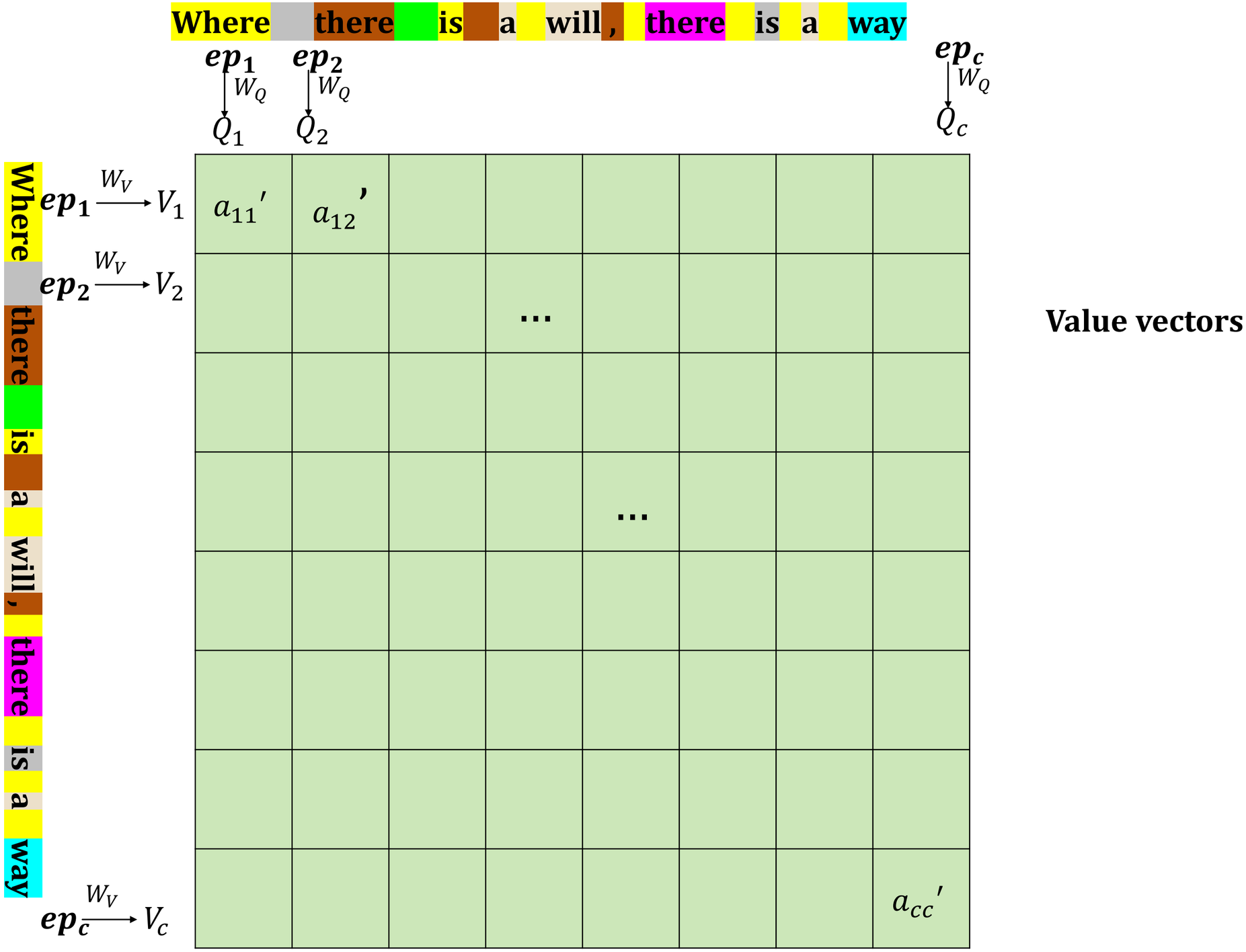
Attention matrix

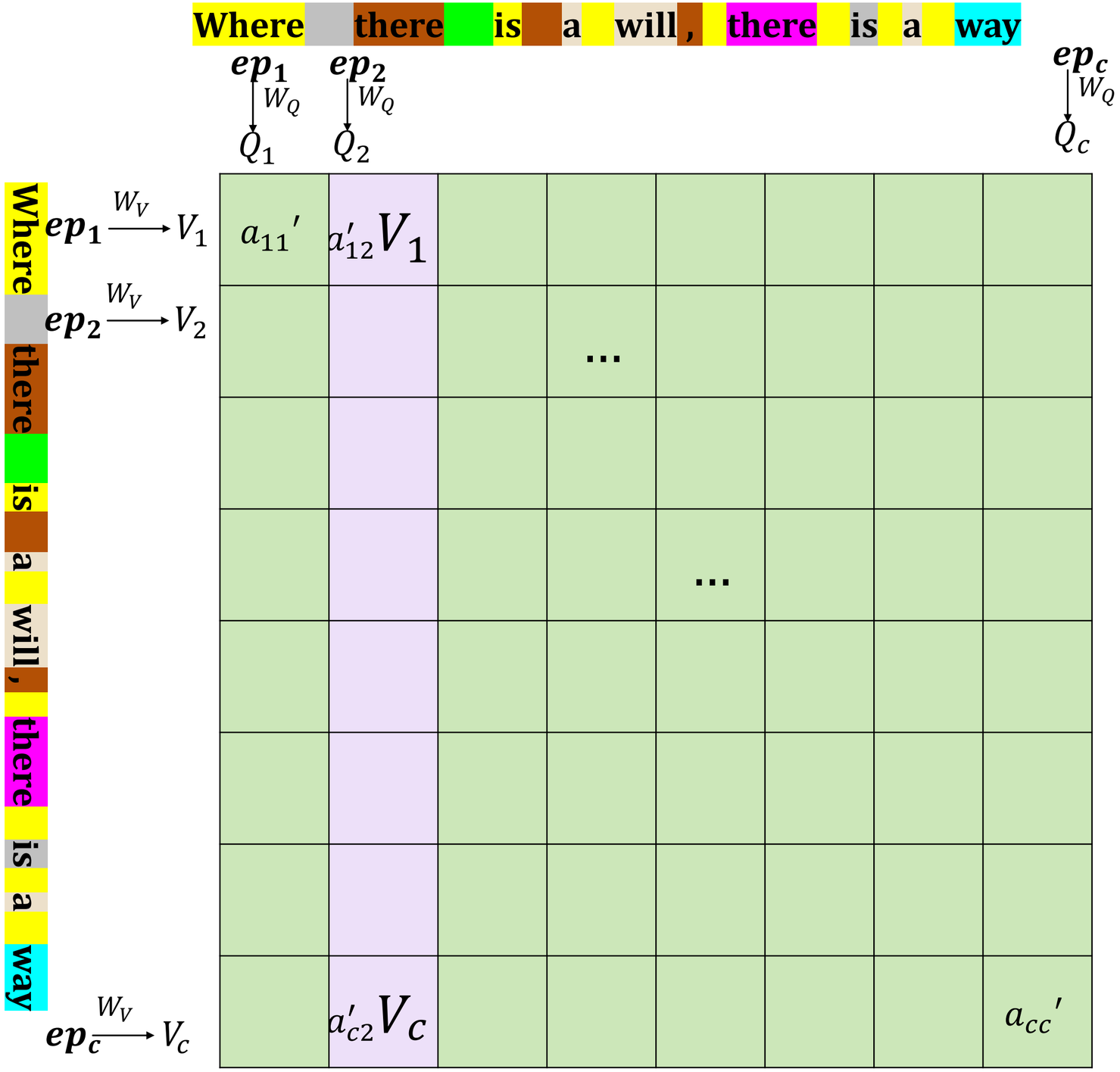
For each query column, we apply softmax. As a result, each column kind of indicates the probabilities of the corresponding query to match with different keys



Attention matrix

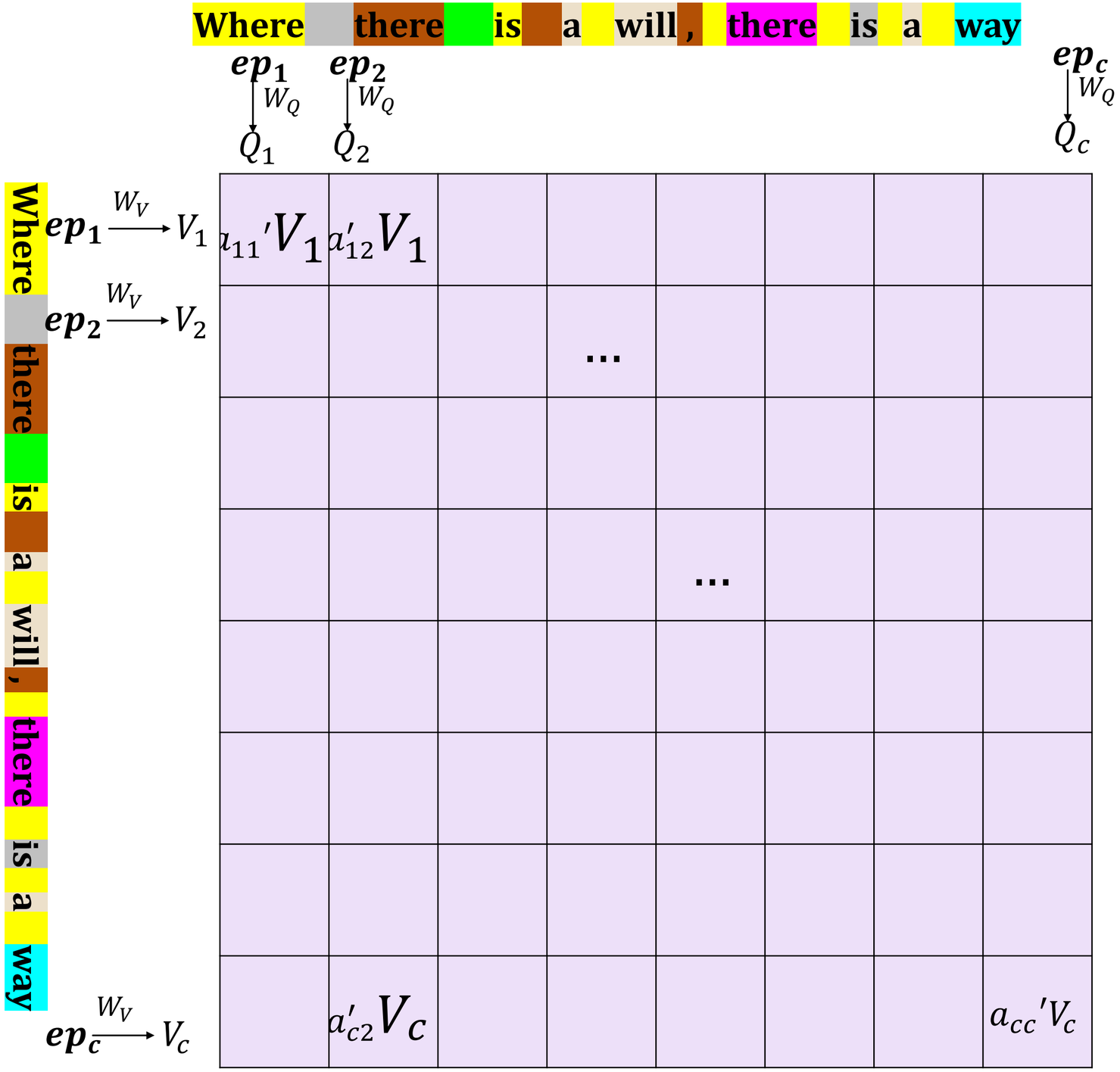
After applying softmax to every column


Where there is a will, there is a way
ep_1
ep_2
ep_c
W_Q
Q_1
Q_2
Q_c
W_V
V_1
V_2
V_c
a_{11}'
a_{12}'
a_{cc}'
...
...
Value vectors



Multiply Value vectors with every column

Consider column 2



Multiply Value vectors with every column

Consider column 2

Similarly, for other columns

	ep_1	ep_2						ep_c
ep_1	$a_{11}{}'V_1$	$a'_{12}V_1$						
ep_2				...				
					...			
ep_c		$a'_{c2}V_c$						$a_{cc}{}'V_c$

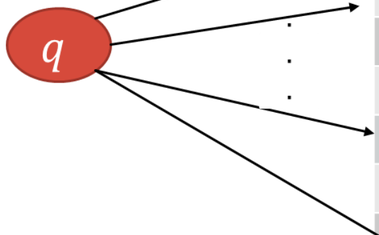
Multiply Value vectors with every column

Consider column 2

Similarly, for other columns

	ep_1	ep_2						ep_c
ep_1	$a_{11}{}'V_1$	$a'_{12}V_1$						
ep_2				...				
					...			
ep_c		$a'_{c2}V_c$						$a_{cc}{}'V_c$
		Δep_2						

Sum up values at each column



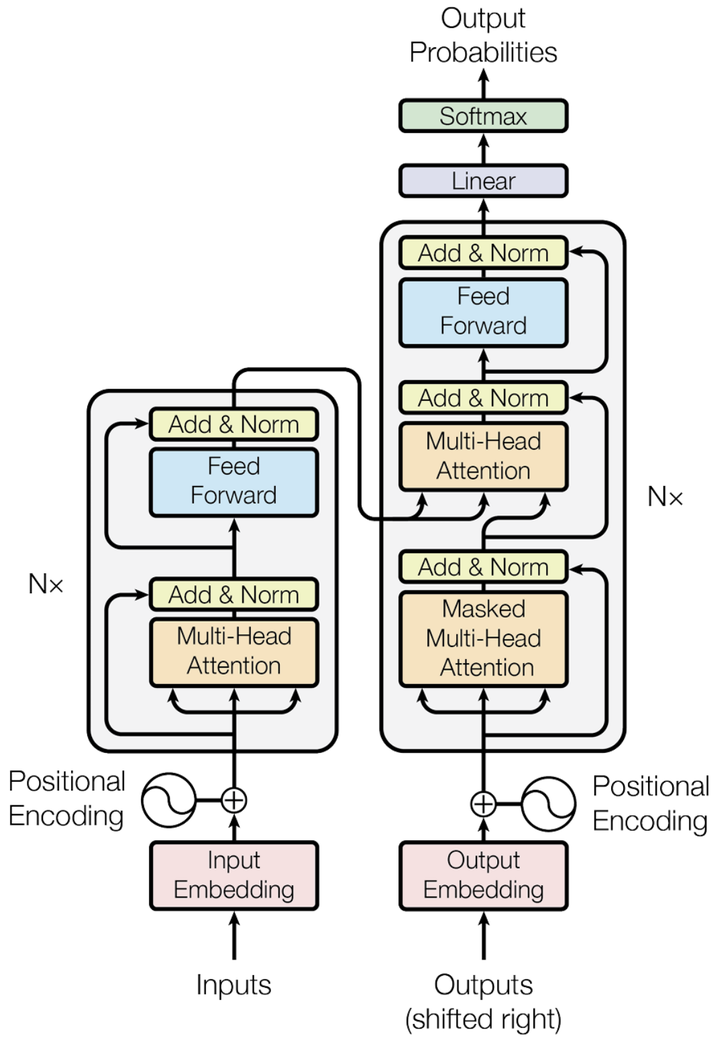
Similarity	Key	Value
s_1	k_1	v_1
s_2	k_2	v_2
.	.	.
.	.	.
s_m	k_m	v_m
.	.	.
s_n	k_n	v_n

$$Attention\ value = \sum_i softmax(s_i)v_i$$

	ep_1	ep_2						ep_c
ep_1	$a_{11}{}'V_1$	$a'_{12}V_1$						
ep_2				...				
					...			
ep_c		$a'_{c2}V_c$						$a_{cc}{}'V_c$
		Δep_2						

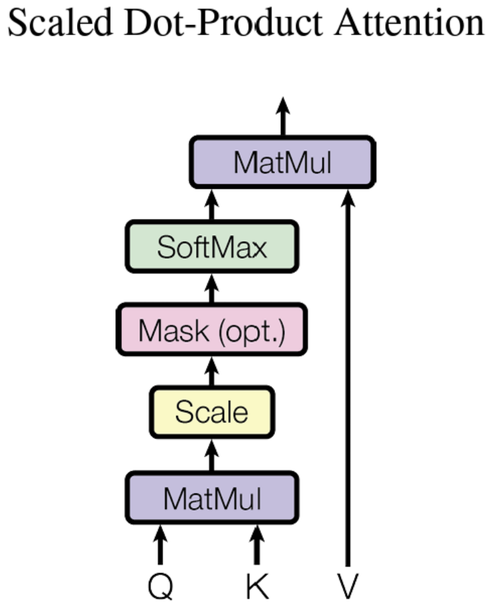
Sum up values at each column

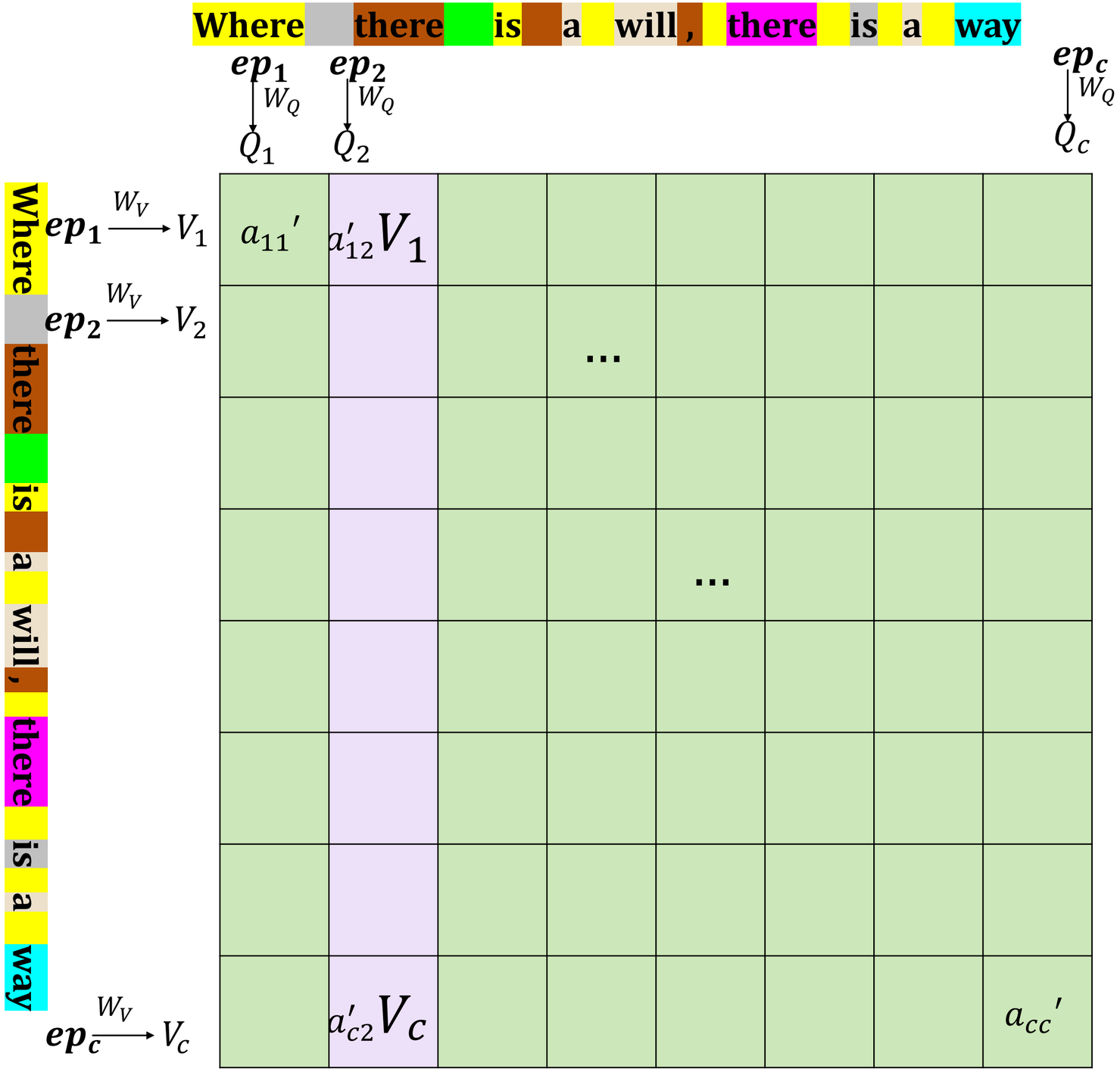
Get $ep'_i = ep_i + \Delta ep_i$
Through residual connection



	ep_1	ep_2						ep_c
ep_1	$a_{11}{}'V_1$	$a'_{12}V_1$						
ep_2				...				
					...			
ep_c		$a'_{c2}V_c$						$a_{cc}{}'V_c$
		Δep_2						

This entire thing is one head of attention

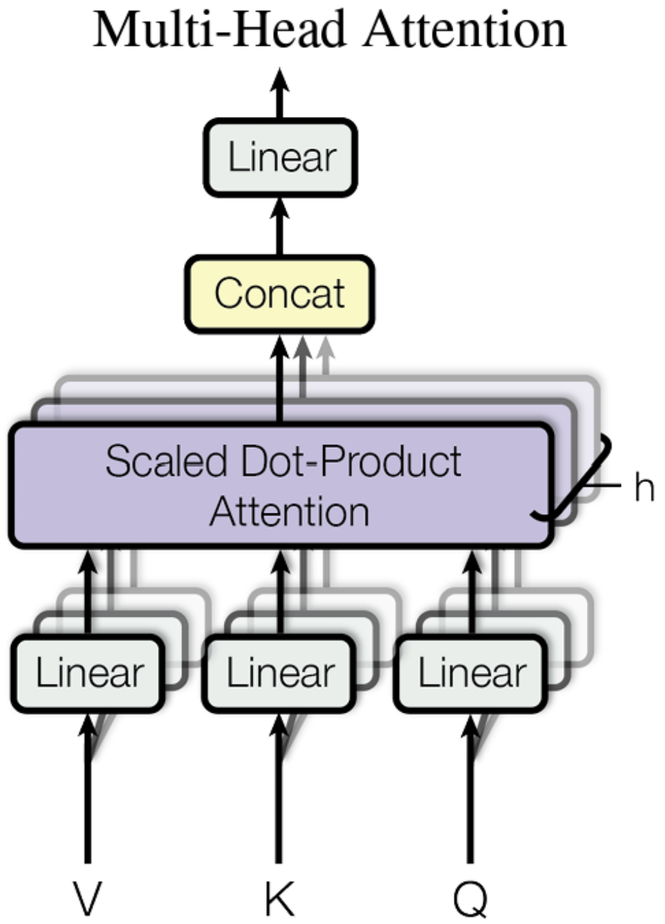


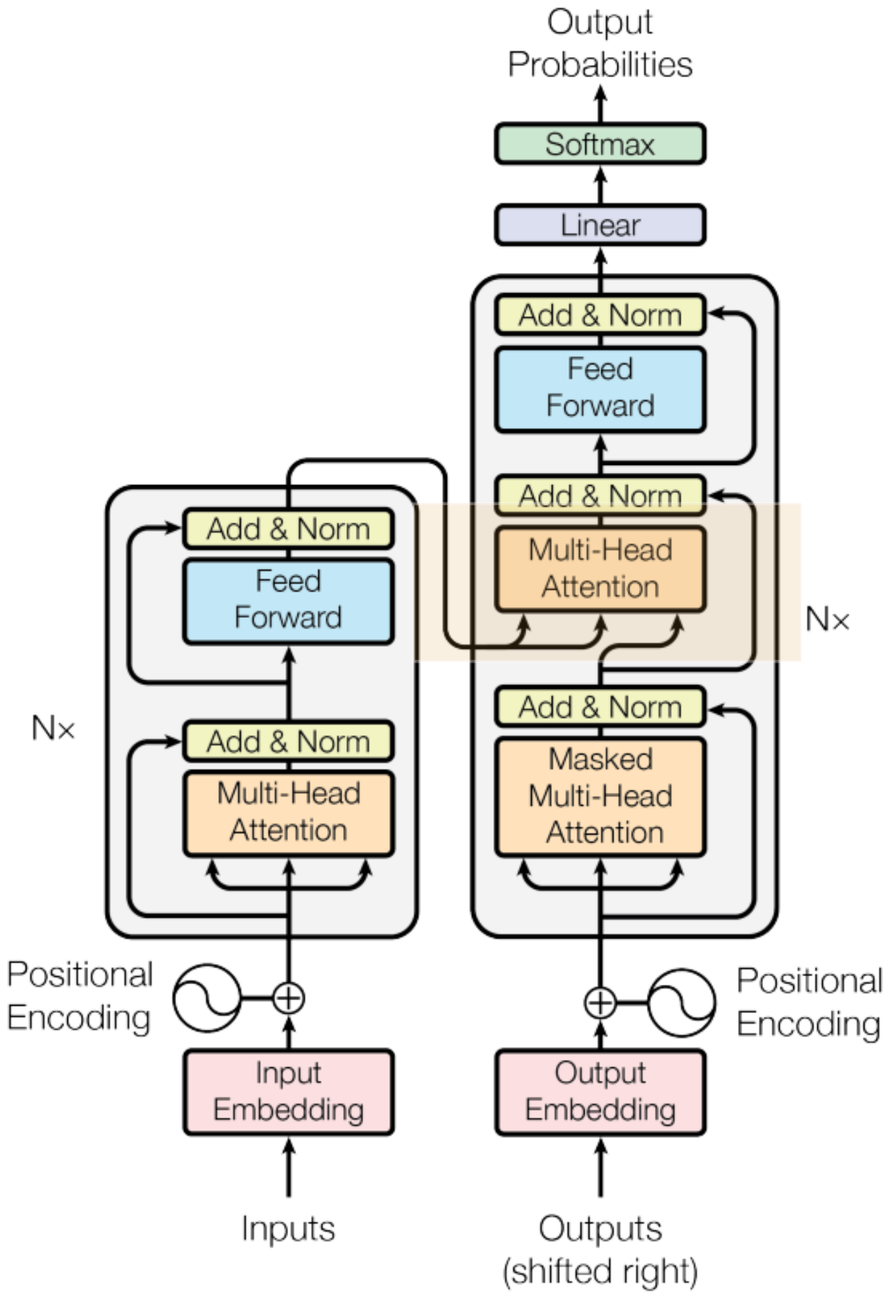


We will have many such heads with different W_V, W_Q, and W_K

	ep_1	ep_2						ep_c
ep_1	$a_{11}{}'V_1$	$a'_{12}V_1$						
ep_2				...				
					...			
ep_c		$a'_{c2}V_c$						$a_{cc}{}'V_c$
		Δep_2						

This entire thing is one head of attention





Encoder-decoder Attention

Key and value comes from the encoder through two paths

Query comes from the decoder

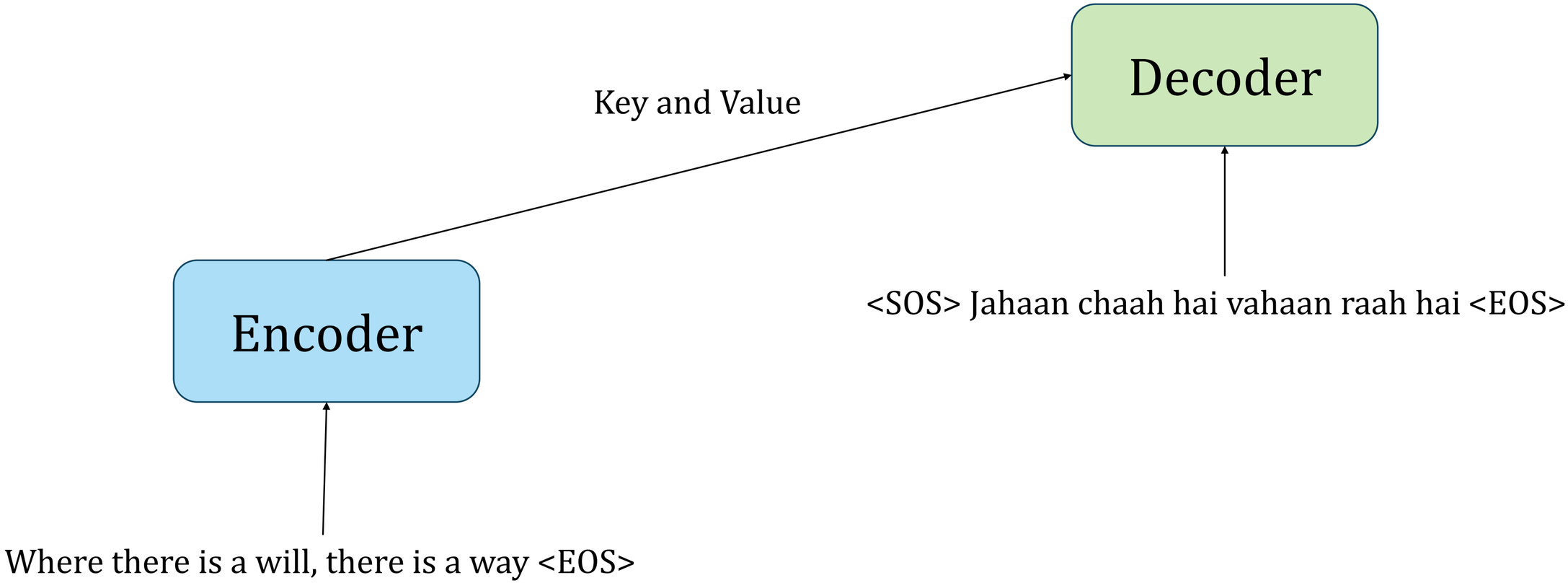
PROGRAMMING

Implement Scaled Dot-product Attention

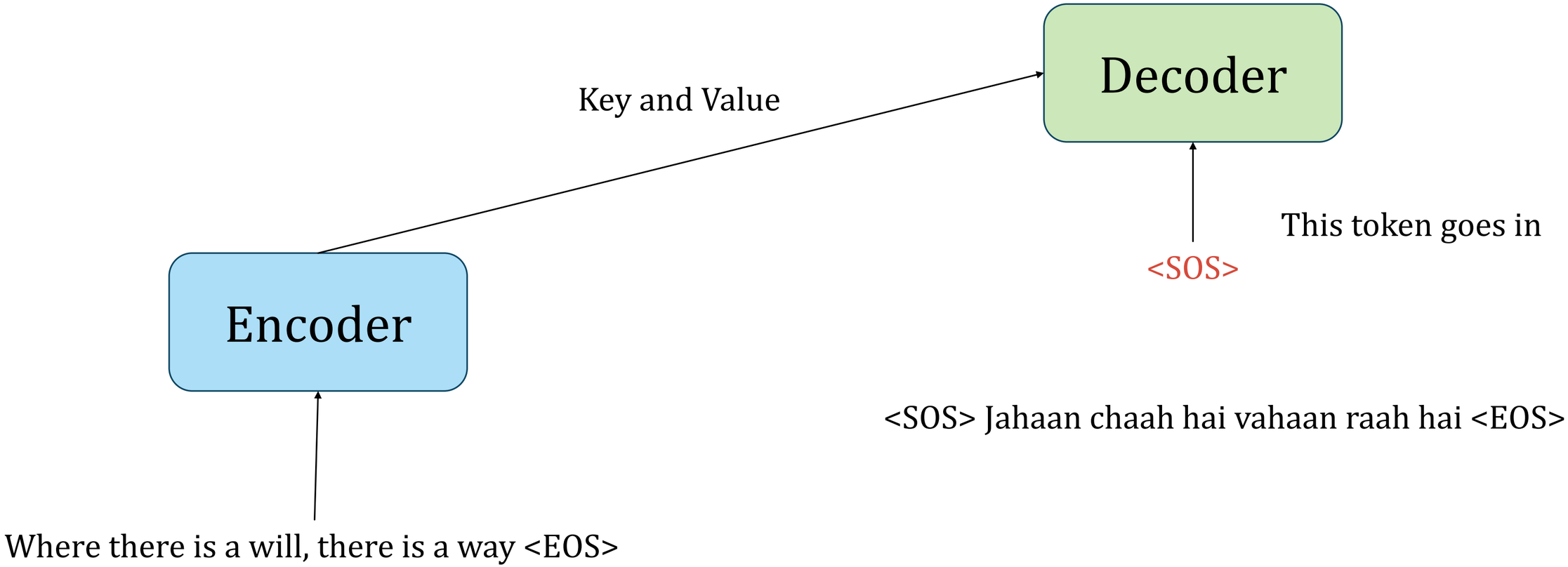
Write a function in NumPy or PyTorch that takes Q, K, V of shape (n, d_k) and an optional causal mask, and returns the attention output and weights.

extend it to batched, multi-head inputs of shape (batch, heads, n, d_k).

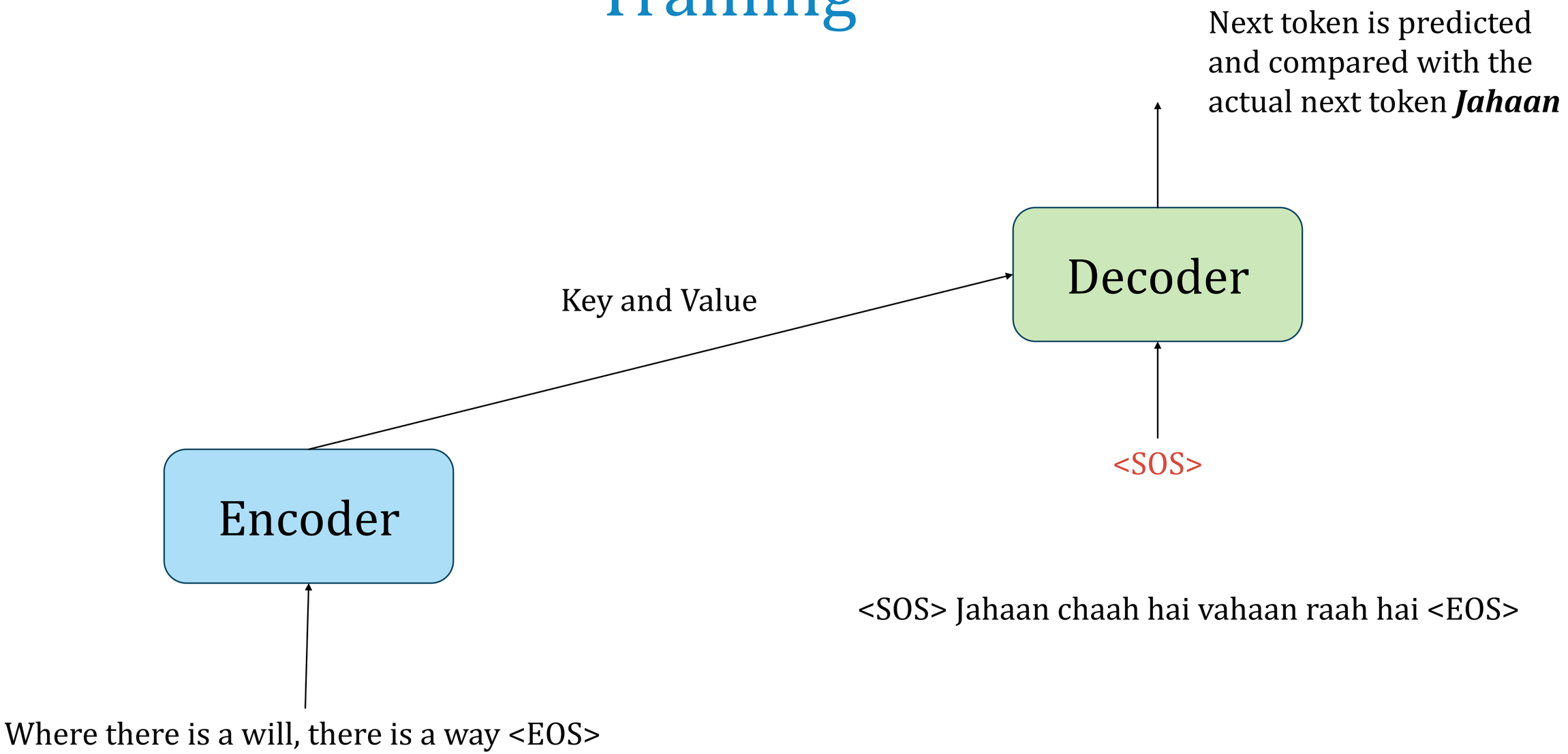
Training


Decoder
Key and Value
Encoder
<SOS> Jahaan chaah hai vahaan raah hai <EOS>
Where there is a will, there is a way <EOS>

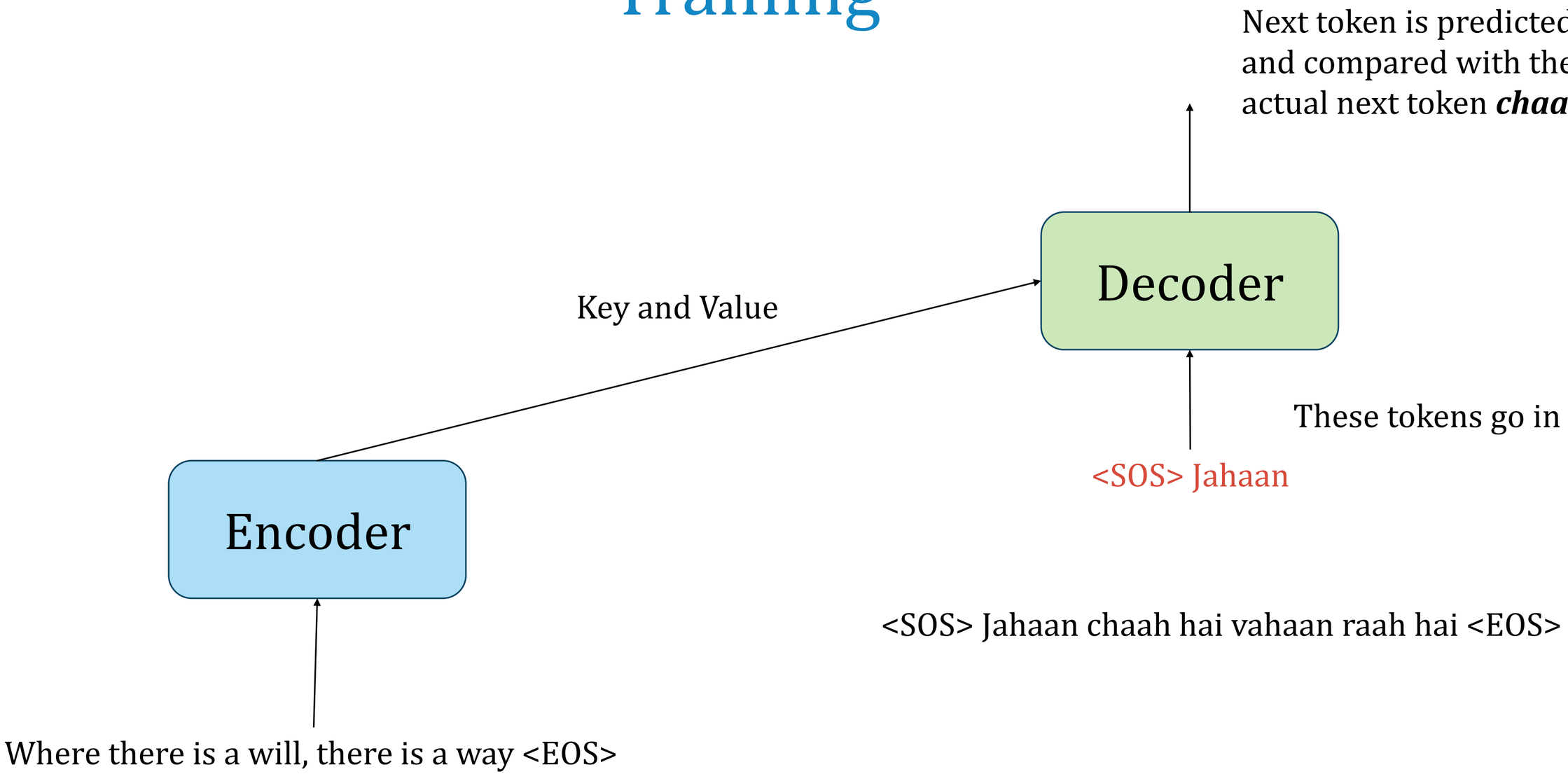
Training


Decoder
Key and Value
This token goes in
<SOS>
Encoder
<SOS> Jahaan chaah hai vahaan raah hai <EOS>
Where there is a will, there is a way <EOS>

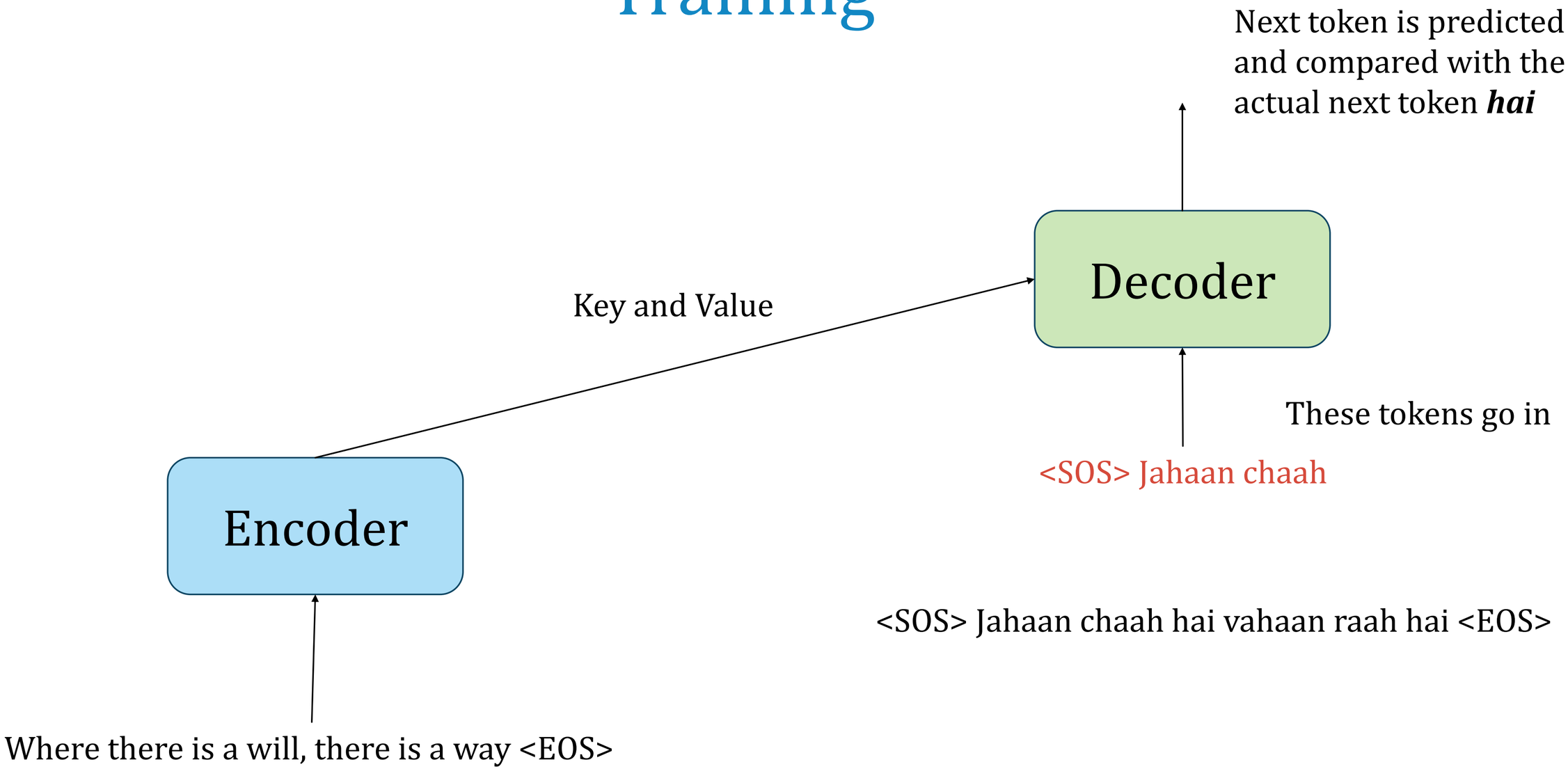
Training


Next token is predicted and compared with the actual next token ***Jahaan***
Decoder
Key and Value
<SOS>
Encoder
<SOS> Jahaan chaah hai vahaan raah hai <EOS>
Where there is a will, there is a way <EOS>

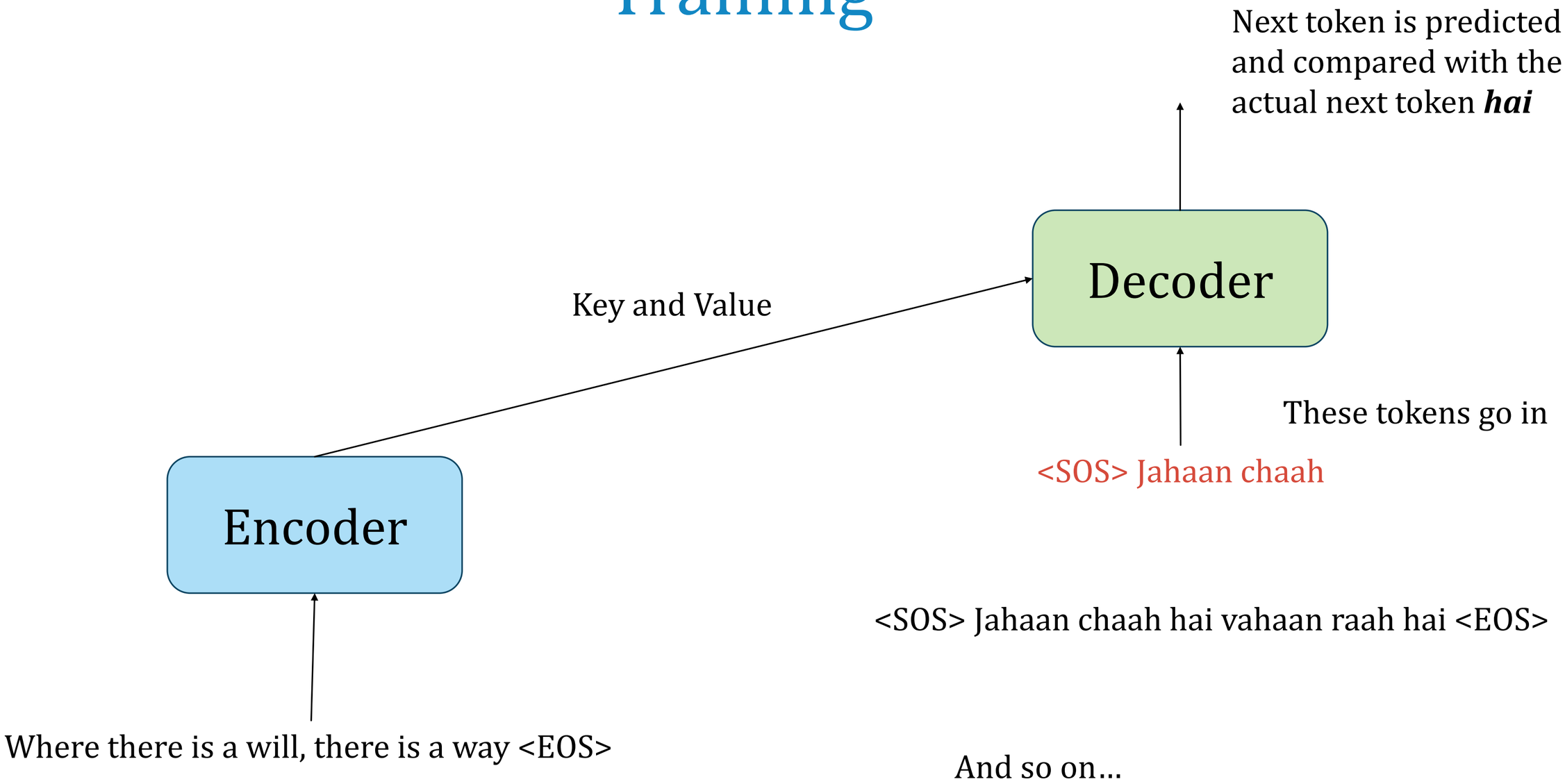
Training


Next token is predicted and compared with the actual next token ***chaah***
Decoder
Key and Value
These tokens go in
<SOS> Jahaan
Encoder
<SOS> Jahaan chaah hai vahaan raah hai <EOS>
Where there is a will, there is a way <EOS>

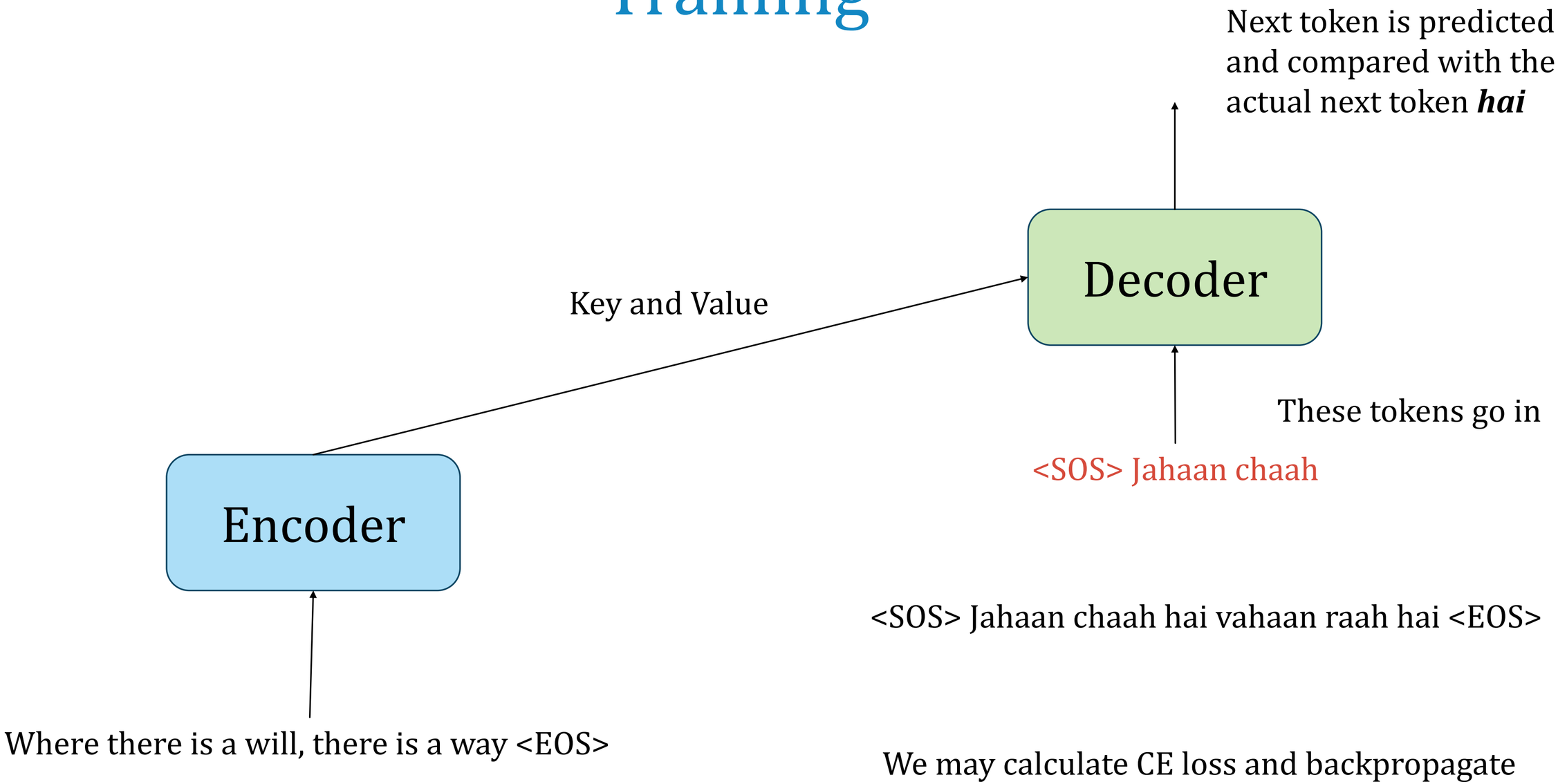
Training


Next token is predicted and compared with the actual next token ***hai***
Decoder
Key and Value
These tokens go in
<SOS> Jahaan chaah
Encoder
<SOS> Jahaan chaah hai vahaan raah hai <EOS>
Where there is a will, there is a way <EOS>

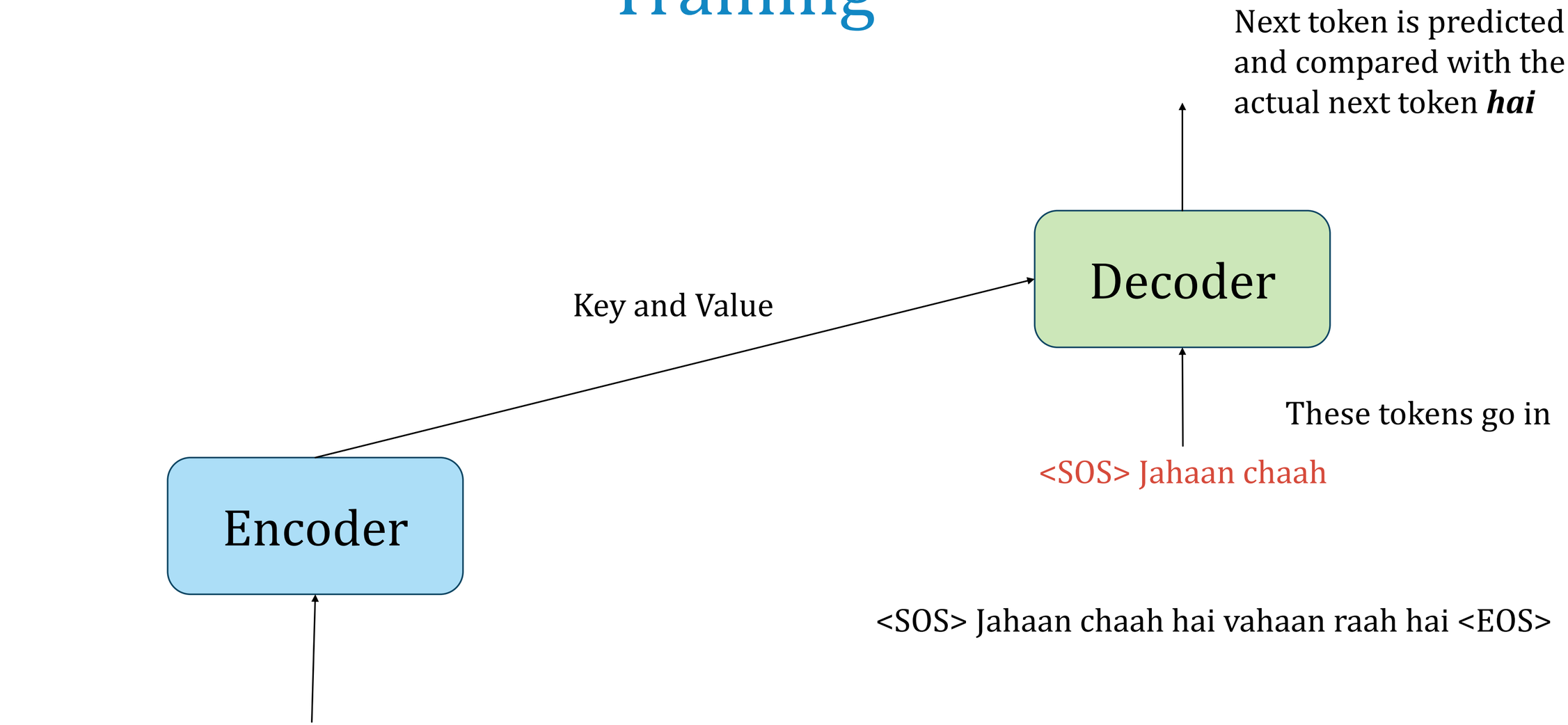
Training


Next token is predicted and compared with the actual next token ***hai***
Decoder
Key and Value
These tokens go in
<SOS> Jahaan chaah
Encoder
<SOS> Jahaan chaah hai vahaan raah hai <EOS>
Where there is a will, there is a way <EOS>
And so on…

Training


Next token is predicted and compared with the actual next token ***hai***
Decoder
Key and Value
These tokens go in
<SOS> Jahaan chaah
Encoder
<SOS> Jahaan chaah hai vahaan raah hai <EOS>
Where there is a will, there is a way <EOS>
We may calculate CE loss and backpropagate

Training


Next token is predicted and compared with the actual next token ***hai***
Decoder
Key and Value
These tokens go in
<SOS> Jahaan chaah
Encoder
<SOS> Jahaan chaah hai vahaan raah hai <EOS>
Where there is a will, there is a way <EOS>
Thus the model learns to predict the next token given the previous tokens

Training: the Whole Sequence, One Pass

Teacher forcing feeds the entire shifted target sequence in at once — nothing is grown incrementally during training.

decoder input (all 4 positions present simultaneously)



The "only see earlier tokens" rule is enforced by the causal mask inside self-attention — not by hiding later tokens from the input.

Every position is physically present in the same forward pass. Masking blocks what each position is allowed to attend to — position i can only look at positions ≤ i, even though later tokens sit right next to it in the tensor.

This is exactly what makes training parallelizable across the whole sequence at once.

Two Masks, Working Together

A real training batch applies two separate masks at once — they solve different problems.

Causal mask

What it blocks

Blocks a position from attending to future positions in the same sequence.

Why it's needed

Enforces autoregressive order — position i only sees positions ≤ i.

Padding mask

What it blocks

Blocks attention to <PAD> tokens added to line sequences up to a common batch length.

Why it's needed

Lets sequences of different lengths be batched into one tensor efficiently.

Mechanically, both work the same way: set the corresponding attention scores to −∞ before softmax, so those positions receive zero weight.

From Decoder Output to a Probability Matrix

Example: sequence length 20, vocabulary size 10,000.

Decoder output

`(20 × d_model)`

one contextual vector per position

Linear projection

`(20 × 10,000)`

project each position up to vocab-size logits

Softmax (row-wise)

`(20 × 10,000)`

each row becomes a valid probability distribution

Row i of this final (20 × 10,000) matrix is the model's predicted probability distribution over the vocabulary for what comes after position i — one full distribution per position, all produced in a single forward pass.

Computing the Loss, Position by Position

The target is a vector of 20 true next-token indices — one per position — compared against the matching row of the predicted matrix.

$$\text{Loss}_i = -\log(P[i, \text{target}_i])$$

— the predicted probability the model assigned to the correct token at row i

$$\text{Total Loss} = (1/20) \times \Sigma_i \ \text{Loss}_i$$

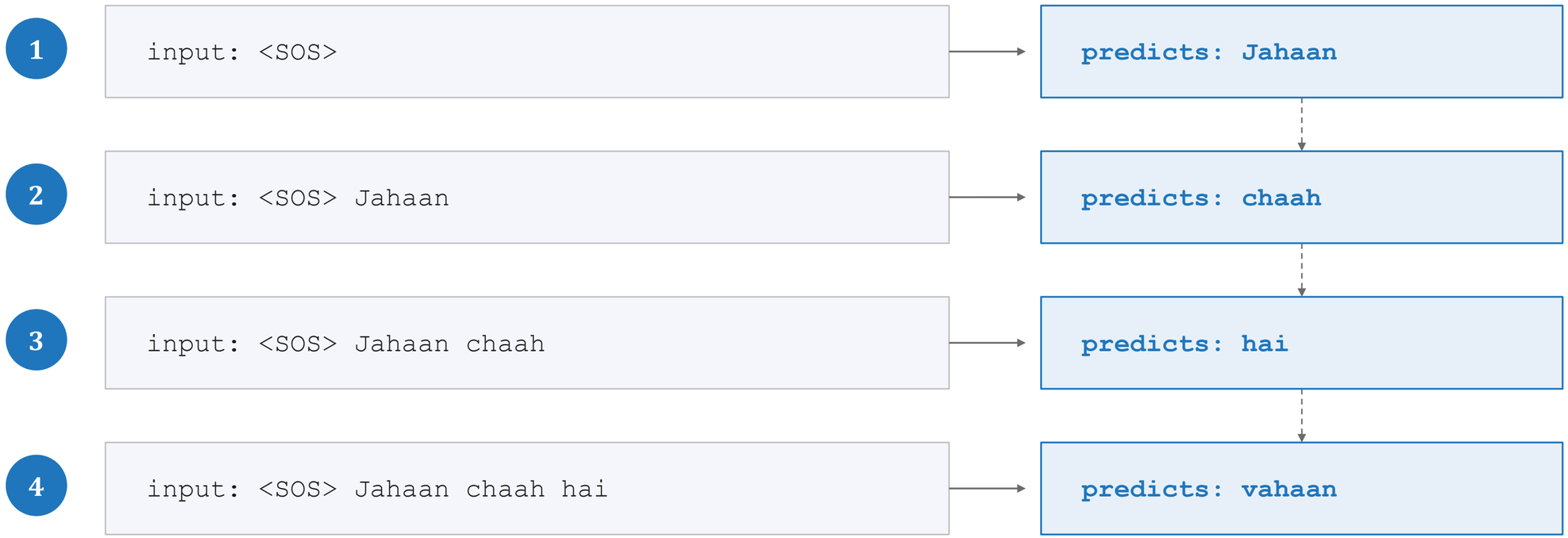
average across all 20 positions → one scalar loss for the whole sequence

Padding positions are excluded

If some of the 20 positions are <PAD> (from batching sequences of different lengths), they're skipped in the loss — typically via an ignore_index passed to the loss function — so padding never affects the average.

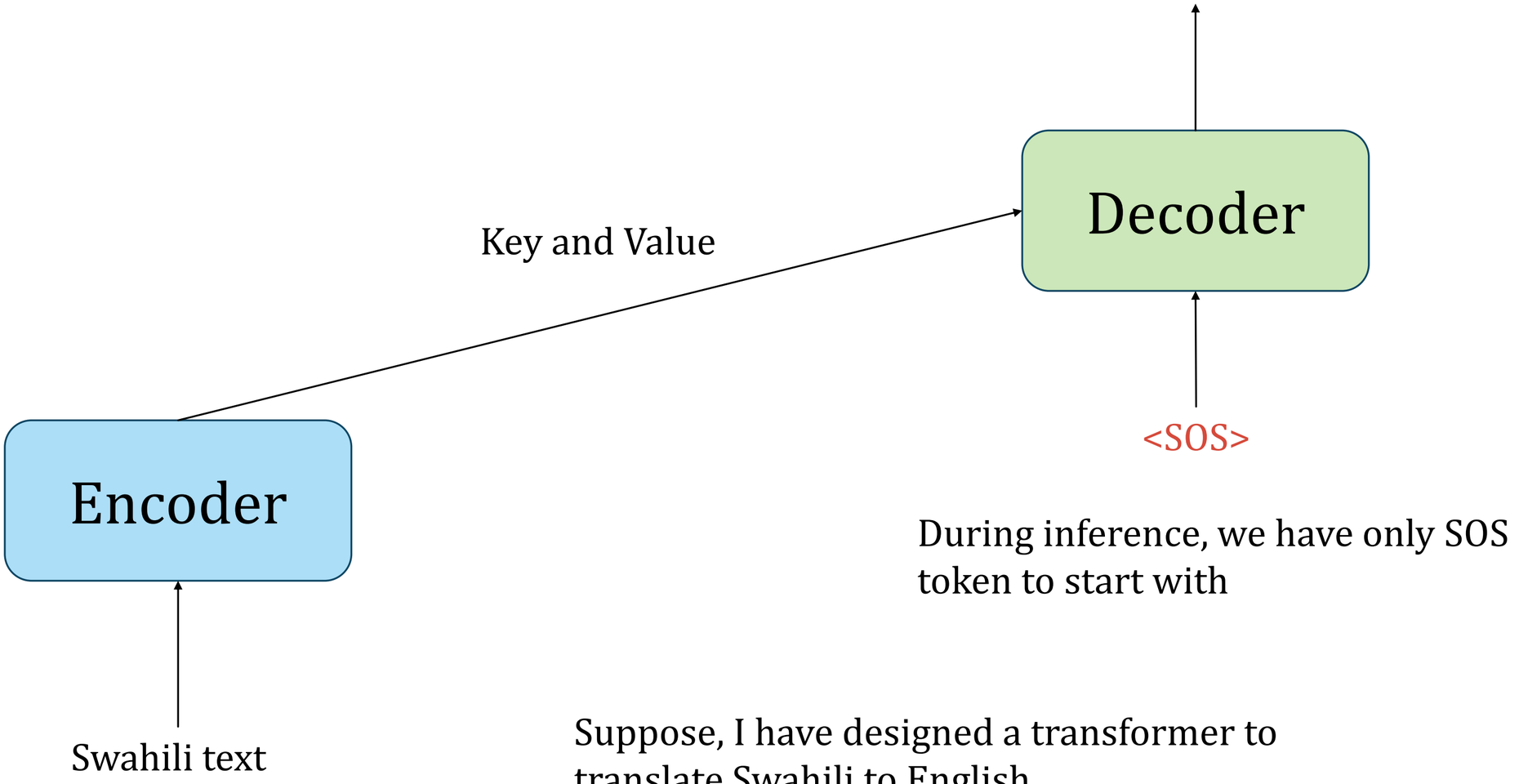
Inference: Sequential

At inference, there's no ground-truth target to feed all at once — each token must be generated before the next input can be built.



The predicted token is fed back in, extending the input by one position — one full forward pass per generated token.

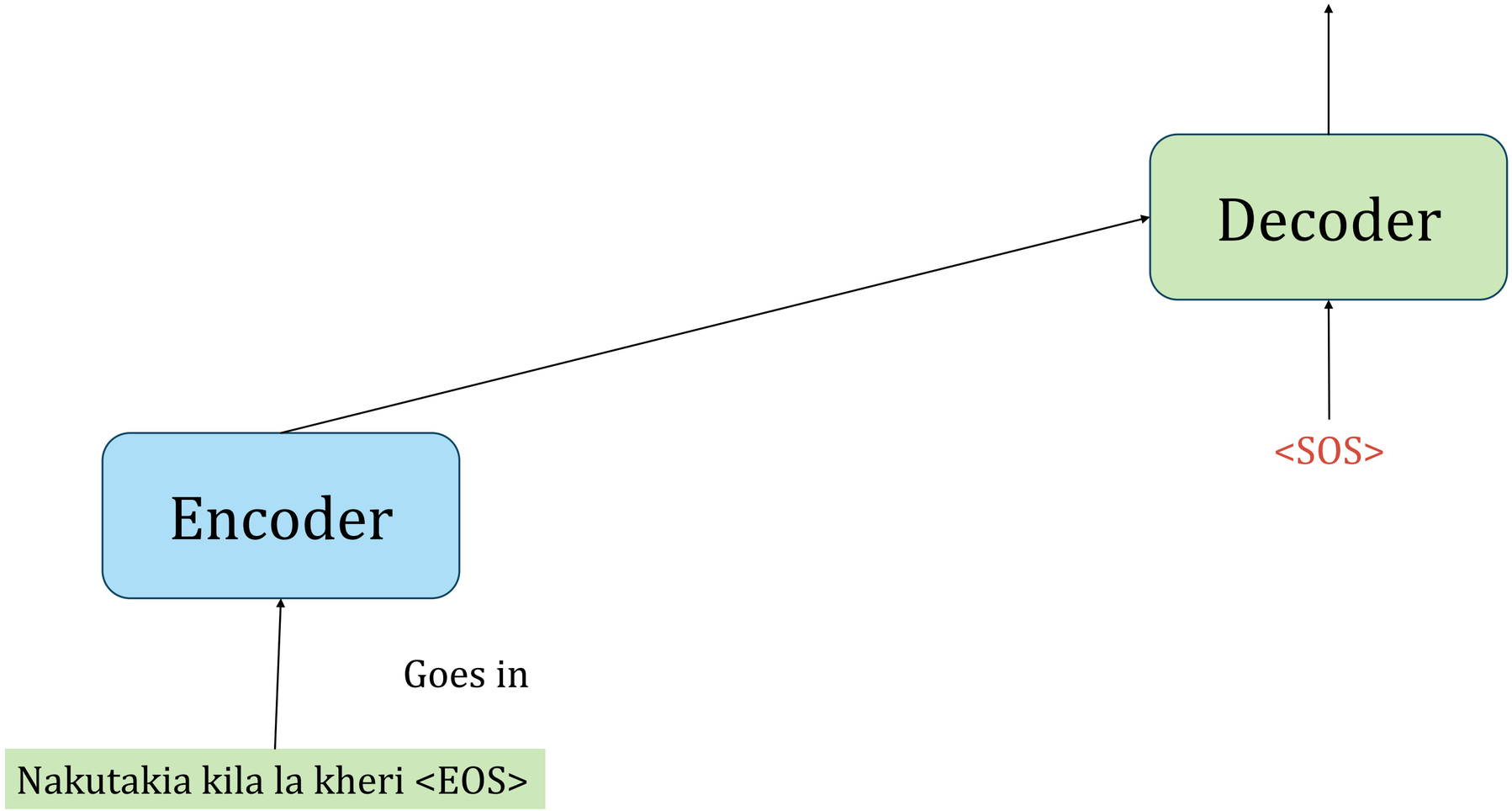
Inference



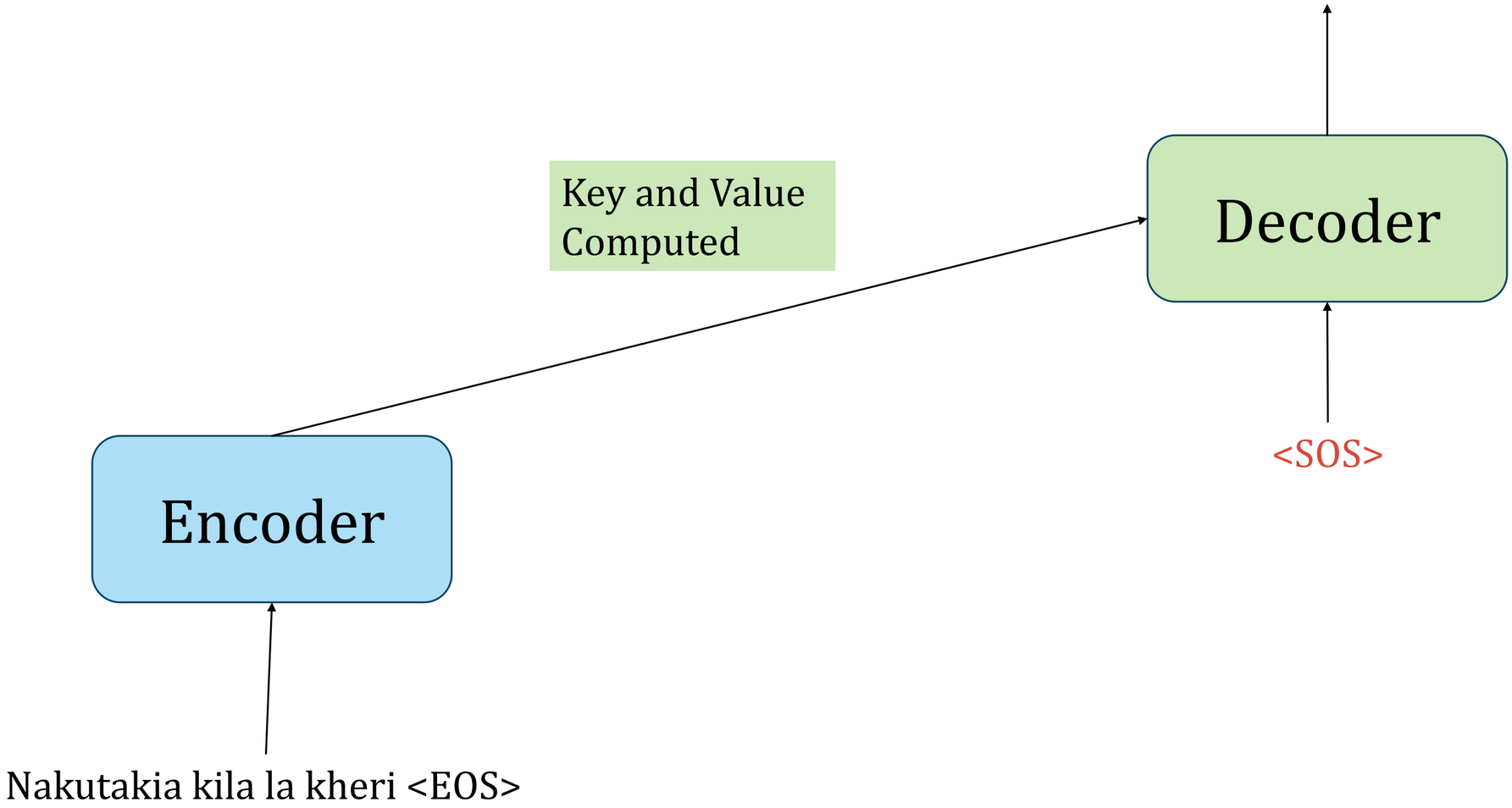
During inference, we have only SOS token to start with

Suppose, I have designed a transformer to translate Swahili to English

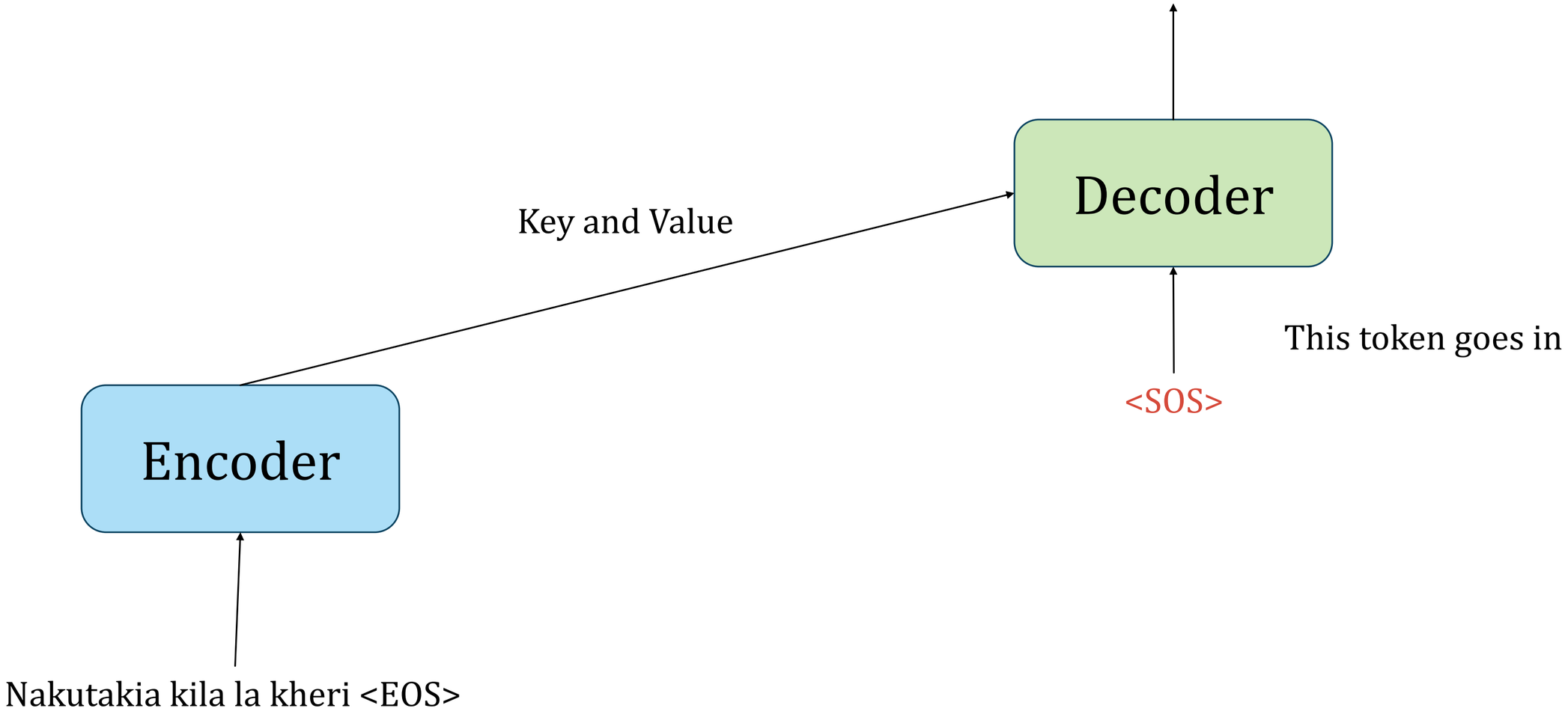
Inference



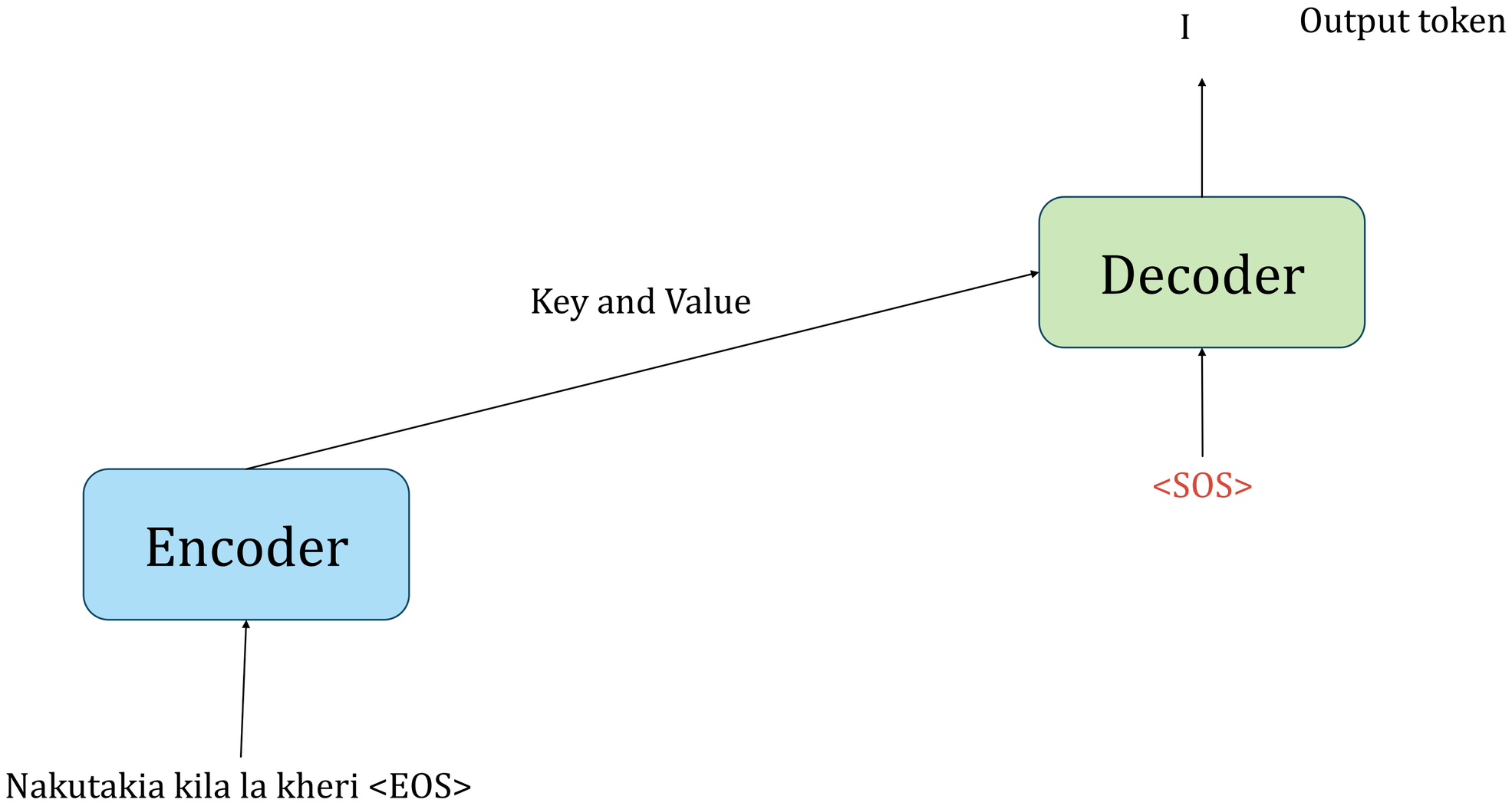
Inference


Key and Value
Computed
Decoder
<SOS>
Encoder
Nakutakia kila la kheri <EOS>

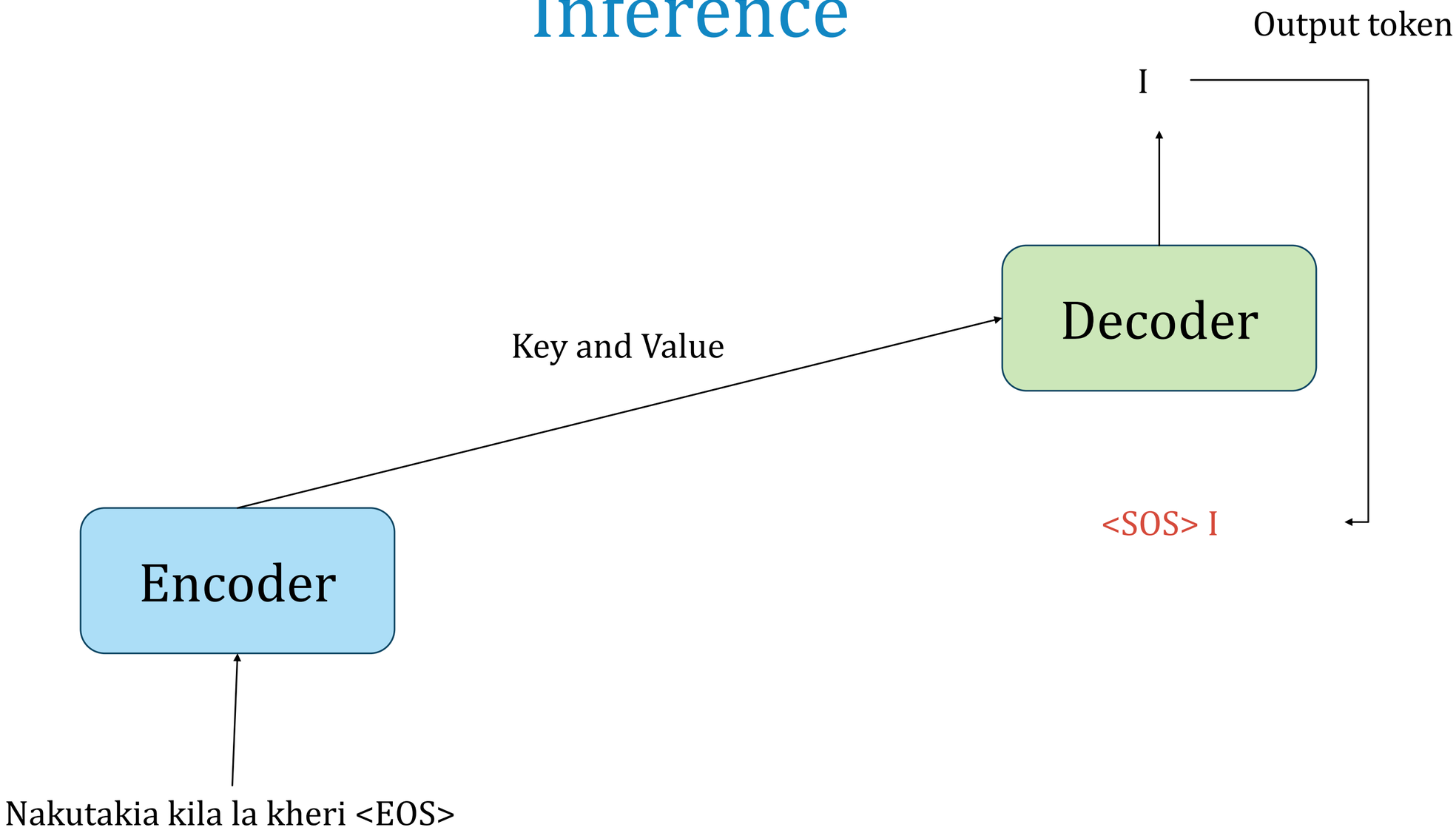
Inference


Decoder
Key and Value
This token goes in
<SOS>
Encoder
Nakutakia kila la kheri <EOS>

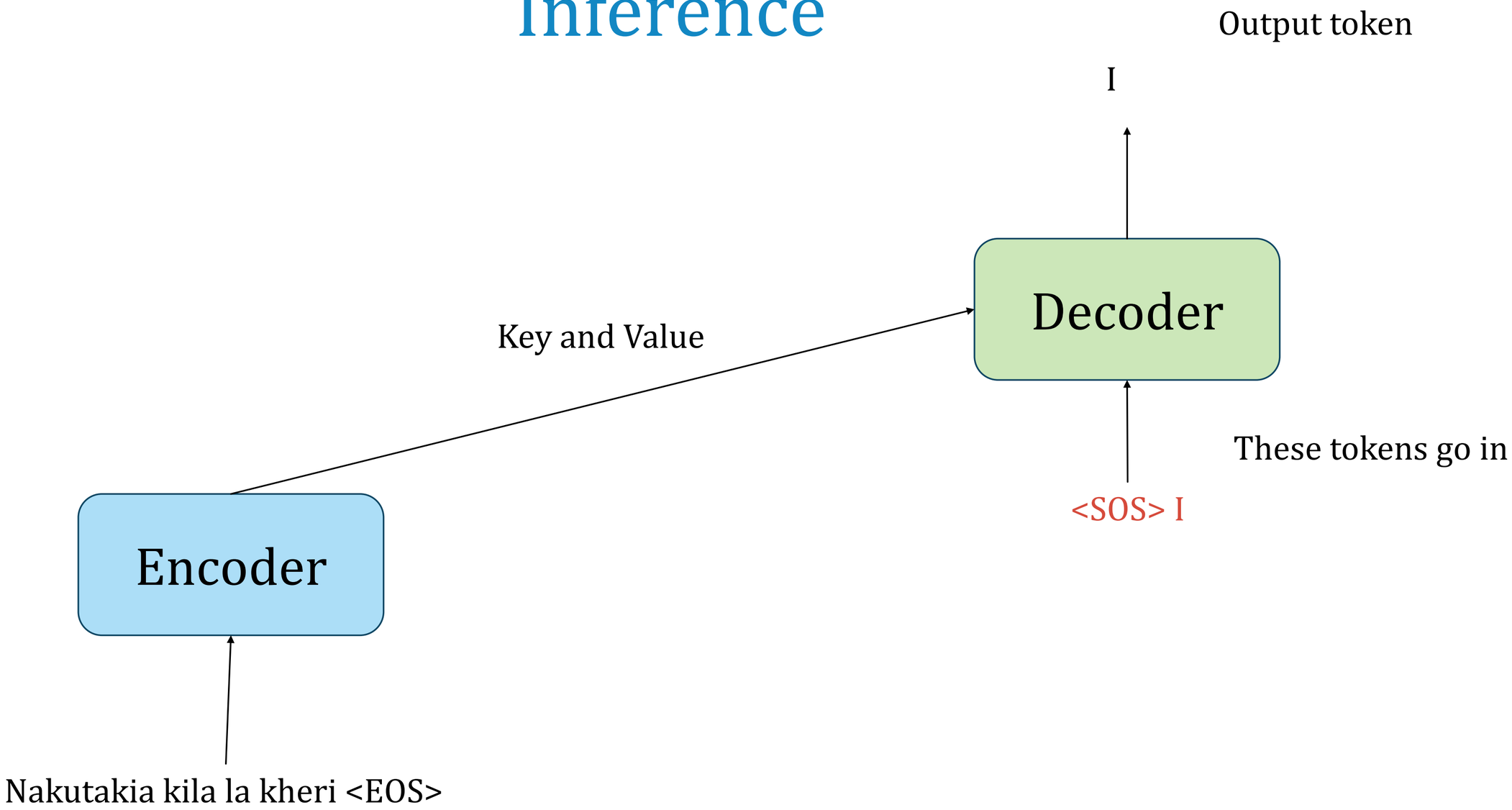
Inference


I
Output token
Decoder
Key and Value
<SOS>
Encoder
Nakutakia kila la kheri <EOS>

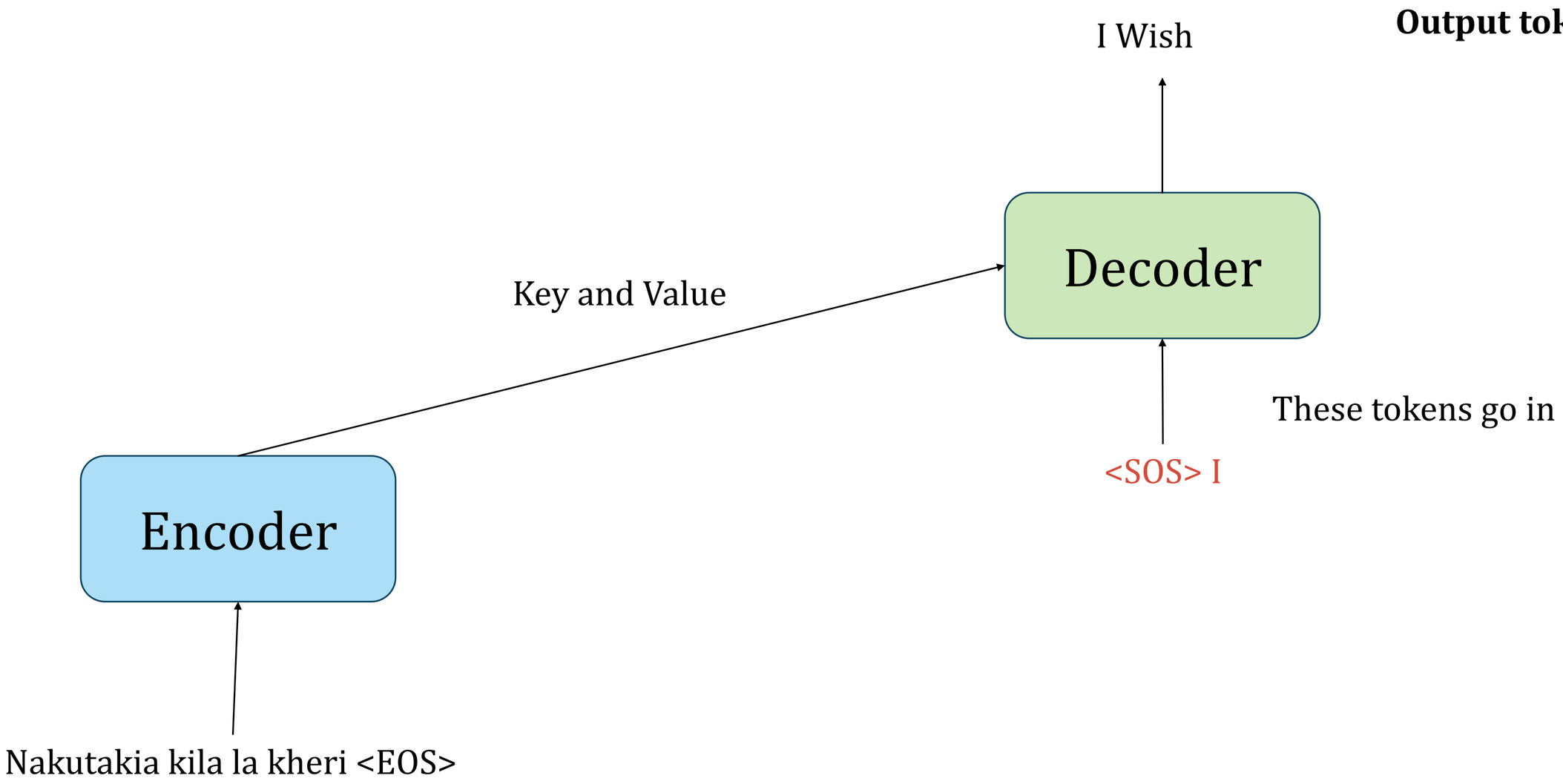
Inference


Output token
I
Decoder
Key and Value
<SOS> I
Encoder
Nakutakia kila la kheri <EOS>

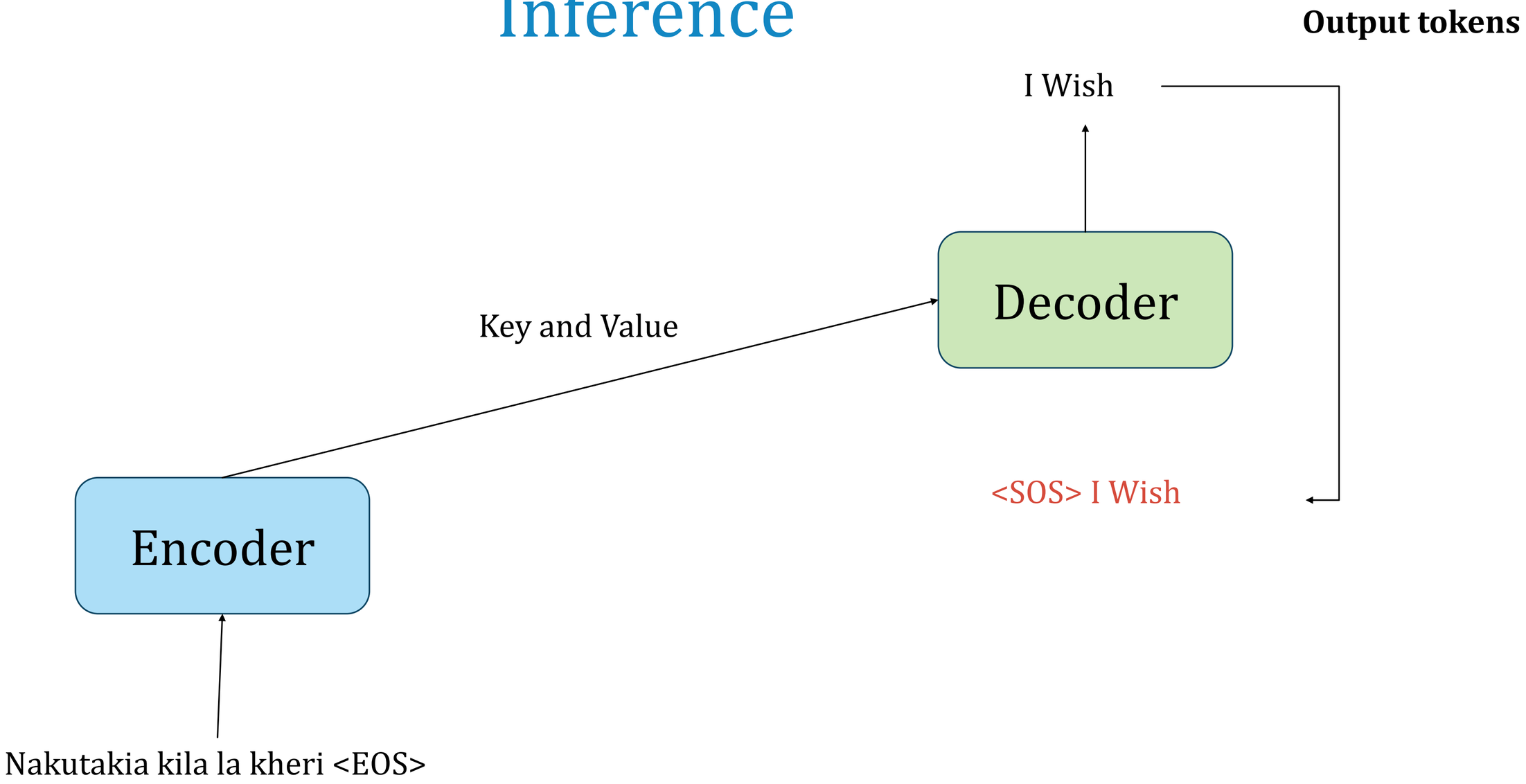
Inference


Output token
I
Decoder
Key and Value
These tokens go in
<SOS> I
Encoder
Nakutakia kila la kheri <EOS>

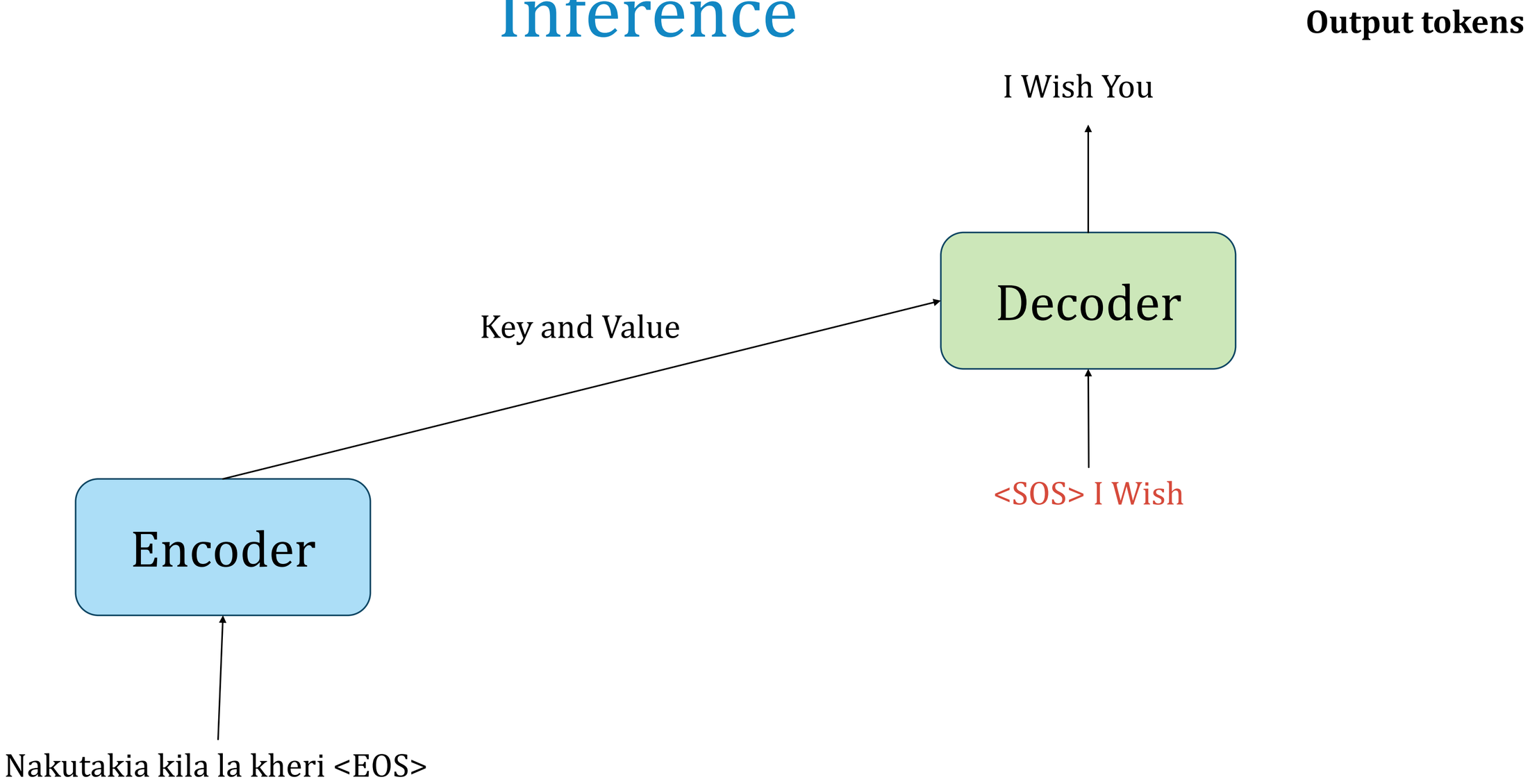
Inference


Output tokens
I Wish
Decoder
Key and Value
These tokens go in
<SOS> I
Encoder
Nakutakia kila la kheri <EOS>

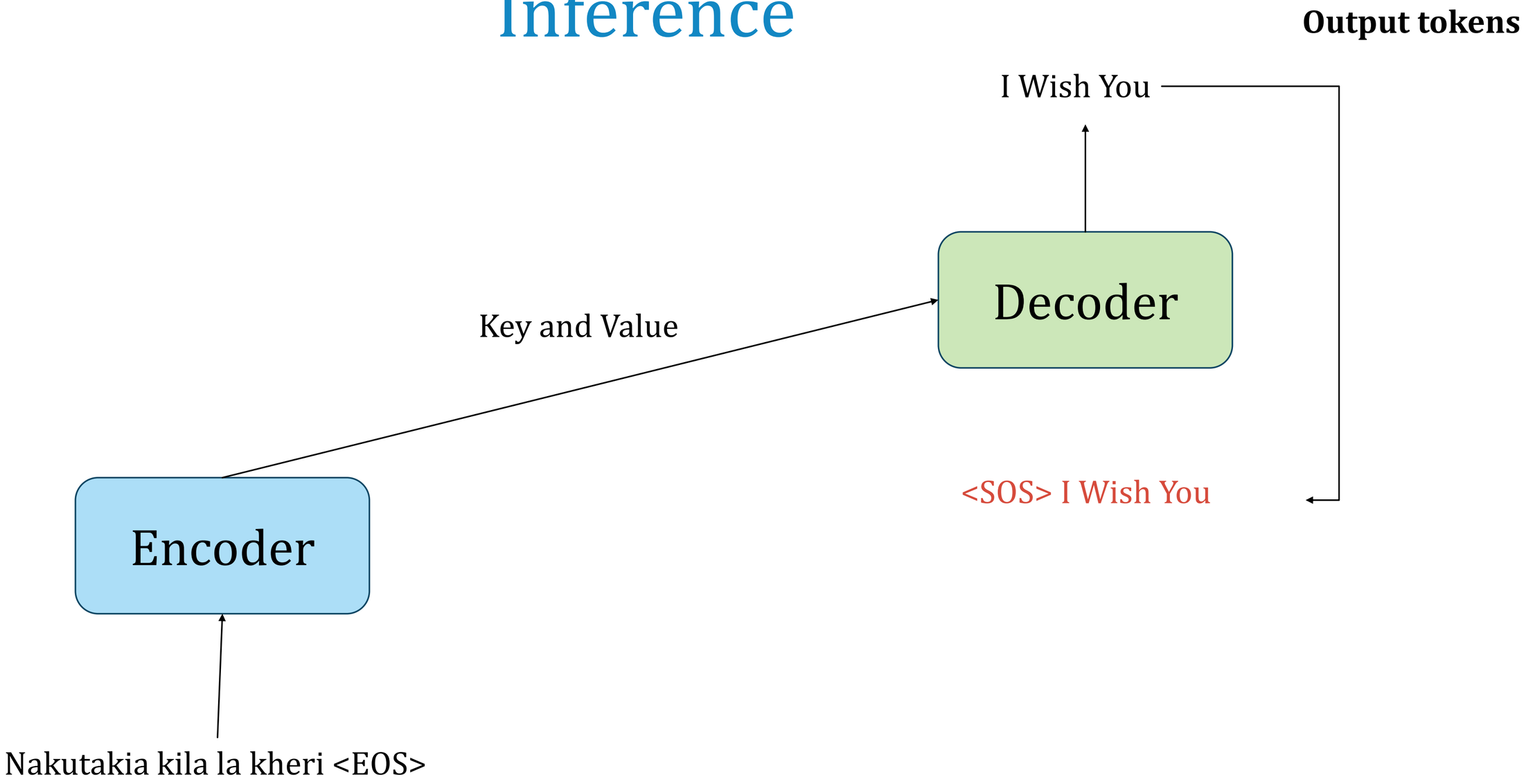
Inference


Output tokens
I Wish
Decoder
Key and Value
<SOS> I Wish
Encoder
Nakutakia kila la kheri <EOS>

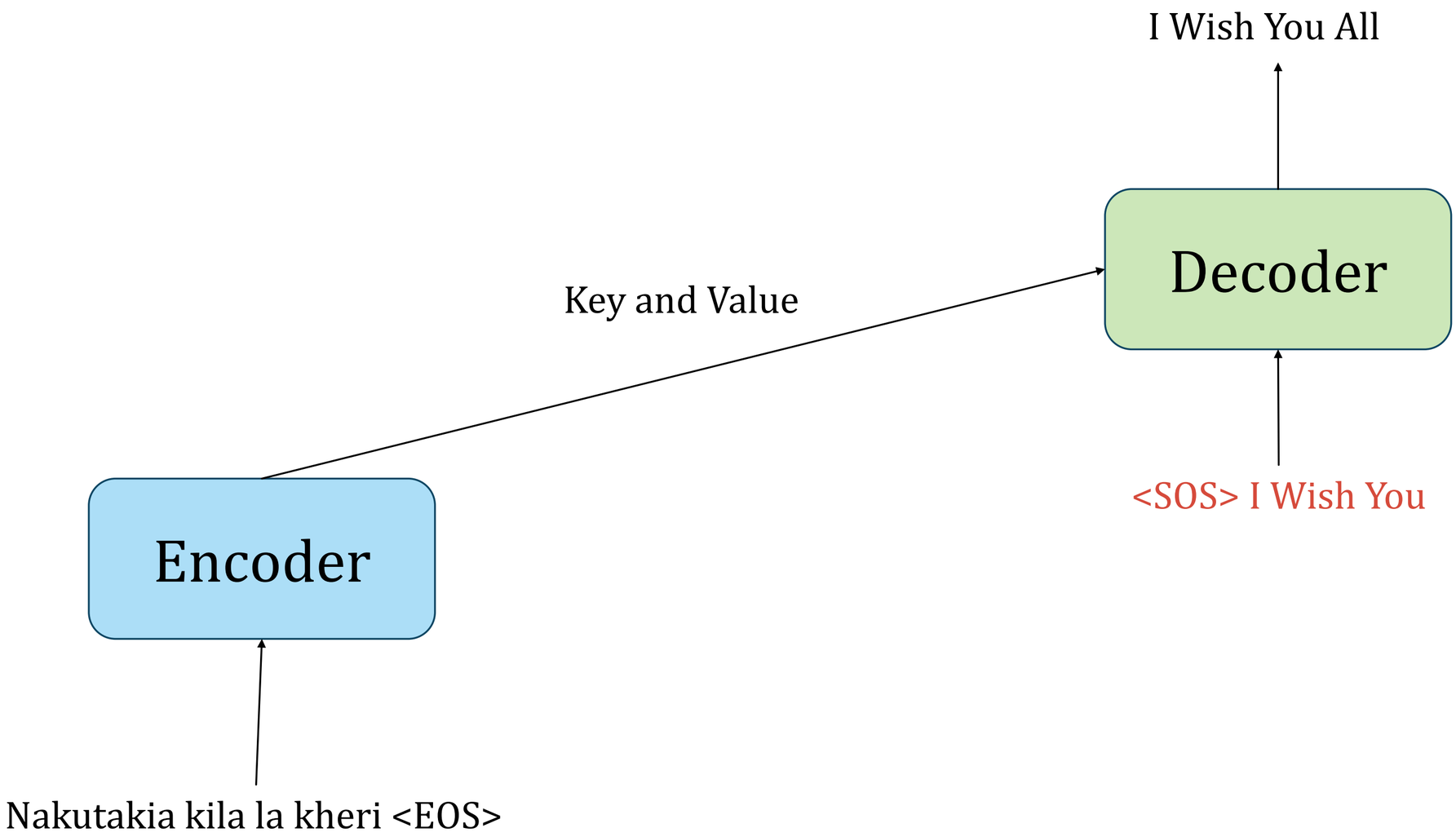
Inference


Output tokens
I Wish You
Decoder
Key and Value
Encoder
<SOS> I Wish
Nakutakia kila la kheri <EOS>

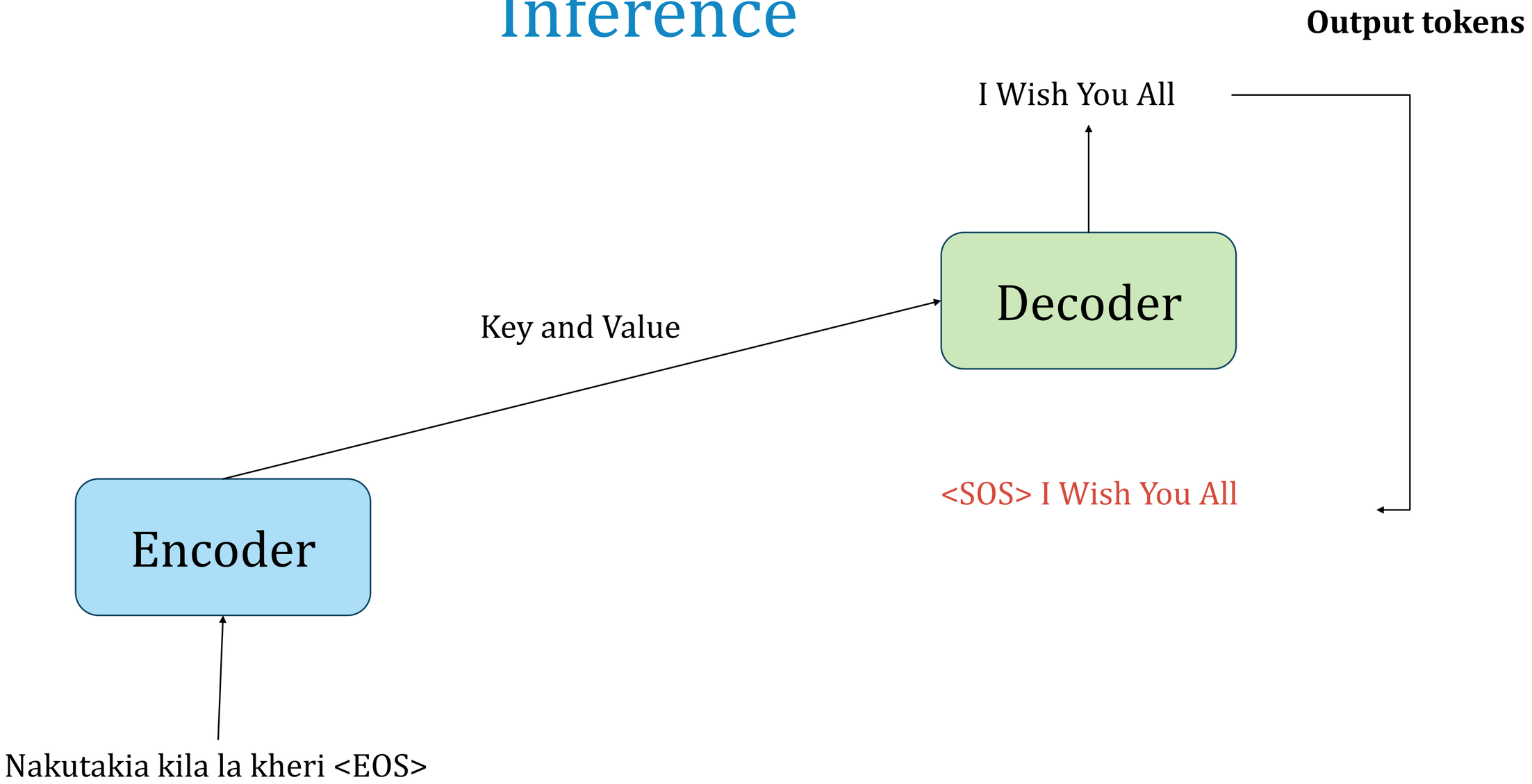
Inference


Output tokens
I Wish You
Decoder
Key and Value
<SOS> I Wish You
Encoder
Nakutakia kila la kheri <EOS>

Inference

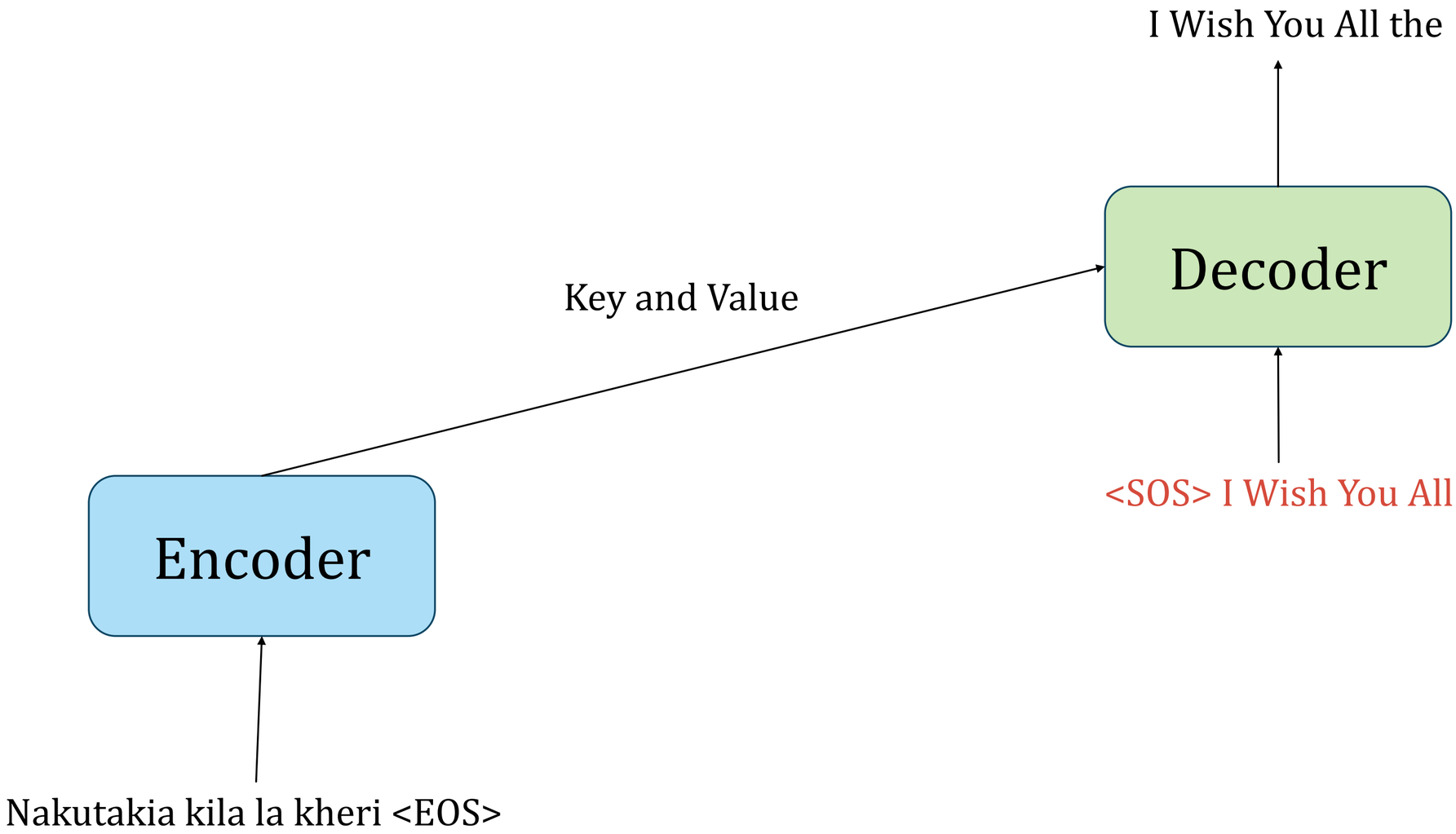

Output tokens
I Wish You All
Decoder
Key and Value
<SOS> I Wish You
Encoder
Nakutakia kila la kheri <EOS>

Inference

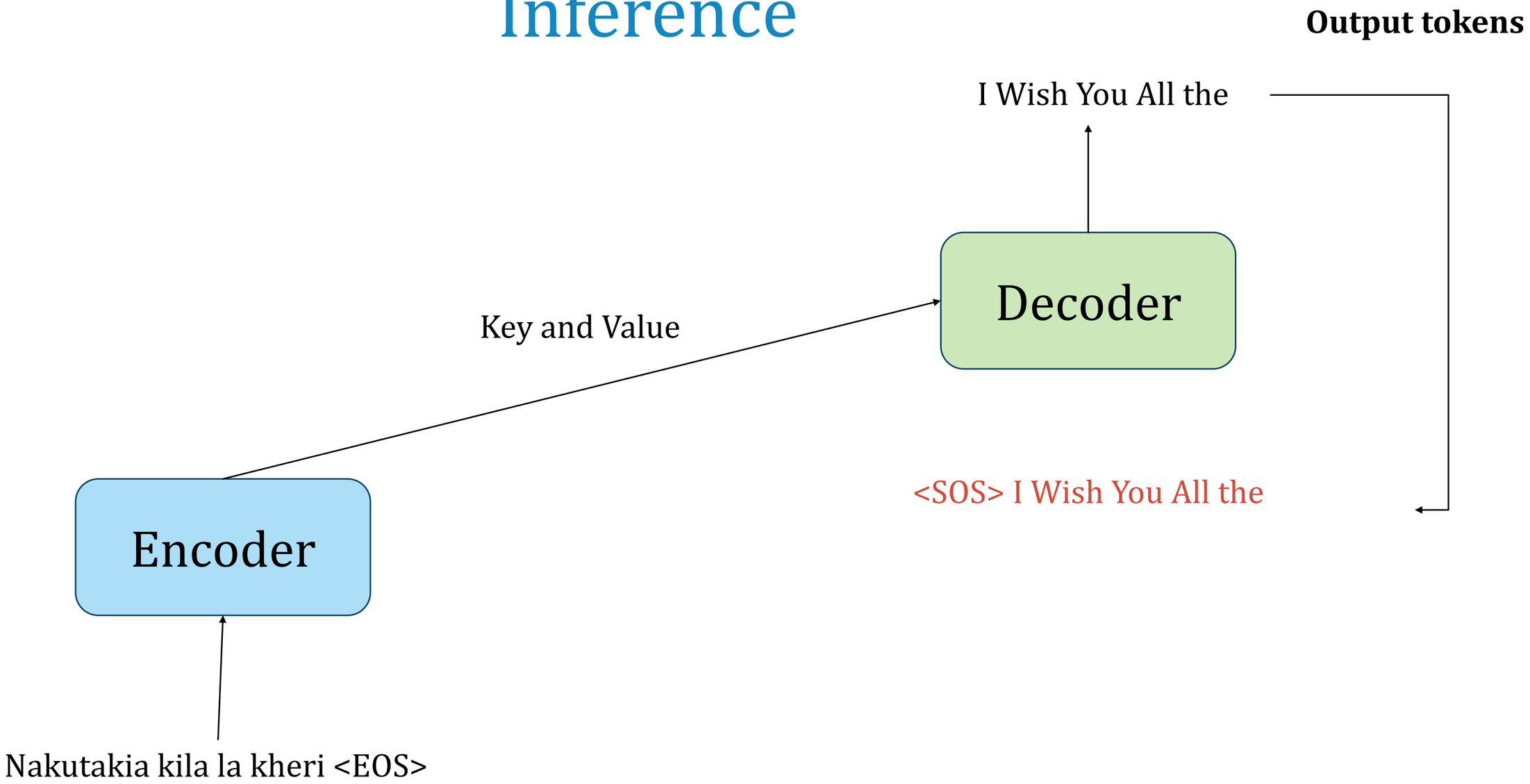

Output tokens
I Wish You All
Decoder
Key and Value
<SOS> I Wish You All
Encoder
Nakutakia kila la kheri <EOS>

Inference

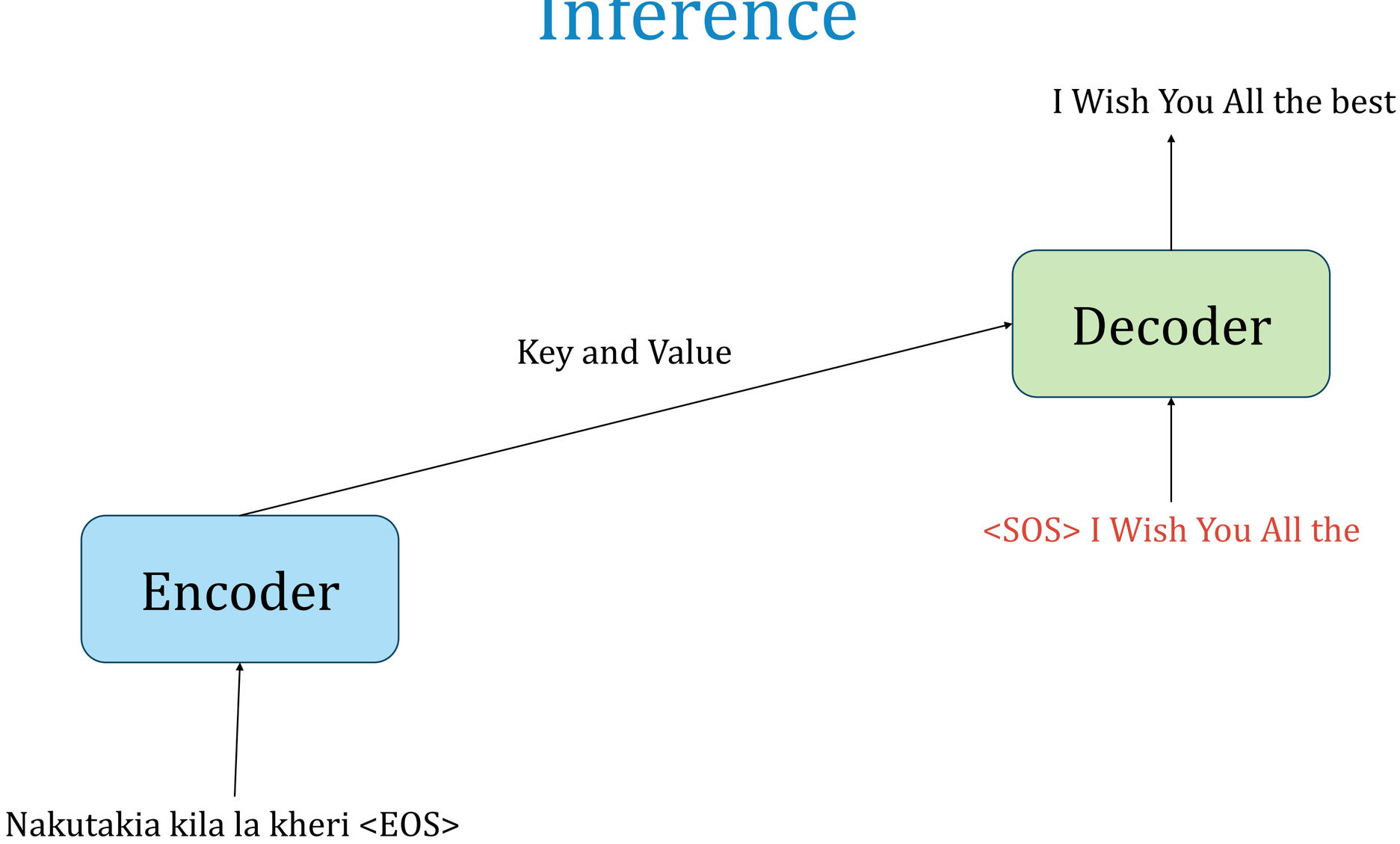
Output tokens



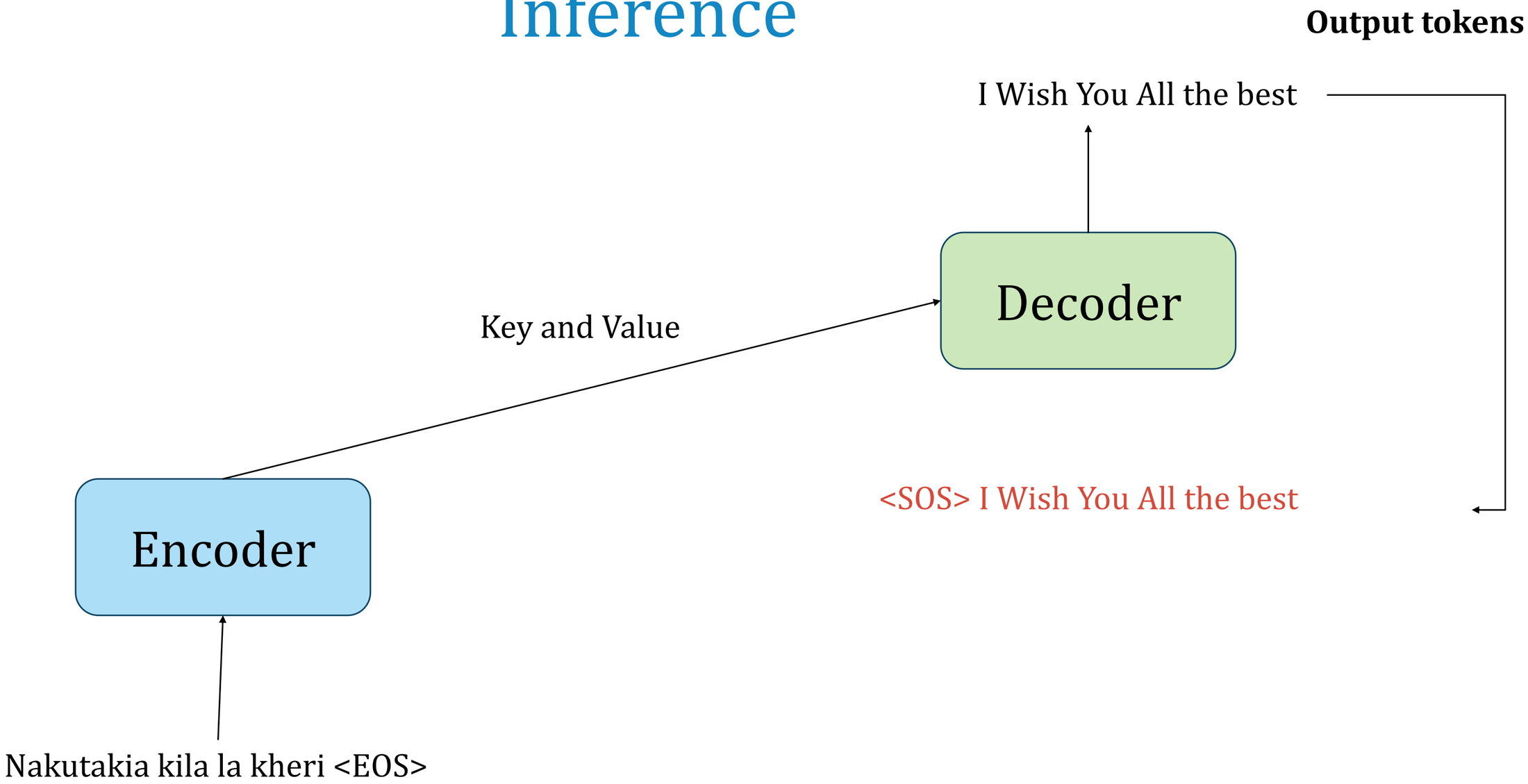
Inference


Output tokens
I Wish You All the
Decoder
Key and Value
Encoder
<SOS> I Wish You All the
Nakutakia kila la kheri <EOS>

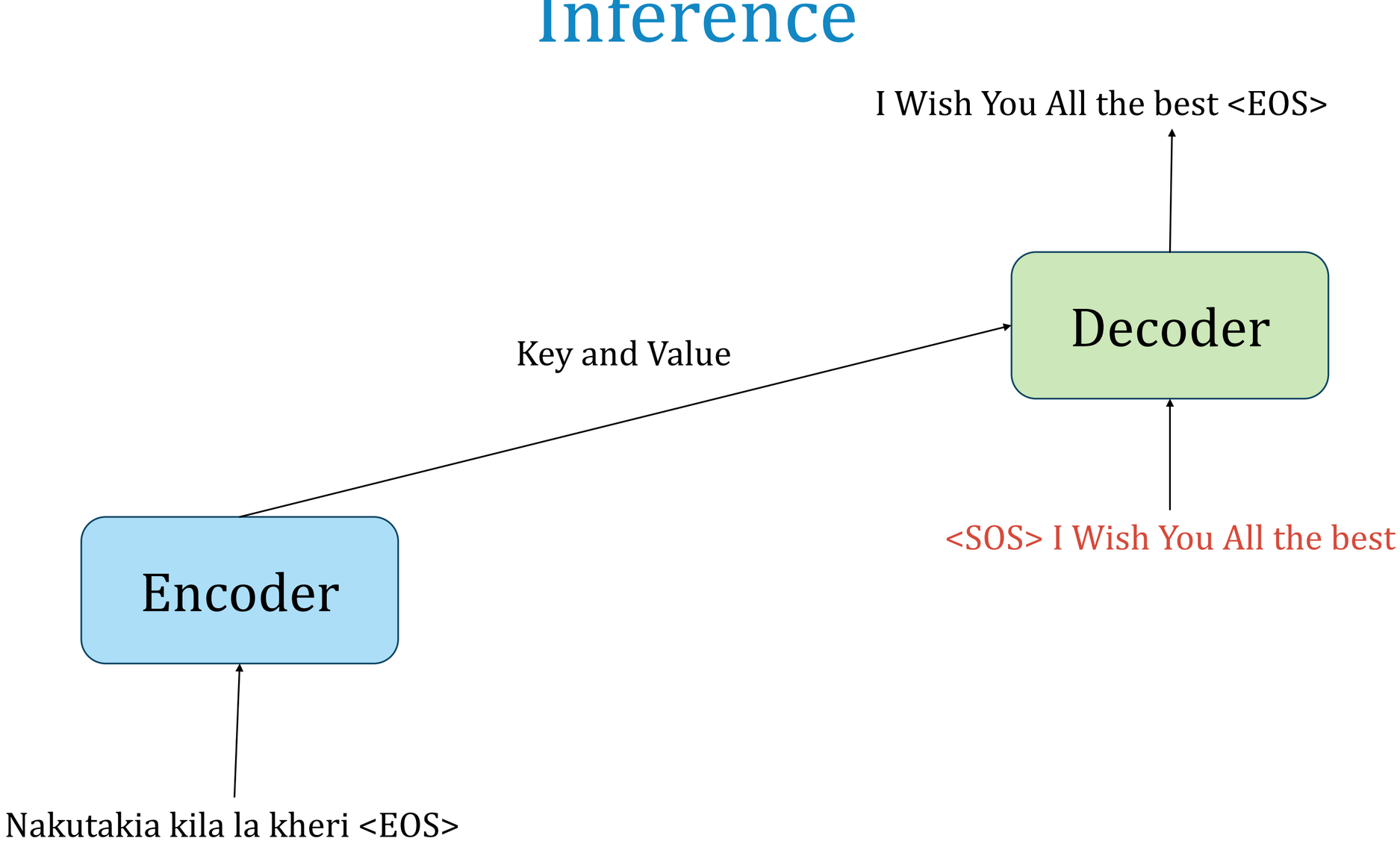
Inference


Output tokens
I Wish You All the best
Decoder
Key and Value
Encoder
<SOS> I Wish You All the
Nakutakia kila la kheri <EOS>

Inference


Output tokens
I Wish You All the best
Decoder
Key and Value
Encoder
<SOS> I Wish You All the best
Nakutakia kila la kheri <EOS>

Inference


Output tokens
I Wish You All the best <EOS>
Decoder
Key and Value
Encoder
<SOS> I Wish You All the best
Nakutakia kila la kheri <EOS>

CHECK YOUR INTUITION

A model is trained on sequences up to 512 tokens. At inference time, someone feeds it 4,000 tokens. What breaks, and why?

Consider both learned and sinusoidal encodings.

Applications: NLP, Vision, Speech

Language

BERT (encoder-only) for understanding; GPT family (decoder-only) for generation.

Vision

Vision Transformer (ViT) splits an image into patches and applies the same self-attention block.

Speech

Encoder-decoder speech recognition models cross-attend from text tokens to audio frames.

Devlin et al., "BERT," NAACL 2019 (arXiv:1810.04805) · Dosovitskiy et al., "ViT," ICLR 2021 (arXiv:2010.11929)

The cost of Attention: $O(n^2)$

Self-attention computes a score for *every pair* of positions: an n×n matrix.

Compute and memory both scale as O(n²) in sequence length.

Doubling the sequence length quadruples the cost of a single attention layer.



When Sequences Get Long and Modalities Multiply

text	image (224^2)	audio (1 min)	video
~2K tokens	**~50K pixels**	**~1M samples**	**tens of millions**

$O(n^2)$ attention over raw video is computationally infeasible.

A Design Problem

You need self-attention to work on a 10,000-token document.

Full self-attention is too expensive to run at that length.

Sketch an approach before we look at how the paper actually solves it.

Why Full Attention Doesn't Scale

Every token attends to every other token — so the number of attention pairs grows with the square of sequence length.

Sequence length	Attention pairs (n^2)	Relative to n=512
512	262,144	1×
2,048	4,194,304	16×
4,096	16,777,216	64×
10,000	**100,000,000**	**381×**

A 10,000-token document needs roughly 380× the attention computation of a 512-token one.

Longformer's Answer: Structured Sparsity

Instead of every token attending to every other token, restrict most attention to a local neighborhood — and grant a few tokens a global view.

Sliding window

Each token attends only to nearby tokens within a fixed-size window.

Cost grows linearly, not quadratically, with sequence length.

Dilated window

Some layers use gaps in the window, widening the receptive field without extra compute.

More context reached, same linear cost.

Global attention

A handful of task-specific tokens attend to and are attended to by the entire sequence.

Preserves the ability to aggregate global information where it's actually needed.

Longformer: The Long-Document Transformer," 2020

Vision Transformers

Can a Transformer Replace Convolutions?

CNNs bake in a strong assumption: nearby pixels matter more than distant ones. ViT asks what happens if you drop that assumption entirely and let attention decide.

Convolutional networks

Local receptive fields, weight sharing, built-in translation equivariance.

Strong inductive bias — works well with less data.

Vision Transformer

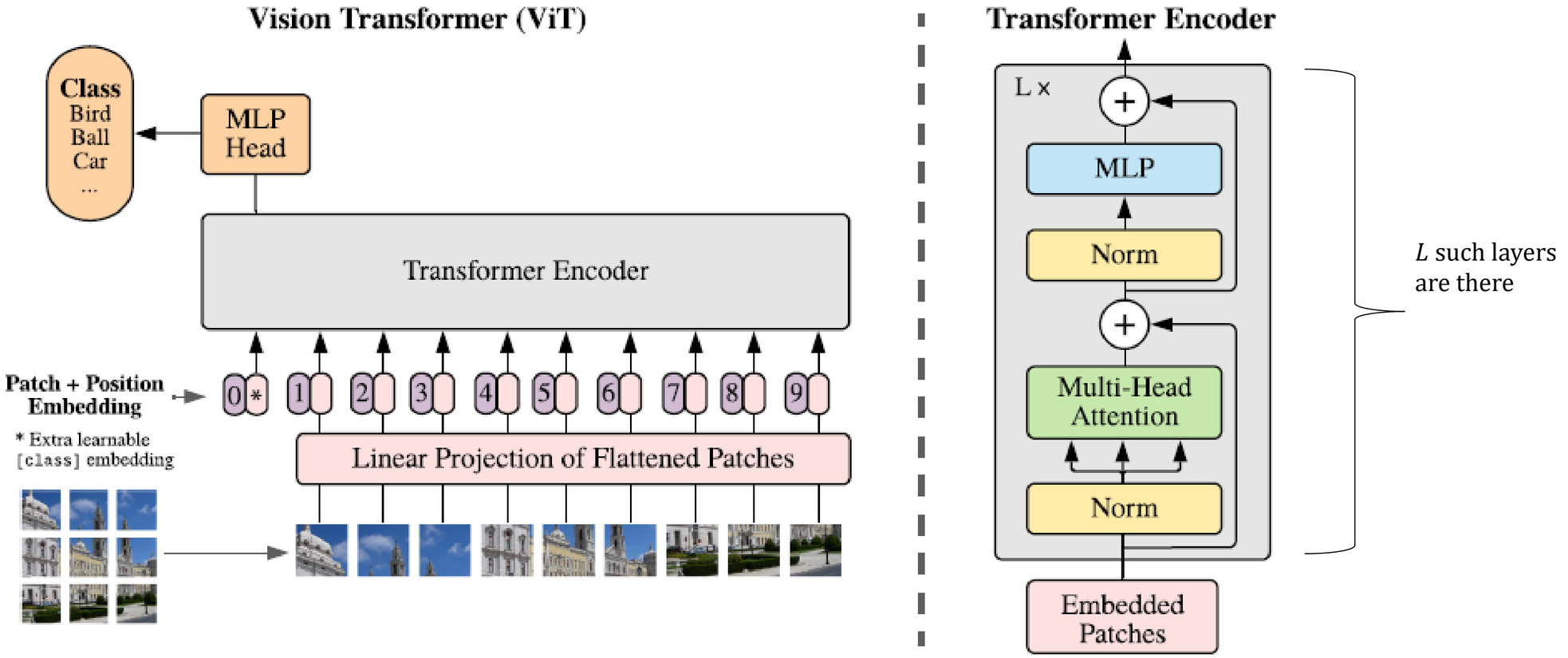
Global self-attention from the very first layer

Every patch can attend to every other patch

Weak inductive bias — needs more data, but scales further.

Dosovitskiy et al., "An Image Is Worth 16x16 Words: Transformers for Image Recognition at Scale," ICLR 2021

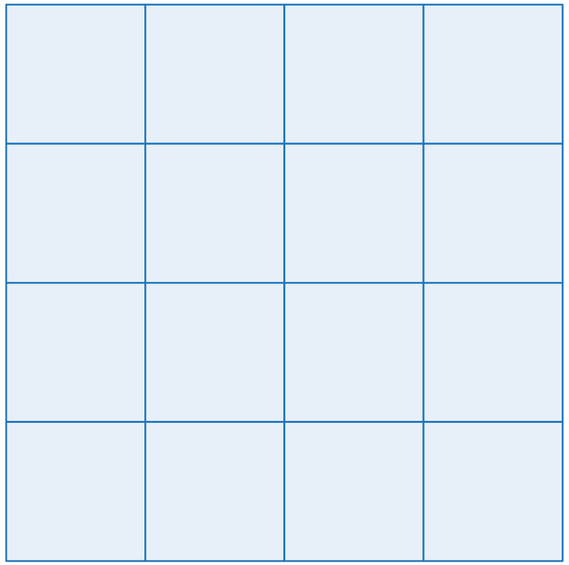
Vision Transformer



Dosovitskiy *et al.*, "AN IMAGE IS WORTH 16X16 WORDS: TRANSFORMERS FOR IMAGE RECOGNITION AT SCALE". ICLR. 2021

Turning an Image Into a Sequence of Tokens

A Transformer expects a sequence. An image is a grid. ViT bridges the gap by cutting the image into fixed-size patches and treating each patch as one token.



224 x 224 image, 16 x 16 patches
→ 14 x 14 = 196 patches

Flatten each patch

Each 16x16xC patch is flattened into a single vector.

Linear projection

A learned linear layer maps each flattened patch to a D-dimensional embedding.

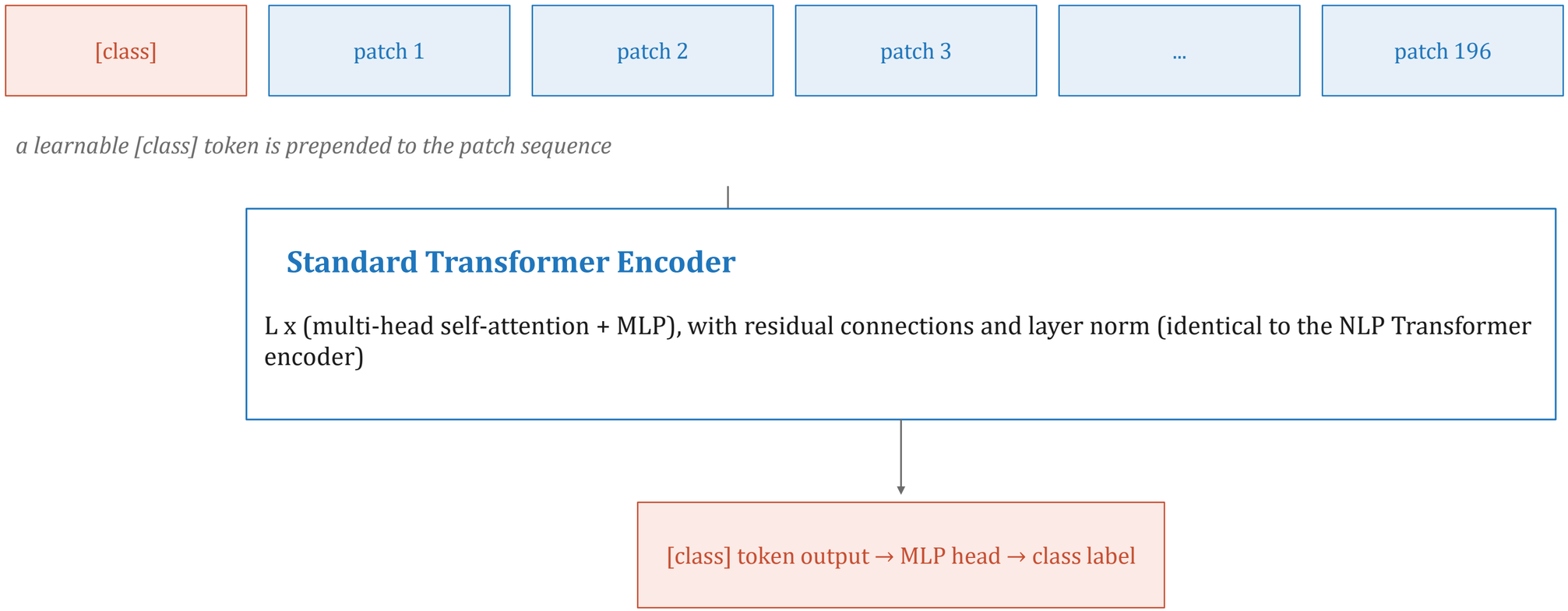
+ Position embedding

A learned vector per position is added, since attention has no sense of layout on its own.

A [class] Token, and a Standard Encoder

ViT reuses the standard Transformer encoder unchanged

The input is prepared in a different manner



What Is the [class] Token?

A Transformer produces one output vector per input token. Classification needs a single vector for the whole image. The [class] token is how ViT bridges that gap.



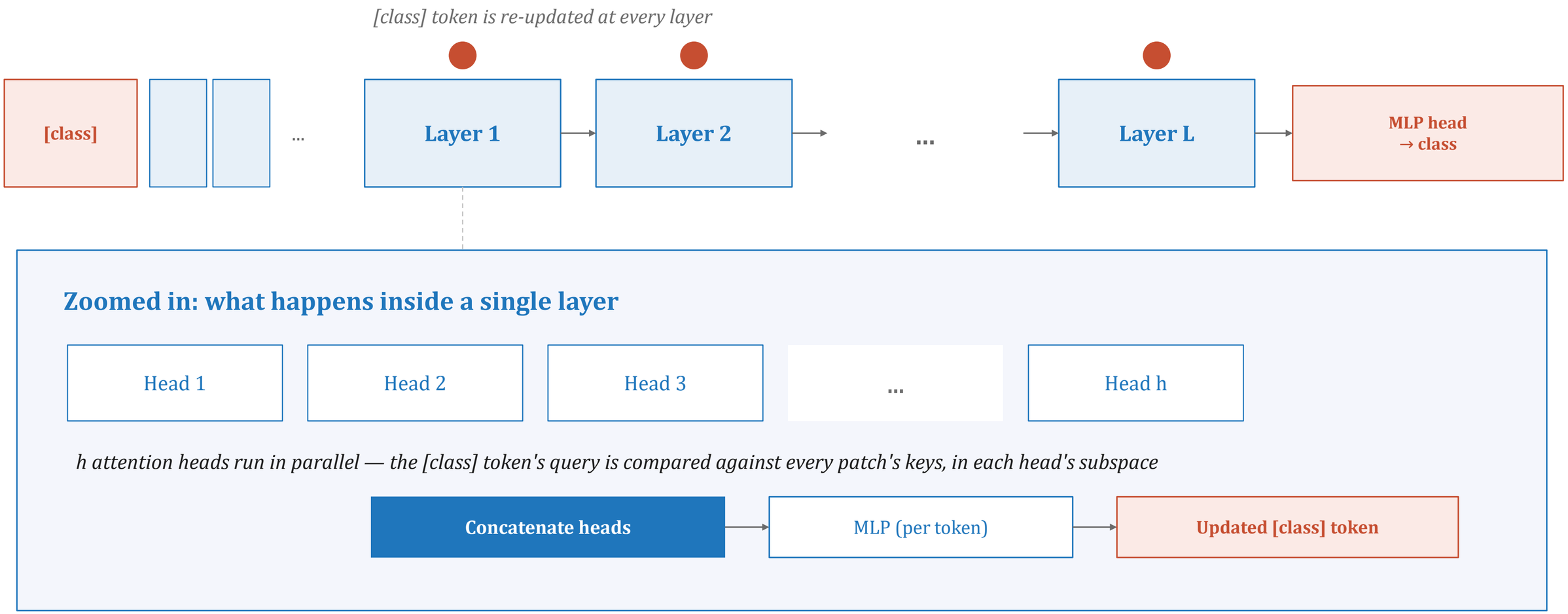
a learnable embedding, prepended to the sequence of flattened patches

A useful analogy

Reading a full book gives you every detail, but sometimes you just want the one-sentence summary. The [class] token is that summary sentence — a slot the model fills in with the gist of everything it has seen.

Dosovitskiy et al., "An Image Is Worth 16x16 Words," ICLR 2021 · Zhang et al., Dive into Deep Learning, Section 11.8 · Sohail, "Understanding the Role of the [class] Token in ViT," 2024

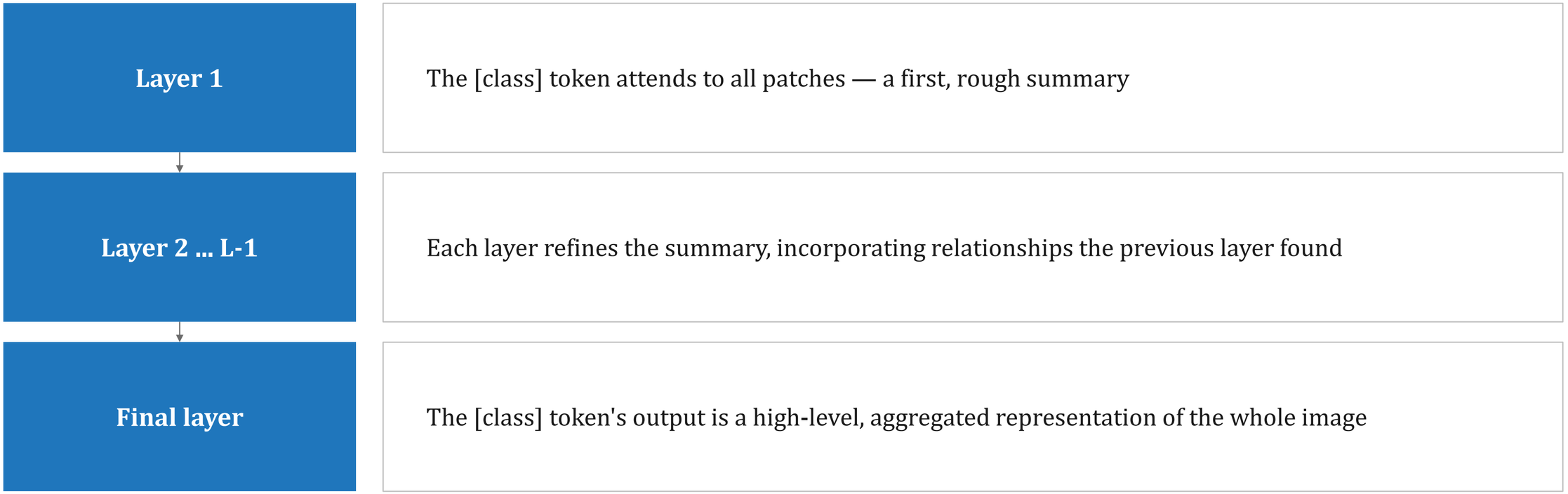
How the [class] Token Moves Through the Encoder



Depth (L) = how many times this whole block repeats. Width (h) = how many parallel views this block uses, each time (number of attention heads).

How the Token Gathers Information

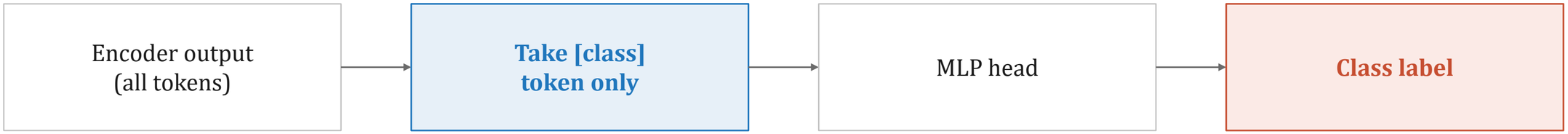
The [class] token starts out knowing nothing about the image. Self-attention is what lets it learn.



Because self-attention connects every token to every other token, nothing stops the [class] token from drawing on any patch, at any layer.

From Token to Prediction

Only the [class] token's final-layer output is used for classification — every patch token's output is discarded at this stage.



This mirrors BERT's [CLS] token in NLP — a special token trained to summarize a sequence for a downstream task.

An alternative worth knowing

Instead of a [class] token, some ViT variants average all patch output vectors (global average pooling) and classify that instead. Results are broadly comparable. The [class] token's advantage is that the model learns how to weight patches, rather than averaging them uniformly.

Why ViT Needs Scale to Win

- Dropping convolution's built-in assumptions is a double-edged move
  - The model becomes more flexible, but loses the inductive bias

On small/medium datasets

ViT underperforms similarly-sized CNNs without the assumption of locality, it has to learn spatial structure from data alone, and there isn't enough data to do that well.

At large scale (100M+ images)

ViT matches or exceeds state-of-the-art CNNs, while using substantially less compute to train. The lack of built-in bias stops being a handicap once there's enough data to learn structure directly.

Takeaway: with enough data, a more general architecture can learn what the bias used to supply, and go further.

If quadratic cost comes from every token attending to every other token, how might you redesign attention so cost grows linearly instead?

Rethinking the bottleneck

Don't let every token attend to every other token. Introduce a small, fixed-size set of *latent* vectors — and let **them** attend into the (arbitrarily large) input instead.

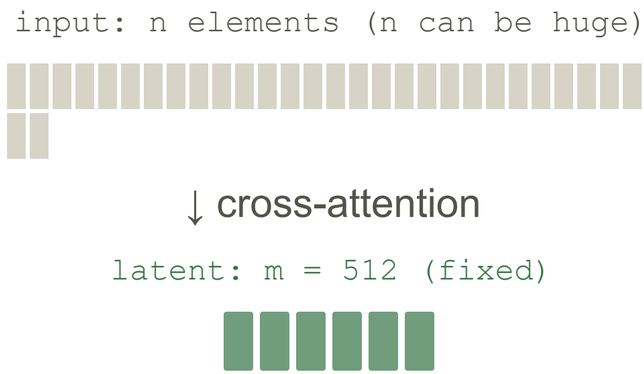
Jaegle et al., "Perceiver: General Perception with Iterative Attention," ICML 2021 — arXiv:2103.03206

The latent array: a fixed-size bottleneck

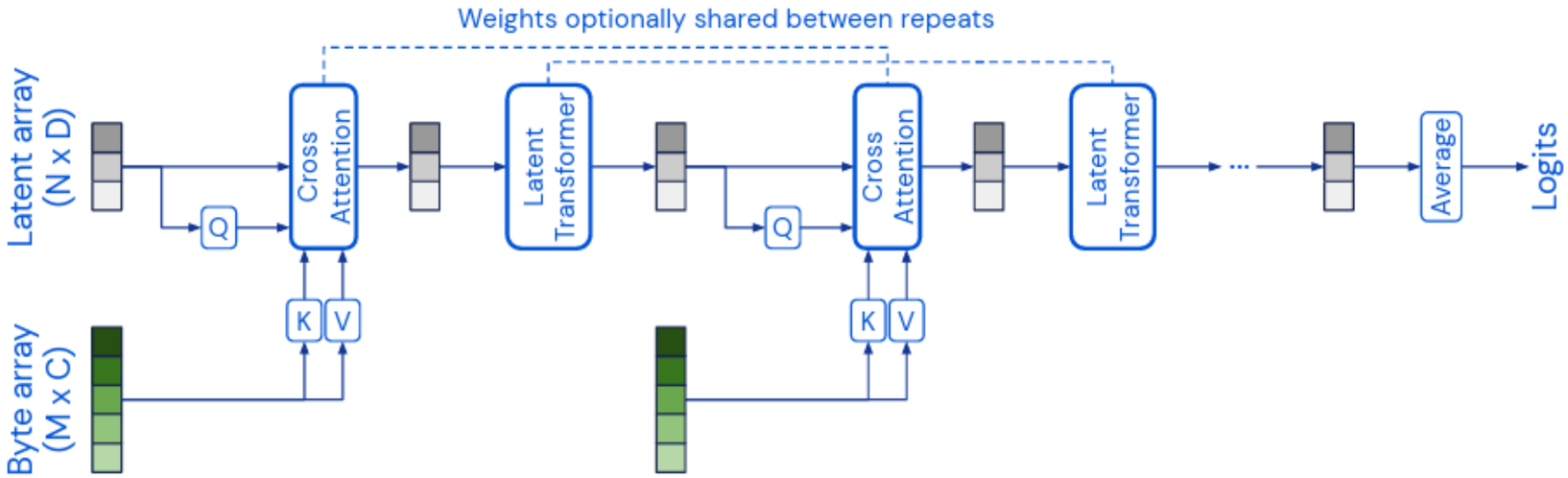
A small set of m learned vectors — say m = 512 — completely independent of input size n.

This latent array is the model's working memory: all deep processing happens here, at fixed cost.

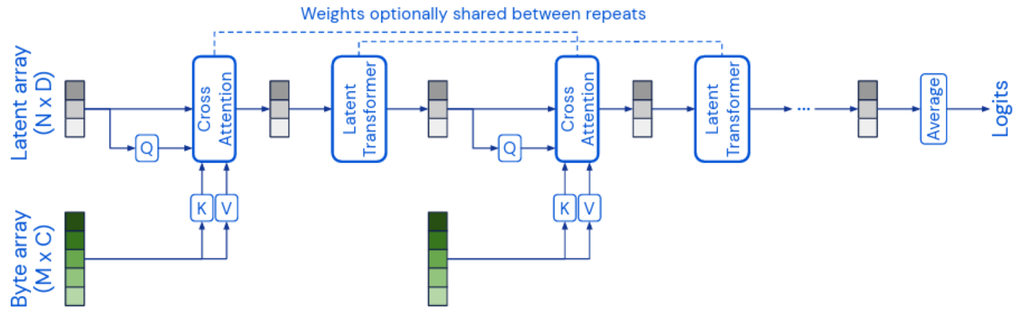
m stays fixed whether the input is a sentence or an hour of video.



Perceiver Architecture



Perceiver Architecture



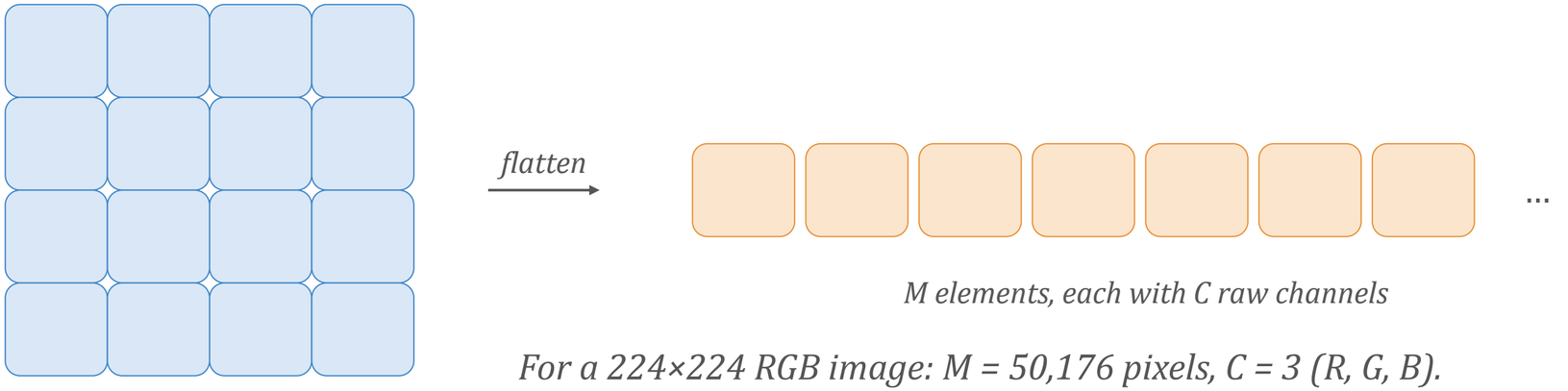
- The model leverages an asymmetric attention mechanism to iteratively distill inputs into a tight latent bottleneck, allowing it to scale to handle very large inputs.
- The Perceiver is an architecture based on attentional principles that can handle high-dimensional inputs like images, videos, audio, point clouds, and multimodal data.
- It does not rely on domain-specific assumptions, making it widely applicable.
- The Perceiver uses a cross-attention module to map high-dimensional input (large number of input indices M) to a fixed-size latent bottleneck (smaller number of latent indices N).
- It processes the latent space using a deep stack of Transformer-style self-attention blocks.
- The architecture alternates between cross-attention and latent self-attention blocks to iteratively attend to the input.

Perceiver Architecture

- The model leverages an asymmetric attention mechanism to iteratively distill inputs into a tight latent bottleneck, allowing it to scale to handle very large inputs.
- The Perceiver is an architecture based on attentional principles that can handle high-dimensional inputs like images, videos, audio, point clouds, and multimodal data.
- It does not rely on domain-specific assumptions, making it widely applicable.
- The Perceiver uses a cross-attention module to map high-dimensional input (large number of input indices M) to a fixed-size latent bottleneck (smaller number of latent indices N).
- It processes the latent space using a deep stack of Transformer-style self-attention blocks.
- The architecture alternates between cross-attention and latent self-attention blocks to iteratively attend to the input.

Handling the Input: Step 1- Flatten the Input

An image is a 2D grid — not a sequence. Perceiver has no built-in notion of grid structure, so it flattens first.



H × W grid

M elements, each with C raw channels

For a 224×224 RGB image: M = 50,176 pixels, C = 3 (R, G, B).

The problem: flattening throws away position. The model can no longer tell which pixels were neighbors in the original grid.

That's exactly what the next step has to fix.

Step 2: Put Position Back — Fourier Features

For every element, encode its original coordinate using sines and cosines at several frequencies:

$$\gamma(v) = [\, v, \sin(f_1 \pi v), \cos(f_1 \pi v), \ldots, \sin(f_k \pi v), \cos(f_k \pi v) \,]$$

v is the coordinate (e.g. x or y position); K frequency bands give both coarse and fine positional detail.

raw value + **γ(position)** → **concatenated**

Result: the M × C input byte array — M elements, each now [raw value, position]

Same recipe for any modality — only the number of coordinate axes changes: 2 for images, 1 for audio, 3 for video.

Setting Up, and Reading the Input

1 Initialize the Latent Array

- Learned array, (initialized randomly)
- Truncated normal init: mean 0, std 0.02, bounds [-2, 2]
- Performance is robust to the exact init scale

2 Input Processing & Prep

- Layer norm applied to all inputs
- Inputs passed through linear projections
- Produces three projections: Q, K, V

3 Cross-Attention (Reads Input)

- Query = projection of the latent array (not the input)
- Key & Value = projections of the input byte array
- Q, K, V share channel count = min(input channels)
- Output passes through a linear projection
- Residual: latent input added back to the output
- First cross-attend is unshared; later ones share weights

Processing the Latent Array

4 Self-Attention Block

- Layer norm applied to latent inputs
- Standard QKV self-attention, computed on the latents
- Output passed through a linear projection
- Residual: input added to output before the next block

5 Dense (MLP) Block

- Layer norm applied to the input
- First linear layer
- GELU nonlinearity
- Second linear layer
- No dropout; earlier experiments showed it hurts performance
- All linear layers preserve input dimensionality

Together, cross-attention + self-attention + the dense block form one full processing block — this is what repeats in Step 6.

Iterating, and Producing an Output

6 Iterative Processing

- Steps 3–5 repeat for multiple iterations (e.g. 8 on ImageNet)
- Weight sharing across repeated blocks mitigates overfitting
- Each iteration refines the latent representation

7 Output Generation

- Average the final latent self-attention output over the index dimension
- Result: a single global summary vector
- Project to the number of target classes with one linear layer
- Analogous to how ResNets turn a feature map into logits

Key architectural principles

- Fully residual design — input is always added back to the output
- Asymmetric attention: $O(MN)$ instead of $O(M^2)$, since $N \ll M$
- Linear layer tiling — every linear layer applied as a 1×1 convolution across the input dimension
- No dropout — removed based on validation performance on ImageNet

Perceiver: Cross-attention into a Latent Space

$$Q = Z\ W^{Q}\ K = X\ W^{K}\ V = X\ W^{V}$$

$$Z' = \mathrm{softmax}\left(QK^{T} / \sqrt{d_k} \right) V$$

Z is the latent array (m vectors), X is the raw input (n elements). The resulting attention matrix has shape $(m \times n)$ — linear in the input size n.

Perceiver Complexity: O(n·m)

cross-attention: $O(m \cdot n)$
latents (m) query the full input (n)

latent self-attention: $O(m^2)$
m is fixed and small — independent of n

total: $O(m \cdot n + m^2) = O(n)$
linear in input size, since m is a constant



Perceiver IO: Arbitrary Outputs

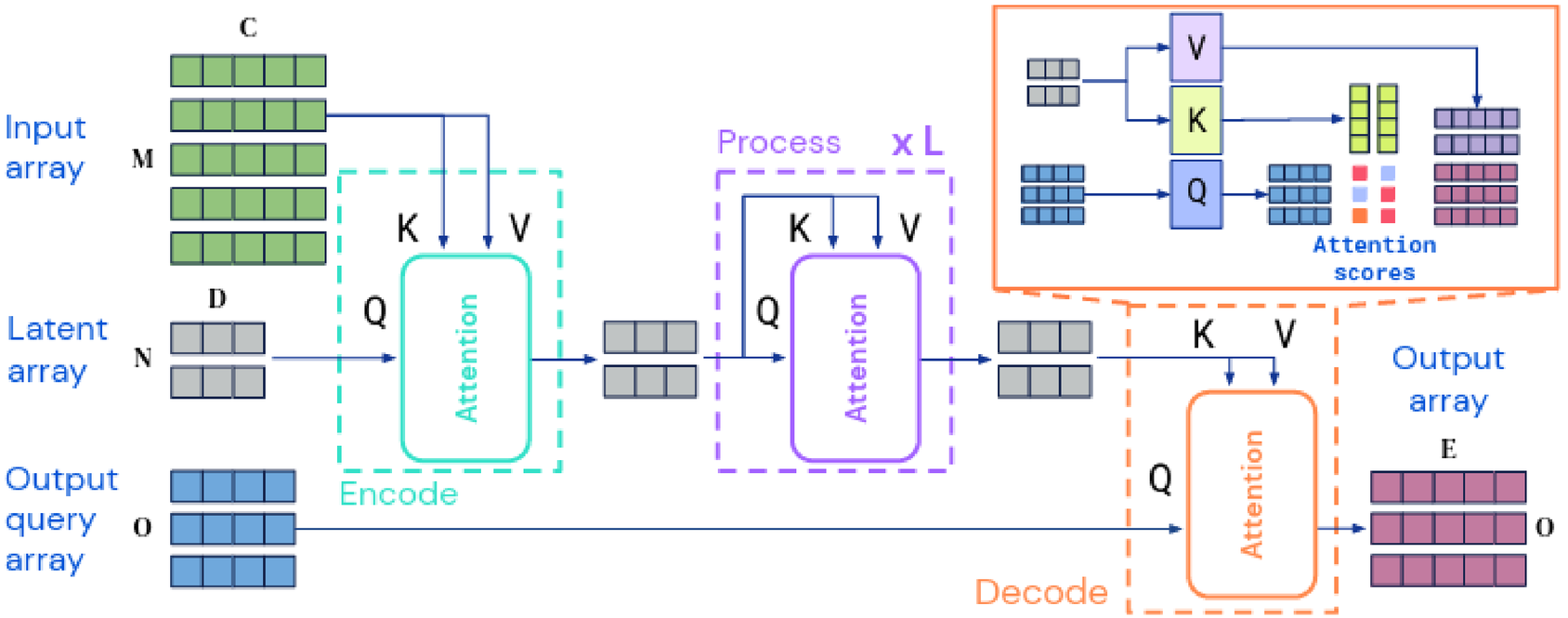
The original Perceiver still ends in a single fixed-size output, useful for classification, limiting for anything structured.

Perceiver IO applies the same trick to the output: cross-attend *out* of the latents using a set of output queries.

Now outputs can be:

- a class label
- a full segmentation mask, per pixel
- a sequence of translated tokens
- multiple modalities at once (e.g. audio + video labels)

Perceiver IO Architecture



Output shape q is set by however many output queries you provide — completely decoupled from input size n.

Perceiver IO: Querying for Any Output Structure

Segmentation

One output query per pixel coordinate → a per-pixel label.

Translation

One output query per target position → a generated token there.

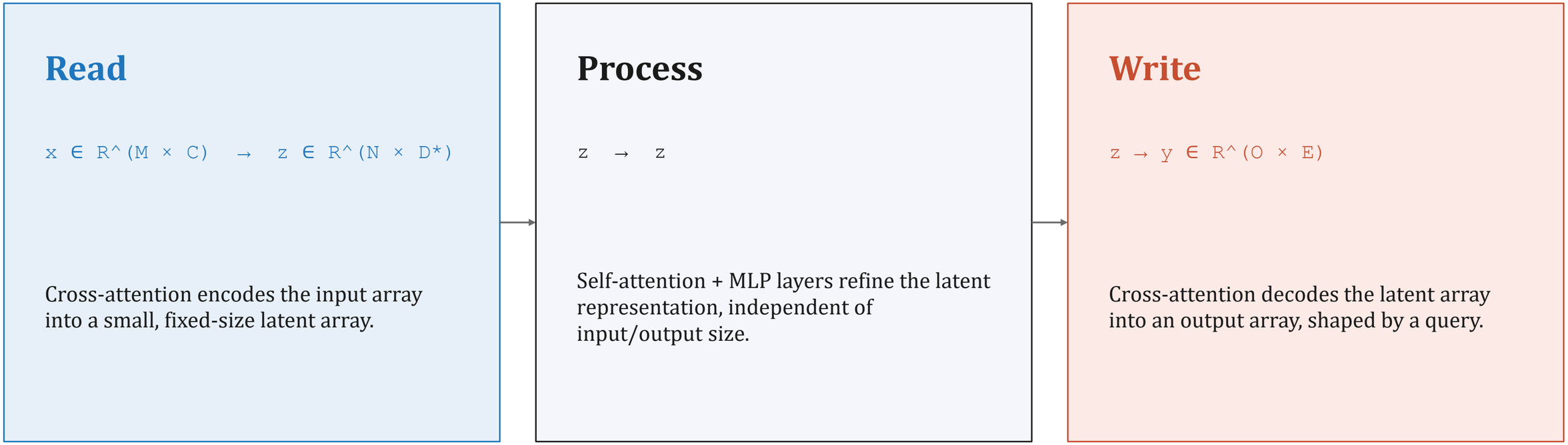
Multi-task output

Different query sets for different tasks, sharing one latent representation.

Each output query encodes "what do I want to know here?" — the model's answer is a cross-attention read from the latents.

A General Architecture: Read, Process, Write

Perceiver IO scales linearly with both input AND output size — the original Perceiver only solved half of that (inputs, not outputs).



M, O can be huge and completely unrelated in size — N stays small and fixed throughout.

Encoding the Input, Processing the Latents

Step 1 · Encoding

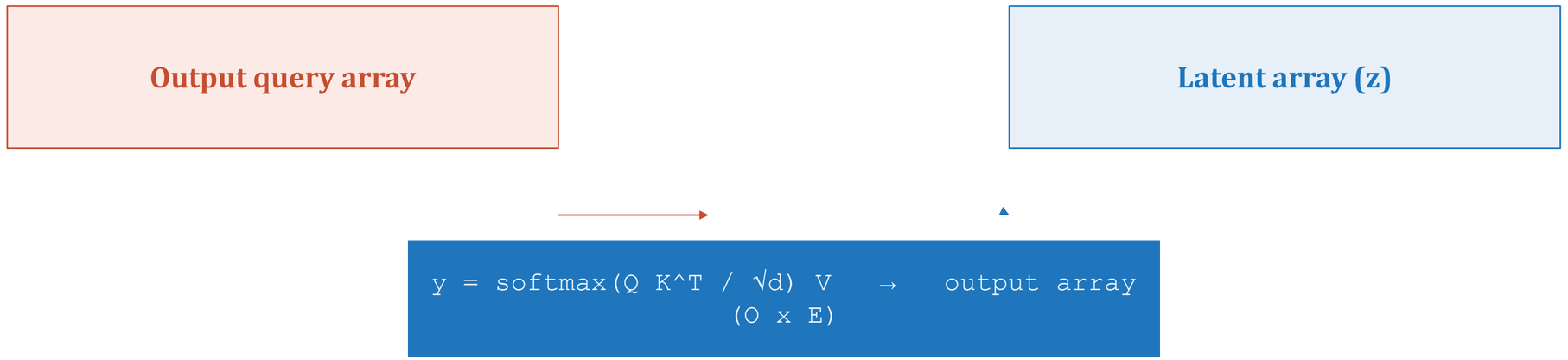
- Input x has shape M x C — M elements (pixels, tokens, patches), C features each
- Latent z has shape N x D* — N learned vectors, N << M
- Cross-attention: input supplies keys & values, latents supply queries
- Cost scales linearly with M — no quadratic blow-up on the input side

Step 2 · Processing

- Each processing layer = self-attention among latents + a per-element MLP
- Cost depends only on N — completely independent of input or output size
- Multiple layers stack to iteratively refine the latent representation
- Result: a latent array that has integrated information from the whole input

Decoding: Querying for Structured Outputs

The decoder is itself a cross-attention step — but here the latent array supplies keys & values, and a query array (shaped like the desired output) supplies the queries.



How the query is built depends on the task:

Classification

One learned embedding, reused for every example

Spatial / sequence outputs

Position encoding marks which output location is being decoded

Multi-task / multi-modal

One learned query per task or modality, so the model can tell them apart

Content-dependent

Include the relevant input feature itself, e.g. optical flow at a queried pixel

Original Perceiver vs. Perceiver IO

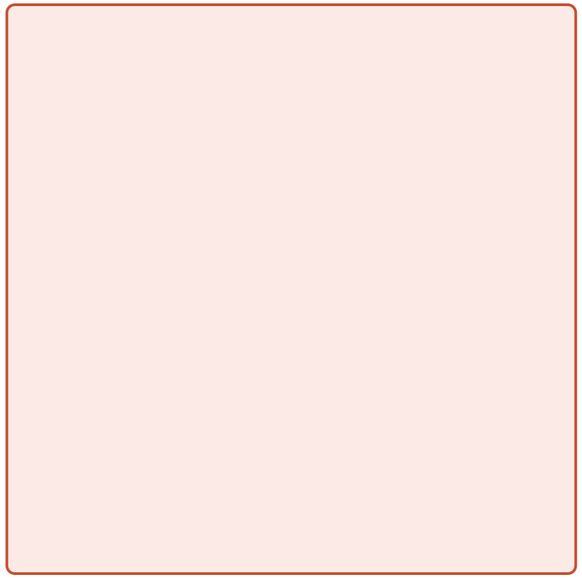
	Original Perceiver	Perceiver IO
Output types	Simple classification only	Arbitrary structured outputs — dense, sequence, multi-modal, multi-task
Output mechanism	Single aggregated logit	Flexible, query-based decoding
Where queries are used	Encoder only	Both encoder and decoder
Generality	Handles diverse inputs	Handles diverse inputs AND outputs

Because decoding is also just cross-attention, each output element depends only on its own query and the shared latent array — outputs can be produced fully in parallel.

Transformer vs. Perceiver vs. Perceiver IO

	Transformer	Perceiver	Perceiver IO
Input handling	self-attends over all n tokens	latents cross-attend into n inputs	same as Perceiver
Output handling	task-specific head on fixed positions	single fixed-size output (e.g. class logits)	arbitrary — output queries cross-attend out
Complexity	$O(n^2)$	$O(n \cdot m)$	$O(n \cdot m + m \cdot q)$

Complexity Comparison: $O(n^2)$ vs. $O(n \cdot m)$



Transformer: n × n



Perceiver: m × n (m fixed, e.g. 512)

At n = 50,000 (an image in raw pixels), $n^2 \approx 2.5$ billion vs. $m \cdot n \approx 25$ million with m = 512 — a ~100× reduction.

PROBLEM · NUMERICAL + CODE

Perceiver latent sizing

An input has n = 100,000 elements. Compare total attention FLOPs (roughly proportional to matrix size) for m = 256 vs. m = 1024 latents, considering both the cross-attention (m·n) and latent self-attention (m²) steps.

Solution

m=256: 256·100k + 256² ≈ 25.7M m=1024: 1024·100k + 1024² ≈ 103.4M → 4× latents costs ~4× compute, dominated by the cross-attn term here.

```
def perceiver_flops(n, m, d):
    cross = m * n * d      # cross-attn
    latent = m * m * d     # latent self-attn
    return cross + latent

# Task: plot perceiver_flops(n, m, d)
# vs. n*n*d for n in [1e3 .. 1e6],
# m fixed at 512. Where do they cross?
```

Building One Query: Segmentation

Goal: predict a class label for a single pixel at (x=80, y=200) — repeated once per pixel across the image.

1 Position encoding

```
fourier_features(x=80, y=200)
```

Identifies which pixel this query is asking about.

2 Task embedding

```
learned_vector["segmentation"]
```

A shared vector telling the model this is a segmentation query.

3 Content feature

```
skipped
```

Not needed — the latent already encodes what's visually at this location.

`query = concat(pos_encoding, task_embedding)` → `dimension 64 + 64 = 128`

$$output = softmax(query \cdot K^T / \sqrt{d}) \cdot V \quad \rightarrow \quad \text{class logits} \rightarrow \text{argmax} \rightarrow \text{"road"}$$

Scaling Up

The same construction repeats for every pixel — same task embedding, different position — and every query is independent, so all of them run in parallel.



one query per pixel — 65,536 for a 256×256 image

	Segmentation	Optical flow
Position encoding	✓ (per pixel)	✓ (per pixel)
Task embedding	✓	✓
Content feature from input	✗ not needed	✓ improves accuracy
Output per query	Class logits	Motion vector (dx, dy)

Same recipe, adapted per task: position + task, and content only where it genuinely helps.

Can you design the query vector for the optical flow problem?

Applications: Perceiver IO Across Modalities

Multimodal audio-video

Raw audio and video frames feed the same latent array — no modality-specific architecture.

Optical flow

Dense per-pixel motion output via output queries at every pixel coordinate.

Point clouds

Unordered 3D point sets — no grid structure required, unlike CNNs.

StarCraft II action modeling

Structured game-state input and structured action output, same architecture.

Jaegle et al., "Perceiver IO," ICLR 2022 — arXiv:2107.14795

The Attention Family Tree

Self-attention	Q, K, V from the same sequence — $O(n^2)$
Multi-head	several attention subspaces run in parallel
Cross-attention	Q from one sequence, K/V from another
Perceiver	fixed-size latents cross-attend into input — $O(n \cdot m)$
Perceiver IO	output queries cross-attend out — arbitrary structured output

Non Parametric Transformers

- Most deep learning models predict using only learned parameters and the features of a single input.
- Direct dependencies between datapoints are ignored entirely — each prediction is made in isolation.
- This paper challenges that assumption directly.
- Goal: let the model use the dataset itself, not just one row of it, at prediction time.

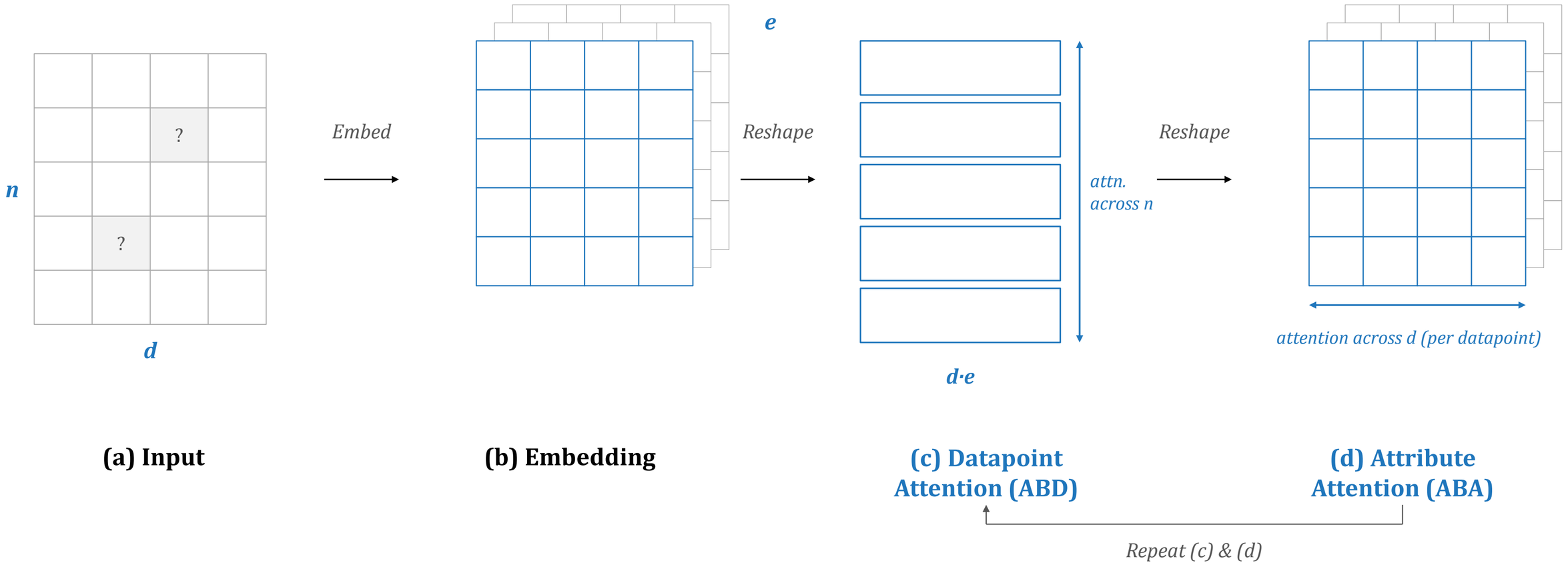
Key Idea: Non-Parametric Transformers (NPTs)

- Input: the entire dataset, not just one datapoint at a time.
- Self-attention runs across datapoints to learn relationships between them.
- Combines parametric learning (standard deep nets) with non-parametric reasoning (dataset lookup, like k-NN).
- End-to-end training decides how much to rely on each mechanism — nothing is hand-coded

Kossen et al., Self-Attention Between Datapoints, NeurIPS 2021 (arXiv:2106.02584)

Architecture: Reshape, Then Alternate Attention

ABD and ABA are not parallel branches — they alternate, each on a different reshape of the same tensor



Kossen et al., Self-Attention Between Datapoints, NeurIPS 2021 (arXiv:2106.02584)

Architecture: Reshape, Then Alternate Attention

ABD and ABA are not parallel branches — they alternate, each on a different reshape of the same tensor

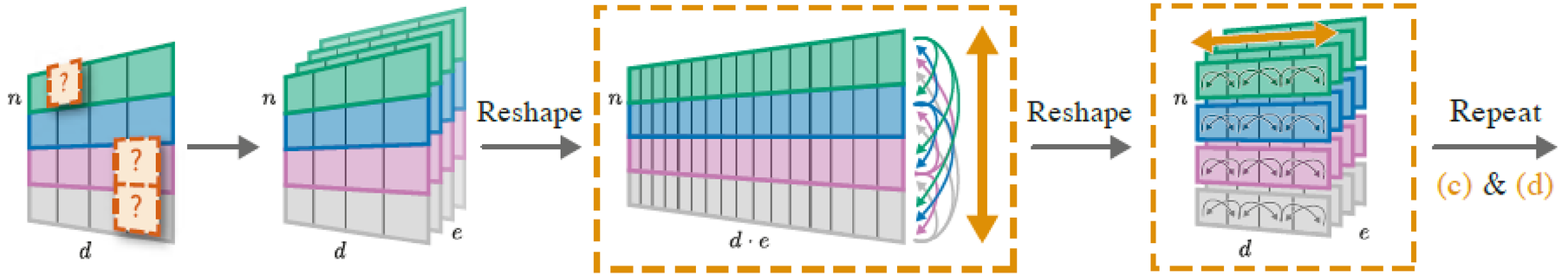


Figure 2: Overview of the Non-Parametric Transformer. (a) The input dataset and mask matrix are stacked and (b) linearly embedded for all datapoints independently. NPT then applies (c) Attention Between Datapoints (ABD, §2.4) across all n samples of hidden dimension $h = d \cdot e$. (d) Attention Between Attributes (ABA, §2.5) then attends between the attributes for each datapoint independently. We repeat steps (c) and (d) and obtain a final prediction from a separate linear projection (not shown).

Kossen et al., Self-Attention Between Datapoints, NeurIPS 2021 (arXiv:2106.02584)

ABA: Attention Between Attributes

$H_i^{(\ell)} \in R^{d \times e}$, i = 1, ..., n

ABA applies multi-head self-attention independently to each row (each datapoint's own attribute vector), batched over the n dimension.

What it does

Transforms attribute representations of single datapoints independently — exactly like standard Transformer self-attention over tokens in a sentence.

Why it matters

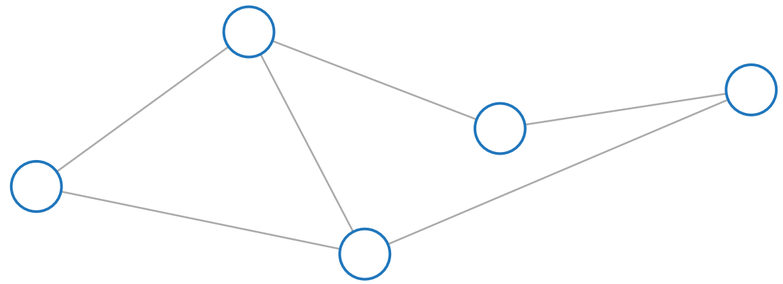
Prepares better per-datapoint representations, learning suitable transformations before the model reasons about between-datapoint interactions in ABD.

In NLP terms: attention is usually applied between the tokens (attributes) of a sentence (datapoint) — never between different sentences. That is exactly what ABA does here.

ABD: Attention Between Datapoints (“Cross-Sample Attention”)

$$ABD(H^{(\ell)}) = MHSA(H^{(\ell)}) = H^{(\ell+1)} \in R^{n \times h}$$

ABD flattens each datapoint's representation to a single row, then applies multi-head self-attention across all n rows at once — letting the model learn which samples should attend to which other samples.



each node = one datapoint; edges = learned attention weights between samples

Key distinction: standard Transformers attend between attributes within one datapoint (tokens in a sentence) and never across datapoints. ABD is the opposite — attention runs between samples, not within one.

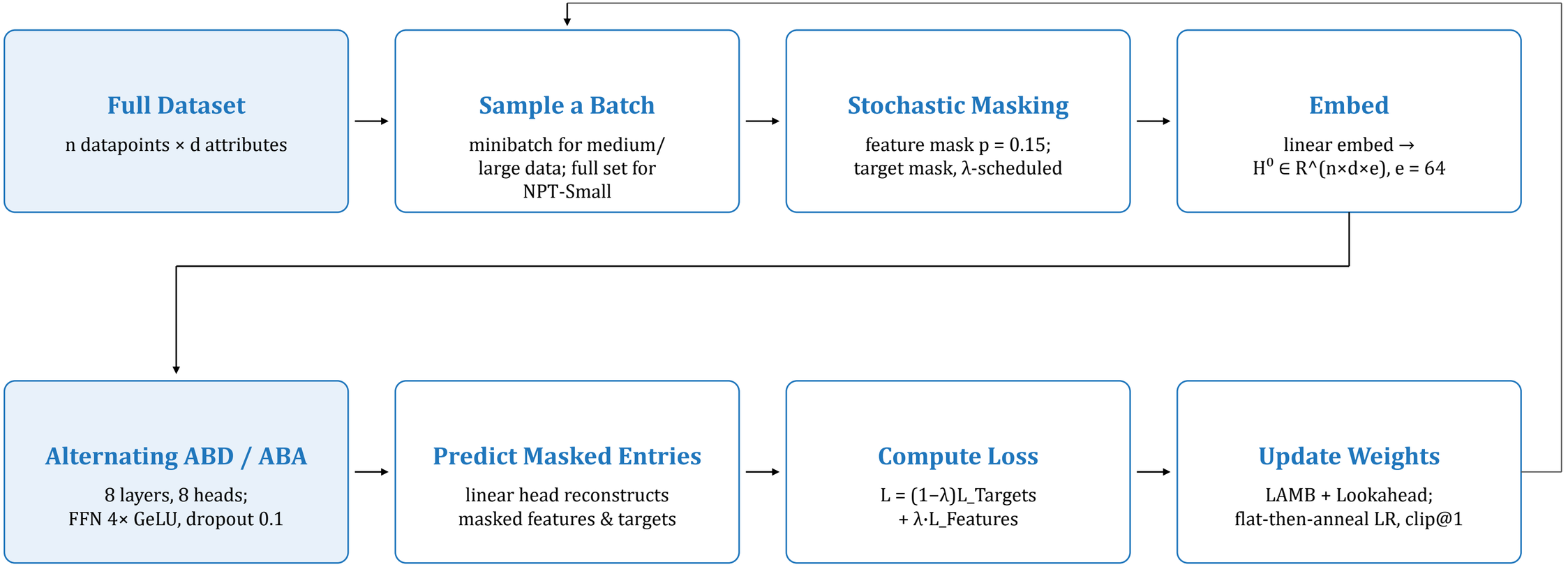
Stacking multiple ABD layers lets NPT learn higher-order interactions between datapoints — not just direct pairwise similarity.

Training Objective: Masked Reconstruction

- Inspired by masked language modeling, as in BERT.
- Features and targets are randomly masked during training — the same masking shown in the dataset table on the architecture slide.
- Loss combines target prediction with feature reconstruction across the masked entries.
- This encourages the model to learn dataset-wide representations, not just a mapping from one row's features to its label.

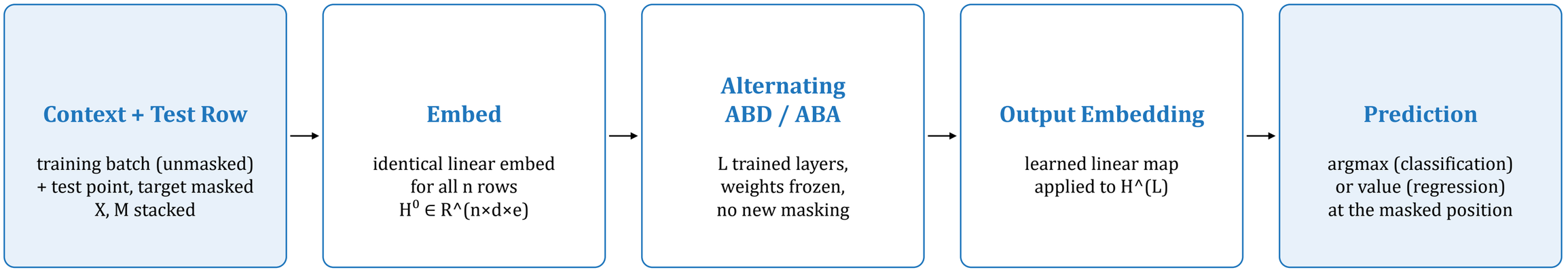
NPT Training Pipeline

Every step: fresh batch, fresh mask, end-to-end update



NPT Inference Pipeline

One forward pass, frozen weights — only the test target is masked



Formally: NPT computes $p(X_M \mid X_O)$ — the distribution over masked entries given everything observed.

Only the test point's target is ever masked — its features, and every entry of every training point, stay visible. This is how NPT avoids test-set leakage.

Large datasets: minibatched, with training and test data always kept in the same batch so the test row can still attend to real training targets.

Does NPT Need Causal Connectivity Between Samples?

No — the model learns end-to-end whether cross-datapoint attention is worth using

Flexible learning	NPT learns from data how much to rely on inter-datapoint relationships — nothing forces it to use them.
Falls back gracefully	*"When NPTs do not find it advantageous to rely on attention between datapoints during training, they can learn to completely ignore other inputs, essentially collapsing into a standard parametric model."*
No architectural assumption	The architecture provides the capacity for cross-datapoint reasoning — it does not assume such relationships exist.

Empirical evidence: on datasets such as Forest Cover, Kick, and Breast Cancer, corrupting the input's cross-datapoint structure does not significantly hurt performance — NPT had learned to lean on individual features there instead.

Takeaway: NPT is a strict generalization of a standard parametric model — it can always fall back to one, and only pays the ABD cost where it helps.

Kossen et al., Self-Attention Between Datapoints, NeurIPS 2021 (arXiv:2106.02584)

Properties of NPTs

- Permutation equivariant: shuffling the order of datapoints shuffles the predictions accordingly — no dependence on row order.
- Can generalize beyond memorization — it learns a rule for using other datapoints, not a fixed lookup table.
- Strictly generalizes parametric models: with attention between datapoints switched off, NPT reduces to a standard per-row predictor.

Kossen et al., Self-Attention Between Datapoints, NeurIPS 2021 (arXiv:2106.02584)

Results

Setting	Finding
Tabular UCI datasets	Strong performance on both classification and regression tasks
vs. gradient-boosted trees	Competitive with CatBoost, XGBoost, and LightGBM — the standard baselines for tabular data
CIFAR-10 and MNIST	Early, promising results outside the tabular setting the architecture was designed for
Interaction analysis	Demonstrates the model actually exploits interactions between datapoints, not just within-row features

Kossen et al., Self-Attention Between Datapoints, NeurIPS 2021 (arXiv:2106.02584)

Contributions

- Introduces NPTs: a general-purpose architecture for dataset-level reasoning.
- Demonstrates competitive results on tabular benchmarks against strong non-deep-learning baselines.
- Provides insight into how and when models make use of datapoint interactions.
- Opens a paradigm beyond purely parametric deep learning — one this course will revisit when we discuss retrieval-augmented methods later in the semester.

Why is it called non-parametric?